1. Instalación de librerías

In [1]:

```
# INSTALACIÓN DE DEPENDENCIAS

%pip install -q \
    ultralytics==8.4.129 \
    opencv-python==5.0.0.93 \
    transformers==5.16.1 \
    torch==2.13.0 \
    torchvision==0.28.0 \
    numpy==2.1.3 \
    pandas==2.2.3 \
    matplotlib==3.9.3 \
    sentencepiece==0.2.2 \
    tqdm==4.70.0 \
    pillow==11.0.0 \
    scikit-learn
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run:
C:\Users\juank\AppData\Local\Programs\Python\Python312\python.exe -m
pip install --upgrade pip
```

2. Importación de librerías

In [3]:

```
from ultralytics import YOLO
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import torch
import cv2
import pandas as pd
import os
```

3.Carga de modelos

In [4]:

```
# Cargar modelos


yolo = YOLO("yolo11n.pt")

clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
```

```
clip_processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-
patch32")

print("Modelos cargados correctamente")
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
Loading weights: 100%|██████████| 398/398 [00:00<00:00, 24824.64it/s]
Modelos cargados correctamente
```

4. Configuración del experimento

In [99]:

```
# 4. CONFIGURACIÓN GENERAL DEL EXPERIMENTO


# Directorio de los vídeos experimentales
VIDEOS_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS"

# Directorio de resultados
OUTPUT_DIR = os.path.join(
    VIDEOS_DIR,
    "resultados_experimentales"
)

os.makedirs(
    OUTPUT_DIR,
    exist_ok=True
)

# Umbrales iniciales de detección
CONFIDENCE_THRESHOLD = 0.40
TRAFFIC_LIGHT_THRESHOLD = 0.10

# Clases relevantes para el escenario peatonal
RELEVANT_CLASSES = {
    "person",
    "car",
    "truck",
    "bus",
    "motorcycle",
    "bicycle",
    "traffic light"
}

# Diez vídeos utilizados en la experimentación
VIDEO_FILES = [
```

```
    "20260611_184739.mp4",
    "20260611_184808.mp4",
    "20260611_185035.mp4",
    "20260611_185443~2.mp4",
    "20260611_185616.mp4",
    "1000179702.mp4",
    "1000179704.mp4",
    "1000179707.mp4",
    "1000179709.mp4",
    "1000179711.mp4"
]

print("Configuración experimental cargada.")
print("Vídeos seleccionados:", len(VIDEO_FILES))
```

```
Configuración experimental cargada.
Vídeos seleccionados: 10
```

5. Funciones auxiliares

In [100]:

```
# 5.1 CLASIFICACIÓN ZERO-SHOT CON CLIP

def classify_with_clip(image_region, prompts):

    inputs = clip_processor(
        text=prompts,
        images=image_region,
        return_tensors="pt",
        padding=True
    )

    with torch.no_grad():

        outputs = clip_model(**inputs)

        probs = (
            outputs.logits_per_image
            .softmax(dim=1)[0]
        )

    best_idx = probs.argmax().item()

    return {
        "label": prompts[best_idx],
        "confidence": float(probs[best_idx]),
        "all_probabilities": {
            prompt: float(prob)
            for prompt, prob in zip(
```

```
                prompts,
                probs
            )
        }
    }
```

6. Procesamiento completo del vídeo con YOLO

6.1 Generación del vídeo anotado y del CSV de detecciones

In [101]:

```
# Función que procesa un vídeo

def process_video_with_yolo(video_path, output_dir):
    """
    Procesa un vídeo completo con YOLO.

    Genera:
    - un vídeo anotado;
    - un CSV con todas las detecciones.

    Devuelve:
    - DataFrame con las detecciones;
    - resumen del procesamiento.
    """

    video_name = os.path.splitext(
        os.path.basename(video_path)
    )[0]

    output_video_path = os.path.join(
        output_dir,
        f"{video_name}_YOLO_anotado.mp4"
    )

    output_csv_path = os.path.join(
        output_dir,
        f"{video_name}_detecciones_YOLO.csv"
    )

    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        raise RuntimeError(
```

```
                f"No se ha podido abrir el vídeo: {video_path}"
            )

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = int(
            cap.get(cv2.CAP_PROP_FRAME_COUNT)
        )
        frame_width = int(
            cap.get(cv2.CAP_PROP_FRAME_WIDTH)
        )
        frame_height = int(
            cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
        )

        if fps <= 0:
            cap.release()
            raise RuntimeError(
                f"FPS no válido en el vídeo: {video_path}"
            )

        print("\n" + "=" * 70)
        print("Procesando:", video_name)
        print("FPS:", round(fps, 2))
        print("Total de frames:", total_frames)
        print(
            "Resolución:",
            frame_width,
            "x",
            frame_height
        )

        fourcc = cv2.VideoWriter_fourcc(*"mp4v")

        writer = cv2.VideoWriter(
            output_video_path,
            fourcc,
            fps,
            (frame_width, frame_height)
        )

        if not writer.isOpened():
            cap.release()
            raise RuntimeError(
                f"No se ha podido crear el vídeo de salida para
{video_name}"
            )

        detections_data = []
        frame_number = 0
```

```
try:
    while True:
        ret, frame = cap.read()

        if not ret:
            break

        result = yolo.predict(
            source=frame,
            conf=CONFIDENCE_THRESHOLD,
            verbose=False
        )[0]

        if result.boxes is not None:
            for detection_number, box in enumerate(
                result.boxes
            ):
                class_id = int(
                    box.cls[0].item()
                )
                class_name = yolo.names[
                    class_id
                ]
                confidence = float(
                    box.conf[0].item()
                )

                if class_name not in RELEVANT_CLASSES:
                    continue

                x1, y1, x2, y2 = map(
                    int,
                    box.xyxy[0].tolist()
                )

                x1 = max(
                    0,
                    min(x1, frame_width - 1)
                )
                y1 = max(
                    0,
                    min(y1, frame_height - 1)
                )
                x2 = max(
                    0,
                    min(x2, frame_width - 1)
                )
                y2 = max(
```

```
    0,
    min(y2, frame_height - 1)
)

bbox_width = max(0, x2 - x1)
bbox_height = max(0, y2 - y1)
bbox_area = (
    bbox_width * bbox_height
)

center_x = (x1 + x2) / 2
center_y = (y1 + y2) / 2

center_x_norm = (
    center_x / frame_width
)
center_y_norm = (
    center_y / frame_height
)
bbox_width_norm = (
    bbox_width / frame_width
)
bbox_height_norm = (
    bbox_height / frame_height
)
bbox_area_norm = (
    bbox_area /
    (frame_width * frame_height)
)

time_seconds = (
    frame_number / fps
)

detections_data.append({
    "video": video_name,
    "frame": frame_number,
    "time_seconds": round(
        time_seconds,
        3
    ),
    "detection_id_frame":
        detection_number,
    "class_id": class_id,
    "class_name": class_name,
    "confidence": round(
        confidence,
        4
    ),
```

```
            "x1": x1,
            "y1": y1,
            "x2": x2,
            "y2": y2,
            "center_x": round(
                center_x,
                2
            ),
            "center_y": round(
                center_y,
                2
            ),
            "bbox_width":
                bbox_width,
            "bbox_height":
                bbox_height,
            "bbox_area":
                bbox_area,
            "center_x_norm": round(
                center_x_norm,
                5
            ),
            "center_y_norm": round(
                center_y_norm,
                5
            ),
            "bbox_width_norm": round(
                bbox_width_norm,
                5
            ),
            "bbox_height_norm": round(
                bbox_height_norm,
                5
            ),
            "bbox_area_norm": round(
                bbox_area_norm,
                6
            )
        })

        cv2.rectangle(
            frame,
            (x1, y1),
            (x2, y2),
            (0, 255, 0),
            2
        )

        label = (
```

```
            f"{class_name} "
            f"{confidence:.2f}"
        )

        cv2.putText(
            frame,
            label,
            (
                x1,
                max(y1 - 8, 20)
            ),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.55,
            (0, 255, 0),
            2,
            cv2.LINE_AA
        )

cv2.putText(
    frame,
    (
        f"Frame: {frame_number} | "
        f"Tiempo: "
        f"{frame_number / fps:.2f} s"
    ),
    (15, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.65,
    (255, 255, 255),
    2,
    cv2.LINE_AA
)

writer.write(frame)
frame_number += 1

if (
    frame_number % 100 == 0
    or frame_number == total_frames
):
    percentage = (
        frame_number /
        max(total_frames, 1)
    ) * 100

    print(
        f"{video_name}: "
        f"{frame_number}/"
        f"{total_frames} frames "
```

```
                    f"({percentage:.1f} %)"
                )

    finally:
        cap.release()
        writer.release()

    df_video = pd.DataFrame(
        detections_data
    )

    df_video.to_csv(
        output_csv_path,
        index=False,
        encoding="utf-8-sig"
    )

    summary = {
        "video": video_name,
        "fps": round(fps, 3),
        "total_frames": total_frames,
        "duration_seconds": round(
            total_frames / fps,
            3
        ),
        "width": frame_width,
        "height": frame_height,
        "total_detections": len(df_video),
        "output_video": output_video_path,
        "output_csv": output_csv_path
    }

    print(
        f"Finalizado {video_name}: "
        f"{len(df_video)} detecciones"
    )

    return df_video, summary
```

In [102]:

```
import os

# ============================================================
# RUTAS Y CONJUNTO FINAL DE VÍDEOS
# ============================================================

VIDEOS_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS"

OUTPUT_DIR = os.path.join(
```

```
    VIDEOS_DIR,
    "resultados_experimentales"
)

os.makedirs(OUTPUT_DIR, exist_ok=True)

# Conjunto definitivo de 10 vídeos
# 1000179710.mp4 queda excluido del conjunto final
VIDEO_FILES = [
    "20260611_184739.mp4",
    "20260611_184808.mp4",
    "20260611_185035.mp4",
    "20260611_185443~2.mp4",
    "20260611_185616.mp4",
    "1000179702.mp4",
    "1000179704.mp4",
    "1000179707.mp4",
    "1000179709.mp4",
    "1000179711.mp4",
]

VIDEO_PATHS = [
    os.path.join(VIDEOS_DIR, video)
    for video in VIDEO_FILES
]

print("Directorio de vídeos:")
print(VIDEOS_DIR)

print("\nVídeos del conjunto final:", len(VIDEO_FILES))

for video_path in VIDEO_PATHS:
    estado = "OK" if os.path.isfile(video_path) else "NO ENCONTRADO"
    print(f"{estado:15} {os.path.basename(video_path)}")

assert len(VIDEO_FILES) == 10, (
    f"ERROR: se esperaban 10 vídeos y hay {len(VIDEO_FILES)}."
)

assert all(os.path.isfile(p) for p in VIDEO_PATHS), (
    "ERROR: falta al menos uno de los vídeos del conjunto final."
)
Directorio de vídeos:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM\VÍDEOS

Vídeos del conjunto final: 10
OK              20260611_184739.mp4
OK              20260611_184808.mp4
OK              20260611_185035.mp4
```

```
OK              20260611_185443~2.mp4
OK              20260611_185616.mp4
OK              1000179702.mp4
OK              1000179704.mp4
OK              1000179707.mp4
OK              1000179709.mp4
OK              1000179711.mp4
```

In [ ]:

```
# Procesa los 10 vídeos

all_detections = []
processing_summaries = []
failed_videos = []

for video_file in VIDEO_FILES:
    video_path = os.path.join(
        VIDEOS_DIR,
        video_file
    )

    if not os.path.exists(video_path):
        print(
            f"\nNo se encuentra el archivo: "
            f"{video_path}"
        )
        failed_videos.append({
            "video": video_file,
            "error": "Archivo no encontrado"
        })
        continue

    try:
        df_video, summary = (
            process_video_with_yolo(
                video_path=video_path,
                output_dir=OUTPUT_DIR
            )
        )

        all_detections.append(df_video)
        processing_summaries.append(
            summary
        )

    except Exception as exc:
        print(
            f"\nError procesando "
            f"{video_file}: {exc}"
        )
```

```
        failed_videos.append({
            "video": video_file,
            "error": str(exc)
        })
```

In [ ]:

```
# Unir las detecciones de los diez vídeos en un solo CSV general

if all_detections:
    df_all_detections = pd.concat(
        all_detections,
        ignore_index=True
    )

    combined_csv_path = os.path.join(
        OUTPUT_DIR,
        "dataset_completo_detecciones_YOLO.csv"
    )

    df_all_detections.to_csv(
        combined_csv_path,
        index=False,
        encoding="utf-8-sig"
    )

    print("\nDataset completo guardado en:")
    print(combined_csv_path)

    print(
        "\nNúmero total de detecciones:",
        len(df_all_detections)
    )
else:
    df_all_detections = pd.DataFrame()
    print("No se ha procesado ningún vídeo.")
```

In [ ]:

```
# Guardar un resumen por vídeo

df_processing_summary = pd.DataFrame(
    processing_summaries
)

summary_csv_path = os.path.join(
    OUTPUT_DIR,
    "resumen_procesamiento_videos.csv"
)

df_processing_summary.to_csv(
    summary_csv_path,
```

```
    index=False,
    encoding="utf-8-sig"
)

df_processing_summary
```

In [93]:

```
# resumen por vídeo y clase

if df_all_detections.empty:
    print("El DataFrame general está vacío.")
else:
    summary_video_class = (
        df_all_detections
        .groupby(["video", "class_name"])
        .agg(
            total_detections=("class_name", "size"),
            mean_confidence=("confidence", "mean"),
            min_confidence=("confidence", "min"),
            max_confidence=("confidence", "max")
        )
        .round(3)
        .reset_index()
        .sort_values(
            ["video", "total_detections"],
            ascending=[True, False]
        )
    )

    print(
        "Resumen generado correctamente. "
        f"Filas: {len(summary_video_class)}"
    )

    display(summary_video_class.head(20))
```

```
---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
Cell In[93], line 3
      1 # resumen por vídeo y clase
      2
----> 3 if df_all_detections.empty:
      4     print("El DataFrame general está vacío.")
      5 else:
      6     summary_video_class = (

NameError: name 'df_all_detections' is not defined
```

In [94]:

```
summary_video_class_path = os.path.join(
    OUTPUT_DIR,
    "resumen_detecciones_por_video_y_clase.csv"
)

summary_video_class.to_csv(
    summary_video_class_path,
    index=False,
    encoding="utf-8-sig"
)

print("Resumen guardado en:")
print(summary_video_class_path)
```

```
---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
Cell In[94], line 6
      2     OUTPUT_DIR,
      3     "resumen_detecciones_por_video_y_clase.csv"
      4 )
      5
----> 6 summary_video_class.to_csv(
      7     summary_video_class_path,
      8     index=False,
      9     encoding="utf-8-sig"

NameError: name 'summary_video_class' is not defined
```

7.Función de Tracking para un vídeo

In [103]:

```
# Clases móviles que queremos seguir
TRACKED_CLASSES = {
    "person",
    "car",
    "truck",
    "bus",
    "motorcycle",
    "bicycle",
}

TRACKING_CONFIDENCE = 0.40


def process_video_with_tracking(video_path, output_dir):
    """
```

```
Realiza tracking de los objetos móviles de un vídeo.

Genera:
- un vídeo anotado con los identificadores de seguimiento;
- un CSV con las trayectorias de cada objeto.

Devuelve:
- DataFrame de tracking;
- resumen del procesamiento.
"""

video_name = os.path.splitext(
    os.path.basename(video_path)
)[0]

output_video_path = os.path.join(
    output_dir,
    f"{video_name}_TRACKING_anotado.mp4"
)

output_csv_path = os.path.join(
    output_dir,
    f"{video_name}_tracking.csv"
)

# Modelo nuevo para cada vídeo.
# Así evitamos que los IDs de un vídeo continúen en el siguiente.
tracking_model = YOLO("yolo11n.pt")

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    raise RuntimeError(
        f"No se ha podido abrir el vídeo: {video_path}"
    )

fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(
    cap.get(cv2.CAP_PROP_FRAME_COUNT)
)
frame_width = int(
    cap.get(cv2.CAP_PROP_FRAME_WIDTH)
)
frame_height = int(
    cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
)

if fps <= 0:
    cap.release()
```

```
    raise RuntimeError(
        f"FPS no válido en {video_name}"
    )

fourcc = cv2.VideoWriter_fourcc(*"mp4v")

writer = cv2.VideoWriter(
    output_video_path,
    fourcc,
    fps,
    (frame_width, frame_height)
)

if not writer.isOpened():
    cap.release()
    raise RuntimeError(
        f"No se pudo crear el vídeo de tracking: {video_name}"
    )

print("\n" + "=" * 70)
print("Tracking del vídeo:", video_name)
print("FPS:", round(fps, 2))
print("Frames:", total_frames)
print("Resolución:", frame_width, "x", frame_height)

tracking_data = []
frame_number = 0

try:
    while True:
        ret, frame = cap.read()

        if not ret:
            break

        # persist=True mantiene los IDs entre frames consecutivos
        result = tracking_model.track(
            source=frame,
            conf=TRACKING_CONFIDENCE,
            persist=True,
            tracker="bytetrack.yaml",
            verbose=False
        )[0]

        if (
            result.boxes is not None
            and result.boxes.id is not None
        ):
            boxes = result.boxes.xyxy.cpu().tolist()
```

```
class_ids = (
    result.boxes.cls
    .int()
    .cpu()
    .tolist()
)
confidences = (
    result.boxes.conf
    .cpu()
    .tolist()
)
track_ids = (
    result.boxes.id
    .int()
    .cpu()
    .tolist()
)

for (
    box,
    class_id,
    confidence,
    track_id
) in zip(
    boxes,
    class_ids,
    confidences,
    track_ids
):
    class_name = tracking_model.names[
        class_id
    ]

    if class_name not in TRACKED_CLASSES:
        continue

    x1, y1, x2, y2 = map(int, box)

    # Limitar coordenadas al tamaño del frame
    x1 = max(0, min(x1, frame_width - 1))
    y1 = max(0, min(y1, frame_height - 1))
    x2 = max(0, min(x2, frame_width - 1))
    y2 = max(0, min(y2, frame_height - 1))

    bbox_width = max(0, x2 - x1)
    bbox_height = max(0, y2 - y1)
    bbox_area = bbox_width * bbox_height

    center_x = (x1 + x2) / 2
```

```
center_y = (y1 + y2) / 2

time_seconds = frame_number / fps

tracking_data.append({
    "video": video_name,
    "frame": frame_number,
    "time_seconds": round(
        time_seconds,
        3
    ),
    "track_id": track_id,
    "class_id": class_id,
    "class_name": class_name,
    "confidence": round(
        float(confidence),
        4
    ),
    "x1": x1,
    "y1": y1,
    "x2": x2,
    "y2": y2,
    "center_x": round(center_x, 2),
    "center_y": round(center_y, 2),
    "center_x_norm": round(
        center_x / frame_width,
        5
    ),
    "center_y_norm": round(
        center_y / frame_height,
        5
    ),
    "bbox_width": bbox_width,
    "bbox_height": bbox_height,
    "bbox_area": bbox_area,
    "bbox_area_norm": round(
        bbox_area
        / (frame_width * frame_height),
        6
    )
})

# Dibujar caja
cv2.rectangle(
    frame,
    (x1, y1),
    (x2, y2),
    (0, 255, 0),
    2
```

```
            )

            label = (
                f"{class_name} "
                f"ID:{track_id} "
                f"{confidence:.2f}"
            )

            cv2.putText(
                frame,
                label,
                (x1, max(y1 - 8, 20)),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.55,
                (0, 255, 0),
                2,
                cv2.LINE_AA
            )

    cv2.putText(
        frame,
        (
            f"Frame: {frame_number} | "
            f"Tiempo: {frame_number / fps:.2f} s"
        ),
        (15, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.65,
        (255, 255, 255),
        2,
        cv2.LINE_AA
    )

    writer.write(frame)
    frame_number += 1

    if (
        frame_number % 100 == 0
        or frame_number == total_frames
    ):
        percentage = (
            frame_number
            / max(total_frames, 1)
        ) * 100

        print(
            f"{video_name}: "
            f"{frame_number}/{total_frames} "
            f"frames ({percentage:.1f} %)"
```

```
            )

    finally:
        cap.release()
        writer.release()

    df_tracking_video = pd.DataFrame(tracking_data)

    df_tracking_video.to_csv(
        output_csv_path,
        index=False,
        encoding="utf-8-sig"
    )

    unique_tracks = (
        df_tracking_video["track_id"].nunique()
        if not df_tracking_video.empty
        else 0
    )

    summary = {
        "video": video_name,
        "fps": round(fps, 3),
        "total_frames": total_frames,
        "duration_seconds": round(
            total_frames / fps,
            3
        ),
        "tracking_records": len(df_tracking_video),
        "unique_track_ids": unique_tracks,
        "output_video": output_video_path,
        "output_csv": output_csv_path
    }

    print(
        f"Tracking finalizado: {video_name}"
    )
    print(
        "IDs distintos generados:",
        unique_tracks
    )

    return df_tracking_video, summary
```

7.1 Probar con un solo vídeo

In [ ]:

```
test_video_path = os.path.join(
    VIDEOS_DIR,
```

```
    VIDEO_FILES[0]
)

df_tracking_test, tracking_test_summary = (
    process_video_with_tracking(
        video_path=test_video_path,
        output_dir=OUTPUT_DIR
    )
)

display(df_tracking_test.head(20))
print(tracking_test_summary)
```

7.1.1 Resumen de los objetos seguidos

In [91]:

```
track_summary_test = (
    df_tracking_test
    .groupby(["class_name", "track_id"])
    .agg(
        first_frame=("frame", "min"),
        last_frame=("frame", "max"),
        frames_detected=("frame", "nunique"),
        first_time=("time_seconds", "min"),
        last_time=("time_seconds", "max"),
        mean_confidence=("confidence", "mean"),
        initial_area=("bbox_area_norm", "first"),
        final_area=("bbox_area_norm", "last")
    )
    .reset_index()
)

track_summary_test["duration_seconds"] = (
    track_summary_test["last_time"]
    - track_summary_test["first_time"]
).round(3)

track_summary_test = (
    track_summary_test
    .sort_values(
        "frames_detected",
        ascending=False
    )
)

display(track_summary_test.head(10))

-------------------------------------------------------------------------
-----
```

```
NameError                                 Traceback (most recent call
last)
Cell In[91], line 2
      1 track_summary_test = (
----> 2     df_tracking_test
      3     .groupby(["class_name", "track_id"])
      4     .agg(
      5         first_frame=("frame", "min"),

NameError: name 'df_tracking_test' is not defined
```

7.2 Tracking automático de los diez vídeos

In [104]:

```
all_tracking_data = []
tracking_summaries = []
tracking_failed_videos = []

for video_file in VIDEO_FILES:
    video_path_tracking = os.path.join(
        VIDEOS_DIR,
        video_file
    )

    if not os.path.exists(video_path_tracking):
        print(f"No se encuentra: {video_path_tracking}")

        tracking_failed_videos.append({
            "video": video_file,
            "error": "Archivo no encontrado"
        })
        continue

    try:
        df_tracking_video, tracking_summary = (
            process_video_with_tracking(
                video_path=video_path_tracking,
                output_dir=OUTPUT_DIR
            )
        )

        all_tracking_data.append(
            df_tracking_video
        )

        tracking_summaries.append(
            tracking_summary
        )
```

```
    except Exception as exc:
        print(
            f"Error procesando el tracking de "
            f"{video_file}: {exc}"
        )

        tracking_failed_videos.append({
            "video": video_file,
            "error": str(exc)
        })
```

```
======================================================================
Tracking del vídeo: 20260611_184739
FPS: 30.0
Frames: 434
Resolución: 1920 x 1080
requirements: Ultralytics requirement ['lap>=0.5.12'] not found,
attempting AutoUpdate...
Collecting lap>=0.5.12
  Downloading lap-0.5.13-cp312-cp312-win_amd64.whl.metadata (6.8 kB)
Requirement already satisfied: numpy>=1.21.6 in
c:\users\juank\appdata\local\programs\python\python312\lib\site-
packages (from lap>=0.5.12) (2.1.3)
Downloading lap-0.5.13-cp312-cp312-win_amd64.whl (1.5 MB)
   ---------------------------------------- 0.0/1.5 MB ? eta -:--:--
   ---------------------------------------- 0.0/1.5 MB ? eta -:--:--
   ---------------------------------------- 1.5/1.5 MB 9.7 MB/s eta
0:00:00
Installing collected packages: lap
Successfully installed lap-0.5.13

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run:
C:\Users\juank\AppData\Local\Programs\Python\Python312\python.exe -m
pip install --upgrade pip

requirements: AutoUpdate success  2.7s
WARNING requirements: Restart runtime or rerun command for updates to
take effect

20260611_184739: 100/434 frames (23.0 %)
20260611_184739: 200/434 frames (46.1 %)
20260611_184739: 300/434 frames (69.1 %)
20260611_184739: 400/434 frames (92.2 %)
Tracking finalizado: 20260611_184739
IDs distintos generados: 50

======================================================================
Tracking del vídeo: 20260611_184808
```

```
FPS: 30.0
Frames: 364
Resolución: 1920 x 1080
20260611_184808: 100/364 frames (27.5 %)
20260611_184808: 200/364 frames (54.9 %)
20260611_184808: 300/364 frames (82.4 %)
20260611_184808: 364/364 frames (100.0 %)
Tracking finalizado: 20260611_184808
IDs distintos generados: 25

========================================================================
Tracking del vídeo: 20260611_185035
FPS: 30.08
Frames: 273
Resolución: 1920 x 1080
20260611_185035: 100/273 frames (36.6 %)
20260611_185035: 200/273 frames (73.3 %)
Tracking finalizado: 20260611_185035
IDs distintos generados: 57

========================================================================
Tracking del vídeo: 20260611_185443~2
FPS: 29.73
Frames: 91
Resolución: 1920 x 1080
20260611_185443~2: 91/91 frames (100.0 %)
Tracking finalizado: 20260611_185443~2
IDs distintos generados: 13

========================================================================
Tracking del vídeo: 20260611_185616
FPS: 30.0
Frames: 611
Resolución: 1920 x 1080
20260611_185616: 100/611 frames (16.4 %)
20260611_185616: 200/611 frames (32.7 %)
20260611_185616: 300/611 frames (49.1 %)
20260611_185616: 400/611 frames (65.5 %)
20260611_185616: 500/611 frames (81.8 %)
20260611_185616: 600/611 frames (98.2 %)
Tracking finalizado: 20260611_185616
IDs distintos generados: 55

========================================================================
Tracking del vídeo: 1000179702
FPS: 29.89
Frames: 607
Resolución: 1080 x 1920
1000179702: 100/607 frames (16.5 %)
```

```
1000179702: 200/607 frames (32.9 %)
1000179702: 300/607 frames (49.4 %)
1000179702: 400/607 frames (65.9 %)
1000179702: 500/607 frames (82.4 %)
1000179702: 600/607 frames (98.8 %)
Tracking finalizado: 1000179702
IDs distintos generados: 37

======================================================================
Tracking del vídeo: 1000179704
FPS: 30.04
Frames: 778
Resolución: 1080 x 1920
1000179704: 100/778 frames (12.9 %)
1000179704: 200/778 frames (25.7 %)
1000179704: 300/778 frames (38.6 %)
1000179704: 400/778 frames (51.4 %)
1000179704: 500/778 frames (64.3 %)
1000179704: 600/778 frames (77.1 %)
1000179704: 700/778 frames (90.0 %)
Tracking finalizado: 1000179704
IDs distintos generados: 22

======================================================================
Tracking del vídeo: 1000179707
FPS: 29.98
Frames: 684
Resolución: 1080 x 1920
1000179707: 100/684 frames (14.6 %)
1000179707: 200/684 frames (29.2 %)
1000179707: 300/684 frames (43.9 %)
1000179707: 400/684 frames (58.5 %)
1000179707: 500/684 frames (73.1 %)
1000179707: 600/684 frames (87.7 %)
Tracking finalizado: 1000179707
IDs distintos generados: 36

======================================================================
Tracking del vídeo: 1000179709
FPS: 30.0
Frames: 536
Resolución: 1080 x 1920
1000179709: 100/536 frames (18.7 %)
1000179709: 200/536 frames (37.3 %)
1000179709: 300/536 frames (56.0 %)
1000179709: 400/536 frames (74.6 %)
1000179709: 500/536 frames (93.3 %)
Tracking finalizado: 1000179709
IDs distintos generados: 39
```

```
======================================================================
Tracking del vídeo: 1000179711
FPS: 30.01
Frames: 553
Resolución: 1080 x 1920
1000179711: 100/553 frames (18.1 %)
1000179711: 200/553 frames (36.2 %)
1000179711: 300/553 frames (54.2 %)
1000179711: 400/553 frames (72.3 %)
1000179711: 500/553 frames (90.4 %)
Tracking finalizado: 1000179711
IDs distintos generados: 14
```

7.2.1 Crear el CSV global de tracking

In [105]:

```
if all_tracking_data:
    df_all_tracking = pd.concat(
        all_tracking_data,
        ignore_index=True
    )

    tracking_combined_csv_path = os.path.join(
        OUTPUT_DIR,
        "dataset_completo_tracking.csv"
    )

    df_all_tracking.to_csv(
        tracking_combined_csv_path,
        index=False,
        encoding="utf-8-sig"
    )

    print("Dataset global de tracking guardado en:")
    print(tracking_combined_csv_path)

    print(
        "\nNúmero total de registros:",
        len(df_all_tracking)
    )

    print(
        "Vídeos incluidos:",
        df_all_tracking["video"].nunique()
    )

else:
    df_all_tracking = pd.DataFrame()
```

```
    print(
        "No se ha generado información de tracking."
    )

###### Este CSV contendrá todas las trayectorias detectadas en los
diez vídeos. ######
Dataset global de tracking guardado en:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\resultados_experimentales\dataset_completo_tracking.csv

Número total de registros: 10098
Vídeos incluidos: 10
```

7.2.2 Guardar el resumen del procesamiento

In [106]:

```
df_tracking_summaries = pd.DataFrame(
    tracking_summaries
)

tracking_summary_path = os.path.join(
    OUTPUT_DIR,
    "resumen_tracking_videos.csv"
)

df_tracking_summaries.to_csv(
    tracking_summary_path,
    index=False,
    encoding="utf-8-sig"
)

display(df_tracking_summaries)
```

	video	fps	total_frames	duration_seconds	tracking_records	unique_track_ids
0	20260611_184739	29.995	434	14.469	1207	50
1	20260611_184808	29.999	364	12.134	1026	25
2	20260611_185035	30.084	273	9.075	1729	57
3	20260611_185443~2	29.734	91	3.060	437	13
4	20260611_185616	29.998	611	20.368	2150	55
5	1000179702	29.888	607	20.309	472	37

	video	fps	total_frames	duration_seconds	tracking_records	unique_track_ids	
6	1000179704	30.041	778	25.898	563	22	
7	1000179707	29.979	684	22.816	1463	36	
8	1000179709	29.998	536	17.868	746	39	
9	1000179711	30.015	553	18.424	305	14	

7.3 Resumen de trayectorias por vídeos y clase

In [76]:

```
if df_all_tracking.empty:
    print("El dataset global de tracking está vacío.")

else:
    tracking_by_video_class = (
        df_all_tracking
        .groupby(["video", "class_name"])
        .agg(
            unique_track_ids=("track_id", "nunique"),
            total_tracking_records=("track_id", "size"),
            mean_confidence=("confidence", "mean")
        )
        .round(3)
        .reset_index()
        .sort_values(
            ["video", "unique_track_ids"],
            ascending=[True, False]
        )
    )

    display(tracking_by_video_class.head(20))



###### Aquí unique_track_ids ya se aproxima mucho más al número de
objetos distintos que el anterior total_detections. ######

###### No obstante, puede sobreestimarlo porque un mismo coche podría
perderse y recibir un ID nuevo. ######
```

	video	class_name	unique_track_ids	total_tracking_records	mean_confidence
1	1000179702	car	28	427	0.763
3	1000179702	person	10	39	0.583
2	1000179702	motorcycle	2	2	0.597

	video	class_name	unique_track_ids	total_tracking_records	mean_confidence
0	1000179702	bus	1	1	0.577
4	1000179702	truck	1	5	0.573
5	1000179704	car	14	536	0.772
6	1000179704	person	7	22	0.618
8	1000179707	car	33	1452	0.697
10	1000179707	truck	4	20	0.472
9	1000179707	person	3	7	0.635
7	1000179707	bus	1	7	0.501
12	1000179709	car	35	721	0.759
15	1000179709	truck	3	18	0.643
13	1000179709	motorcycle	2	11	0.716
14	1000179709	person	2	4	0.646
11	1000179709	bicycle	1	1	0.448
18	1000179711	car	7	219	0.791
19	1000179711	motorcycle	4	30	0.589
20	1000179711	person	4	43	0.643
16	1000179711	bicycle	1	12	0.628

7.3.1 Resumen de cada trayectoria

In [77]:

```
if not df_all_tracking.empty:
    track_summary_all = (
        df_all_tracking
        .sort_values(
            ["video", "track_id", "frame"]
        )
        .groupby(
            ["video", "class_name", "track_id"]
        )
        .agg(
            first_frame=("frame", "min"),
            last_frame=("frame", "max"),
            frames_detected=("frame", "nunique"),
            first_time=("time_seconds", "min"),
            last_time=("time_seconds", "max"),
            mean_confidence=("confidence", "mean"),
            initial_center_x=("center_x_norm", "first"),
            final_center_x=("center_x_norm", "last"),
            initial_center_y=("center_y_norm", "first"),
            final_center_y=("center_y_norm", "last"),
            initial_area=("bbox_area_norm", "first"),
            final_area=("bbox_area_norm", "last")
```

```
    )
    .reset_index()
)

track_summary_all["duration_seconds"] = (
    track_summary_all["last_time"]
    - track_summary_all["first_time"]
).round(3)

track_summary_all["area_change"] = (
    track_summary_all["final_area"]
    - track_summary_all["initial_area"]
).round(6)

track_summary_all = (
    track_summary_all
    .sort_values(
        ["video", "frames_detected"],
        ascending=[True, False]
    )
)

display(track_summary_all.head(20))
```

	video	class_name	track_id	first_frame	last_frame	frames_detected	first_time	last_time
6	1000179702	car	28	169	238	69	5.654	7.963
1	1000179702	car	1	0	63	64	0.000	2.108
2	1000179702	car	2	0	61	62	0.000	2.041
23	1000179702	car	163	467	531	61	15.625	17.766
9	1000179702	car	38	204	256	47	6.825	8.565
32	1000179702	person	73	251	269	18	8.398	9.000
26	1000179702	car	177	525	547	17	17.566	18.302
19	1000179702	car	141	392	411	15	13.116	13.751
7	1000179702	car	31	171	212	13	5.721	7.093
3	1000179702	car	9	108	157	11	3.613	5.253
35	1000179702	person	137	380	397	10	12.714	13.283
5	1000179702	car	23	166	182	8	5.554	6.089
16	1000179702	car	100	279	289	8	9.335	9.669
24	1000179702	car	164	470	479	7	15.725	16.026
21	1000179702	car	158	460	465	5	15.391	15.558
22	1000179702	car	162	465	471	5	15.558	15.759
41	1000179702	truck	132	308	315	5	10.305	10.539
10	1000179702	car	44	218	221	4	7.294	7.394
15	1000179702	car	99	279	282	4	9.335	9.435

	video	class_name	track_id	first_frame	last_frame	frames_detected	first_time	last_time
17	1000179702	car	105	283	286	4	9.469	9.569

7.3.2 Guardar el resumen de trayectorias

In [78]:

```
track_summary_all_path = os.path.join(
    OUTPUT_DIR,
    "resumen_trayectorias_objetos.csv"
)

track_summary_all.to_csv(
    track_summary_all_path,
    index=False,
    encoding="utf-8-sig"
)

print("Resumen de trayectorias guardado en:")
print(track_summary_all_path)
```

```
Resumen de trayectorias guardado en:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\resultados_experimentales\resumen_trayectorias_objetos.c
sv
```

8. Variables dinámicas

8.1 Cálculo del movimiento y la aproximación por fotograma

In [90]:

```
### 1. Preparar y ordenar los datos ###

import numpy as np

if "df_all_tracking" not in globals():
    raise RuntimeError(
        "df_all_tracking no existe. "
        "Ejecuta primero el tracking de los diez vídeos."
    )

if df_all_tracking.empty:
    raise RuntimeError(
        "El dataset global de tracking está vacío."
    )

df_dynamic = (
```

```
    df_all_tracking
    .copy()
    .sort_values(
        ["video", "track_id", "frame"]
    )
    .reset_index(drop=True)
)

print("Registros disponibles:", len(df_dynamic))
print("Vídeos:", df_dynamic["video"].nunique())
print("Trayectorias:", df_dynamic.groupby(
    ["video", "track_id"]
).ngroups)
```

```
---------------------------------------------------------------------------
-----
RuntimeError                              Traceback (most recent call
last)
Cell In[90], line 6
      2
      3 import numpy as np
      4
      5 if "df_all_tracking" not in globals():
----> 6     raise RuntimeError(
      7         "df_all_tracking no existe. "
      8         "Ejecuta primero el tracking de los diez vídeos."
      9     )

RuntimeError: df_all_tracking no existe. Ejecuta primero el tracking
de los diez vídeos.
```

In [80]:

```
### 2. Calcular variaciones entre fotogramas ###

track_groups = df_dynamic.groupby(
    ["video", "track_id"],
    sort=False
)

# Diferencias temporales
df_dynamic["delta_time"] = (
    track_groups["time_seconds"]
    .diff()
)

# Desplazamiento del centro de la caja
df_dynamic["delta_x_norm"] = (
    track_groups["center_x_norm"]
    .diff()
)
```

```
df_dynamic["delta_y_norm"] = (
    track_groups["center_y_norm"]
    .diff()
)

# Cambio del área relativa de la caja
df_dynamic["delta_area_norm"] = (
    track_groups["bbox_area_norm"]
    .diff()
)

# Distancia recorrida entre dos observaciones consecutivas
df_dynamic["displacement_norm"] = np.sqrt(
    df_dynamic["delta_x_norm"] ** 2
    + df_dynamic["delta_y_norm"] ** 2
)

# Velocidad aparente normalizada por segundo
valid_time = df_dynamic["delta_time"] > 0

df_dynamic["apparent_speed_norm_s"] = np.nan

df_dynamic.loc[
    valid_time,
    "apparent_speed_norm_s"
] = (
    df_dynamic.loc[
        valid_time,
        "displacement_norm"
    ]
    / df_dynamic.loc[
        valid_time,
        "delta_time"
    ]
)

# Componentes horizontal y vertical de la velocidad
df_dynamic["velocity_x_norm_s"] = np.nan
df_dynamic["velocity_y_norm_s"] = np.nan

df_dynamic.loc[
    valid_time,
    "velocity_x_norm_s"
] = (
    df_dynamic.loc[
        valid_time,
        "delta_x_norm"
    ]
```

```
    / df_dynamic.loc[
        valid_time,
        "delta_time"
    ]
)

df_dynamic.loc[
    valid_time,
    "velocity_y_norm_s"
] = (
    df_dynamic.loc[
        valid_time,
        "delta_y_norm"
    ]
    / df_dynamic.loc[
        valid_time,
        "delta_time"
    ]
)

# Tasa relativa de crecimiento de la caja por segundo
df_dynamic["area_growth_rate_s"] = np.nan

previous_area = (
    track_groups["bbox_area_norm"]
    .shift(1)
)

valid_area = (
    valid_time
    & previous_area.notna()
    & (previous_area > 0)
)

df_dynamic.loc[
    valid_area,
    "area_growth_rate_s"
] = (
    df_dynamic.loc[
        valid_area,
        "delta_area_norm"
    ]
    / previous_area.loc[valid_area]
    / df_dynamic.loc[
        valid_area,
        "delta_time"
    ]
)
```

In [81]:

```
### 3. Suavizar las variables ###

ROLLING_WINDOW = 5

dynamic_groups = df_dynamic.groupby(
    ["video", "track_id"],
    sort=False
)

df_dynamic["speed_smooth"] = (
    dynamic_groups["apparent_speed_norm_s"]
    .transform(
        lambda series: series.rolling(
            window=ROLLING_WINDOW,
            min_periods=2
        ).median()
    )
)

df_dynamic["velocity_x_smooth"] = (
    dynamic_groups["velocity_x_norm_s"]
    .transform(
        lambda series: series.rolling(
            window=ROLLING_WINDOW,
            min_periods=2
        ).median()
    )
)

df_dynamic["velocity_y_smooth"] = (
    dynamic_groups["velocity_y_norm_s"]
    .transform(
        lambda series: series.rolling(
            window=ROLLING_WINDOW,
            min_periods=2
        ).median()
    )
)

df_dynamic["area_growth_smooth"] = (
    dynamic_groups["area_growth_rate_s"]
    .transform(
        lambda series: series.rolling(
            window=ROLLING_WINDOW,
            min_periods=2
        ).median()
    )
)
```

In [82]:

```
### Clasificar la dirección aparente ###

DIRECTION_THRESHOLD = 0.01

def classify_direction(row):
    vx = row["velocity_x_smooth"]
    vy = row["velocity_y_smooth"]

    if pd.isna(vx) or pd.isna(vy):
        return "indeterminada"

    if (
        abs(vx) < DIRECTION_THRESHOLD
        and abs(vy) < DIRECTION_THRESHOLD
    ):
        return "detenido_o_movimiento_minimo"

    if abs(vx) >= abs(vy):
        return (
            "derecha"
            if vx > 0
            else "izquierda"
        )

    return (
        "abajo"
        if vy > 0
        else "arriba"
    )

df_dynamic["movement_direction"] = df_dynamic.apply(
    classify_direction,
    axis=1
)
```

In [83]:

```
### 5. Clasificar aproximación o alejamiento ###

APPROACH_THRESHOLD = 0.08

def classify_proximity_change(value):
    if pd.isna(value):
        return "indeterminada"

    if value > APPROACH_THRESHOLD:
        return "aproximandose"

    if value < -APPROACH_THRESHOLD:
        return "alejandose"
```

```
    return "distancia_aparente_estable"

df_dynamic["proximity_trend"] = (
    df_dynamic["area_growth_smooth"]
    .apply(classify_proximity_change)
)
```

In [84]:

```
### 6. Clasificar proximidad relativa ###

display(
    df_dynamic["bbox_area_norm"]
    .describe(
        percentiles=[0.25, 0.50, 0.75, 0.90]
    )
)
```

```
count    10130.000000
mean         0.035159
std          0.061466
min          0.000495
25%          0.005001
50%          0.013464
75%          0.044365
90%          0.076715
max          0.606083
Name: bbox_area_norm, dtype: float64
```

In [85]:

```
#### Como primera aproximación experimental, puedes utilizar los
percentiles del propio conjunto: ####

AREA_MEDIUM_THRESHOLD = (
    df_dynamic["bbox_area_norm"]
    .quantile(0.50)
)

AREA_CLOSE_THRESHOLD = (
    df_dynamic["bbox_area_norm"]
    .quantile(0.85)
)

def classify_relative_proximity(area):
    if pd.isna(area):
        return "indeterminada"

    if area >= AREA_CLOSE_THRESHOLD:
        return "cerca"

    if area >= AREA_MEDIUM_THRESHOLD:
        return "distancia_media"
```

```
    return "lejos"

df_dynamic["relative_proximity"] = (
    df_dynamic["bbox_area_norm"]
    .apply(classify_relative_proximity)
)

print(
    "Umbral distancia media:",
    round(AREA_MEDIUM_THRESHOLD, 6)
)

print(
    "Umbral cerca:",
    round(AREA_CLOSE_THRESHOLD, 6)
)
```

```
Umbral distancia media: 0.013464
Umbral cerca: 0.060953
```

In [86]:

```
### 7. Mostrar resultados ###

dynamic_columns = [
    "video",
    "frame",
    "time_seconds",
    "track_id",
    "class_name",
    "confidence",
    "center_x_norm",
    "center_y_norm",
    "bbox_area_norm",
    "speed_smooth",
    "movement_direction",
    "area_growth_smooth",
    "proximity_trend",
    "relative_proximity"
]

display(
    df_dynamic[dynamic_columns]
    .dropna(
        subset=["speed_smooth"]
    )
    .head(20)
)
```

	video	frame	time_seconds	track_id	class_name	confidence	center_x_norm	center_y_nor
2	1000179702	2	0.067	1	car	0.9211	0.61481	0.54036
3	1000179702	3	0.100	1	car	0.9111	0.61574	0.54479

	video	frame	time_seconds	track_id	class_name	confidence	center_x_norm	center_y_nor
4	1000179702	4	0.134	1	car	0.8956	0.61713	0.55521
5	1000179702	5	0.167	1	car	0.9156	0.61667	0.55755
6	1000179702	6	0.201	1	car	0.9246	0.61898	0.55885
7	1000179702	7	0.234	1	car	0.9141	0.61898	0.56641
8	1000179702	8	0.268	1	car	0.9052	0.61343	0.58255
9	1000179702	9	0.301	1	car	0.9208	0.60185	0.60677
10	1000179702	10	0.335	1	car	0.9294	0.61481	0.59219
11	1000179702	11	0.368	1	car	0.9075	0.62222	0.59583
12	1000179702	12	0.401	1	car	0.9259	0.63333	0.59375
13	1000179702	13	0.435	1	car	0.9106	0.63889	0.59401
14	1000179702	14	0.468	1	car	0.9109	0.64259	0.59323
15	1000179702	15	0.502	1	car	0.9123	0.64491	0.59427
16	1000179702	16	0.535	1	car	0.9313	0.64769	0.59297
17	1000179702	17	0.569	1	car	0.9255	0.65139	0.59010
18	1000179702	18	0.602	1	car	0.9186	0.65417	0.58802
19	1000179702	19	0.636	1	car	0.9222	0.65694	0.58672
20	1000179702	20	0.669	1	car	0.9375	0.66065	0.58542
21	1000179702	21	0.703	1	car	0.9301	0.66250	0.58672

In [87]:

```
### 8. Guardar el dataset enriquecido ###

dynamic_dataset_path = os.path.join(
    OUTPUT_DIR,
    "dataset_tracking_variables_dinamicas.csv"
)

df_dynamic.to_csv(
    dynamic_dataset_path,
    index=False,
    encoding="utf-8-sig"
)

print("Dataset dinámico guardado en:")
print(dynamic_dataset_path)
```

```
Dataset dinámico guardado en:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\resultados_experimentales\dataset_tracking_variables_din
amicas.csv
```

8.2 Resumen dinámico por trayectoria

In [88]:

```
track_dynamic_summary = (
    df_dynamic
    .sort_values(
        ["video", "track_id", "frame"]
    )
    .groupby(
        ["video", "class_name", "track_id"]
    )
    .agg(
        first_frame=("frame", "min"),
        last_frame=("frame", "max"),
        frames_detected=("frame", "nunique"),
        first_time=("time_seconds", "min"),
        last_time=("time_seconds", "max"),
        mean_confidence=("confidence", "mean"),
        initial_x=("center_x_norm", "first"),
        final_x=("center_x_norm", "last"),
        initial_y=("center_y_norm", "first"),
        final_y=("center_y_norm", "last"),
        initial_area=("bbox_area_norm", "first"),
        final_area=("bbox_area_norm", "last"),
        mean_speed=("speed_smooth", "mean"),
        max_speed=("speed_smooth", "max"),
        mean_area_growth=(
            "area_growth_smooth",
            "mean"
        ),
        maximum_area=("bbox_area_norm", "max")
    )
    .reset_index()
)

track_dynamic_summary["duration_seconds"] = (
    track_dynamic_summary["last_time"]
    - track_dynamic_summary["first_time"]
).round(3)

track_dynamic_summary["total_displacement_x"] = (
    track_dynamic_summary["final_x"]
    - track_dynamic_summary["initial_x"]
).round(5)

track_dynamic_summary["total_displacement_y"] = (
    track_dynamic_summary["final_y"]
    - track_dynamic_summary["initial_y"]
).round(5)

track_dynamic_summary["relative_area_change"] = np.where(
    track_dynamic_summary["initial_area"] > 0,
```

```
    (
        track_dynamic_summary["final_area"]
        - track_dynamic_summary["initial_area"]
    )
    / track_dynamic_summary["initial_area"],
    np.nan
)

track_dynamic_summary["relative_area_change"] = (
    track_dynamic_summary[
        "relative_area_change"
    ].round(3)
)
```

In [89]:

```
#### Clasificación de la evolución general de cada trayectoria  ####

def classify_track_evolution(value):
    if pd.isna(value):
        return "indeterminada"

    if value > 0.20:
        return "aproximacion_global"

    if value < -0.20:
        return "alejamiento_global"

    return "distancia_global_estable"

track_dynamic_summary["track_evolution"] = (
    track_dynamic_summary[
        "relative_area_change"
    ].apply(classify_track_evolution)
)

track_dynamic_summary = (
    track_dynamic_summary
    .sort_values(
        ["video", "frames_detected"],
        ascending=[True, False]
    )
)

display(track_dynamic_summary.head(20))
```

	video	class_name	track_id	first_frame	last_frame	frames_detected	first_time	last_time
6	1000179702	car	28	169	238	69	5.654	7.963

	video	class_name	track_id	first_frame	last_frame	frames_detected	first_time	last_time
1	1000179702	car	1	0	63	64	0.000	2.108
2	1000179702	car	2	0	61	62	0.000	2.041
23	1000179702	car	163	467	531	61	15.625	17.766
9	1000179702	car	38	204	256	47	6.825	8.565
32	1000179702	person	73	251	269	18	8.398	9.000
26	1000179702	car	177	525	547	17	17.566	18.302
19	1000179702	car	141	392	411	15	13.116	13.751
7	1000179702	car	31	171	212	13	5.721	7.093
3	1000179702	car	9	108	157	11	3.613	5.253
35	1000179702	person	137	380	397	10	12.714	13.283
5	1000179702	car	23	166	182	8	5.554	6.089
16	1000179702	car	100	279	289	8	9.335	9.669
24	1000179702	car	164	470	479	7	15.725	16.026
21	1000179702	car	158	460	465	5	15.391	15.558
22	1000179702	car	162	465	471	5	15.558	15.759
41	1000179702	truck	132	308	315	5	10.305	10.539
10	1000179702	car	44	218	221	4	7.294	7.394
15	1000179702	car	99	279	282	4	9.335	9.435
17	1000179702	car	105	283	286	4	9.469	9.569

20 rows × 24 columns

In [90]:

```
#### Guardar: ####

track_dynamic_summary_path = os.path.join(
    OUTPUT_DIR,
    "resumen_variables_dinamicas_por_trayectoria.csv"
)

track_dynamic_summary.to_csv(
    track_dynamic_summary_path,
    index=False,
    encoding="utf-8-sig"
)

print("Resumen dinámico guardado en:")
print(track_dynamic_summary_path)
```

```
Resumen dinámico guardado en:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\resultados_experimentales\resumen_variables_dinamicas_po
r_trayectoria.csv
```

9. Comprensión contextual mediante CLIP

9.1 Definición de hipótesis textuales

Las hipótesis textuales utilizadas por CLIP se definen específicamente en los experimentos correspondientes. Se emplean descripciones semánticas asociadas a los distintos estados y tipos de señales relevantes de la escena.

Estas hipótesis permiten realizar clasificación zero-shot sobre las regiones de interés detectadas previamente por YOLOv11.

9.3 Detección y extracción de candidatos de semáforo con YOLOv11

In [101]:

```
TEST_FRAME = 60

cap = cv2.VideoCapture(video_path)
cap.set(cv2.CAP_PROP_POS_FRAMES, TEST_FRAME)

ret, frame = cap.read()
cap.release()

if not ret:
```

```
    raise RuntimeError(
        f"No se pudo leer el frame {TEST_FRAME}"
    )

frame_rgb = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2RGB
)

frame_pil = Image.fromarray(frame_rgb)

print("Frame utilizado:", TEST_FRAME)
display(frame_pil)


result_test = yolo.predict(
    source=frame,
    conf=0.10,
    verbose=False
)[0]

traffic_light_crops = []

for box in result_test.boxes:

    class_id = int(box.cls[0].item())
    class_name = yolo.names[class_id]
    confidence = float(box.conf[0].item())

    if class_name != "traffic light":
        continue

    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0].tolist()
    )

    # Ligero margen alrededor de la detección
    margin = 20

    x1_crop = max(0, x1 - margin)
    y1_crop = max(0, y1 - margin)

    x2_crop = min(
        frame_pil.width,
        x2 + margin
    )

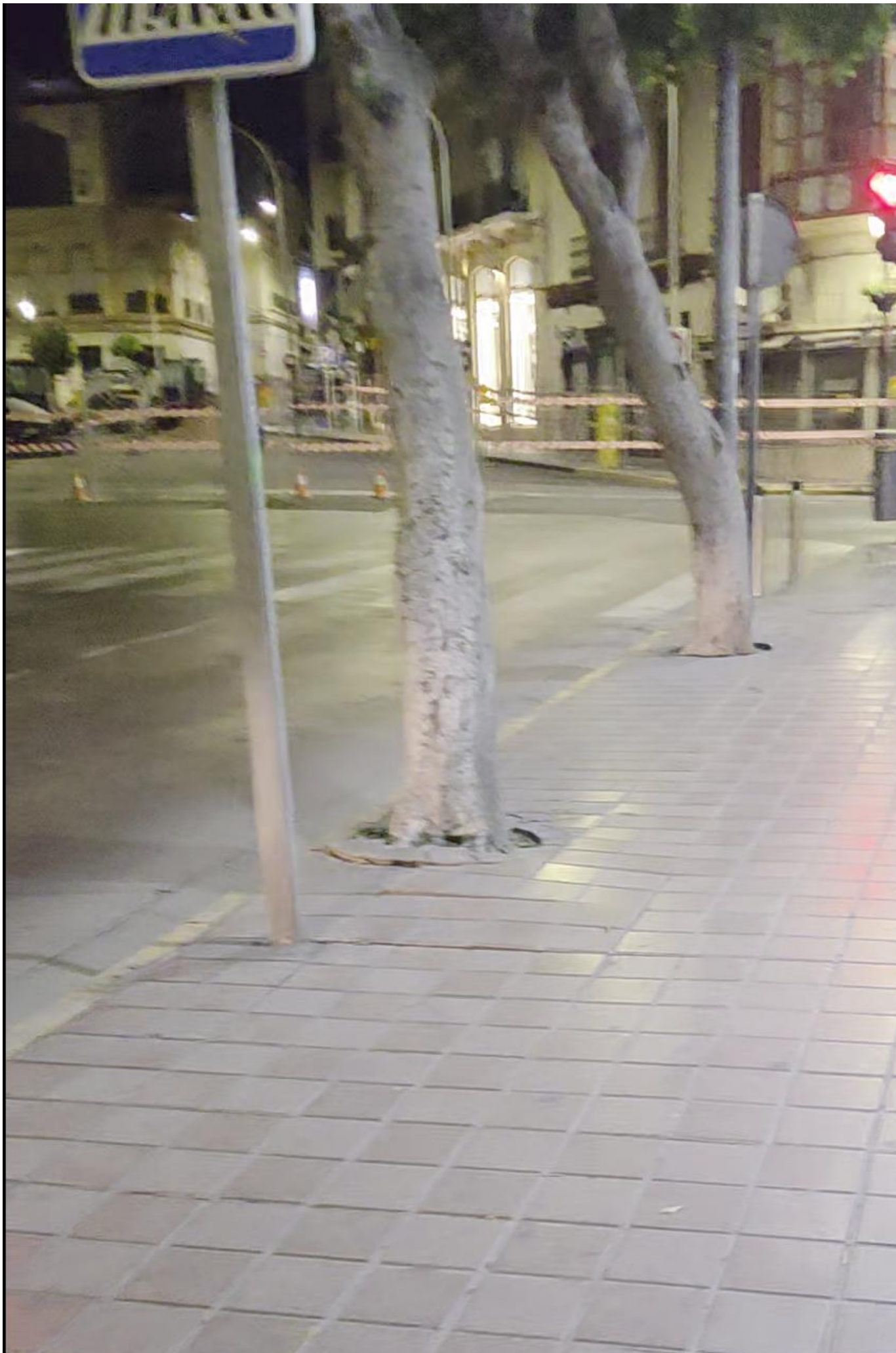
    y2_crop = min(
```

```
            frame_pil.height,
            y2 + margin
        )

        crop = frame_pil.crop(
            (x1_crop, y1_crop, x2_crop, y2_crop)
        )

        traffic_light_crops.append({
            "confidence": confidence,
            "bbox": (x1, y1, x2, y2),
            "crop": crop
        })

print(
    "Semáforos candidatos detectados:",
    len(traffic_light_crops)
)
Frame utilizado: 60
```



```
Semáforos candidatos detectados: 1
```

9.4 Validación del estado del semáforo peatonal: RED y GREEN

In [105]:

```
# 9.14 VALIDACIÓN DEL ESTADO DEL SEMÁFORO PEATONAL
#      CASOS RED Y GREEN

import os
import cv2

from pathlib import Path
from PIL import Image
from IPython.display import display


# ============================================================
# 1. RUTA ROBUSTA DE LOS VÍDEOS
# ============================================================

VIDEO_DIR_CANDIDATES = [

    Path(
        r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM\VÍDEOS"
    ),

    Path(
        r"C:\Users\juank\Documents\CARLOS\TFM\VÍDEOS"
    )
]


VIDEOS_DIR = next(

    (
        path
        for path
        in VIDEO_DIR_CANDIDATES
        if path.is_dir()
    ),

    None
)


if VIDEOS_DIR is None:
```

```
    raise FileNotFoundError(
        "No se ha encontrado la carpeta VÍDEOS "
        "en ninguna de las rutas conocidas."
    )


print("✓ Carpeta de vídeos:")
print(VIDEOS_DIR)


# ============================================================
# 2. COMPROBACIÓN DE DEPENDENCIAS
# ============================================================

if "yolo" not in globals():

    raise RuntimeError(
        "No existe el modelo 'yolo' en memoria. "
        "Ejecuta previamente la celda donde se carga YOLO11."
    )


if "classify_with_clip" not in globals():

    raise RuntimeError(
        "No existe classify_with_clip en memoria. "
        "Ejecuta previamente el módulo CLIP."
    )


# ============================================================
# 3. PROMPTS CLIP
# ============================================================

GREEN_PROMPTS = [

    "a green pedestrian traffic light",

    "a green pedestrian crossing signal",

    "a green walking pedestrian signal",

    "a green walking person traffic signal",

    "a green walk signal"
]


RED_PROMPTS = [
```

```
    "a red pedestrian traffic light",

    "a red pedestrian crossing signal",

    "a red standing pedestrian signal",

    "a red stop pedestrian signal"
]


PEDESTRIAN_PROMPTS = (
    GREEN_PROMPTS
    +
    RED_PROMPTS
)


# ============================================================
# 4. FUNCIÓN DE VALIDACIÓN
# ============================================================

def validate_pedestrian_traffic_light(
    video_path,
    frame_number,
    expected_state
):
    """
    Extrae un frame real, detecta candidatos 'traffic light'
    con YOLO11 y clasifica su estado peatonal mediante CLIP.

    Incluye lectura robusta del vídeo y del frame solicitado.
    """

    video_path = Path(
        video_path
    )


    # ------------------------------------------------------
    # 4.1 Comprobación del vídeo
    # ------------------------------------------------------

    if not video_path.is_file():

        raise FileNotFoundError(
            f"No existe el vídeo:\n{video_path}"
        )
```

```
# ------------------------------------------------------
# 4.2 Apertura del vídeo
# ------------------------------------------------------

cap = cv2.VideoCapture(
    str(
        video_path
    )
)


if not cap.isOpened():

    raise RuntimeError(
        f"No se pudo abrir el vídeo:\n{video_path}"
    )


total_frames = int(
    cap.get(
        cv2.CAP_PROP_FRAME_COUNT
    )
)


if total_frames <= 0:

    cap.release()

    raise RuntimeError(
        "No se pudo determinar el número de frames de:\n"
        f"{video_path}"
    )


# ------------------------------------------------------
# 4.3 Proteger frente a frames fuera de rango
# ------------------------------------------------------

frame_number_safe = min(
    int(
        frame_number
    ),
    total_frames - 1
)


if frame_number_safe != frame_number:
```

```
    print(
        f"⚠ Frame solicitado {frame_number} fuera de rango. "
        f"Se utilizará el frame {frame_number_safe}."
    )


# ------------------------------------------------------------
# 4.4 Primer intento: acceso directo
# ------------------------------------------------------------

cap.set(
    cv2.CAP_PROP_POS_FRAMES,
    frame_number_safe
)


ret, frame = (
    cap.read()
)


# ------------------------------------------------------------
# 4.5 Segundo intento: lectura secuencial
#     Algunos MP4 fallan con acceso aleatorio.
# ------------------------------------------------------------

if not ret:

    cap.release()

    cap = cv2.VideoCapture(
        str(
            video_path
        )
    )


    if not cap.isOpened():

        raise RuntimeError(
            f"No se pudo reabrir el vídeo:\n{video_path}"
        )


    frame = None


    for i in range(
```

```
        frame_number_safe + 1
    ):

        ret_seq, current_frame = (
            cap.read()
        )


        if not ret_seq:
            break


        if i == frame_number_safe:

            frame = (
                current_frame
            )

            break


    ret = (
        frame is not None
    )


cap.release()


if not ret:

    raise RuntimeError(
        f"No se pudo leer el frame "
        f"{frame_number_safe} "
        f"de {total_frames} frames."
    )


# ----------------------------------------------------------
# 4.6 Conversión BGR -> RGB -> PIL
# ----------------------------------------------------------

frame_rgb = cv2.cvtColor(

    frame,

    cv2.COLOR_BGR2RGB
)
```

```
frame_pil = Image.fromarray(
    frame_rgb
)


# ============================================================
# 5. DETECCIÓN YOLO11
# ============================================================

result = yolo.predict(

    source=frame,

    conf=0.10,

    verbose=False

)[0]


candidates = []


if result.boxes is None:

    print(
        f"\nFrame {frame_number_safe}: "
        "YOLO no devolvió detecciones."
    )

    return None


for box in result.boxes:

    class_id = int(
        box.cls[
            0
        ].item()
    )


    class_name = (
        yolo.names[
            class_id
        ]
    )
```

```
if class_name != "traffic light":

    continue


yolo_confidence = float(
    box.conf[
        0
    ].item()
)


x1, y1, x2, y2 = map(

    int,

    box.xyxy[
        0
    ].tolist()
)


margin = 15


crop = frame_pil.crop((

    max(
        0,
        x1 - margin
    ),

    max(
        0,
        y1 - margin
    ),

    min(
        frame_pil.width,
        x2 + margin
    ),

    min(
        frame_pil.height,
        y2 + margin
    )
))
```

```
# ================================================
# 6. CLASIFICACIÓN CLIP
# ================================================

clip_result = classify_with_clip(

    crop,

    PEDESTRIAN_PROMPTS
)


probs = (
    clip_result[
        "all_probabilities"
    ]
)


green_score = sum(

    probs[p]

    for p
    in GREEN_PROMPTS
)


red_score = sum(

    probs[p]

    for p
    in RED_PROMPTS
)


total = (
    green_score
    +
    red_score
)


if total > 0:

    green_norm = (
        green_score
```

```
        /
        total
    )

    red_norm = (
        red_score
        /
        total
    )


else:

    green_norm = 0.0

    red_norm = 0.0


state = (

    "GREEN"

    if green_norm > red_norm

    else "RED"
)


semantic_confidence = max(

    green_norm,

    red_norm
)


candidates.append({

    "crop":
        crop,

    "yolo_confidence":
        yolo_confidence,

    "green_score":
        green_norm,

    "red_score":
        red_norm,
```

```
        "state":
            state,

        "semantic_confidence":
            semantic_confidence
    })


# ============================================================
# 7. COMPROBAR CANDIDATOS
# ============================================================

if not candidates:

    print(
        f"\nFrame {frame_number_safe}: "
        "no se detectaron semáforos."
    )

    return None


# Candidato con mayor evidencia semántica RED/GREEN
best_candidate = max(

    candidates,

    key=lambda x:
        x[
            "semantic_confidence"
        ]
)


# ============================================================
# 8. RESULTADO
# ============================================================

print(
    "\n"
    +
    "=" * 55
)


print(
    f"Vídeo: {video_path.name}"
)
```

```
print(
    f"Frame analizado: "
    f"{frame_number_safe}/{total_frames - 1}"
)


print(
    f"Estado esperado: "
    f"{expected_state}"
)


print(
    "Estado estimado: "
    f"{best_candidate['state']}"
)


print(
    "Confianza YOLO: "
    f"{best_candidate['yolo_confidence']:.3f}"
)


print(
    "GREEN: "
    f"{best_candidate['green_score']:.3f}"
)


print(
    "RED: "
    f"{best_candidate['red_score']:.3f}"
)


print(
    "=" * 55
)


display(
    best_candidate[
        "crop"
    ]
)
```

```
    return (
        best_candidate
    )


# ============================================================
# 9. CASO 1 — SEMÁFORO PEATONAL ROJO
# ============================================================

red_video_path = (
    VIDEOS_DIR
    /
    "20260611_184808.mp4"
)


print()
print(
    "Vídeo RED:"
)
print(
    red_video_path
)
print(
    "¿Existe?:",
    red_video_path.is_file()
)


result_red = (
    validate_pedestrian_traffic_light(

        video_path=red_video_path,

        frame_number=120,

        expected_state="RED"
    )
)


# ============================================================
# 10. CASO 2 — SEMÁFORO PEATONAL VERDE
# ============================================================

green_video_path = (
    VIDEOS_DIR
    /
```

```
    "20260611_184739.mp4"
)


print()
print(
    "Vídeo GREEN:"
)
print(
    green_video_path
)
print(
    "¿Existe?:",
    green_video_path.is_file()
)


result_green = (
    validate_pedestrian_traffic_light(

        video_path=green_video_path,

        frame_number=60,

        expected_state="GREEN"
    )
)
```

```
✓ Carpeta de vídeos:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM\VÍDEOS

Vídeo RED:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184808.mp4
¿Existe?: True

============================================================
Vídeo: 20260611_184808.mp4
Frame analizado: 120/363
Estado esperado: RED
Estado estimado: RED
Confianza YOLO: 0.470
GREEN: 0.426
RED: 0.574
============================================================
```



```
Vídeo GREEN:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184739.mp4
¿Existe?: True

=======================================================
Vídeo: 20260611_184739.mp4
Frame analizado: 60/433
Estado esperado: GREEN
Estado estimado: GREEN
Confianza YOLO: 0.122
GREEN: 0.705
RED: 0.295
=======================================================
```



10. Fusión del estado del semáforo con las variables dinámicas de los vehículos para estimar riesgo y generar una recomendación al peatón.

10.1 Fusión de información y estimación del riesgo de cruce

En esta etapa se integra la información obtenida mediante los diferentes módulos del sistema. El estado del semáforo peatonal se combina con las variables dinámicas derivadas del seguimiento de los vehículos para estimar el nivel de riesgo asociado a la situación de cruce.

La decisión no depende únicamente del estado del semáforo. Incluso ante una señal peatonal favorable, la presencia de un vehículo aproximándose o situado en una posición potencialmente conflictiva puede incrementar el riesgo. El sistema adopta por tanto un enfoque conservador orientado a la asistencia al usuario.

In [106]:

```
# 10.1 FUSIÓN DE INFORMACIÓN Y ESTIMACIÓN DEL RIESGO


def estimate_crossing_risk(
    traffic_light_state,
    vehicle_approaching=False,
```

```
    vehicle_close=False,
    vehicle_moving=False
):
    """
    Estima el riesgo de cruce combinando:

    - estado del semáforo peatonal;
    - aproximación de vehículos;
    - proximidad relativa;
    - movimiento del vehículo.

    Devuelve:
        risk_level
        recommendation
    """

    # ------------------------------------------------------
    # 1. Semáforo peatonal rojo
    # ------------------------------------------------------
    if traffic_light_state == "RED":
        return (
            "HIGH",
            "WAIT"
        )

    # ------------------------------------------------------
    # 2. Semáforo peatonal verde
    # ------------------------------------------------------
    if traffic_light_state == "GREEN":

        # Vehículo próximo y aproximándose:
        # situación de mayor riesgo
        if vehicle_approaching and vehicle_close:
            return (
                "HIGH",
                "WAIT"
            )

        # Vehículo aproximándose,
        # pero todavía no clasificado como próximo
        if vehicle_approaching:
            return (
                "MEDIUM",
                "CAUTION"
            )

        # Vehículo próximo y en movimiento,
        # pero sin evidencia clara de aproximación
        if vehicle_close and vehicle_moving:
```

```
            return (
                "MEDIUM",
                "CAUTION"
            )

        # Vehículo próximo pero prácticamente detenido
        if vehicle_close:
            return (
                "MEDIUM",
                "CAUTION"
            )

        # Sin evidencia dinámica de peligro
        return (
            "LOW",
            "CROSS WITH CAUTION"
        )

    # ----------------------------------------------------------
    # 3. Estado del semáforo desconocido/no disponible
    # ----------------------------------------------------------
    return (
        "MEDIUM",
        "WAIT / CHECK ENVIRONMENT"
    )


# ============================================================
# PRUEBAS DE LAS REGLAS DE RIESGO
# ============================================================

test_cases = [
    # RED
    ("RED", False, False, False),

    # GREEN: próximo y aproximándose
    ("GREEN", True, True, True),

    # GREEN: aproximándose pero no próximo
    ("GREEN", True, False, True),

    # GREEN: próximo y moviéndose
    ("GREEN", False, True, True),

    # GREEN: próximo pero detenido
    ("GREEN", False, True, False),

    # GREEN: sin peligro dinámico
    ("GREEN", False, False, False),
```

```
    # Estado desconocido
    ("UNKNOWN", False, False, False)
]


for state, approaching, close, moving in test_cases:

    risk, recommendation = estimate_crossing_risk(
        traffic_light_state=state,
        vehicle_approaching=approaching,
        vehicle_close=close,
        vehicle_moving=moving
    )

    print(
        f"Semáforo={state:7} | "
        f"Aproxima={str(approaching):5} | "
        f"Cerca={str(close):5} | "
        f"Movimiento={str(moving):5} "
        f"→ Riesgo={risk:6} | "
        f"Recomendación={recommendation}"
    )
```

```
Semáforo=RED     | Aproxima=False | Cerca=False | Movimiento=False →
Riesgo=HIGH   | Recomendación=WAIT
Semáforo=GREEN   | Aproxima=True  | Cerca=True  | Movimiento=True  →
Riesgo=HIGH   | Recomendación=WAIT
Semáforo=GREEN   | Aproxima=True  | Cerca=False | Movimiento=True  →
Riesgo=MEDIUM | Recomendación=CAUTION
Semáforo=GREEN   | Aproxima=False | Cerca=True  | Movimiento=True  →
Riesgo=MEDIUM | Recomendación=CAUTION
Semáforo=GREEN   | Aproxima=False | Cerca=True  | Movimiento=False →
Riesgo=MEDIUM | Recomendación=CAUTION
Semáforo=GREEN   | Aproxima=False | Cerca=False | Movimiento=False →
Riesgo=LOW    | Recomendación=CROSS WITH CAUTION
Semáforo=UNKNOWN | Aproxima=False | Cerca=False | Movimiento=False →
Riesgo=MEDIUM | Recomendación=WAIT / CHECK ENVIRONMENT
```

In [107]:

```
# PRUEBAS DE LAS REGLAS DE RIESGO


test_cases = [
    ("RED", False, False, False),
    ("GREEN", True, True, True),
    ("GREEN", False, True, True),
    ("GREEN", False, True, False),
    ("GREEN", False, False, False),
    ("UNKNOWN", False, False, False)
]
```

```
for state, approaching, close, moving in test_cases:

    risk, recommendation = estimate_crossing_risk(
        traffic_light_state=state,
        vehicle_approaching=approaching,
        vehicle_close=close,
        vehicle_moving=moving
    )

    print(
        f"Semáforo={state:7} | "
        f"Aproxima={str(approaching):5} | "
        f"Cerca={str(close):5} | "
        f"Movimiento={str(moving):5} "
        f"→ Riesgo={risk:6} | "
        f"Recomendación={recommendation}"
    )
```

```
Semáforo=RED     | Aproxima=False | Cerca=False | Movimiento=False →
Riesgo=HIGH   | Recomendación=WAIT
Semáforo=GREEN   | Aproxima=True  | Cerca=True  | Movimiento=True  →
Riesgo=HIGH   | Recomendación=WAIT
Semáforo=GREEN   | Aproxima=False | Cerca=True  | Movimiento=True  →
Riesgo=MEDIUM | Recomendación=CAUTION
Semáforo=GREEN   | Aproxima=False | Cerca=True  | Movimiento=False →
Riesgo=MEDIUM | Recomendación=CAUTION
Semáforo=GREEN   | Aproxima=False | Cerca=False | Movimiento=False →
Riesgo=LOW    | Recomendación=CROSS WITH CAUTION
Semáforo=UNKNOWN | Aproxima=False | Cerca=False | Movimiento=False →
Riesgo=MEDIUM | Recomendación=WAIT / CHECK ENVIRONMENT
```

10.2 CONVERSIÓN DE VARIABLES DINÁMICAS EN INDICADORES DE RIESGO

In [108]:

```
VEHICLE_CLASSES = [
    "car",
    "truck",
    "bus",
    "motorcycle",
    "bicycle"
]


def get_dynamic_vehicle_indicators(row):
```

```
    """
    Convierte las variables dinámicas calculadas previamente
    en indicadores booleanos utilizados por el módulo de riesgo.
    """

    vehicle_approaching = (
        row["proximity_trend"] == "aproximandose"
    )

    vehicle_close = (
        row["relative_proximity"] == "cerca"
    )

    vehicle_moving = (
        row["movement_direction"]
        not in [
            "detenido_o_movimiento_minimo",
            "indeterminada"
        ]
    )

    return (
        vehicle_approaching,
        vehicle_close,
        vehicle_moving
    )


# Trabajamos únicamente con vehículos relevantes
df_vehicle_risk = df_dynamic[
    df_dynamic["class_name"].isin(VEHICLE_CLASSES)
].copy()


# Generación de indicadores
indicators = df_vehicle_risk.apply(
    get_dynamic_vehicle_indicators,
    axis=1,
    result_type="expand"
)

indicators.columns = [
    "vehicle_approaching",
    "vehicle_close",
    "vehicle_moving"
]


df_vehicle_risk[
```

```
    [
        "vehicle_approaching",
        "vehicle_close",
        "vehicle_moving"
    ]
] = indicators


# Visualización de las variables relevantes
display(
    df_vehicle_risk[
        [
            "video",
            "frame",
            "track_id",
            "class_name",
            "movement_direction",
            "proximity_trend",
            "relative_proximity",
            "vehicle_approaching",
            "vehicle_close",
            "vehicle_moving"
        ]
    ].head(10)
)
```

	video	frame	track_id	class_name	movement_direction	proximity_trend	relative_
0	1000179702	0	1	car	indeterminada	indeterminada	cerca
1	1000179702	1	1	car	indeterminada	indeterminada	cerca
2	1000179702	2	1	car	abajo	aproximandose	cerca
3	1000179702	3	1	car	abajo	distancia_aparente_estable	cerca
4	1000179702	4	1	car	abajo	distancia_aparente_estable	cerca
5	1000179702	5	1	car	abajo	distancia_aparente_estable	cerca
6	1000179702	6	1	car	abajo	alejandose	cerca
7	1000179702	7	1	car	abajo	alejandose	cerca
8	1000179702	8	1	car	abajo	aproximandose	cerca
9	1000179702	9	1	car	abajo	aproximandose	cerca

10.3 Integración de variables dinámicas con la estimación del riesgo

En esta etapa se integran las variables dinámicas obtenidas mediante el seguimiento de los vehículos con las reglas de decisión definidas para la estimación del riesgo de cruce. Las variables 'vehicle_approaching', 'vehicle_close'

y 'vehicle_moving' representan respectivamente la aproximación, proximidad y movimiento de los vehículos detectados.

In [109]:

```
# 10.3 INTEGRACIÓN DE VARIABLES DINÁMICAS CON LA ESTIMACIÓN
#      DEL RIESGO

# Partimos de los indicadores dinámicos calculados en 10.2
df_risk = df_vehicle_risk.copy()

# Estado del semáforo utilizado para esta prueba de integración.
# Se utiliza GREEN para comprobar que la presencia de vehículos
# puede modificar el nivel de riesgo incluso con señal favorable.
TRAFFIC_LIGHT_STATE_TEST = "GREEN"


def apply_crossing_risk(row):

    risk, recommendation = estimate_crossing_risk(
        traffic_light_state=TRAFFIC_LIGHT_STATE_TEST,
        vehicle_approaching=row["vehicle_approaching"],
        vehicle_close=row["vehicle_close"],
        vehicle_moving=row["vehicle_moving"]
    )

    return pd.Series({
        "risk": risk,
        "recommendation": recommendation
    })


df_risk[
    ["risk", "recommendation"]
] = df_risk.apply(
    apply_crossing_risk,
    axis=1
)

display(
    df_risk[
        [
            "video",
            "frame",
            "track_id",
            "class_name",
            "vehicle_approaching",
            "vehicle_close",
            "vehicle_moving",
            "risk",
```

```
            "recommendation"
        ]
    ].head(10)
)
```

	video	frame	track_id	class_name	vehicle_approaching	vehicle_close	vehicle_moving	
0	1000179702	0	1	car	False	True	False	MEI
1	1000179702	1	1	car	False	True	False	MEI
2	1000179702	2	1	car	True	True	True	HIG
3	1000179702	3	1	car	False	True	True	MEI
4	1000179702	4	1	car	False	True	True	MEI
5	1000179702	5	1	car	False	True	True	MEI
6	1000179702	6	1	car	False	True	True	MEI
7	1000179702	7	1	car	False	True	True	MEI
8	1000179702	8	1	car	True	True	True	HIG
9	1000179702	9	1	car	True	True	True	HIG

10.4 Agregación temporal del riesgo por vídeo

Las estimaciones obtenidas frame a frame permiten analizar la evolución temporal del riesgo. Para obtener una interpretación global de cada vídeo, se agregan las decisiones individuales considerando el nivel máximo de riesgo observado y la proporción de frames asociados a cada categoría.

In [110]:

```
# ============================================================
# 10.4 AGREGACIÓN TEMPORAL DEL RIESGO POR VÍDEO
# ============================================================

RISK_ORDER = {
    "LOW": 1,
    "MEDIUM": 2,
    "HIGH": 3
}

INVERSE_RISK_ORDER = {
    1: "LOW",
    2: "MEDIUM",
    3: "HIGH"
}

# ------------------------------------------------------------
# 1. Convertir el nivel de riesgo en valor numérico
# ------------------------------------------------------------

df_risk["risk_value"] = (
```

```
    df_risk["risk"]
    .map(RISK_ORDER)
)

# ------------------------------------------------------------
# 2. Obtener un único riesgo por frame
#    Se conserva el nivel más alto detectado en la escena
# ------------------------------------------------------------

frame_risk = (
    df_risk
    .groupby(
        ["video", "frame"],
        as_index=False
    )["risk_value"]
    .max()
)

frame_risk["risk"] = (
    frame_risk["risk_value"]
    .map(INVERSE_RISK_ORDER)
)

# ------------------------------------------------------------
# 3. Resumen temporal por vídeo
# ------------------------------------------------------------

video_risk_summary = (
    frame_risk
    .groupby("video")
    .agg(
        total_frames=("frame", "count"),
        high_frames=(
            "risk",
            lambda x: (x == "HIGH").sum()
        ),
        medium_frames=(
            "risk",
            lambda x: (x == "MEDIUM").sum()
        ),
        low_frames=(
            "risk",
            lambda x: (x == "LOW").sum()
        ),
        max_risk_value=(
            "risk_value",
            "max"
        )
    )
```

```
    .reset_index()
)

video_risk_summary["max_risk"] = (
    video_risk_summary["max_risk_value"]
    .map(INVERSE_RISK_ORDER)
)

# ------------------------------------------------------------
# 4. Proporción temporal de cada nivel de riesgo
# ------------------------------------------------------------

video_risk_summary["high_ratio"] = (
    video_risk_summary["high_frames"]
    / video_risk_summary["total_frames"]
)

video_risk_summary["medium_ratio"] = (
    video_risk_summary["medium_frames"]
    / video_risk_summary["total_frames"]
)

video_risk_summary["low_ratio"] = (
    video_risk_summary["low_frames"]
    / video_risk_summary["total_frames"]
)

display(
    video_risk_summary[
        [
            "video",
            "total_frames",
            "high_frames",
            "medium_frames",
            "low_frames",
            "high_ratio",
            "medium_ratio",
            "low_ratio",
            "max_risk"
        ]
    ]
)
```

	video	total_frames	high_frames	medium_frames	low_frames	high_ratio	medium
0	1000179702	279	100	107	72	0.358423	0.383513
1	1000179704	323	23	204	96	0.071207	0.631579
2	1000179707	538	88	389	61	0.163569	0.723048
3	1000179709	464	164	225	75	0.353448	0.484914

	video	total_frames	high_frames	medium_frames	low_frames	high_ratio	medium
4	1000179711	253	16	105	132	0.063241	0.41502
5	20260611_184739	420	41	301	78	0.097619	0.71666
6	20260611_184808	364	69	223	72	0.189560	0.61263
7	20260611_185035	272	43	222	7	0.158088	0.81617
8	20260611_185443~2	91	42	46	3	0.461538	0.50549
9	20260611_185616	573	121	401	51	0.211169	0.69982

10.5 DECISIÓN GLOBAL ROBUSTA POR VÍDEO

In [111]:

```
# ============================================================
# 10.5 DECISIÓN GLOBAL ROBUSTA POR VÍDEO
# ============================================================

def robust_global_decision(row):
    """
    Determina el nivel global de riesgo del vídeo teniendo
    en cuenta la persistencia temporal de HIGH, MEDIUM y LOW.

    Criterio:
    - HIGH: presencia elevada y persistente de riesgo alto.
    - LOW: predominio claro de situaciones de bajo riesgo
           y muy poca presencia de riesgo alto.
    - MEDIUM: situaciones intermedias o mixtas.
    """

    high_ratio = row["high_ratio"]
    medium_ratio = row["medium_ratio"]
    low_ratio = row["low_ratio"]

    # ----------------------------------------------------------
    # 1. RIESGO ALTO
    # Riesgo HIGH claramente persistente en el vídeo
    # ----------------------------------------------------------
    if high_ratio >= 0.45:
        return pd.Series({
            "global_risk": "HIGH",
            "global_recommendation": "WAIT"
        })

    # ----------------------------------------------------------
    # 2. RIESGO BAJO
    # Predominio de LOW y escasa presencia de HIGH
    # ----------------------------------------------------------
    if low_ratio >= 0.50 and high_ratio < 0.15:
```

```
        return pd.Series({
            "global_risk": "LOW",
            "global_recommendation": "CROSS WITH CAUTION"
        })

    # ------------------------------------------------------
    # 3. RIESGO MEDIO
    # Escenario intermedio o con evidencia contradictoria
    # ------------------------------------------------------
    return pd.Series({
        "global_risk": "MEDIUM",
        "global_recommendation": "CAUTION"
    })


# Aplicar decisión global
video_risk_summary[
    ["global_risk", "global_recommendation"]
] = video_risk_summary.apply(
    robust_global_decision,
    axis=1
)


# Mostrar resumen final
display(
    video_risk_summary[
        [
            "video",
            "total_frames",
            "high_ratio",
            "medium_ratio",
            "low_ratio",
            "global_risk",
            "global_recommendation"
        ]
    ]
)
```

	video	total_frames	high_ratio	medium_ratio	low_ratio	global_risk	global_recom
0	1000179702	279	0.358423	0.383513	0.258065	MEDIUM	CAUTION
1	1000179704	323	0.071207	0.631579	0.297214	MEDIUM	CAUTION
2	1000179707	538	0.163569	0.723048	0.113383	MEDIUM	CAUTION
3	1000179709	464	0.353448	0.484914	0.161638	MEDIUM	CAUTION
4	1000179711	253	0.063241	0.415020	0.521739	LOW	CROSS WITH CAUTION
5	20260611_184739	420	0.097619	0.716667	0.185714	MEDIUM	CAUTION

	video	total_frames	high_ratio	medium_ratio	low_ratio	global_risk	global_recom
6	20260611_184808	364	0.189560	0.612637	0.197802	MEDIUM	CAUTION
7	20260611_185035	272	0.158088	0.816176	0.025735	MEDIUM	CAUTION
8	20260611_185443~2	91	0.461538	0.505495	0.032967	HIGH	WAIT
9	20260611_185616	573	0.211169	0.699825	0.089005	MEDIUM	CAUTION

10.6 Validación manual de los escenarios de prueba

Con el objetivo de evaluar la coherencia de la estimación automática del riesgo, se establece una referencia manual para cada uno de los vídeos analizados.

La anotación manual se realiza mediante inspección visual de las secuencias, considerando el estado del semáforo peatonal y el comportamiento de los vehículos presentes en la escena. Esta referencia permitirá posteriormente comparar el nivel de riesgo esperado con el estimado automáticamente por el sistema.

In [112]:

```
# 10.6 REFERENCIA MANUAL DE LOS ESCENARIOS DE CRUCE


manual_validation = pd.DataFrame([
    {
        "video": "20260611_184739",
        "crossing_id": 1,
        "pedestrian_light_manual": "GREEN",
        "vehicle_signal_manual":
            "AMBER_FROM_LEFT",
        "vehicle_behavior_manual":
            "TRAFFIC_PRESENT",
        "pedestrian_action_manual": "CROSSING"
    },
    {
        "video": "20260611_184739",
        "crossing_id": 2,
        "pedestrian_light_manual":
            "GREEN_TO_RED",
        "vehicle_signal_manual":
            "AMBER_FROM_RIGHT_LEFT_TURN",
        "vehicle_behavior_manual":
            "APPROACHING_AND_TURNING",
        "pedestrian_action_manual": "CROSSING"
    },
    {
        "video": "20260611_184808",
        "crossing_id": 1,
```

```
    "pedestrian_light_manual": "RED",
    "vehicle_signal_manual": "NOT_ANNOTATED",
    "vehicle_behavior_manual":
        "APPROACHING_AND_CROSSING",
    "pedestrian_action_manual": "WAIT"
},
{
    "video": "20260611_185035",
    "crossing_id": 1,
    "pedestrian_light_manual":
        "NO_TRAFFIC_LIGHT",
    "vehicle_signal_manual": "NOT_APPLICABLE",
    "vehicle_behavior_manual":
        "APPROACHING_AND_CROSSING",
    "pedestrian_action_manual": "OBSERVED"
},
{
    "video": "20260611_185443~2",
    "crossing_id": 1,
    "pedestrian_light_manual": "GREEN",
    "vehicle_signal_manual": "NOT_ANNOTATED",
    "vehicle_behavior_manual": "STOPPED",
    "pedestrian_action_manual": "CROSSING"
},
{
    "video": "20260611_185616",
    "crossing_id": 1,
    "pedestrian_light_manual": "RED",
    "vehicle_signal_manual": "NOT_ANNOTATED",
    "vehicle_behavior_manual":
        "APPROACHING_AND_CROSSING; "
        "E_SCOOTER_WHEN_SAFE",
    "pedestrian_action_manual": "WAIT"
},
{
    "video": "1000179702",
    "crossing_id": 1,
    "pedestrian_light_manual": "GREEN",
    "vehicle_signal_manual": "AMBER",
    "vehicle_behavior_manual":
        "APPROACHING_AND_CROSSING",
    "pedestrian_action_manual": "CROSSING"
},
{
    "video": "1000179704",
    "crossing_id": 1,
    "pedestrian_light_manual": "GREEN",
    "vehicle_signal_manual":
        "AMBER_FROM_LEFT",
```

```
        "vehicle_behavior_manual":
            "TRAFFIC_PRESENT",
        "pedestrian_action_manual": "CROSSING"
    },
    {
        "video": "1000179704",
        "crossing_id": 2,
        "pedestrian_light_manual":
            "GREEN_TO_RED",
        "vehicle_signal_manual":
            "AMBER_FROM_RIGHT_LEFT_TURN",
        "vehicle_behavior_manual":
            "APPROACHING_AND_TURNING",
        "pedestrian_action_manual": "CROSSING"
    },
    {
        "video": "1000179707",
        "crossing_id": 1,
        "pedestrian_light_manual": "RED",
        "vehicle_signal_manual": "NOT_ANNOTATED",
        "vehicle_behavior_manual":
            "NO_APPROACHING_VEHICLES",
        "pedestrian_action_manual":
            "EVENTUALLY_CROSSES"
    },
    {
        "video": "1000179709",
        "crossing_id": 1,
        "pedestrian_light_manual": "RED",
        "vehicle_signal_manual": "NOT_ANNOTATED",
        "vehicle_behavior_manual":
            "MULTIPLE_APPROACHING_AND_CROSSING",
        "pedestrian_action_manual": "WAIT"
    },
    {
        "video": "1000179711",
        "crossing_id": 1,
        "pedestrian_light_manual": "GREEN",
        "vehicle_signal_manual":
            "DIRECTION_DEPENDENT_SIGNALS",
        "vehicle_behavior_manual":
            "MULTIPLE_TRAFFIC_FLOWS",
        "pedestrian_action_manual": "CROSSING"
    }
])

display(manual_validation)
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manu
0	20260611_184739	1	GREEN	AMBER_FROM_LEFT
1	20260611_184739	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
2	20260611_184808	1	RED	NOT_ANNOTATED
3	20260611_185035	1	NO_TRAFFIC_LIGHT	NOT_APPLICABLE
4	20260611_185443~2	1	GREEN	NOT_ANNOTATED
5	20260611_185616	1	RED	NOT_ANNOTATED
6	1000179702	1	GREEN	AMBER
7	1000179704	1	GREEN	AMBER_FROM_LEFT
8	1000179704	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
9	1000179707	1	RED	NOT_ANNOTATED
10	1000179709	1	RED	NOT_ANNOTATED
11	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNAL

In [113]:

```
# 10.7 DEFINICIÓN DEL RIESGO ESPERADO EN LOS ESCENARIOS DE PRUEBA

expected_risks = [
    "MEDIUM",  # 20260611_184739 - cruce 1
    "HIGH",    # 20260611_184739 - cruce 2
    "HIGH",    # 20260611_184808
    "HIGH",    # 20260611_185035
    "LOW",     # 20260611_185443~2
    "HIGH",    # 20260611_185616
    "HIGH",    # 1000179702
    "MEDIUM",  # 1000179704 - cruce 1
    "HIGH",    # 1000179704 - cruce 2
    "LOW",     # 1000179707
    "HIGH",    # 1000179709
    "MEDIUM"   # 1000179711
]

if len(expected_risks) != len(manual_validation):
    raise ValueError(
        "El número de riesgos esperados no coincide "
        "con el número de escenarios manuales."
    )

manual_validation["expected_risk"] = expected_risks

display(
    manual_validation[
        [
            "video",
            "crossing_id",
```

```
            "pedestrian_light_manual",
            "vehicle_behavior_manual",
            "pedestrian_action_manual",
            "expected_risk"
        ]
    ]
)
```

	video	crossing_id	pedestrian_light_manual	vehicle_behav
0	20260611_184739	1	GREEN	TRAFFIC_PRESENT
1	20260611_184739	2	GREEN_TO_RED	APPROACHING_AND_TURNING
2	20260611_184808	1	RED	APPROACHING_AND_CROSSING
3	20260611_185035	1	NO_TRAFFIC_LIGHT	APPROACHING_AND_CROSSING
4	20260611_185443~2	1	GREEN	STOPPED
5	20260611_185616	1	RED	APPROACHING_AND_CROSSING; E_SCOOTER_WHEN_SAFE
6	1000179702	1	GREEN	APPROACHING_AND_CROSSING
7	1000179704	1	GREEN	TRAFFIC_PRESENT
8	1000179704	2	GREEN_TO_RED	APPROACHING_AND_TURNING
9	1000179707	1	RED	NO_APPROACHING_VEHICLES
10	1000179709	1	RED	MULTIPLE_APPROACHING_AND_
11	1000179711	1	GREEN	MULTIPLE_TRAFFIC_FLOWS

10.8 Delimitación temporal de los vídeos con múltiples situaciones de cruce

Durante la inspección manual de los vídeos se identificaron dos grabaciones que contienen dos situaciones de cruce diferenciadas. Para permitir una evaluación independiente de cada situación, se delimitaron manualmente sus intervalos temporales a nivel de fotograma.

Los intervalos obtenidos se utilizan posteriormente para realizar la evaluación automática por situación de cruce.

In [114]:

```
# ============================================================
# 10.8 INTERVALOS DE LOS VÍDEOS CON MÚLTIPLES CRUCES
# ============================================================

crossing_intervals = {
    "20260611_184739": {
        1: (0, 400),
        2: (401, 433)
    },
    "1000179704": {
```

```
        1: (0, 350),
        2: (351, 777)
    }
}

crossing_intervals
```

Out[114]:

```
{'20260611_184739': {1: (0, 400), 2: (401, 433)},
 '1000179704': {1: (0, 350), 2: (351, 777)}}
```

10.9 Evaluación automática por situación de cruce

In [115]:

```
# 10.9 EVALUACIÓN AUTOMÁTICA POR SITUACIÓN DE CRUCE

# 1. Construir intervalos para las 12 situaciones

evaluation_intervals = [
    ("20260611_184739", 1, 0, 400),
    ("20260611_184739", 2, 401, 433),

    ("20260611_184808", 1, None, None),
    ("20260611_185035", 1, None, None),
    ("20260611_185443~2", 1, None, None),
    ("20260611_185616", 1, None, None),

    ("1000179702", 1, None, None),

    ("1000179704", 1, 0, 350),
    ("1000179704", 2, 351, 777),

    ("1000179707", 1, None, None),
    ("1000179709", 1, None, None),
    ("1000179711", 1, None, None)
]


# ------------------------------------------------------------
# 2. Función de decisión por intervalo
# ------------------------------------------------------------

def classify_interval(video, crossing_id, start_frame=None,
end_frame=None):

    subset = frame_risk[
        frame_risk["video"].astype(str) == str(video)
    ].copy()

    if start_frame is not None:
```

```
    subset = subset[
        subset["frame"] >= start_frame
    ]

if end_frame is not None:
    subset = subset[
        subset["frame"] <= end_frame
    ]

if subset.empty:
    return {
        "video": video,
        "crossing_id": crossing_id,
        "frames_evaluated": 0,
        "high_ratio_auto": None,
        "medium_ratio_auto": None,
        "low_ratio_auto": None,
        "predicted_risk": "NO_DATA"
    }

total = len(subset)

high_ratio = (
    (subset["risk"] == "HIGH").sum() / total
)

medium_ratio = (
    (subset["risk"] == "MEDIUM").sum() / total
)

low_ratio = (
    (subset["risk"] == "LOW").sum() / total
)

# Mismo criterio temporal definido en 10.5
if high_ratio >= 0.45:
    predicted_risk = "HIGH"

elif low_ratio >= 0.50 and high_ratio < 0.15:
    predicted_risk = "LOW"

else:
    predicted_risk = "MEDIUM"

return {
    "video": video,
    "crossing_id": crossing_id,
    "frames_evaluated": total,
    "high_ratio_auto": high_ratio,
```

```
        "medium_ratio_auto": medium_ratio,
        "low_ratio_auto": low_ratio,
        "predicted_risk": predicted_risk
    }


# ------------------------------------------------------------
# 3. Evaluar las 12 situaciones
# ------------------------------------------------------------

automatic_crossing_results = pd.DataFrame([
    classify_interval(
        video,
        crossing_id,
        start_frame,
        end_frame
    )
    for video, crossing_id, start_frame, end_frame
    in evaluation_intervals
])


# ------------------------------------------------------------
# 4. Comparar con el Ground Truth manual
# ------------------------------------------------------------

evaluation_results = manual_validation.merge(
    automatic_crossing_results,
    on=["video", "crossing_id"],
    how="left"
)

evaluation_results["correct"] = (
    evaluation_results["expected_risk"]
    == evaluation_results["predicted_risk"]
)


# ------------------------------------------------------------
# 5. Mostrar comparación
# ------------------------------------------------------------

display(
    evaluation_results[
        [
            "video",
            "crossing_id",
            "expected_risk",
            "predicted_risk",
```

```
            "correct",
            "frames_evaluated",
            "high_ratio_auto",
            "medium_ratio_auto",
            "low_ratio_auto"
        ]
    ]
)
```

	video	crossing_id	expected_risk	predicted_risk	correct	frames_evaluated	high_
0	20260611_184739	1	MEDIUM	MEDIUM	True	388	0.1056
1	20260611_184739	2	HIGH	LOW	False	32	0.0000
2	20260611_184808	1	HIGH	MEDIUM	False	364	0.1895
3	20260611_185035	1	HIGH	MEDIUM	False	272	0.1580
4	20260611_185443~2	1	LOW	HIGH	False	91	0.4615
5	20260611_185616	1	HIGH	MEDIUM	False	573	0.2111
6	1000179702	1	HIGH	MEDIUM	False	279	0.3584
7	1000179704	1	MEDIUM	MEDIUM	True	128	0.0000
8	1000179704	2	HIGH	MEDIUM	False	195	0.1179
9	1000179707	1	LOW	MEDIUM	False	538	0.1635
10	1000179709	1	HIGH	MEDIUM	False	464	0.3534
11	1000179711	1	MEDIUM	LOW	False	253	0.0632

10.10 PRIMERA FUSIÓN DEL RIESGORiesgo dinámico + estado del semáforo peatonal

In [116]:

```
# ============================================================
# 10.10 PRIMERA FUSIÓN DEL RIESGO
# Riesgo dinámico + estado del semáforo peatonal
# ============================================================

# ------------------------------------------------------------
# 1. Orden de los niveles de riesgo
# ------------------------------------------------------------

RISK_LEVEL = {
    "LOW": 0,
    "MEDIUM": 1,
    "HIGH": 2
}

RISK_NAME = {
    0: "LOW",
    1: "MEDIUM",
```

```
    2: "HIGH"
}


# ------------------------------------------------------------
# 2. Función de fusión
# ------------------------------------------------------------

def fuse_crossing_risk(row):

    dynamic_risk = str(row["predicted_risk"]).upper()
    pedestrian_light = str(row["pedestrian_light_manual"]).upper()

    # Riesgo dinámico calculado automáticamente
    dynamic_level = RISK_LEVEL.get(dynamic_risk, 1)

    # ------------------------------------------------------------
    # REGLA A
    # Semáforo peatonal rojo
    #
    # El rojo no implica por sí solo que exista un vehículo
    # peligroso, pero impide considerar LOW una situación
    # donde además existe riesgo dinámico.
    # ------------------------------------------------------------

    if pedestrian_light == "RED":

        if dynamic_level >= 1:
            return "HIGH"

        return "MEDIUM"


    # ------------------------------------------------------------
    # REGLA B
    # Transición verde -> rojo
    #
    # Es una situación temporalmente crítica.
    # ------------------------------------------------------------

    if pedestrian_light == "GREEN_TO_RED":

        if dynamic_level >= 1:
            return "HIGH"

        return "MEDIUM"


    # ------------------------------------------------------------
```

```
    # REGLA C
    # Semáforo peatonal verde
    #
    # El verde NO elimina el riesgo vehicular.
    # Si los vehículos generan peligro, prevalece.
    # ----------------------------------------------------------

    if pedestrian_light == "GREEN":

        return RISK_NAME[dynamic_level]


    # ----------------------------------------------------------
    # REGLA D
    # Cruce sin semáforo peatonal
    #
    # La decisión depende fundamentalmente del tráfico.
    # ----------------------------------------------------------

    if pedestrian_light == "NO_TRAFFIC_LIGHT":

        return RISK_NAME[dynamic_level]


    # ----------------------------------------------------------
    # Estado desconocido/no reconocido:
    # conservar riesgo dinámico
    # ----------------------------------------------------------

    return RISK_NAME[dynamic_level]


# ------------------------------------------------------------
# 3. Aplicar fusión
# ------------------------------------------------------------

evaluation_results["fused_risk"] = evaluation_results.apply(
    fuse_crossing_risk,
    axis=1
)


# Comparación exclusivamente experimental.
# El estado manual del semáforo no forma parte
# de la entrada del sistema automático final.

evaluation_results["correct_fused"] = (
    evaluation_results["expected_risk"]
    == evaluation_results["fused_risk"]
```

```
)


# ------------------------------------------------------------
# 5. Métricas antes y después de la fusión
# ------------------------------------------------------------

accuracy_dynamic = evaluation_results["correct"].mean()
accuracy_fused = evaluation_results["correct_fused"].mean()

print("=" * 65)
print("EVALUACIÓN DE LA FUSIÓN DE RIESGO")
print("=" * 65)

print(f"\nSituaciones evaluadas: {len(evaluation_results)}")

print(
    f"Accuracy riesgo dinámico: "
    f"{accuracy_dynamic:.2%}"
)

print(
    f"Accuracy después de la fusión: "
    f"{accuracy_fused:.2%}"
)

print(
    f"Mejora absoluta: "
    f"{(accuracy_fused - accuracy_dynamic):.2%}"
)


# ------------------------------------------------------------
# 6. Mostrar resultados situación por situación
# ------------------------------------------------------------

display(
    evaluation_results[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_signal_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "fused_risk",
            "correct_fused"
        ]
```

```
    ]
)
================================================================
EVALUACIÓN DE LA FUSIÓN DE RIESGO
================================================================

Situaciones evaluadas: 12
Accuracy riesgo dinámico: 16.67%
Accuracy después de la fusión: 50.00%
Mejora absoluta: 33.33%
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manu
0	20260611_184739	1	GREEN	AMBER_FROM_LEFT
1	20260611_184739	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
2	20260611_184808	1	RED	NOT_ANNOTATED
3	20260611_185035	1	NO_TRAFFIC_LIGHT	NOT_APPLICABLE
4	20260611_185443~2	1	GREEN	NOT_ANNOTATED
5	20260611_185616	1	RED	NOT_ANNOTATED
6	1000179702	1	GREEN	AMBER
7	1000179704	1	GREEN	AMBER_FROM_LEFT
8	1000179704	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
9	1000179707	1	RED	NOT_ANNOTATED
10	1000179709	1	RED	NOT_ANNOTATED
11	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNAL

In [117]:

```
# 10.11 ANÁLISIS DE ERRORES TRAS LA PRIMERA FUSIÓN

fusion_errors = evaluation_results[
    ~evaluation_results["correct_fused"]
].copy()

print("=" * 70)
print("ANÁLISIS DE ERRORES TRAS LA PRIMERA FUSIÓN")
print("=" * 70)

print(f"Situaciones evaluadas: {len(evaluation_results)}")
print(
    "Situaciones correctas:",
    int(evaluation_results["correct_fused"].sum())
)
print(
    "Situaciones incorrectas:",
    len(fusion_errors)
)
```

```
display(
    fusion_errors[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_signal_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "fused_risk",
            "high_ratio_auto",
            "medium_ratio_auto",
            "low_ratio_auto"
        ]
    ].reset_index(drop=True)
)
```

```
======================================================================
ANÁLISIS DE ERRORES TRAS LA PRIMERA FUSIÓN
======================================================================
Situaciones evaluadas: 12
Situaciones correctas: 6
Situaciones incorrectas: 6
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manua
0	20260611_184739	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TURN
1	20260611_185035	1	NO_TRAFFIC_LIGHT	NOT_APPLICABLE
2	20260611_185443~2	1	GREEN	NOT_ANNOTATED
3	1000179702	1	GREEN	AMBER
4	1000179707	1	RED	NOT_ANNOTATED
5	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNALS

10.12 Segunda fusión contextual del riesgo. Experimento con información manual

In [118]:

```
# 10.12 SEGUNDA FUSIÓN CONTEXTUAL DEL RIESGO
#       EXPERIMENTO CON INFORMACIÓN MANUAL

# Combina:

#   - riesgo dinámico estimado automáticamente
#   - semáforo peatonal
#   - señalización para vehículos
#   - comportamiento observado de los vehículos
#
# IMPORTANTE:
```

```
# El riesgo físico se mantiene separado de la recomendación
# final de cruce al peatón.
# Esta etapa es experimental y utiliza anotaciones manuales
# del semáforo y del contexto vehicular. Por tanto, no
# representa todavía el sistema automático final.

def contextual_risk_fusion(row):

    dynamic = str(row["predicted_risk"]).upper()
    pedestrian = str(row["pedestrian_light_manual"]).upper()
    vehicle_signal = str(row["vehicle_signal_manual"]).upper()
    behavior = str(row["vehicle_behavior_manual"]).upper()

    # ---------------------------------------------------------
    # 1. Evidencia de vehículos aproximándose o cruzando
    # ---------------------------------------------------------
    active_vehicle = (
        "APPROACHING" in behavior
        or "CROSSING" in behavior
        or "TURNING" in behavior
        or "MULTIPLE_TRAFFIC_FLOWS" in behavior
    )

    # ---------------------------------------------------------
    # 2. Ausencia explícita de vehículos aproximándose
    # ---------------------------------------------------------
    no_approaching_vehicle = (
        "NO_APPROACHING_VEHICLES" in behavior
    )

    # ---------------------------------------------------------
    # 3. Vehículos detenidos
    # ---------------------------------------------------------
    stopped_vehicle = (
        behavior == "STOPPED"
    )

    # ---------------------------------------------------------
    # 4. Situaciones de señalización vehicular conflictiva
    # ---------------------------------------------------------
    vehicle_conflict = (
        "AMBER" in vehicle_signal
        or "DIRECTION_DEPENDENT" in vehicle_signal
    )

    # ---------------------------------------------------------
    # REGLAS CONTEXTUALES
    # ---------------------------------------------------------
```

```
    # Vehículos detenidos: la presencia visual del vehículo
    # no debe interpretarse automáticamente como peligro alto.
    if stopped_vehicle:
        return "LOW"

    # Si se ha observado explícitamente ausencia de vehículos
    # aproximándose, reducimos el riesgo físico.
    if no_approaching_vehicle:
        return "LOW"

    # Vehículos aproximándose/cruzando con señalización
    # potencialmente conflictiva.
    if active_vehicle and vehicle_conflict:
        return "HIGH"

    # Vehículos activos sin semáforo peatonal disponible.
    if active_vehicle and pedestrian == "NO_TRAFFIC_LIGHT":
        return "HIGH"

    # Vehículos activos cuando el semáforo peatonal está
    # cambiando de verde a rojo.
    if active_vehicle and pedestrian == "GREEN_TO_RED":
        return "HIGH"

    # Múltiples flujos de tráfico requieren al menos
    # consideración de riesgo medio.
    if "MULTIPLE_TRAFFIC_FLOWS" in behavior:
        return "MEDIUM"

    # En cualquier otro caso conservamos la estimación
    # obtenida mediante las variables dinámicas.
    return dynamic


# Aplicar la fusión contextual
evaluation_results["contextual_risk"] = evaluation_results.apply(
    contextual_risk_fusion,
    axis=1
)

# Comparación con el Ground Truth
evaluation_results["correct_contextual"] = (
    evaluation_results["expected_risk"]
    == evaluation_results["contextual_risk"]
)

contextual_accuracy = (
    evaluation_results["correct_contextual"].mean() * 100
)
```

```
print("=" * 65)
print("EVALUACIÓN DEL RIESGO CONTEXTUAL")
print("=" * 65)
print(f"Situaciones evaluadas: {len(evaluation_results)}")
print(f"Accuracy riesgo dinámico: "
      f"{evaluation_results['correct'].mean() * 100:.2f}%")
print(f"Accuracy fusión anterior: "
      f"{evaluation_results['correct_fused'].mean() * 100:.2f}%")
print(f"Accuracy contextual: {contextual_accuracy:.2f}%")
print("=" * 65)

display(
    evaluation_results[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_signal_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "fused_risk",
            "contextual_risk",
            "correct_contextual"
        ]
    ]
)
```

```
==================================================================
EVALUACIÓN DEL RIESGO CONTEXTUAL
==================================================================
Situaciones evaluadas: 12
Accuracy riesgo dinámico: 16.67%
Accuracy fusión anterior: 50.00%
Accuracy contextual: 66.67%
==================================================================
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manu
0	20260611_184739	1	GREEN	AMBER_FROM_LEFT
1	20260611_184739	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
2	20260611_184808	1	RED	NOT_ANNOTATED
3	20260611_185035	1	NO_TRAFFIC_LIGHT	NOT_APPLICABLE
4	20260611_185443~2	1	GREEN	NOT_ANNOTATED
5	20260611_185616	1	RED	NOT_ANNOTATED
6	1000179702	1	GREEN	AMBER
7	1000179704	1	GREEN	AMBER_FROM_LEFT

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manu
8	1000179704	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
9	1000179707	1	RED	NOT_ANNOTATED
10	1000179709	1	RED	NOT_ANNOTATED
11	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNAL

In [119]:

```
# 10.13 ANÁLISIS DE LOS ERRORES CONTEXTUALES RESTANTES

contextual_errors = evaluation_results[
    ~evaluation_results["correct_contextual"]
].copy()

print("=" * 70)
print("ERRORES RESTANTES TRAS LA FUSIÓN CONTEXTUAL")
print("=" * 70)

print(f"\nSituaciones totales: {len(evaluation_results)}")
print(f"Situaciones correctas:
{evaluation_results['correct_contextual'].sum()}")
print(f"Situaciones incorrectas: {len(contextual_errors)}")

display(
    contextual_errors[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_signal_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "fused_risk",
            "contextual_risk",
            "frames_evaluated",
            "high_ratio_auto",
            "medium_ratio_auto",
            "low_ratio_auto"
        ]
    ].reset_index(drop=True)
)
```

```
======================================================================
ERRORES RESTANTES TRAS LA FUSIÓN CONTEXTUAL
======================================================================

Situaciones totales: 12
Situaciones correctas: 8
Situaciones incorrectas: 4
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manual	
0	20260611_184808	1	RED	NOT_ANNOTATED	A
1	20260611_185616	1	RED	NOT_ANNOTATED	A B
2	1000179709	1	RED	NOT_ANNOTATED	N
3	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNALS	N

In [120]:

```
# ============================================================
# 10.14 REFINAMIENTO TEMPORAL TRAS EL ANÁLISIS DE ERRORES
# ============================================================

crossing_intervals["20260611_185616"] = {
    1: (0, 270),      # tráfico en conflicto
    2: (271, 612)     # calzada libre
}

crossing_intervals["1000179709"] = {
    1: (0, 440),      # tráfico en conflicto
    2: (441, 517)     # calzada libre
}
```

In [121]:

```
# ============================================================
# 10.15 ACTUALIZACIÓN DE LA REFERENCIA MANUAL
#       A 14 SITUACIONES TEMPORALES
# ============================================================


# ------------------------------------------------------------
# 1. Partimos de las situaciones ya validadas,
#    eliminando los dos vídeos que ahora se dividen en fases
# ------------------------------------------------------------

manual_validation_v2 = manual_validation[
    ~manual_validation["video"].isin([
        "20260611_185616",
        "1000179709"
    ])
].copy()


# ------------------------------------------------------------
# 2. Añadir las cuatro situaciones temporales revisadas
# ------------------------------------------------------------

new_temporal_situations = pd.DataFrame([

    # ---------------------------------------------------
    # 20260611_185616
```

```
    # ------------------------------------------------------
    {
        "video": "20260611_185616",
        "crossing_id": 1,
        "pedestrian_light_manual": "RED",
        "vehicle_signal_manual": "NOT_ANNOTATED",
        "vehicle_behavior_manual":
            "APPROACHING_AND_CROSSING; E_SCOOTER_AND_VEHICLE",
        "pedestrian_action_manual": "WAIT",
        "expected_risk": "HIGH"
    },

    {
        "video": "20260611_185616",
        "crossing_id": 2,
        "pedestrian_light_manual": "RED",
        "vehicle_signal_manual": "NOT_ANNOTATED",
        "vehicle_behavior_manual":
            "NO_APPROACHING_VEHICLES",
        "pedestrian_action_manual": "WAIT",
        "expected_risk": "LOW"
    },

    # ------------------------------------------------------
    # 1000179709
    # ------------------------------------------------------
    {
        "video": "1000179709",
        "crossing_id": 1,
        "pedestrian_light_manual": "RED",
        "vehicle_signal_manual": "NOT_ANNOTATED",
        "vehicle_behavior_manual":
            "MULTIPLE_APPROACHING_AND_CROSSING",
        "pedestrian_action_manual": "WAIT",
        "expected_risk": "HIGH"
    },

    {
        "video": "1000179709",
        "crossing_id": 2,
        "pedestrian_light_manual": "RED",
        "vehicle_signal_manual": "NOT_ANNOTATED",
        "vehicle_behavior_manual":
            "NO_APPROACHING_VEHICLES",
        "pedestrian_action_manual": "WAIT",
        "expected_risk": "LOW"
    }

])
```

```
manual_validation_v2 = pd.concat(
    [
        manual_validation_v2,
        new_temporal_situations
    ],
    ignore_index=True
)


# Ordenar para facilitar la lectura
manual_validation_v2 = (
    manual_validation_v2
    .sort_values(
        ["video", "crossing_id"]
    )
    .reset_index(drop=True)
)


# ------------------------------------------------------------
# 3. Intervalos de las 14 situaciones
# ------------------------------------------------------------

evaluation_intervals_v2 = [

    ("20260611_184739", 1, 0, 400),
    ("20260611_184739", 2, 401, 433),

    ("20260611_184808", 1, None, None),
    ("20260611_185035", 1, None, None),
    ("20260611_185443~2", 1, None, None),

    # Vídeo dividido temporalmente
    ("20260611_185616", 1, 0, 270),
    ("20260611_185616", 2, 271, 612),

    ("1000179702", 1, None, None),

    ("1000179704", 1, 0, 350),
    ("1000179704", 2, 351, 777),

    ("1000179707", 1, None, None),

    # Vídeo dividido temporalmente
    ("1000179709", 1, 0, 440),
    ("1000179709", 2, 441, 517),
```

```
    ("1000179711", 1, None, None)
]


# ------------------------------------------------------------
# 4. Recalcular riesgo dinámico para cada situación
# ------------------------------------------------------------

automatic_crossing_results_v2 = pd.DataFrame([

    classify_interval(
        video,
        crossing_id,
        start_frame,
        end_frame
    )

    for (
        video,
        crossing_id,
        start_frame,
        end_frame
    ) in evaluation_intervals_v2
])


# ------------------------------------------------------------
# 5. Combinar referencia manual y predicción automática
# ------------------------------------------------------------

evaluation_results_v2 = manual_validation_v2.merge(
    automatic_crossing_results_v2,
    on=["video", "crossing_id"],
    how="left"
)


# ------------------------------------------------------------
# 6. Accuracy del riesgo dinámico
# ------------------------------------------------------------

evaluation_results_v2["correct_dynamic"] = (
    evaluation_results_v2["expected_risk"]
    == evaluation_results_v2["predicted_risk"]
)


# ------------------------------------------------------------
# 7. Mostrar resultado
```

```
# ------------------------------------------------------------

print("=" * 70)
print("EVALUACIÓN ACTUALIZADA - 14 SITUACIONES")
print("=" * 70)

print(
    "Situaciones evaluadas:",
    len(evaluation_results_v2)
)

print(
    "Accuracy dinámico:",
    f"{evaluation_results_v2['correct_dynamic'].mean() * 100:.2f}%"
)

print("=" * 70)

display(
    evaluation_results_v2[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "correct_dynamic",
            "frames_evaluated",
            "high_ratio_auto",
            "medium_ratio_auto",
            "low_ratio_auto"
        ]
    ]
)
```

```
======================================================================
EVALUACIÓN ACTUALIZADA - 14 SITUACIONES
======================================================================
Situaciones evaluadas: 14
Accuracy dinámico: 21.43%
======================================================================
```

	video	crossing_id	pedestrian_light_manual	vehicle_behav
0	1000179702	1	GREEN	APPROACHING_AND_CROSSING
1	1000179704	1	GREEN	TRAFFIC_PRESENT
2	1000179704	2	GREEN_TO_RED	APPROACHING_AND_TURNING
3	1000179707	1	RED	NO_APPROACHING_VEHICLES
4	1000179709	1	RED	MULTIPLE_APPROACHING_AND_

	video	crossing_id	pedestrian_light_manual	vehicle_behav
5	1000179709	2	RED	NO_APPROACHING_VEHICLES
6	1000179711	1	GREEN	MULTIPLE_TRAFFIC_FLOWS
7	20260611_184739	1	GREEN	TRAFFIC_PRESENT
8	20260611_184739	2	GREEN_TO_RED	APPROACHING_AND_TURNING
9	20260611_184808	1	RED	APPROACHING_AND_CROSSING
10	20260611_185035	1	NO_TRAFFIC_LIGHT	APPROACHING_AND_CROSSING
11	20260611_185443~2	1	GREEN	STOPPED
12	20260611_185616	1	RED	APPROACHING_AND_CROSSING; E_SCOOTER_AND_VEHICLE
13	20260611_185616	2	RED	NO_APPROACHING_VEHICLES

In [122]:

```
# ============================================================
# 10.16 FUSIÓN CONTEXTUAL SOBRE LAS 14 SITUACIONES REFINADAS
# ============================================================

# Reutilizamos exactamente la misma función contextual
# definida anteriormente:
# contextual_risk_fusion(row)

evaluation_results_v2["contextual_risk"] = (
    evaluation_results_v2.apply(
        contextual_risk_fusion,
        axis=1
    )
)

# Comparación con el Ground Truth actualizado
evaluation_results_v2["correct_contextual"] = (
    evaluation_results_v2["expected_risk"]
    == evaluation_results_v2["contextual_risk"]
)

# Accuracy contextual actualizado
contextual_accuracy_v2 = (
    evaluation_results_v2["correct_contextual"].mean() * 100
)

print("=" * 70)
print("EVALUACIÓN CONTEXTUAL ACTUALIZADA - 14 SITUACIONES")
print("=" * 70)

print(
    f"Situaciones evaluadas: "
    f"{len(evaluation_results_v2)}"
)
```

```
print(
    f"Accuracy riesgo dinámico: "
    f"{evaluation_results_v2['correct_dynamic'].mean() * 100:.2f}%"
)

print(
    f"Accuracy contextual: "
    f"{contextual_accuracy_v2:.2f}%"
)

print(
    f"Mejora absoluta: "
    f"{contextual_accuracy_v2 -
evaluation_results_v2['correct_dynamic'].mean() * 100:.2f} puntos"
)

print("=" * 70)

display(
    evaluation_results_v2[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_signal_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "contextual_risk",
            "correct_contextual",
            "frames_evaluated",
            "high_ratio_auto",
            "medium_ratio_auto",
            "low_ratio_auto"
        ]
    ]
)
```

```
======================================================================
EVALUACIÓN CONTEXTUAL ACTUALIZADA - 14 SITUACIONES
======================================================================
Situaciones evaluadas: 14
Accuracy riesgo dinámico: 21.43%
Accuracy contextual: 71.43%
Mejora absoluta: 50.00 puntos
======================================================================
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manu
0	1000179702	1	GREEN	AMBER

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manu
1	1000179704	1	GREEN	AMBER_FROM_LEFT
2	1000179704	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
3	1000179707	1	RED	NOT_ANNOTATED
4	1000179709	1	RED	NOT_ANNOTATED
5	1000179709	2	RED	NOT_ANNOTATED
6	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNAL
7	20260611_184739	1	GREEN	AMBER_FROM_LEFT
8	20260611_184739	2	GREEN_TO_RED	AMBER_FROM_RIGHT_LEFT_TUR
9	20260611_184808	1	RED	NOT_ANNOTATED
10	20260611_185035	1	NO_TRAFFIC_LIGHT	NOT_APPLICABLE
11	20260611_185443~2	1	GREEN	NOT_ANNOTATED
12	20260611_185616	1	RED	NOT_ANNOTATED
13	20260611_185616	2	RED	NOT_ANNOTATED

In [123]:

```
# ============================================================
# 10.17 DIAGNÓSTICO FINAL DE ERRORES CONTEXTUALES
# ============================================================

contextual_errors_v2 = evaluation_results_v2[
    ~evaluation_results_v2["correct_contextual"]
].copy()

print(
    f"Situaciones: {len(evaluation_results_v2)} | "
    f"Correctas:
{int(evaluation_results_v2['correct_contextual'].sum())} | "
    f"Incorrectas: {len(contextual_errors_v2)}"
)

display(
    contextual_errors_v2[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "vehicle_signal_manual",
            "vehicle_behavior_manual",
            "expected_risk",
            "predicted_risk",
            "contextual_risk",
            "frames_evaluated",
            "high_ratio_auto",
            "medium_ratio_auto",
```

```
            "low_ratio_auto"
        ]
    ].reset_index(drop=True)
)
Situaciones: 14 | Correctas: 10 | Incorrectas: 4
```

	video	crossing_id	pedestrian_light_manual	vehicle_signal_manual	
0	1000179709	1	RED	NOT_ANNOTATED	N
1	1000179711	1	GREEN	DIRECTION_DEPENDENT_SIGNALS	N
2	20260611_184808	1	RED	NOT_ANNOTATED	A
3	20260611_185616	1	RED	NOT_ANNOTATED	A E

11. Detección y reconocimiento automático del semáforo peatonal

In [124]:

```
# ============================================================
# 11.1 INFERENCIA AUTOMÁTICA DEL SEMÁFORO PEATONAL
# ============================================================
# Versión para evaluación automática.
# NO utiliza el estado manual/esperado.
# ============================================================

def infer_pedestrian_traffic_light(video_path, frame_number):

    # ------------------------------------------------------
    # 1. Extracción del frame
    # ------------------------------------------------------
    cap = cv2.VideoCapture(video_path)

    cap.set(
        cv2.CAP_PROP_POS_FRAMES,
        frame_number
    )

    ret, frame = cap.read()
    cap.release()

    if not ret:
        return None

    frame_rgb = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2RGB
    )

    frame_pil = Image.fromarray(frame_rgb)
```

```
# ---------------------------------------------------------
# 2. Detección YOLOv11
# ---------------------------------------------------------
result = yolo.predict(
    source=frame,
    conf=0.10,
    verbose=False
)[0]

candidates = []

for box in result.boxes:

    class_id = int(box.cls[0].item())
    class_name = yolo.names[class_id]

    if class_name != "traffic light":
        continue

    yolo_confidence = float(
        box.conf[0].item()
    )

    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0].tolist()
    )

    margin = 15

    crop = frame_pil.crop((
        max(0, x1 - margin),
        max(0, y1 - margin),
        min(frame_pil.width, x2 + margin),
        min(frame_pil.height, y2 + margin)
    ))

    # -----------------------------------------------------
    # 3. Clasificación mediante CLIP
    # -----------------------------------------------------
    clip_result = classify_with_clip(
        crop,
        PEDESTRIAN_PROMPTS
    )

    probs = clip_result["all_probabilities"]

    green_score = sum(
```

```
            probs[p]
            for p in GREEN_PROMPTS
        )

        red_score = sum(
            probs[p]
            for p in RED_PROMPTS
        )

        total = green_score + red_score

        if total > 0:
            green_norm = green_score / total
            red_norm = red_score / total
        else:
            green_norm = 0
            red_norm = 0

        state = (
            "GREEN"
            if green_norm > red_norm
            else "RED"
        )

        semantic_confidence = max(
            green_norm,
            red_norm
        )

        candidates.append({
            "state": state,
            "semantic_confidence": semantic_confidence,
            "yolo_confidence": yolo_confidence,
            "green_score": green_norm,
            "red_score": red_norm
        })

    # ------------------------------------------------------
    # 4. Si YOLO no encuentra ningún semáforo
    # ------------------------------------------------------
    if not candidates:
        return {
            "state": "NOT_DETECTED",
            "semantic_confidence": 0.0,
            "yolo_confidence": 0.0,
            "green_score": 0.0,
            "red_score": 0.0
        }
```

```
    # ----------------------------------------------------------
    # 5. Selección del candidato
    # ----------------------------------------------------------
    best_candidate = max(
        candidates,
        key=lambda x: x["semantic_confidence"]
    )

    return best_candidate
```

In [125]:

```
# ============================================================
# 11.2 INFERENCIA TEMPORAL DEL SEMÁFORO EN LAS 14 SITUACIONES
# ============================================================



def infer_light_over_interval(
    video,
    crossing_id,
    start_frame=None,
    end_frame=None,
    n_samples=5
):

    video_path = os.path.join(
        VIDEOS_DIR,
        video + ".mp4"
    )

    # Si no hay límites explícitos, usamos los frames disponibles
    # en frame_risk para ese vídeo
    if start_frame is None or end_frame is None:

        subset = frame_risk[
            frame_risk["video"].astype(str) == str(video)
        ]

        if subset.empty:
            return {
                "video": video,
                "crossing_id": crossing_id,
                "light_state_auto": "NO_DATA",
                "light_confidence_auto": 0.0,
                "samples_used": 0
            }

        start_frame = int(subset["frame"].min())
        end_frame = int(subset["frame"].max())
```

```
if end_frame <= start_frame:
    sample_frames = [start_frame]
else:
    sample_frames = np.linspace(
        start_frame,
        end_frame,
        n_samples,
        dtype=int
    ).tolist()

results = []

for frame_number in sample_frames:

    result = infer_pedestrian_traffic_light(
        video_path,
        frame_number
    )

    if result is None:
        continue

    results.append({
        "frame": frame_number,
        "state": result["state"],
        "semantic_confidence":
            result["semantic_confidence"]
    })

if not results:
    return {
        "video": video,
        "crossing_id": crossing_id,
        "light_state_auto": "NO_DATA",
        "light_confidence_auto": 0.0,
        "samples_used": 0
    }

states = [r["state"] for r in results]

valid_states = [
    s for s in states
    if s in ["RED", "GREEN"]
]

if not valid_states:
    final_state = "NOT_DETECTED"
    confidence = 0.0
```

```
    else:
        red_count = valid_states.count("RED")
        green_count = valid_states.count("GREEN")

        if red_count > green_count:
            final_state = "RED"
        elif green_count > red_count:
            final_state = "GREEN"
        else:
            final_state = "UNKNOWN"

        confidence = np.mean([
            r["semantic_confidence"]
            for r in results
            if r["state"] == final_state
        ]) if final_state in ["RED", "GREEN"] else 0.0

    return {
        "video": video,
        "crossing_id": crossing_id,
        "light_state_auto": final_state,
        "light_confidence_auto": confidence,
        "samples_used": len(results)
    }


# ------------------------------------------------------------
# Aplicar a las 14 situaciones
# ------------------------------------------------------------

automatic_light_results = pd.DataFrame([
    infer_light_over_interval(
        video,
        crossing_id,
        start_frame,
        end_frame,
        n_samples=5
    )
    for (
        video,
        crossing_id,
        start_frame,
        end_frame
    ) in evaluation_intervals_v2
])


display(automatic_light_results)
```

	video	crossing_id	light_state_auto	light_confidence_auto	samples_used
0	20260611_184739	1	GREEN	0.854189	5
1	20260611_184739	2	GREEN	0.737826	4
2	20260611_184808	1	RED	0.633852	5
3	20260611_185035	1	NOT_DETECTED	0.000000	5
4	20260611_185443~2	1	UNKNOWN	0.000000	5
5	20260611_185616	1	GREEN	0.796752	5
6	20260611_185616	2	GREEN	0.714446	4
7	1000179702	1	RED	0.931926	5
8	1000179704	1	NOT_DETECTED	0.000000	5
9	1000179704	2	GREEN	0.765337	4
10	1000179707	1	RED	0.788815	5
11	1000179709	1	NOT_DETECTED	0.000000	5
12	1000179709	2	RED	0.932608	5
13	1000179711	1	RED	0.745882	5

In [126]:

```
# ============================================================
# 11.3 EVALUACIÓN DEL ESTADO AUTOMÁTICO DEL SEMÁFORO PEATONAL
# ============================================================

light_eval = manual_validation_v2.merge(
    automatic_light_results,
    on=["video", "crossing_id"],
    how="left"
)

# Normalización mínima para comparar
def normalize_manual_light(x):
    x = str(x).upper()

    # GREEN_TO_RED no puede compararse como estado único;
    # lo trataremos como transición.
    if x == "GREEN_TO_RED":
        return "TRANSITION"

    if x in ["GREEN", "RED", "NO_TRAFFIC_LIGHT"]:
        return x

    return "OTHER"


light_eval["manual_light_norm"] = (
    light_eval["pedestrian_light_manual"]
    .apply(normalize_manual_light)
)
```

```
# Casos directamente comparables RED/GREEN
comparable_mask = (
    light_eval["manual_light_norm"].isin(["RED", "GREEN"])
)

light_eval["light_correct"] = False

light_eval.loc[comparable_mask, "light_correct"] = (
    light_eval.loc[comparable_mask, "manual_light_norm"]
    ==
    light_eval.loc[comparable_mask, "light_state_auto"]
)

# Resumen
n_total = len(light_eval)

n_comparable = comparable_mask.sum()

n_correct = (
    light_eval.loc[comparable_mask, "light_correct"]
    .sum()
)

accuracy_light = (
    n_correct / n_comparable * 100
    if n_comparable > 0
    else 0
)

n_not_detected = (
    light_eval["light_state_auto"]
    .eq("NOT_DETECTED")
    .sum()
)

n_unknown = (
    light_eval["light_state_auto"]
    .eq("UNKNOWN")
    .sum()
)

print("=" * 70)
print("EVALUACIÓN AUTOMÁTICA DEL SEMÁFORO PEATONAL")
print("=" * 70)

print(f"Situaciones totales: {n_total}")
print(f"Situaciones comparables RED/GREEN: {n_comparable}")
print(f"Aciertos RED/GREEN: {n_correct}")
```

```
print(f"Accuracy RED/GREEN: {accuracy_light:.2f}%")
print(f"NOT_DETECTED: {n_not_detected}")
print(f"UNKNOWN: {n_unknown}")

print("=" * 70)

display(
    light_eval[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "manual_light_norm",
            "light_state_auto",
            "light_confidence_auto",
            "samples_used",
            "light_correct"
        ]
    ]
)
```

```
======================================================================
EVALUACIÓN AUTOMÁTICA DEL SEMÁFORO PEATONAL
======================================================================
Situaciones totales: 14
Situaciones comparables RED/GREEN: 11
Aciertos RED/GREEN: 4
Accuracy RED/GREEN: 36.36%
NOT_DETECTED: 3
UNKNOWN: 1
======================================================================
```

	video	crossing_id	pedestrian_light_manual	manual_light_norm	light_state_
0	1000179702	1	GREEN	GREEN	RED
1	1000179704	1	GREEN	GREEN	NOT_DETEC
2	1000179704	2	GREEN_TO_RED	TRANSITION	GREEN
3	1000179707	1	RED	RED	RED
4	1000179709	1	RED	RED	NOT_DETEC
5	1000179709	2	RED	RED	RED
6	1000179711	1	GREEN	GREEN	RED
7	20260611_184739	1	GREEN	GREEN	GREEN
8	20260611_184739	2	GREEN_TO_RED	TRANSITION	GREEN
9	20260611_184808	1	RED	RED	RED
10	20260611_185035	1	NO_TRAFFIC_LIGHT	NO_TRAFFIC_LIGHT	NOT_DETEC
11	20260611_185443~2	1	GREEN	GREEN	UNKNOWN
12	20260611_185616	1	RED	RED	GREEN

	video	crossing_id	pedestrian_light_manual	manual_light_norm	light_state_
13	20260611_185616	2	RED	RED	GREEN

In [127]:

```
# ============================================================
# 11.4 DIAGNÓSTICO DEL MÓDULO AUTOMÁTICO DE SEMÁFORO
# ============================================================

diag_light = light_eval.copy()

def light_result_type(row):

    manual = row["manual_light_norm"]
    auto = row["light_state_auto"]

    # Caso especial: no existe semáforo peatonal
    if manual == "NO_TRAFFIC_LIGHT":
        if auto == "NOT_DETECTED":
            return "CORRECT_NO_TRAFFIC_LIGHT"
        return "FALSE_DETECTION"

    # Transiciones no se evalúan como RED/GREEN puro
    if manual == "TRANSITION":
        return "TRANSITION_NOT_BINARY"

    # Fallo de detección
    if auto == "NOT_DETECTED":
        return "DETECTION_FAILURE"

    # Resultado indeterminado
    if auto == "UNKNOWN":
        return "UNKNOWN"

    # Clasificación correcta
    if manual == auto:
        return "CORRECT_CLASSIFICATION"

    # Semáforo detectado pero color incorrecto
    if (
        manual in ["RED", "GREEN"]
        and auto in ["RED", "GREEN"]
    ):
        return "CLASSIFICATION_ERROR"

    return "OTHER"


diag_light["diagnosis"] = diag_light.apply(
    light_result_type,
```

```
    axis=1
)

print("=" * 70)
print("DIAGNÓSTICO YOLO + CLIP DEL SEMÁFORO PEATONAL")
print("=" * 70)

print(
    diag_light["diagnosis"]
    .value_counts()
)

print("=" * 70)

display(
    diag_light[
        [
            "video",
            "crossing_id",
            "pedestrian_light_manual",
            "light_state_auto",
            "light_confidence_auto",
            "samples_used",
            "diagnosis"
        ]
    ]
)
```

```
======================================================================
DIAGNÓSTICO YOLO + CLIP DEL SEMÁFORO PEATONAL
======================================================================
diagnosis
CLASSIFICATION_ERROR        4
CORRECT_CLASSIFICATION      4
DETECTION_FAILURE           2
TRANSITION_NOT_BINARY       2
CORRECT_NO_TRAFFIC_LIGHT    1
UNKNOWN                     1
Name: count, dtype: int64
======================================================================
```

	video	crossing_id	pedestrian_light_manual	light_state_auto	light_confidence_
0	1000179702	1	GREEN	RED	0.931926
1	1000179704	1	GREEN	NOT_DETECTED	0.000000
2	1000179704	2	GREEN_TO_RED	GREEN	0.765337
3	1000179707	1	RED	RED	0.788815
4	1000179709	1	RED	NOT_DETECTED	0.000000
5	1000179709	2	RED	RED	0.932608

	video	crossing_id	pedestrian_light_manual	light_state_auto	light_confidence_
6	1000179711	1	GREEN	RED	0.745882
7	20260611_184739	1	GREEN	GREEN	0.854189
8	20260611_184739	2	GREEN_TO_RED	GREEN	0.737826
9	20260611_184808	1	RED	RED	0.633852
10	20260611_185035	1	NO_TRAFFIC_LIGHT	NOT_DETECTED	0.000000
11	20260611_185443~2	1	GREEN	UNKNOWN	0.000000
12	20260611_185616	1	RED	GREEN	0.796752
13	20260611_185616	2	RED	GREEN	0.714446

12. Integración automática del sistema

In [128]:

```
# ============================================================
# 12.1 REGLA DE FUSIÓN AUTOMÁTICA DEL RIESGO
# ============================================================

def safe_automatic_fusion(row):

    dynamic = str(row["predicted_risk"]).upper()
    light = str(row["light_state_auto"]).upper()

    try:
        confidence = float(row["light_confidence_auto"])
    except:
        confidence = 0.0

    # RED automático suficientemente fiable:
    # puede aumentar el riesgo, pero nunca reducirlo.
    if light == "RED" and confidence >= 0.60:

        if dynamic == "LOW":
            return "MEDIUM"

        if dynamic == "MEDIUM":
            return "HIGH"

        return "HIGH"

    # Debido a los falsos GREEN observados,
    # GREEN no se utiliza para reducir el riesgo dinámico.
    if light == "GREEN":
        return dynamic

    # Estados indeterminados o no detectados:
    # se conserva el riesgo dinámico.
```

```
    return dynamic
```

In [129]:

```
# ============================================================
# 12.2 FUSIÓN AUTOMÁTICA SOBRE LAS 14 SITUACIONES
# ============================================================

auto_fusion_v2 = evaluation_results_v2.merge(
    automatic_light_results[
        [
            "video",
            "crossing_id",
            "light_state_auto",
            "light_confidence_auto",
            "samples_used"
        ]
    ],
    on=["video", "crossing_id"],
    how="left"
)

# Aplicar la regla de fusión automática definida en 12.1
auto_fusion_v2["automatic_fused_risk"] = (
    auto_fusion_v2.apply(
        safe_automatic_fusion,
        axis=1
    )
)

# Comparación con la referencia manual de riesgo
auto_fusion_v2["correct_automatic_fusion"] = (
    auto_fusion_v2["automatic_fused_risk"]
    == auto_fusion_v2["expected_risk"]
)

# ------------------------------------------------------------
# Métricas
# ------------------------------------------------------------

dynamic_accuracy_v2 = (
    auto_fusion_v2["correct_dynamic"].mean() * 100
)

automatic_accuracy_v2 = (
    auto_fusion_v2["correct_automatic_fusion"].mean() * 100
)

print(
    f"Situaciones evaluadas: {len(auto_fusion_v2)} | "
    f"Accuracy dinámica: {dynamic_accuracy_v2:.2f}% | "
```

```
    f"Accuracy fusión automática: {automatic_accuracy_v2:.2f}% | "
    f"Mejora: {automatic_accuracy_v2 - dynamic_accuracy_v2:.2f} pp"
)

display(
    auto_fusion_v2[
        [
            "video",
            "crossing_id",
            "expected_risk",
            "predicted_risk",
            "light_state_auto",
            "light_confidence_auto",
            "automatic_fused_risk",
            "correct_automatic_fusion"
        ]
    ]
)
```

```
Situaciones evaluadas: 14 | Accuracy dinámica: 21.43% | Accuracy
fusión automática: 35.71% | Mejora: 14.29 pp
```

	video	crossing_id	expected_risk	predicted_risk	light_state_auto	light_confider
0	1000179702	1	HIGH	MEDIUM	RED	0.931926
1	1000179704	1	MEDIUM	MEDIUM	NOT_DETECTED	0.000000
2	1000179704	2	HIGH	MEDIUM	GREEN	0.765337
3	1000179707	1	LOW	MEDIUM	RED	0.788815
4	1000179709	1	HIGH	MEDIUM	NOT_DETECTED	0.000000
5	1000179709	2	LOW	LOW	RED	0.932608
6	1000179711	1	MEDIUM	LOW	RED	0.745882
7	20260611_184739	1	MEDIUM	MEDIUM	GREEN	0.854189
8	20260611_184739	2	HIGH	LOW	GREEN	0.737826
9	20260611_184808	1	HIGH	MEDIUM	RED	0.633852
10	20260611_185035	1	HIGH	MEDIUM	NOT_DETECTED	0.000000
11	20260611_185443~2	1	LOW	HIGH	UNKNOWN	0.000000
12	20260611_185616	1	HIGH	MEDIUM	GREEN	0.796752
13	20260611_185616	2	LOW	MEDIUM	GREEN	0.714446

In [130]:

```
# ============================================================
# 12.3 RECOMENDACIÓN FINAL AUTOMÁTICA AL PEATÓN
# ============================================================
# Se mantiene separada la estimación de riesgo físico
# de la recomendación de comportamiento.
# ============================================================

def generate_final_recommendation(row):
```

```
risk = str(
    row["automatic_fused_risk"]
).upper()

light = str(
    row["light_state_auto"]
).upper()

try:
    confidence = float(
        row["light_confidence_auto"]
    )
except:
    confidence = 0.0

# ------------------------------------------------------
# 1. Riesgo físico alto
# ------------------------------------------------------
if risk == "HIGH":
    return "WAIT"

# ------------------------------------------------------
# 2. Semáforo rojo detectado
# ------------------------------------------------------
# Aunque el riesgo físico sea bajo,
# la recomendación normativa sigue siendo esperar.
if light == "RED" and confidence >= 0.60:
    return "WAIT"

# ------------------------------------------------------
# 3. Información semafórica no fiable/disponible
# ------------------------------------------------------
if light in [
    "NOT_DETECTED",
    "UNKNOWN",
    "NO_DATA"
]:
    return "WAIT / CHECK ENVIRONMENT"

# ------------------------------------------------------
# 4. Riesgo medio
# ------------------------------------------------------
if risk == "MEDIUM":
    return "CAUTION"

# ------------------------------------------------------
# 5. Riesgo bajo + verde
# ------------------------------------------------------
if risk == "LOW" and light == "GREEN":
```

```
        return "CROSS WITH CAUTION"

    # ----------------------------------------------------------
    # 6. Situación restante
    # ----------------------------------------------------------
    return "WAIT / CHECK ENVIRONMENT"


auto_fusion_v2["final_recommendation"] = (
    auto_fusion_v2.apply(
        generate_final_recommendation,
        axis=1
    )
)


print("=" * 70)
print("SALIDA FINAL DEL SISTEMA")
print("=" * 70)

display(
    auto_fusion_v2[
        [
            "video",
            "crossing_id",
            "predicted_risk",
            "light_state_auto",
            "light_confidence_auto",
            "automatic_fused_risk",
            "final_recommendation"
        ]
    ]
)
```

```
======================================================================
SALIDA FINAL DEL SISTEMA
======================================================================
```

	video	crossing_id	predicted_risk	light_state_auto	light_confidence_auto	auton
0	1000179702	1	MEDIUM	RED	0.931926	HIGH
1	1000179704	1	MEDIUM	NOT_DETECTED	0.000000	MED
2	1000179704	2	MEDIUM	GREEN	0.765337	MED
3	1000179707	1	MEDIUM	RED	0.788815	HIGH
4	1000179709	1	MEDIUM	NOT_DETECTED	0.000000	MED
5	1000179709	2	LOW	RED	0.932608	MED
6	1000179711	1	LOW	RED	0.745882	MED

	video	crossing_id	predicted_risk	light_state_auto	light_confidence_auto	auton
7	20260611_184739	1	MEDIUM	GREEN	0.854189	MED
8	20260611_184739	2	LOW	GREEN	0.737826	LOW
9	20260611_184808	1	MEDIUM	RED	0.633852	HIGH
10	20260611_185035	1	MEDIUM	NOT_DETECTED	0.000000	MED
11	20260611_185443~2	1	HIGH	UNKNOWN	0.000000	HIGH
12	20260611_185616	1	MEDIUM	GREEN	0.796752	MED
13	20260611_185616	2	MEDIUM	GREEN	0.714446	MED

In [131]:

```
# 12.4 EVALUACIÓN DE LA FUSIÓN AUTOMÁTICA


import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix
)

labels = ["LOW", "MEDIUM", "HIGH"]

y_true = auto_fusion_v2["expected_risk"]

y_dynamic = auto_fusion_v2["predicted_risk"]

y_fused = auto_fusion_v2["automatic_fused_risk"]


# ------------------------------------------------------------
# Función de métricas
# ------------------------------------------------------------

def calculate_metrics(y_true, y_pred, model_name):

    accuracy = accuracy_score(
        y_true,
        y_pred
    )

    precision, recall, f1, _ = (
        precision_recall_fscore_support(
            y_true,
            y_pred,
            labels=labels,
            average="macro",
```

```
            zero_division=0
        )
    )

    return {
        "modelo": model_name,
        "accuracy": accuracy,
        "precision_macro": precision,
        "recall_macro": recall,
        "f1_macro": f1
    }


# ------------------------------------------------------------
# 1. Comparación de rendimiento
# ------------------------------------------------------------

metrics_summary = pd.DataFrame([
    calculate_metrics(
        y_true,
        y_dynamic,
        "Riesgo dinámico"
    ),

    calculate_metrics(
        y_true,
        y_fused,
        "Fusión automática"
    )
])

print("=" * 70)
print("COMPARACIÓN DE MÉTRICAS: RIESGO DINÁMICO VS FUSIÓN AUTOMÁTICA")
print("=" * 70)

display(
    metrics_summary.style.format({
        "accuracy": "{:.3f}",
        "precision_macro": "{:.3f}",
        "recall_macro": "{:.3f}",
        "f1_macro": "{:.3f}"
    })
)


# ------------------------------------------------------------
# 2. Matriz de confusión - riesgo dinámico
# ------------------------------------------------------------
```

```
cm_dynamic = confusion_matrix(
    y_true,
    y_dynamic,
    labels=labels
)

cm_dynamic_df = pd.DataFrame(
    cm_dynamic,
    index=[
        f"Real_{x}"
        for x in labels
    ],
    columns=[
        f"Pred_{x}"
        for x in labels
    ]
)

print("\nMATRIZ DE CONFUSIÓN - RIESGO DINÁMICO")
display(cm_dynamic_df)


# ------------------------------------------------------------
# 3. Matriz de confusión - fusión automática
# ------------------------------------------------------------

cm_fused = confusion_matrix(
    y_true,
    y_fused,
    labels=labels
)

cm_fused_df = pd.DataFrame(
    cm_fused,
    index=[
        f"Real_{x}"
        for x in labels
    ],
    columns=[
        f"Pred_{x}"
        for x in labels
    ]
)

print("\nMATRIZ DE CONFUSIÓN - FUSIÓN AUTOMÁTICA")
display(cm_fused_df)
======================================================================
COMPARACIÓN DE MÉTRICAS: RIESGO DINÁMICO VS FUSIÓN AUTOMÁTICA
======================================================================
```

	modelo	accuracy	precision_macro	recall_macro	f1_macro
0	Riesgo dinámico	0.214	0.178	0.306	0.198
1	Fusión automática	0.357	0.278	0.429	0.288

MATRIZ DE CONFUSIÓN - RIESGO DINÁMICO

	Pred_LOW	Pred_MEDIUM	Pred_HIGH
Real_LOW	1	2	1
Real_MEDIUM	1	2	0
Real_HIGH	1	6	0

MATRIZ DE CONFUSIÓN - FUSIÓN AUTOMÁTICA

	Pred_LOW	Pred_MEDIUM	Pred_HIGH
Real_LOW	0	2	2
Real_MEDIUM	0	3	0
Real_HIGH	1	4	2

In [132]:

```
# ============================================================
# 12.5 Evaluación específica del semáforo peatonal
# ============================================================

from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd

# Solo situaciones cuyo Ground Truth es inequívocamente
# RED o GREEN
light_binary_eval = light_eval[
    light_eval["manual_light_norm"].isin(
        ["RED", "GREEN"]
    )
].copy()

y_true_light = light_binary_eval[
    "manual_light_norm"
]

y_pred_light = light_binary_eval[
    "light_state_auto"
]

# ------------------------------------------------------------
# Métricas RED / GREEN
# ------------------------------------------------------------

print("=" * 70)
print("MÉTRICAS DEL SEMÁFORO PEATONAL AUTOMÁTICO")
print("=" * 70)
```

```
print(
    classification_report(
        y_true_light,
        y_pred_light,
        labels=["RED", "GREEN"],
        zero_division=0,
        digits=3
    )
)

# Matriz de confusión RED / GREEN
# Solo sobre predicciones concluyentes.
# UNKNOWN y NOT_DETECTED se contabilizan por separado.

binary_predictions = light_binary_eval[
    light_binary_eval["light_state_auto"].isin(
        ["RED", "GREEN"]
    )
]

cm_light = confusion_matrix(
    binary_predictions["manual_light_norm"],
    binary_predictions["light_state_auto"],
    labels=["RED", "GREEN"]
)

cm_light_df = pd.DataFrame(
    cm_light,
    index=["Real_RED", "Real_GREEN"],
    columns=["Pred_RED", "Pred_GREEN"]
)

print("\nMATRIZ DE CONFUSIÓN RED / GREEN")
display(cm_light_df)

# ------------------------------------------------------------
# Estados no concluyentes
# ------------------------------------------------------------

n_total = len(light_binary_eval)

n_unknown = (
    light_binary_eval["light_state_auto"]
    .eq("UNKNOWN")
    .sum()
)

n_not_detected = (
```

```
    light_binary_eval["light_state_auto"]
    .eq("NOT_DETECTED")
    .sum()
)

print("\nRESULTADOS NO CONCLUYENTES")
print(
    f"UNKNOWN: {n_unknown}/{n_total} "
    f"({n_unknown/n_total*100:.2f}%)"
)

print(
    f"NOT_DETECTED: {n_not_detected}/{n_total} "
    f"({n_not_detected/n_total*100:.2f}%)"
)
======================================================================
MÉTRICAS DEL SEMÁFORO PEATONAL AUTOMÁTICO
======================================================================
              precision    recall  f1-score   support

         RED      0.600     0.500     0.545         6
       GREEN      0.333     0.200     0.250         5

   micro avg      0.500     0.364     0.421        11
   macro avg      0.467     0.350     0.398        11
weighted avg      0.479     0.364     0.411        11


MATRIZ DE CONFUSIÓN RED / GREEN
```

	Pred_RED	**Pred_GREEN**
Real_RED	3	2
Real_GREEN	2	1

```
RESULTADOS NO CONCLUYENTES
UNKNOWN: 1/11 (9.09%)
NOT_DETECTED: 2/11 (18.18%)
```

Diagnóstico inicial del módulo de semáforo Se realizaron pruebas preliminares para distinguir candidatos de semáforo peatonal frente a semáforos vehiculares mediante YOLO11 COCO y CLIP. Estas pruebas mostraron que la clase genérica traffic light de COCO no permitía identificar de forma robusta el semáforo peatonal, especialmente cuando el objeto aparecía pequeño o con baja resolución. Este resultado motivó la exploración posterior de filtros adicionales y, finalmente, el desarrollo del pipeline mejorado de semáforo.

Posteriormente se equilibraron los prompts positivos y negativos de CLIP y se realizaron controles sobre ejemplos reales. Aunque la separación semántica mejoró en algunos candidatos, persistieron ambigüedades y falsos positivos,

especialmente por la presencia de semáforos vehiculares, contadores y regiones visualmente similares. Esto motivó la incorporación de criterios adicionales de filtrado. También se comprobó que la evidencia semántica de figura peatonal podía variar significativamente entre frames del mismo semáforo, por lo que se exploró una validación específica de presencia de figura humana mediante CLIP. Esta estrategia tampoco se mantuvo como criterio independiente en el pipeline definitivo.

Se desarrolló además una variante híbrida del clasificador que combinaba filtrado morfológico, validación semántica mediante CLIP y evidencia cromática RED/GREEN. Esta aproximación permitía introducir comprobaciones explícitas sobre la forma y el color de los candidatos, pero aumentaba la dependencia de umbrales y heurísticas ajustadas manualmente. Por este motivo no se mantuvo como arquitectura definitiva y se continuó explorando una solución más robusta.

Se analizó también la estabilidad espacial del candidato seleccionado a lo largo de varios frames de un mismo intervalo. El diagnóstico evidenció que la selección independiente por frame podía producir variaciones entre candidatos, lo que motivó posteriormente la incorporación de mecanismos de consistencia temporal.

También se exploró la estabilidad espacial del candidato entre distintos frames de un mismo intervalo. Esta línea de diagnóstico dependía del clasificador híbrido experimental y no se mantuvo; posteriormente la consistencia entre frames se abordó de forma explícita mediante seguimiento temporal.

También se exploró la estabilidad espacial del candidato entre distintos frames de un mismo intervalo. Este diagnóstico puso de manifiesto las limitaciones de una selección independiente por frame y motivó que la consistencia entre observaciones se abordara posteriormente mediante mecanismos explícitos de seguimiento temporal.

También se ensayó una agregación temporal preliminar basada en el muestreo de varios frames, votación ponderada por confianza y detección heurística de transiciones RED/GREEN. Esta estrategia dependía del clasificador V2 experimental y mostró una elevada sensibilidad a las predicciones ambiguas y al estado de ejecución de las distintas etapas, por lo que no se incorporó al sistema definitivo. La consistencia temporal se abordó posteriormente mediante un mecanismo explícito de seguimiento sobre los vídeos completos.

La reevaluación de los casos conflictivos confirmó que la clasificación podía variar notablemente entre frames del mismo semáforo. Se probaron prompts equilibrados y una verificación cromática explícita en espacio HSV para reforzar la decisión RED/GREEN, pero estas mejoras seguían dependiendo de umbrales heurísticos y no resolvían de forma suficientemente robusta la variabilidad entre recortes y condiciones visuales.

También se evaluó una estrategia de inferencia por regiones solapadas para aumentar la sensibilidad de YOLOv11 ante objetos pequeños. Aunque permitió recuperar detecciones con mayor confianza en algunos frames, incrementó el

número de candidatos espurios y la complejidad del procesamiento. Por ello no se incorporó al pipeline definitivo y se optó posteriormente por especializar el detector mediante fine-tuning.

13. Fine-tuning de YOLOv11 para detección de semáforos peatonales

13.1 Extracción de frames para el dataset

In [133]:

```
# ============================================================
# 13.1 EXTRACCIÓN DE FRAMES PARA EL DATASET
# ============================================================

import os
import cv2

DATASET_DIR = os.path.join(
    os.path.dirname(VIDEOS_DIR),
    "dataset_pedestrian_traffic_light"
)

IMAGES_POOL_DIR = os.path.join(
    DATASET_DIR,
    "images_pool"
)

os.makedirs(
    IMAGES_POOL_DIR,
    exist_ok=True
)

# Extraer aproximadamente un frame cada 0.5 segundos
SECONDS_BETWEEN_FRAMES = 0.5

extracted_images = []

for video_file in VIDEO_FILES:

    # Admitimos que VIDEO_FILES pueda contener
    # nombres completos o rutas
    if os.path.isabs(video_file):
        video_path = video_file
        video_name = os.path.splitext(
            os.path.basename(video_file)
        )[0]
    else:
        video_path = os.path.join(
```

```
        VIDEOS_DIR,
        video_file
    )

    video_name = os.path.splitext(
        os.path.basename(video_file)
    )[0]

cap = cv2.VideoCapture(
    video_path
)

if not cap.isOpened():

    print(
        f"No se pudo abrir: {video_path}"
    )

    continue

fps = cap.get(
    cv2.CAP_PROP_FPS
)

total_frames = int(
    cap.get(
        cv2.CAP_PROP_FRAME_COUNT
    )
)

if fps <= 0:
    fps = 30.0

frame_step = max(
    1,
    int(
        fps * SECONDS_BETWEEN_FRAMES
    )
)

saved = 0

for frame_number in range(
    0,
    total_frames,
    frame_step
):

    cap.set(
```

```
            cv2.CAP_PROP_POS_FRAMES,
            frame_number
        )

        ret, frame = cap.read()

        if not ret:
            continue

        image_name = (
            f"{video_name}"
            f"_frame_{frame_number:06d}.jpg"
        )

        image_path = os.path.join(
            IMAGES_POOL_DIR,
            image_name
        )

        cv2.imwrite(
            image_path,
            frame
        )

        extracted_images.append({
            "video": video_name,
            "frame": frame_number,
            "image": image_name
        })

        saved += 1

    cap.release()

    print(
        f"{video_name}: "
        f"{saved} imágenes extraídas"
    )

print("\n" + "=" * 60)
print(
    "TOTAL DE IMÁGENES EXTRAÍDAS:",
    len(extracted_images)
)
print(
    "Carpeta:",
    IMAGES_POOL_DIR
)
print("=" * 60)
```

```
20260611_184739: 31 imágenes extraídas
20260611_184808: 26 imágenes extraídas
20260611_185035: 19 imágenes extraídas
20260611_185443~2: 7 imágenes extraídas
20260611_185616: 44 imágenes extraídas
1000179702: 44 imágenes extraídas
1000179704: 52 imágenes extraídas
1000179707: 49 imágenes extraídas
1000179709: 38 imágenes extraídas
1000179711: 37 imágenes extraídas

============================================================
TOTAL DE IMÁGENES EXTRAÍDAS: 347
Carpeta: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\images_pool
============================================================
```

In [134]:

```
# ============================================================
# 13.2 DIVISIÓN DEL DATASET POR VÍDEOS
# ============================================================

import os
import shutil

TRAIN_VIDEOS = [
    "20260611_184739",
    "20260611_184808",
    "20260611_185035",
    "20260611_185443~2",
    "1000179702",
    "1000179704",
    "1000179707"
]

VAL_VIDEOS = [
    "20260611_185616"
]

TEST_VIDEOS = [
    "1000179709",
    "1000179711"
]

for split in ["train", "val", "test"]:

    os.makedirs(
        os.path.join(
            DATASET_DIR,
            "images",
```

```
            split
        ),
        exist_ok=True
    )

    os.makedirs(
        os.path.join(
            DATASET_DIR,
            "labels",
            split
        ),
        exist_ok=True
    )


def get_split(video):

    if video in TRAIN_VIDEOS:
        return "train"

    if video in VAL_VIDEOS:
        return "val"

    if video in TEST_VIDEOS:
        return "test"

    return None


split_counts = {
    "train": 0,
    "val": 0,
    "test": 0
}

for item in extracted_images:

    video = item["video"]
    image_name = item["image"]

    split = get_split(video)

    if split is None:
        print(
            "Vídeo sin asignar:",
            video
        )
        continue
```

```
    source = os.path.join(
        IMAGES_POOL_DIR,
        image_name
    )

    destination = os.path.join(
        DATASET_DIR,
        "images",
        split,
        image_name
    )

    shutil.copy2(
        source,
        destination
    )

    split_counts[split] += 1


print("\nDIVISIÓN DEL DATASET")
print("=" * 50)

for split, count in split_counts.items():

    print(
        f"{split.upper()}: "
        f"{count} imágenes"
    )
```

```
DIVISIÓN DEL DATASET
==================================================
TRAIN: 228 imágenes
VAL: 44 imágenes
TEST: 75 imágenes
```

In [135]:

```
# ============================================================
# 13.3 SELECCIÓN DE IMÁGENES PARA ANOTACIÓN
# ============================================================

import os
import shutil

ANNOTATION_DIR = os.path.join(
    DATASET_DIR,
    "annotation_selection"
)

os.makedirs(
```

```
    ANNOTATION_DIR,
    exist_ok=True
)

SELECT_PER_VIDEO = 15

selected_for_annotation = []

for video_name in sorted(
    set(item["video"] for item in extracted_images)
):

    video_images = [
        item
        for item in extracted_images
        if item["video"] == video_name
    ]

    # Selección uniforme a lo largo del vídeo
    if len(video_images) <= SELECT_PER_VIDEO:

        selected = video_images

    else:

        indices = np.linspace(
            0,
            len(video_images) - 1,
            SELECT_PER_VIDEO,
            dtype=int
        )

        selected = [
            video_images[i]
            for i in indices
        ]

    for item in selected:

        source = os.path.join(
            IMAGES_POOL_DIR,
            item["image"]
        )

        destination = os.path.join(
            ANNOTATION_DIR,
            item["image"]
        )
```

```
        shutil.copy2(
            source,
            destination
        )

        selected_for_annotation.append(
            item
        )

    print(
        f"{video_name}: "
        f"{len(selected)} seleccionadas"
    )

print("\n" + "=" * 60)
print(
    "TOTAL A ANOTAR:",
    len(selected_for_annotation)
)
print(
    "Carpeta:",
    ANNOTATION_DIR
)
print("=" * 60)
```

```
1000179702: 15 seleccionadas
1000179704: 15 seleccionadas
1000179707: 15 seleccionadas
1000179709: 15 seleccionadas
1000179711: 15 seleccionadas
20260611_184739: 15 seleccionadas
20260611_184808: 15 seleccionadas
20260611_185035: 15 seleccionadas
20260611_185443~2: 7 seleccionadas
20260611_185616: 15 seleccionadas

============================================================
TOTAL A ANOTAR: 142
Carpeta: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\annotation_selection
============================================================
```

13.4 Anotación manual

In [136]:

```
# ============================================================
# COMPROBACIÓN FINAL DE LAS ANOTACIONES YOLO
# ============================================================

import os
```

```
BASE_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light"

IMAGES_ANNOTATION_DIR = os.path.join(
    BASE_DIR,
    "annotation_selection"
)

LABELS_ANNOTATION_DIR = os.path.join(
    BASE_DIR,
    "annotation_labels"
)

# ------------------------------------------------------------
# Imágenes seleccionadas
# ------------------------------------------------------------

images = sorted([
    f for f in os.listdir(IMAGES_ANNOTATION_DIR)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])

# ------------------------------------------------------------
# Etiquetas creadas
# ------------------------------------------------------------

labels = sorted([
    f for f in os.listdir(LABELS_ANNOTATION_DIR)
    if f.lower().endswith(".txt")
    and f.lower() != "classes.txt"
])

image_stems = {
    os.path.splitext(f)[0]
    for f in images
}

label_stems = {
    os.path.splitext(f)[0]
    for f in labels
}

# Imágenes sin archivo TXT = ejemplos negativos
negative_images = sorted(
    image_stems - label_stems
)

# ------------------------------------------------------------
```

```
# Validación del formato YOLO
# ------------------------------------------------------------

errors = []

total_boxes = 0

for label_file in labels:

    path = os.path.join(
        LABELS_ANNOTATION_DIR,
        label_file
    )

    with open(path, "r", encoding="utf-8") as f:

        lines = [
            line.strip()
            for line in f
            if line.strip()
        ]

    total_boxes += len(lines)

    for line_number, line in enumerate(lines, start=1):

        parts = line.split()

        if len(parts) != 5:
            errors.append(
                f"{label_file} línea {line_number}: "
                f"debe contener 5 valores"
            )
            continue

        try:
            class_id = int(parts[0])
            values = list(
                map(float, parts[1:])
            )

            if class_id != 0:
                errors.append(
                    f"{label_file}: clase distinta de 0"
                )

            if not all(
                0.0 <= x <= 1.0
                for x in values
```

```
            ):
                errors.append(
                    f"{label_file}: coordenadas fuera de [0,1]"
                )

        except ValueError:

            errors.append(
                f"{label_file}: formato numérico incorrecto"
            )

# ------------------------------------------------------------
# RESUMEN
# ------------------------------------------------------------

print("=" * 60)
print("COMPROBACIÓN DEL DATASET ANOTADO")
print("=" * 60)

print("Imágenes revisadas:", len(images))
print("Imágenes con anotación:", len(labels))
print("Imágenes negativas:", len(negative_images))
print("Bounding boxes totales:", total_boxes)
print("Errores encontrados:", len(errors))

if errors:

    print("\nERRORES:")

    for error in errors:
        print("-", error)

else:

    print("\n✓ Todas las anotaciones YOLO tienen formato correcto.")
```

```
============================================================
COMPROBACIÓN DEL DATASET ANOTADO
============================================================
Imágenes revisadas: 142
Imágenes con anotación: 92
Imágenes negativas: 50
Bounding boxes totales: 137
Errores encontrados: 0

✓ Todas las anotaciones YOLO tienen formato correcto.
```

In [137]:

```
# 13.5 CREACIÓN DEL DATASET FINAL YOLO
# DIVISIÓN POR VÍDEO: TRAIN / VAL / TEST
```

```
import os
import shutil
import random
from collections import Counter

# ------------------------------------------------------------
# 1. Rutas
# ------------------------------------------------------------

BASE_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light"

SOURCE_IMAGES = os.path.join(
    BASE_DIR,
    "annotation_selection"
)

SOURCE_LABELS = os.path.join(
    BASE_DIR,
    "annotation_labels"
)

FINAL_DATASET_DIR = os.path.join(
    BASE_DIR,
    "dataset_yolo_final"
)

# ------------------------------------------------------------
# 2. Obtener las 142 imágenes revisadas
# ------------------------------------------------------------

images = sorted([
    f for f in os.listdir(SOURCE_IMAGES)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])

print("Imágenes encontradas:", len(images))

# ------------------------------------------------------------
# 3. Obtener vídeo de origen
#
# Ejemplo:
# 20260611_184739_frame_000060.jpg
#            ↓
# 20260611_184739
# ------------------------------------------------------------

def get_video_name(filename):
```

```
    stem = os.path.splitext(filename)[0]

    if "_frame_" not in stem:
        raise ValueError(
            f"No se puede identificar el vídeo en: {filename}"
        )

    return stem.split("_frame_")[0]


videos = sorted({
    get_video_name(image)
    for image in images
})

print("Vídeos encontrados:", len(videos))

for video in videos:

    count = sum(
        get_video_name(img) == video
        for img in images
    )

    print(
        f"{video}: {count} imágenes"
    )

# ------------------------------------------------------------
# 4. División POR VÍDEO
#
# Aproximadamente:
# 70 % train
# 10 % validation
# 20 % test
#
# Con 10 vídeos:
# 7 train
# 1 val
# 2 test
# ------------------------------------------------------------

random.seed(42)

videos_shuffled = videos.copy()
random.shuffle(videos_shuffled)

n_videos = len(videos_shuffled)
```

```
n_train = max(
    1,
    round(n_videos * 0.70)
)

n_val = max(
    1,
    round(n_videos * 0.10)
)

# El resto va a test
train_videos = set(
    videos_shuffled[:n_train]
)

val_videos = set(
    videos_shuffled[
        n_train:n_train + n_val
    ]
)

test_videos = set(
    videos_shuffled[
        n_train + n_val:
    ]
)

print("\n" + "=" * 60)
print("DIVISIÓN POR VÍDEO")
print("=" * 60)

print("\nTRAIN:")
for v in sorted(train_videos):
    print(" ", v)

print("\nVAL:")
for v in sorted(val_videos):
    print(" ", v)

print("\nTEST:")
for v in sorted(test_videos):
    print(" ", v)

# ------------------------------------------------------------
# 5. Crear estructura YOLO limpia
# ------------------------------------------------------------

if os.path.exists(FINAL_DATASET_DIR):
    shutil.rmtree(FINAL_DATASET_DIR)
```

```
for split in ["train", "val", "test"]:

    os.makedirs(
        os.path.join(
            FINAL_DATASET_DIR,
            "images",
            split
        ),
        exist_ok=True
    )

    os.makedirs(
        os.path.join(
            FINAL_DATASET_DIR,
            "labels",
            split
        ),
        exist_ok=True
    )

# ------------------------------------------------------------
# 6. Determinar split de cada imagen
# ------------------------------------------------------------

def get_split(video):

    if video in train_videos:
        return "train"

    if video in val_videos:
        return "val"

    if video in test_videos:
        return "test"

    raise ValueError(
        f"Vídeo sin split: {video}"
    )

# ------------------------------------------------------------
# 7. Copiar imágenes y etiquetas
# ------------------------------------------------------------

stats = {
    "train": {
        "images": 0,
        "positive": 0,
        "negative": 0,
```

```
            "boxes": 0
        },
        "val": {
            "images": 0,
            "positive": 0,
            "negative": 0,
            "boxes": 0
        },
        "test": {
            "images": 0,
            "positive": 0,
            "negative": 0,
            "boxes": 0
        }
    }

    for image_name in images:

        video = get_video_name(
            image_name
        )

        split = get_split(
            video
        )

        # ----------------------------------------------------------
        # Imagen
        # ----------------------------------------------------------

        source_image = os.path.join(
            SOURCE_IMAGES,
            image_name
        )

        destination_image = os.path.join(
            FINAL_DATASET_DIR,
            "images",
            split,
            image_name
        )

        shutil.copy2(
            source_image,
            destination_image
        )

        stats[split]["images"] += 1
```

```
# ---------------------------------------------------------
# Etiqueta
# ---------------------------------------------------------

stem = os.path.splitext(
    image_name
)[0]

label_name = stem + ".txt"

source_label = os.path.join(
    SOURCE_LABELS,
    label_name
)

if os.path.exists(source_label):

    destination_label = os.path.join(
        FINAL_DATASET_DIR,
        "labels",
        split,
        label_name
    )

    shutil.copy2(
        source_label,
        destination_label
    )

    stats[split]["positive"] += 1

    with open(
        source_label,
        "r",
        encoding="utf-8"
    ) as f:

        boxes = [
            line
            for line in f
            if line.strip()
        ]

    stats[split]["boxes"] += len(
        boxes
    )

else:
```

```
        # Imagen revisada sin semáforo peatonal
        stats[split]["negative"] += 1

# ------------------------------------------------------------
# 8. Crear data.yaml
# ------------------------------------------------------------

yaml_path = os.path.join(
    FINAL_DATASET_DIR,
    "data.yaml"
)

yaml_content = f"""path: {FINAL_DATASET_DIR.replace(os.sep, "/")}
train: images/train
val: images/val
test: images/test

names:
  0: pedestrian_traffic_light
"""

with open(
    yaml_path,
    "w",
    encoding="utf-8"
) as f:

    f.write(yaml_content)

# ------------------------------------------------------------
# 9. Resumen final
# ------------------------------------------------------------

print("\n" + "=" * 60)
print("DATASET YOLO FINAL")
print("=" * 60)

total_images = 0
total_positive = 0
total_negative = 0
total_boxes = 0

for split in [
    "train",
    "val",
    "test"
]:

    s = stats[split]
```

```
    total_images += s["images"]
    total_positive += s["positive"]
    total_negative += s["negative"]
    total_boxes += s["boxes"]

    print(
        f"\n{split.upper()}"
    )

    print(
        " Imágenes:",
        s["images"]
    )

    print(
        " Positivas:",
        s["positive"]
    )

    print(
        " Negativas:",
        s["negative"]
    )

    print(
        " Bounding boxes:",
        s["boxes"]
    )

print("\n" + "-" * 60)

print(
    "TOTAL imágenes:",
    total_images
)

print(
    "TOTAL positivas:",
    total_positive
)

print(
    "TOTAL negativas:",
    total_negative
)

print(
    "TOTAL bounding boxes:",
```

```
    total_boxes
)

print(
    "\nDataset creado en:"
)

print(
    FINAL_DATASET_DIR
)

print(
    "\ndata.yaml:"
)

print(
    yaml_path
)
Imágenes encontradas: 142
Vídeos encontrados: 10
1000179702: 15 imágenes
1000179704: 15 imágenes
1000179707: 15 imágenes
1000179709: 15 imágenes
1000179711: 15 imágenes
20260611_184739: 15 imágenes
20260611_184808: 15 imágenes
20260611_185035: 15 imágenes
20260611_185443~2: 7 imágenes
20260611_185616: 15 imágenes

============================================================
DIVISIÓN POR VÍDEO
============================================================

TRAIN:
  1000179707
  1000179709
  20260611_184739
  20260611_184808
  20260611_185035
  20260611_185443~2
  20260611_185616

VAL:
  1000179711

TEST:
  1000179702
```

```
  1000179704

============================================================
DATASET YOLO FINAL
============================================================

TRAIN
 Imágenes: 97
 Positivas: 69
 Negativas: 28
 Bounding boxes: 99

VAL
 Imágenes: 15
 Positivas: 7
 Negativas: 8
 Bounding boxes: 7

TEST
 Imágenes: 30
 Positivas: 16
 Negativas: 14
 Bounding boxes: 31

------------------------------------------------------------
TOTAL imágenes: 142
TOTAL positivas: 92
TOTAL negativas: 50
TOTAL bounding boxes: 137

Dataset creado en:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final

data.yaml:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\data.yaml
```

In [138]:

```
# 13.6 COMPROBACIÓN FINAL DEL DATASET YOLO

import os
import yaml

FINAL_DATASET_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light"

# ------------------------------------------------------------
# 1. Comprobar estructura
# ------------------------------------------------------------
```

```
required_dirs = [
    "images/train",
    "images/val",
    "images/test",
    "labels/train",
    "labels/val",
    "labels/test"
]

print("=" * 60)
print("COMPROBACIÓN FINAL DEL DATASET YOLO")
print("=" * 60)

structure_ok = True

for relative_path in required_dirs:

    full_path = os.path.join(
        FINAL_DATASET_DIR,
        relative_path
    )

    exists = os.path.isdir(full_path)

    print(
        f"{relative_path}:",
        "OK" if exists else "FALTA"
    )

    if not exists:
        structure_ok = False

# ------------------------------------------------------------
# 2. Contar imágenes y etiquetas
# ------------------------------------------------------------

split_info = {}

for split in ["train", "val", "test"]:

    images_dir = os.path.join(
        FINAL_DATASET_DIR,
        "images",
        split
    )

    labels_dir = os.path.join(
        FINAL_DATASET_DIR,
```

```
        "labels",
        split
    )

    images = sorted([
        f for f in os.listdir(images_dir)
        if f.lower().endswith(
            (".jpg", ".jpeg", ".png")
        )
    ])

    labels = sorted([
        f for f in os.listdir(labels_dir)
        if f.lower().endswith(".txt")
    ])

    split_info[split] = {
        "images": images,
        "labels": labels
    }

    print(
        f"\n{split.upper()}: "
        f"{len(images)} imágenes | "
        f"{len(labels)} etiquetas"
    )

# ------------------------------------------------------------
# 3. Comprobar que no hay vídeos mezclados entre splits
# ------------------------------------------------------------

def get_video_name(filename):

    stem = os.path.splitext(filename)[0]

    return stem.split("_frame_")[0]


split_videos = {}

for split in ["train", "val", "test"]:

    split_videos[split] = {
        get_video_name(f)
        for f in split_info[split]["images"]
    }

print("\n" + "=" * 60)
print("VÍDEOS POR SPLIT")
```

```
print("=" * 60)

for split in ["train", "val", "test"]:

    print(
        f"\n{split.upper()}:",
        sorted(split_videos[split])
    )

train_val_overlap = (
    split_videos["train"]
    & split_videos["val"]
)

train_test_overlap = (
    split_videos["train"]
    & split_videos["test"]
)

val_test_overlap = (
    split_videos["val"]
    & split_videos["test"]
)

print("\nSolapamiento TRAIN-VAL:",
      train_val_overlap)

print("Solapamiento TRAIN-TEST:",
      train_test_overlap)

print("Solapamiento VAL-TEST:",
      val_test_overlap)

# ------------------------------------------------------------
# 4. Comprobar data.yaml
# ------------------------------------------------------------

yaml_path = os.path.join(
    FINAL_DATASET_DIR,
    "data.yaml"
)

print("\n" + "=" * 60)
print("DATA.YAML")
print("=" * 60)

if os.path.exists(yaml_path):

    with open(
```

```
        yaml_path,
        "r",
        encoding="utf-8"
    ) as f:

        data_yaml = yaml.safe_load(f)

    print(data_yaml)

else:

    print("ERROR: data.yaml no existe.")

# ------------------------------------------------------------
# 5. Resultado final
# ------------------------------------------------------------

no_overlap = (
    len(train_val_overlap) == 0
    and len(train_test_overlap) == 0
    and len(val_test_overlap) == 0
)

print("\n" + "=" * 60)

if structure_ok and no_overlap and os.path.exists(yaml_path):

    print("✓ DATASET PREPARADO CORRECTAMENTE PARA ENTRENAMIENTO")

else:

    print("⚠ REVISAR EL DATASET ANTES DE ENTRENAR")

print("=" * 60)
============================================================
COMPROBACIÓN FINAL DEL DATASET YOLO
============================================================
images/train: OK
images/val: OK
images/test: OK
labels/train: OK
labels/val: OK
labels/test: OK

TRAIN: 228 imágenes | 0 etiquetas

VAL: 44 imágenes | 0 etiquetas

TEST: 75 imágenes | 0 etiquetas
```

```
============================================================
VÍDEOS POR SPLIT
============================================================

TRAIN: ['1000179702', '1000179704', '1000179707', '20260611_184739',
'20260611_184808', '20260611_185035', '20260611_185443~2']

VAL: ['20260611_185616']

TEST: ['1000179709', '1000179711']

Solapamiento TRAIN-VAL: set()
Solapamiento TRAIN-TEST: set()
Solapamiento VAL-TEST: set()

============================================================
DATA.YAML
============================================================
ERROR: data.yaml no existe.

============================================================
⚠ REVISAR EL DATASET ANTES DE ENTRENAR
============================================================
```

In [ ]:

```
# 13.7 ENTRENAMIENTO YOLOv11
# DETECTOR ESPECÍFICO DE SEMÁFORO PEATONAL

from ultralytics import YOLO
import torch
import os

DATASET_DIR = (
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
    r"\dataset_pedestrian_traffic_light"
    r"\dataset_yolo_final"
)

DATA_YAML = os.path.join(
    DATASET_DIR,
    "data.yaml"
)

print("DATASET_DIR:", DATASET_DIR)
print("DATA_YAML:", DATA_YAML)
print("¿Existe dataset?:", os.path.isdir(DATASET_DIR))
print("¿Existe data.yaml?:", os.path.isfile(DATA_YAML))

# ------------------------------------------------------------
```

```
# 2. Dispositivo
# ------------------------------------------------------------

device = 0 if torch.cuda.is_available() else "cpu"

print("Dispositivo utilizado:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("GPU no disponible. Se utilizará CPU.")

# ------------------------------------------------------------
# 3. Cargar YOLOv11 preentrenado
# ------------------------------------------------------------

pedestrian_light_model = YOLO(
    "yolo11n.pt"
)

# ------------------------------------------------------------
# 4. Fine-tuning
# ------------------------------------------------------------

training_results = pedestrian_light_model.train(
    data=DATA_YAML,

    epochs=100,

    imgsz=960,

    batch=8,

    patience=20,

    device=device,

    workers=0,

    project=os.path.join(
        DATASET_DIR,
        "training"
    ),

    name="yolo11_pedestrian_light",

    pretrained=True,

    verbose=True,
```

```
    plots=True,

    seed=42
)
```

In [169]:

```
# ============================================================
# 13.7.1 CONSERVACIÓN PERMANENTE DE LOS MODELOS ENTRENADOS
#        VERIFICACIÓN DE LAS COPIAS ESTABLES
# ============================================================

from pathlib import Path
import hashlib
import json
from datetime import datetime


# ============================================================
# 1. CARPETA ESTABLE DEFINITIVA
# ============================================================

TFM_ROOT = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)

MODELS_DIR = (
    TFM_ROOT
    / "MODELOS_YOLO_FINALES_TFM"
)


if not MODELS_DIR.is_dir():

    raise FileNotFoundError(
        f"No existe la carpeta estable de modelos:\n"
        f"{MODELS_DIR}"
    )


# ============================================================
# 2. MODELOS DEFINITIVOS
# ============================================================

MODELS = {

    "V1_best":
        MODELS_DIR
        / "pedestrian_light_v1_best.pt",
```

```
    "V1_last":
        MODELS_DIR
        / "pedestrian_light_v1_last.pt",

    "V2_best":
        MODELS_DIR
        / "pedestrian_light_v2_best.pt",

    "V2_last":
        MODELS_DIR
        / "pedestrian_light_v2_last.pt"
}


# ============================================================
# 3. FUNCIÓN SHA-256
# ============================================================

def sha256_file(path):

    h = hashlib.sha256()

    with open(
        path,
        "rb"
    ) as f:

        for block in iter(
            lambda:
                f.read(
                    1024 * 1024
                ),
            b""
        ):

            h.update(
                block
            )

    return h.hexdigest()


# ============================================================
# 4. VERIFICACIÓN DE LOS MODELOS
# ============================================================

print("=" * 72)
print("VERIFICACIÓN DE MODELOS YOLO CONSERVADOS")
print("=" * 72)
```

```
print()
print("Carpeta estable:")
print(MODELS_DIR)

print()


manifest = {

    "verification_date":
        datetime.now().isoformat(),

    "models_directory":
        str(
            MODELS_DIR
        ),

    "models":
        {}
}


missing_models = []


for model_name, model_path in MODELS.items():

    exists = (
        model_path.is_file()
    )

    print(
        f"{model_name}:"
    )

    print(
        f"  {model_path}"
    )

    print(
        f"  Existe: {exists}"
    )


    if not exists:

        missing_models.append(
            model_name
```

```
    )

    print()

    continue


size_mb = (
    model_path.stat().st_size
    /
    (1024 ** 2)
)


sha256 = (
    sha256_file(
        model_path
    )
)


print(
    f"  Tamaño: {size_mb:.2f} MB"
)

print(
    f"  SHA-256: {sha256}"
)

print()


manifest[
    "models"
][
    model_name
] = {

    "path":
        str(
            model_path
        ),

    "size_mb":
        round(
            size_mb,
            3
        ),
```

```
        "sha256":
            sha256
    }


# ============================================================
# 5. COMPROBACIÓN FINAL
# ============================================================

if missing_models:

    raise FileNotFoundError(
        "Faltan los siguientes modelos conservados: "
        + ", ".join(
            missing_models
        )
    )


# ============================================================
# 6. GUARDAR MANIFIESTO DE VERIFICACIÓN
# ============================================================

manifest_path = (
    MODELS_DIR
    / "manifest_modelos.json"
)


with open(
    manifest_path,
    "w",
    encoding="utf-8"
) as f:

    json.dump(
        manifest,
        f,
        indent=4,
        ensure_ascii=False
    )


print("=" * 72)
print("MODELOS CONSERVADOS Y VERIFICADOS")
print("=" * 72)

print(
    "✓ pedestrian_light_v1_best.pt"
```

```
)

print(
    "✓ pedestrian_light_v1_last.pt"
)

print(
    "✓ pedestrian_light_v2_best.pt"
)

print(
    "✓ pedestrian_light_v2_last.pt"
)

print()
print(
    "✓ No se ha realizado ningún entrenamiento."
)

print(
    "✓ No se depende de la antigua carpeta training."
)

print()
print(
    "Archivo de control:"
)

print(
    manifest_path
)
======================================================================
==
VERIFICACIÓN DE MODELOS YOLO CONSERVADOS
======================================================================
==

Carpeta estable:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM

V1_best:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_best.pt
  Existe: True
  Tamaño: 5.26 MB
  SHA-256:
d339fb04e489385dc76c664cd3a039e0155e568358d25d33ff3240f2ddf4d2c0
```

```
V1_last:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_last.pt
  Existe: True
  Tamaño: 5.26 MB
  SHA-256:
3e23c15ebe3263a9f7559c66a97e5b9fb8b4c34dafef52c3e67722147fe2df20

V2_best:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_best.pt
  Existe: True
  Tamaño: 5.30 MB
  SHA-256:
07ce72073add35eb0d7f8165e77d211612706716cc7c25980dee7e98ab0c273c

V2_last:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_last.pt
  Existe: True
  Tamaño: 5.30 MB
  SHA-256:
df02244123e9f0072b556984958b6a2ed693bddb848b45330736c7c5f5f107a0

================================================================
==
MODELOS CONSERVADOS Y VERIFICADOS
================================================================
==
✓ pedestrian_light_v1_best.pt
✓ pedestrian_light_v1_last.pt
✓ pedestrian_light_v2_best.pt
✓ pedestrian_light_v2_last.pt

✓ No se ha realizado ningún entrenamiento.
✓ No se depende de la antigua carpeta training.

Archivo de control:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\manifest_modelos.json
```

In [178]:

```
# ============================================================
# 13.8 EVALUACIÓN FINAL DEL MODELO V1 SOBRE TEST
#      CARGA DEL MODELO YA ENTRENADO — SIN REENTRENAR
# ============================================================

from ultralytics import YOLO
```

```
from pathlib import Path
import torch


# ============================================================
# 1. RUTAS DEFINITIVAS
# ============================================================

TFM_DIR = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)


# ------------------------------------------------------------
# Dataset
# ------------------------------------------------------------

DATASET_DIR = (
    TFM_DIR
    / "dataset_pedestrian_traffic_light"
    / "dataset_yolo_final"
)

DATA_YAML = (
    DATASET_DIR
    / "data.yaml"
)


# ------------------------------------------------------------
# Modelo V1 ya entrenado
# IMPORTANTE:
# No se utiliza la antigua carpeta training.
# ------------------------------------------------------------

V1_MODEL_PATH_FINAL = (
    TFM_DIR
    / "MODELOS_YOLO_FINALES_TFM"
    / "pedestrian_light_v1_best.pt"
)


# ============================================================
# 2. COMPROBACIONES
# ============================================================

print("=" * 75)
print("13.8 — EVALUACIÓN FINAL DEL MODELO V1 SOBRE TEST")
print("=" * 75)
```

```
print()

print("Dataset:")
print(DATASET_DIR)

print()

print("data.yaml:")
print(DATA_YAML)

print()

print("Modelo V1 definitivo:")
print(V1_MODEL_PATH_FINAL)

print()

print(
    "¿Existe dataset?:",
    DATASET_DIR.is_dir()
)

print(
    "¿Existe data.yaml?:",
    DATA_YAML.is_file()
)

print(
    "¿Existe modelo V1?:",
    V1_MODEL_PATH_FINAL.is_file()
)


# ============================================================
# 3. VALIDAR RUTAS
# ============================================================

if not DATASET_DIR.is_dir():

    raise FileNotFoundError(
        f"No se encuentra el dataset:\n"
        f"{DATASET_DIR}"
    )


if not DATA_YAML.is_file():

    raise FileNotFoundError(
```

```
        f"No se encuentra data.yaml:\n"
        f"{DATA_YAML}"
    )


if not V1_MODEL_PATH_FINAL.is_file():

    raise FileNotFoundError(
        f"No se encuentra el modelo V1:\n"
        f"{V1_MODEL_PATH_FINAL}"
    )


# ============================================================
# 4. CARGAR MODELO YA ENTRENADO
# ============================================================

pedestrian_light_best = YOLO(
    str(
        V1_MODEL_PATH_FINAL
    )
)


device = (
    0
    if torch.cuda.is_available()
    else "cpu"
)


print()

print(
    "✓ Modelo V1 cargado correctamente."
)

print(
    "✓ Pesos cargados desde MODELOS_YOLO_FINALES_TFM."
)

print(
    "✓ NO se realizará ningún entrenamiento."
)

print(
    f"✓ Dispositivo utilizado: {device}"
)
```

```
# ============================================================
# 5. EVALUACIÓN EXCLUSIVAMENTE SOBRE TEST
# ============================================================

print()

print("=" * 75)
print("INICIANDO EVALUACIÓN SOBRE EL CONJUNTO TEST")
print("=" * 75)


test_metrics = pedestrian_light_best.val(

    data=str(
        DATA_YAML
    ),

    split="test",

    imgsz=960,

    device=device,

    workers=0,

    plots=True
)


# ============================================================
# 6. RESULTADOS
# ============================================================

print()

print("=" * 60)
print("RESULTADOS FINALES SOBRE TEST — MODELO V1")
print("=" * 60)


print(
    f"Precision:   "
    f"{test_metrics.box.mp:.4f}"
)

print(
    f"Recall:      "
```

```
    f"{test_metrics.box.mr:.4f}"
)

print(
    f"mAP50:       "
    f"{test_metrics.box.map50:.4f}"
)

print(
    f"mAP50-95:    "
    f"{test_metrics.box.map:.4f}"
)


print("=" * 60)

print()

print(
    "✓ Evaluación del modelo V1 finalizada."
)

print(
    "✓ No se ha realizado ningún entrenamiento."
)

print(
    "✓ Modelo utilizado:"
)

print(
    V1_MODEL_PATH_FINAL
)
============================================================================
=====
13.8 — EVALUACIÓN FINAL DEL MODELO V1 SOBRE TEST
============================================================================
=====

Dataset:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final

data.yaml:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\data.yaml

Modelo V1 definitivo:
```

```
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_best.pt

¿Existe dataset?: True
¿Existe data.yaml?: True
¿Existe modelo V1?: True

✓ Modelo V1 cargado correctamente.
✓ Pesos cargados desde MODELOS_YOLO_FINALES_TFM.
✓ NO se realizará ningún entrenamiento.
✓ Dispositivo utilizado: cpu

================================================================
=====
INICIANDO EVALUACIÓN SOBRE EL CONJUNTO TEST
================================================================
=====
Ultralytics 8.4.33  Python-3.14.3 torch-2.11.0+cpu CPU (AMD Ryzen AI 7
350 w/ Radeon 860M)
YOLO11n summary (fused): 101 layers, 2,582,347 parameters, 0
gradients, 6.3 GFLOPs
val: Fast image access  (ping: 0.10.0 ms, read: 2159.6832.1 MB/s,
size: 447.9 KB)
val: Scanning C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\labels\test
.cache... 16 images, 14 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━━━━━
━━ 30/30 7.9Mit/s 0.0s
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.0it/s 2.0s3.5s
                   all         30         31      0.357      0.323
0.272      0.107
Speed: 0.6ms preprocess, 46.7ms inference, 0.0ms loss, 4.0ms
postprocess per image
Results saved to C:\Windows\System32\runs\detect\val7

============================================================
RESULTADOS FINALES SOBRE TEST — MODELO V1
============================================================
Precision:   0.3567
Recall:      0.3226
mAP50:       0.2720
mAP50-95:    0.1069
============================================================

✓ Evaluación del modelo V1 finalizada.
✓ No se ha realizado ningún entrenamiento.
✓ Modelo utilizado:
```

C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_best.pt

In [179]:

```
# ============================================================
# 13.9 DIAGNÓSTICO DEL TAMAÑO DE LOS SEMÁFOROS PEATONALES
# ============================================================

import os
import cv2
import pandas as pd

DATASET_DIR = (
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
    r"\dataset_pedestrian_traffic_light"
    r"\dataset_yolo_final"
)

TARGET_IMGSZ = 960

rows = []

for split in ["train", "val", "test"]:

    images_dir = os.path.join(
        DATASET_DIR,
        "images",
        split
    )

    labels_dir = os.path.join(
        DATASET_DIR,
        "labels",
        split
    )

    for image_name in os.listdir(images_dir):

        if not image_name.lower().endswith(
            (".jpg", ".jpeg", ".png")
        ):
            continue

        image_path = os.path.join(
            images_dir,
            image_name
        )

        image = cv2.imread(image_path)
```

```
if image is None:
    continue

h, w = image.shape[:2]

stem = os.path.splitext(
    image_name
)[0]

label_path = os.path.join(
    labels_dir,
    stem + ".txt"
)

if not os.path.exists(label_path):
    continue

with open(
    label_path,
    "r",
    encoding="utf-8"
) as f:

    lines = [
        line.strip()
        for line in f
        if line.strip()
    ]

# Escala aproximada aplicada por YOLO
scale = TARGET_IMGSZ / max(w, h)

for line in lines:

    parts = line.split()

    class_id = int(parts[0])

    x_center = float(parts[1])
    y_center = float(parts[2])
    bw_norm = float(parts[3])
    bh_norm = float(parts[4])

    bw_px = bw_norm * w
    bh_px = bh_norm * h

    bw_960 = bw_px * scale
    bh_960 = bh_px * scale
```

```
            area_960 = (
                bw_960 * bh_960
            )

            rows.append({
                "split": split,
                "image": image_name,
                "original_width": w,
                "original_height": h,
                "bbox_width_px": bw_px,
                "bbox_height_px": bh_px,
                "width_at_960": bw_960,
                "height_at_960": bh_960,
                "area_at_960": area_960
            })

bbox_analysis = pd.DataFrame(rows)

print("=" * 65)
print("ANÁLISIS DE TAMAÑO DE LOS OBJETOS")
print("=" * 65)

print(
    "\nNúmero de bounding boxes:",
    len(bbox_analysis)
)

print("\nTamaño aproximado después del resize a 960:")
print(
    bbox_analysis[
        [
            "width_at_960",
            "height_at_960",
            "area_at_960"
        ]
    ].describe()
)

# ------------------------------------------------------------
# Clasificación orientativa por tamaño
# ------------------------------------------------------------

very_small = (
    (
        bbox_analysis["width_at_960"] < 16
    )
    |
    (
        bbox_analysis["height_at_960"] < 16
```

```
    )
).sum()

small = (
    (
        bbox_analysis["width_at_960"] < 32
    )
    |
    (
        bbox_analysis["height_at_960"] < 32
    )
).sum()

print("\n" + "=" * 65)

print(
    "Objetos con alguna dimensión < 16 px:",
    very_small,
    f"({very_small / len(bbox_analysis) * 100:.1f} %)"
)

print(
    "Objetos con alguna dimensión < 32 px:",
    small,
    f"({small / len(bbox_analysis) * 100:.1f} %)"
)

print("=" * 65)

display(
    bbox_analysis.sort_values(
        "area_at_960"
    ).head(20)
)
```

```
=================================================================
ANÁLISIS DE TAMAÑO DE LOS OBJETOS
=================================================================

Número de bounding boxes: 137

Tamaño aproximado después del resize a 960:
       width_at_960  height_at_960   area_at_960
count    137.000000     137.000000    137.000000
mean      35.532867      53.817522   2282.525952
std       18.609558      24.712974   2218.340063
min        9.000180       9.000000    103.500630
25%       22.500180      37.499520    923.996160
50%       32.499840      48.500160   1638.727890
75%       43.000200      69.500160   2773.492260
```

```
max        94.499520     139.999680  12039.952880

============================================================
Objetos con alguna dimensión < 16 px: 14 (10.2 %)
Objetos con alguna dimensión < 32 px: 69 (50.4 %)
============================================================
```

	split	image	original_width	original_height	bbox_width_px	bbox
109	test	1000179702_frame_000042.jpg	1080	1920	18.00036	22.99
99	val	1000179711_frame_000000.jpg	1080	1920	29.00016	21.99
107	test	1000179702_frame_000000.jpg	1080	1920	20.00052	33.00
24	train	1000179709_frame_000518.jpg	1080	1920	41.00004	18.00
115	test	1000179702_frame_000168.jpg	1080	1920	19.00044	43.00
113	test	1000179702_frame_000126.jpg	1080	1920	25.99992	32.00
27	train	20260611_184739_frame_000000.jpg	1920	1080	27.00096	31.00
125	test	1000179704_frame_000000.jpg	1080	1920	27.00000	33.00
128	test	1000179704_frame_000045.jpg	1080	1920	25.99992	39.99
30	train	20260611_184739_frame_000028.jpg	1920	1080	30.00000	39.99
23	train	1000179709_frame_000476.jpg	1080	1920	27.00000	52.99
117	test	1000179702_frame_000210.jpg	1080	1920	34.99956	40.99
33	train	20260611_184739_frame_000056.jpg	1920	1080	26.00064	59.00
100	val	1000179711_frame_000075.jpg	1080	1920	31.00032	50.99
111	test	1000179702_frame_000084.jpg	1080	1920	38.99988	43.00
0	train	1000179707_frame_000000.jpg	1080	1920	29.00016	61.00
21	train	1000179709_frame_000294.jpg	1080	1920	32.00040	56.00
1	train	1000179707_frame_000000.jpg	1080	1920	32.00040	61.00
47	train	20260611_184739_frame_000294.jpg	1920	1080	39.99936	50.99
132	test	1000179704_frame_000435.jpg	1080	1920	34.99956	58.99

In [180]:

```
# ============================================================
# 13.10 PRUEBA DE CONTROL:
# SEMÁFORO PEATONAL CLARAMENTE VISIBLE
# ============================================================

TRAIN_IMAGES_DIR = os.path.join(
    DATASET_DIR,
    "images",
    "train"
)

video_test = "20260611_184739"
target_frame = 60

# Buscar los frames disponibles de ese vídeo
```

```
candidate_images = [
    f for f in os.listdir(TRAIN_IMAGES_DIR)
    if f.startswith(video_test + "_frame_")
    and f.lower().endswith((".jpg", ".jpeg", ".png"))
]

def get_frame_number(filename):
    stem = os.path.splitext(filename)[0]
    return int(
        stem.split("_frame_")[1]
    )

# Elegir el frame disponible más cercano al 60
selected_image = min(
    candidate_images,
    key=lambda f:
        abs(get_frame_number(f) - target_frame)
)

selected_path = os.path.join(
    TRAIN_IMAGES_DIR,
    selected_image
)

print("Imagen seleccionada:", selected_image)
print(
    "Frame:",
    get_frame_number(selected_image)
)

display(
    Image.open(selected_path)
)

# ============================================================
# 1. INFERENCIA NORMAL
# ============================================================

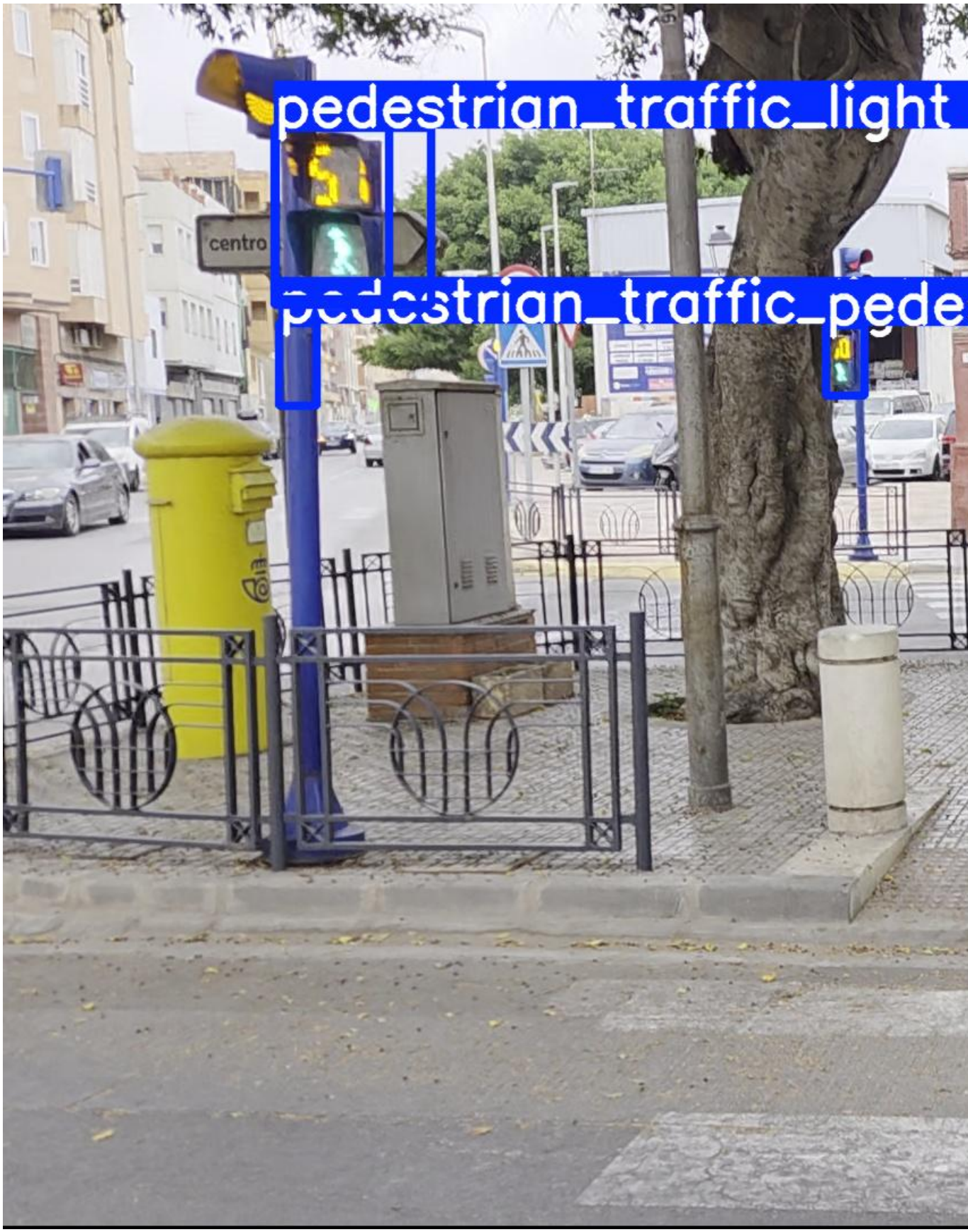
normal_control = pedestrian_light_best.predict(
    source=selected_path,
    imgsz=960,
    conf=0.05,
    verbose=False
)[0]

print(
    "\nDetecciones normales:",
    len(normal_control.boxes)
)
```

```
display(
    Image.fromarray(
        cv2.cvtColor(
            normal_control.plot(),
            cv2.COLOR_BGR2RGB
        )
    )
)
Imagen seleccionada: 20260611_184739_frame_000056.jpg
Frame: 56
```



Detecciones normales: 5



In [181]:

```
# ============================================================
# 13.11 INSPECCIÓN VISUAL DE LAS ANOTACIONES DE TEST
```

```
# ============================================================

import os
import cv2
from PIL import Image

TEST_IMAGES_DIR = os.path.join(
    DATASET_DIR,
    "images",
    "test"
)

TEST_LABELS_DIR = os.path.join(
    DATASET_DIR,
    "labels",
    "test"
)

test_positive_images = []

for image_name in sorted(
    os.listdir(TEST_IMAGES_DIR)
):

    if not image_name.lower().endswith(
        (".jpg", ".jpeg", ".png")
    ):
        continue

    stem = os.path.splitext(
        image_name
    )[0]

    label_path = os.path.join(
        TEST_LABELS_DIR,
        stem + ".txt"
    )

    if not os.path.exists(label_path):
        continue

    with open(
        label_path,
        "r",
        encoding="utf-8"
    ) as f:

        lines = [
            line.strip()
```

```
            for line in f
            if line.strip()
        ]

    if len(lines) > 0:
        test_positive_images.append(
            image_name
        )


print(
    "Imágenes positivas de TEST:",
    len(test_positive_images)
)

print("=" * 70)


for image_name in test_positive_images:

    image_path = os.path.join(
        TEST_IMAGES_DIR,
        image_name
    )

    stem = os.path.splitext(
        image_name
    )[0]

    label_path = os.path.join(
        TEST_LABELS_DIR,
        stem + ".txt"
    )

    image = cv2.imread(
        image_path
    )

    if image is None:
        continue

    image_rgb = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )

    h, w = image.shape[:2]

    with open(
```

```
    label_path,
    "r",
    encoding="utf-8"
) as f:

    lines = [
        line.strip()
        for line in f
        if line.strip()
    ]

print("\n" + "=" * 70)
print(image_name)
print(
    "Semáforos anotados:",
    len(lines)
)

for i, line in enumerate(
    lines,
    start=1
):

    parts = line.split()

    xc = float(parts[1])
    yc = float(parts[2])
    bw = float(parts[3])
    bh = float(parts[4])

    x1 = int(
        (xc - bw / 2) * w
    )

    y1 = int(
        (yc - bh / 2) * h
    )

    x2 = int(
        (xc + bw / 2) * w
    )

    y2 = int(
        (yc + bh / 2) * h
    )

    bbox_width = x2 - x1
    bbox_height = y2 - y1
```

```
        # Margen para facilitar la inspección
        margin = 40

        cx1 = max(
            0,
            x1 - margin
        )

        cy1 = max(
            0,
            y1 - margin
        )

        cx2 = min(
            w,
            x2 + margin
        )

        cy2 = min(
            h,
            y2 + margin
        )

        crop = image_rgb[
            cy1:cy2,
            cx1:cx2
        ]

        print(
            f"\nObjeto {i} | "
            f"{bbox_width} x {bbox_height} px"
        )

        display(
            Image.fromarray(crop)
        )
Imágenes positivas de TEST: 16
================================================================================

================================================================================
1000179702_frame_000000.jpg
Semáforos anotados: 2

Objeto 1 | 55 x 89 px
```



Objeto 2 | 20 x 34 px



==========

1000179702_frame_000042.jpg
Semáforos anotados: 2

Objeto 1 | 72 x 94 px



Objeto 2 | 18 x 23 px



==========

1000179702_frame_000084.jpg
Semáforos anotados: 2

Objeto 1 | 75 x 124 px



Objeto 2 | 39 x 43 px



==========

1000179702_frame_000126.jpg
Semáforos anotados: 2

Objeto 1 | 69 x 135 px



Objeto 2 | 26 x 32 px



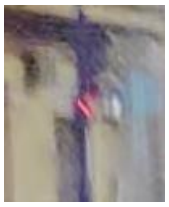
==========

1000179702_frame_000168.jpg
Semáforos anotados: 2

Objeto 1 | 69 x 149 px



Objeto 2 | 20 x 43 px



==========
1000179702_frame_000210.jpg
Semáforos anotados: 2

Objeto 1 | 75 x 155 px



Objeto 2 | 35 x 41 px



==========
1000179702_frame_000252.jpg
Semáforos anotados: 1

Objeto 1 | 87 x 164 px



==========
1000179702_frame_000462.jpg
Semáforos anotados: 1

Objeto 1 | 80 x 97 px



==========
1000179702_frame_000504.jpg
Semáforos anotados: 1

Objeto 1 | 96 x 142 px



==========
1000179702_frame_000546.jpg
Semáforos anotados: 1

Objeto 1 | 106 x 145 px



==========
1000179704_frame_000000.jpg
Semáforos anotados: 4

Objeto 1 | 75 x 92 px



Objeto 2 | 172 x 280 px



Objeto 3 | 40 x 76 px



Objeto 4 | 27 x 33 px



==========

1000179704_frame_000045.jpg

Semáforos anotados: 4

Objeto 1 | 161 x 238 px



Objeto 2 | 39 x 68 px



Objeto 3 | 26 x 40 px



Objeto 4 | 47 x 73 px



==========

1000179704_frame_000435.jpg
Semáforos anotados: 3

Objeto 1 | 68 x 102 px



Objeto 2 | 94 x 196 px



Objeto 3 | 34 x 58 px



==========

1000179704_frame_000480.jpg

Semáforos anotados: 2

Objeto 1 | 72 x 121 px



Objeto 2 | 47 x 68 px



```
================================================================================
1000179704_frame_000540.jpg
Semáforos anotados: 1

Objeto 1 | 88 x 133 px
```



```
================================================================================
1000179704_frame_000600.jpg
Semáforos anotados: 1

Objeto 1 | 109 x 188 px
```



In [182]:

```
# ============================================================
# 13.12 GRID DE ANOTACIONES TEST
# PARA REVISIÓN DE VISIBILIDAD
# ============================================================

import matplotlib.pyplot as plt
import math

gt_crops = []
```

```
global_id = 0

for image_name in test_positive_images:

    image_path = os.path.join(
        TEST_IMAGES_DIR,
        image_name
    )

    stem = os.path.splitext(image_name)[0]

    label_path = os.path.join(
        TEST_LABELS_DIR,
        stem + ".txt"
    )

    image = cv2.imread(image_path)

    if image is None:
        continue

    image_rgb = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )

    h, w = image.shape[:2]

    with open(
        label_path,
        "r",
        encoding="utf-8"
    ) as f:

        lines = [
            line.strip()
            for line in f
            if line.strip()
        ]

    for line in lines:

        parts = line.split()

        xc = float(parts[1])
        yc = float(parts[2])
        bw = float(parts[3])
        bh = float(parts[4])
```

```
        x1 = int((xc - bw / 2) * w)
        y1 = int((yc - bh / 2) * h)
        x2 = int((xc + bw / 2) * w)
        y2 = int((yc + bh / 2) * h)

        # margen visual
        margin = 25

        x1m = max(0, x1 - margin)
        y1m = max(0, y1 - margin)
        x2m = min(w, x2 + margin)
        y2m = min(h, y2 + margin)

        crop = image_rgb[
            y1m:y2m,
            x1m:x2m
        ]

        gt_crops.append({
            "id": global_id,
            "image": image_name,
            "crop": crop
        })

        global_id += 1


# ------------------------------------------------------------
# Mostrar grid
# ------------------------------------------------------------

cols = 5
rows = math.ceil(
    len(gt_crops) / cols
)

plt.figure(
    figsize=(15, rows * 3)
)

for i, item in enumerate(gt_crops):

    plt.subplot(
        rows,
        cols,
        i + 1
    )

    plt.imshow(
```

```
        item["crop"]
    )

    plt.title(
        f"ID {item['id']}",
        fontsize=10
    )

    plt.axis("off")

plt.tight_layout()
plt.show()

print(
    "Total objetos:",
    len(gt_crops)
)
```

```
<Figure size 1500x2100 with 31 Axes>
Total objetos: 31
```

In [183]:

```
# ============================================================
# 13.13 Evaluación funcional sobre semáforos visualmente relevantes de
TEST
# ============================================================

import os
import cv2
import pandas as pd
import numpy as np

# IDs que hemos considerado útiles tras la inspección visual
USEFUL_IDS = {
    0, 1, 2, 3, 4,
    5, 6, 7, 8, 9,
    10, 11, 12, 13, 14,
    15, 16, 18,
    21, 24, 27, 29, 30
}

# ------------------------------------------------------------
# Función IoU
# ------------------------------------------------------------

def calculate_iou(box_a, box_b):

    ax1, ay1, ax2, ay2 = box_a
    bx1, by1, bx2, by2 = box_b

    inter_x1 = max(ax1, bx1)
```

```
    inter_y1 = max(ay1, by1)
    inter_x2 = min(ax2, bx2)
    inter_y2 = min(ay2, by2)

    inter_w = max(0, inter_x2 - inter_x1)
    inter_h = max(0, inter_y2 - inter_y1)

    intersection = inter_w * inter_h

    area_a = max(0, ax2 - ax1) * max(0, ay2 - ay1)
    area_b = max(0, bx2 - bx1) * max(0, by2 - by1)

    union = area_a + area_b - intersection

    if union <= 0:
        return 0.0

    return intersection / union


# ------------------------------------------------------------
# Reconstruir las 31 anotaciones manteniendo los mismos IDs
# que usamos en el grid anterior
# ------------------------------------------------------------

gt_objects = []

global_id = 0

for image_name in test_positive_images:

    image_path = os.path.join(
        TEST_IMAGES_DIR,
        image_name
    )

    stem = os.path.splitext(image_name)[0]

    label_path = os.path.join(
        TEST_LABELS_DIR,
        stem + ".txt"
    )

    image = cv2.imread(image_path)

    if image is None:
        continue

    h, w = image.shape[:2]
```

```
    with open(
        label_path,
        "r",
        encoding="utf-8"
    ) as f:

        lines = [
            line.strip()
            for line in f
            if line.strip()
        ]

    for line in lines:

        parts = line.split()

        xc = float(parts[1])
        yc = float(parts[2])
        bw = float(parts[3])
        bh = float(parts[4])

        x1 = (xc - bw / 2) * w
        y1 = (yc - bh / 2) * h
        x2 = (xc + bw / 2) * w
        y2 = (yc + bh / 2) * h

        gt_objects.append({
            "id": global_id,
            "image": image_name,
            "bbox": (x1, y1, x2, y2),
            "useful": global_id in USEFUL_IDS
        })

        global_id += 1


# ------------------------------------------------------------
# Inferencia del modelo sobre cada imagen positiva de TEST
# ------------------------------------------------------------

predictions_by_image = {}

for image_name in test_positive_images:

    image_path = os.path.join(
        TEST_IMAGES_DIR,
        image_name
    )
```

```
    result = pedestrian_light_best.predict(
        source=image_path,
        imgsz=960,
        conf=0.05,
        verbose=False
    )[0]

    predictions = []

    for box in result.boxes:

        x1, y1, x2, y2 = (
            box.xyxy[0]
            .cpu()
            .numpy()
        )

        confidence = float(
            box.conf[0].item()
        )

        predictions.append({
            "bbox": (
                float(x1),
                float(y1),
                float(x2),
                float(y2)
            ),
            "confidence": confidence
        })

    predictions_by_image[
        image_name
    ] = predictions


# ------------------------------------------------------------
# Comparar cada GT útil con las predicciones
# ------------------------------------------------------------

rows = []

IOU_THRESHOLD = 0.50

for obj in gt_objects:

    if not obj["useful"]:
        continue
```

```
    predictions = predictions_by_image[
        obj["image"]
    ]

    best_iou = 0.0
    best_confidence = 0.0

    for pred in predictions:

        iou = calculate_iou(
            obj["bbox"],
            pred["bbox"]
        )

        if iou > best_iou:

            best_iou = iou
            best_confidence = pred[
                "confidence"
            ]

    detected = (
        best_iou >= IOU_THRESHOLD
    )

    rows.append({
        "ID": obj["id"],
        "image": obj["image"],
        "detected": detected,
        "best_iou": best_iou,
        "confidence": best_confidence
    })


functional_results = pd.DataFrame(rows)

# ------------------------------------------------------------
# Resumen
# ------------------------------------------------------------

detected_count = int(
    functional_results[
        "detected"
    ].sum()
)

total_useful = len(
    functional_results
```

```
)

functional_recall = (
    detected_count /
    total_useful
    if total_useful > 0
    else 0
)

print("=" * 65)
print("EVALUACIÓN FUNCIONAL - TEST")
print("=" * 65)

print(
    "Semáforos útiles:",
    total_useful
)

print(
    "Semáforos útiles detectados:",
    detected_count
)

print(
    f"Recall funcional: "
    f"{functional_recall:.3f} "
    f"({functional_recall * 100:.1f} %)"
)

print(
    "IoU mínimo considerado correcto:",
    IOU_THRESHOLD
)

print("=" * 65)

display(
    functional_results.sort_values(
        ["detected", "best_iou"],
        ascending=[False, False]
    )
)
```

```
=================================================================
EVALUACIÓN FUNCIONAL - TEST
=================================================================
Semáforos útiles: 23
Semáforos útiles detectados: 7
Recall funcional: 0.304 (30.4 %)
IoU mínimo considerado correcto: 0.5
```

==========

	ID	image	detected	best_iou	confidence
22	30	1000179704_frame_000600.jpg	True	0.804434	0.229821
16	16	1000179704_frame_000000.jpg	True	0.744800	0.086574
21	29	1000179704_frame_000540.jpg	True	0.743705	0.063083
4	4	1000179702_frame_000084.jpg	True	0.736659	0.060907
13	13	1000179702_frame_000462.jpg	True	0.637125	0.496062
0	0	1000179702_frame_000000.jpg	True	0.559318	0.068671
18	21	1000179704_frame_000045.jpg	True	0.558319	0.068687
6	6	1000179702_frame_000126.jpg	False	0.433047	0.062110
14	14	1000179702_frame_000504.jpg	False	0.417166	0.331502
15	15	1000179702_frame_000546.jpg	False	0.388965	0.074980
1	1	1000179702_frame_000000.jpg	False	0.000000	0.000000
2	2	1000179702_frame_000042.jpg	False	0.000000	0.000000
3	3	1000179702_frame_000042.jpg	False	0.000000	0.000000
5	5	1000179702_frame_000084.jpg	False	0.000000	0.000000
7	7	1000179702_frame_000126.jpg	False	0.000000	0.000000
8	8	1000179702_frame_000168.jpg	False	0.000000	0.000000
9	9	1000179702_frame_000168.jpg	False	0.000000	0.000000
10	10	1000179702_frame_000210.jpg	False	0.000000	0.000000
11	11	1000179702_frame_000210.jpg	False	0.000000	0.000000
12	12	1000179702_frame_000252.jpg	False	0.000000	0.000000
17	18	1000179704_frame_000000.jpg	False	0.000000	0.000000
19	24	1000179704_frame_000435.jpg	False	0.000000	0.000000
20	27	1000179704_frame_000480.jpg	False	0.000000	0.000000

In [184]:

```
# ============================================================
# 13.14 DIAGNÓSTICO DE CONFIANZA E IoU SOBRE LOS 23 CASOS ÚTILES
# ============================================================

CONF_MIN = 0.001

# ------------------------------------------------------------
# Obtener predicciones con un umbral MUY bajo
# ------------------------------------------------------------

predictions_low_conf = {}

for image_name in test_positive_images:

    image_path = os.path.join(
        TEST_IMAGES_DIR,
```

```
        image_name
    )

    result = pedestrian_light_best.predict(
        source=image_path,
        imgsz=960,
        conf=CONF_MIN,
        verbose=False
    )[0]

    preds = []

    for box in result.boxes:

        x1, y1, x2, y2 = (
            box.xyxy[0]
            .cpu()
            .numpy()
        )

        preds.append({
            "bbox": (
                float(x1),
                float(y1),
                float(x2),
                float(y2)
            ),
            "confidence":
                float(box.conf[0].item())
        })

    predictions_low_conf[
        image_name
    ] = preds


# ------------------------------------------------------------
# Mejor coincidencia para cada semáforo útil
# ------------------------------------------------------------

rows_low = []

for obj in gt_objects:

    if not obj["useful"]:
        continue

    best_iou = 0.0
    best_confidence = 0.0
```

```
    for pred in predictions_low_conf[
        obj["image"]
    ]:

        iou = calculate_iou(
            obj["bbox"],
            pred["bbox"]
        )

        if iou > best_iou:

            best_iou = iou

            best_confidence = pred[
                "confidence"
            ]

    rows_low.append({
        "ID": obj["id"],
        "image": obj["image"],
        "best_iou": best_iou,
        "confidence": best_confidence
    })


low_conf_results = pd.DataFrame(
    rows_low
)

# ------------------------------------------------------------
# Recall para varios umbrales
# ------------------------------------------------------------

thresholds = [
    0.001,
    0.005,
    0.01,
    0.02,
    0.05,
    0.10
]

print("=" * 70)
print("SENSIBILIDAD DEL MODELO SEGÚN UMBRAL")
print("=" * 70)

for conf_threshold in thresholds:
```

```
    detected_05 = (
        (
            low_conf_results["best_iou"] >= 0.50
        )
        &
        (
            low_conf_results["confidence"]
            >= conf_threshold
        )
    ).sum()

    detected_03 = (
        (
            low_conf_results["best_iou"] >= 0.30
        )
        &
        (
            low_conf_results["confidence"]
            >= conf_threshold
        )
    ).sum()

    print(
        f"conf >= {conf_threshold:0.3f} | "
        f"IoU>=0.50: {detected_05:2d}/23 "
        f"({detected_05/23*100:5.1f} %) | "
        f"IoU>=0.30: {detected_03:2d}/23 "
        f"({detected_03/23*100:5.1f} %)"
    )

print("=" * 70)

display(
    low_conf_results.sort_values(
        "best_iou",
        ascending=False
    )
)
```

```
======================================================================
SENSIBILIDAD DEL MODELO SEGÚN UMBRAL
======================================================================
conf >= 0.001 | IoU>=0.50: 10/23 ( 43.5 %) | IoU>=0.30: 15/23 ( 65.2
%)
conf >= 0.005 | IoU>=0.50: 10/23 ( 43.5 %) | IoU>=0.30: 15/23 ( 65.2
%)
conf >= 0.010 | IoU>=0.50:  9/23 ( 39.1 %) | IoU>=0.30: 11/23 ( 47.8
%)
conf >= 0.020 | IoU>=0.50:  6/23 ( 26.1 %) | IoU>=0.30:  8/23 ( 34.8
%)
```

```
conf >= 0.050 | IoU>=0.50:  6/23 ( 26.1 %) | IoU>=0.30:  7/23 ( 30.4
%)
conf >= 0.100 | IoU>=0.50:  2/23 (  8.7 %) | IoU>=0.30:  2/23 (  8.7
%)
======================================================================
```

	ID	image	best_iou	confidence
4	4	1000179702_frame_000084.jpg	0.829153	0.015970
22	30	1000179704_frame_000600.jpg	0.804434	0.229821
20	27	1000179704_frame_000480.jpg	0.751814	0.015335
16	16	1000179704_frame_000000.jpg	0.744800	0.086574
21	29	1000179704_frame_000540.jpg	0.743705	0.063083
13	13	1000179702_frame_000462.jpg	0.637125	0.496062
14	14	1000179702_frame_000504.jpg	0.577977	0.011533
15	15	1000179702_frame_000546.jpg	0.564257	0.008292
0	0	1000179702_frame_000000.jpg	0.559318	0.068671
18	21	1000179704_frame_000045.jpg	0.558319	0.068687
19	24	1000179704_frame_000435.jpg	0.485993	0.005049
8	8	1000179702_frame_000168.jpg	0.454743	0.007438
6	6	1000179702_frame_000126.jpg	0.433047	0.062110
17	18	1000179704_frame_000000.jpg	0.347527	0.027040
12	12	1000179702_frame_000252.jpg	0.338107	0.005840
10	10	1000179702_frame_000210.jpg	0.253700	0.004444
5	5	1000179702_frame_000084.jpg	0.128571	0.005082
2	2	1000179702_frame_000042.jpg	0.064873	0.004340
11	11	1000179702_frame_000210.jpg	0.011653	0.004465
9	9	1000179702_frame_000168.jpg	0.007134	0.004618
7	7	1000179702_frame_000126.jpg	0.007099	0.004889
1	1	1000179702_frame_000000.jpg	0.006773	0.004654
3	3	1000179702_frame_000042.jpg	0.004794	0.004238

In [186]:

```
# ============================================================
# 13.15 INFERENCIA POR TILES CON YOLO11 ESPECIALIZADO
#       MODELO V1 YA ENTRENADO — SIN REENTRENAR
# ============================================================

import cv2
import numpy as np

from pathlib import Path
from PIL import Image
from ultralytics import YOLO
from IPython.display import display
```

```
# ============================================================
# 1. RUTAS DEFINITIVAS
# ============================================================

TFM_DIR = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)


DATASET_DIR = (
    TFM_DIR
    / "dataset_pedestrian_traffic_light"
    / "dataset_yolo_final"
)


V1_MODEL_PATH_FINAL = (
    TFM_DIR
    / "MODELOS_YOLO_FINALES_TFM"
    / "pedestrian_light_v1_best.pt"
)


# ============================================================
# 2. COMPROBACIONES
# ============================================================

print("=" * 72)
print("13.15 — INFERENCIA POR TILES")
print("=" * 72)

print()
print("Dataset:")
print(DATASET_DIR)

print()
print("Modelo V1:")
print(V1_MODEL_PATH_FINAL)

print()
print(
    "¿Existe dataset?:",
    DATASET_DIR.is_dir()
)

print(
    "¿Existe modelo V1?:",
```

```
    V1_MODEL_PATH_FINAL.is_file()
)


if not DATASET_DIR.is_dir():

    raise FileNotFoundError(
        f"No existe el dataset:\n"
        f"{DATASET_DIR}"
    )


if not V1_MODEL_PATH_FINAL.is_file():

    raise FileNotFoundError(
        f"No existe el modelo V1:\n"
        f"{V1_MODEL_PATH_FINAL}"
    )


# ============================================================
# 3. CARGAR MODELO V1 YA ENTRENADO
# ============================================================

pedestrian_light_best = YOLO(
    str(
        V1_MODEL_PATH_FINAL
    )
)


print()
print(
    "✓ Modelo V1 cargado correctamente."
)

print(
    "✓ No se utiliza la antigua carpeta training."
)

print(
    "✓ No se realizará ningún entrenamiento."
)


# ============================================================
# 4. FUNCIÓN DE INFERENCIA POR TILES
# ============================================================
```

```
def tiled_pedestrian_light_detection(
    image_path,
    tile_size=960,
    overlap=0.25,
    conf=0.05
):

    image_path = Path(
        image_path
    )


    if not image_path.is_file():

        raise FileNotFoundError(
            f"No existe la imagen:\n"
            f"{image_path}"
        )


    image = cv2.imread(
        str(
            image_path
        )
    )


    if image is None:

        raise ValueError(
            f"No se pudo cargar la imagen:\n"
            f"{image_path}"
        )


    image_rgb = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )


    h, w = (
        image.shape[:2]
    )


    # ---------------------------------------------------------
    # Protección de parámetros
    # ---------------------------------------------------------
```

```
tile_size = int(
    tile_size
)


overlap = float(
    overlap
)


if tile_size <= 0:

    raise ValueError(
        "tile_size debe ser mayor que 0."
    )


if not (
    0 <= overlap < 1
):

    raise ValueError(
        "overlap debe estar en el intervalo [0, 1)."
    )


step = max(
    1,
    int(
        tile_size
        *
        (
            1 - overlap
        )
    )
)


detections = []


# ============================================================
# 5. POSICIONES DE LOS TILES
# ============================================================

x_positions = list(
    range(
        0,
```

```
        max(
            w - tile_size + 1,
            1
        ),
        step
    )
)


y_positions = list(
    range(
        0,
        max(
            h - tile_size + 1,
            1
        ),
        step
    )
)


# ------------------------------------------------------------
# Asegurar cobertura de bordes
# ------------------------------------------------------------

last_x = max(
    0,
    w - tile_size
)


last_y = max(
    0,
    h - tile_size
)


if last_x not in x_positions:

    x_positions.append(
        last_x
    )


if last_y not in y_positions:

    y_positions.append(
        last_y
    )
```

```
# Eliminar posibles duplicados
x_positions = sorted(
    set(
        x_positions
    )
)


y_positions = sorted(
    set(
        y_positions
    )
)


# ============================================================
# 6. INFERENCIA TILE POR TILE
# ============================================================

for y0 in y_positions:

    for x0 in x_positions:

        x1 = (
            x0
        )

        y1 = (
            y0
        )


        x2 = min(
            w,
            x0 + tile_size
        )


        y2 = min(
            h,
            y0 + tile_size
        )


        tile = image[
            y1:y2,
            x1:x2
```

```
]


if (
    tile.size == 0
):

    continue


result = pedestrian_light_best.predict(

    source=tile,

    imgsz=960,

    conf=conf,

    verbose=False

)[0]


if (
    result.boxes is None
):

    continue


for box in (
    result.boxes
):

    confidence = float(
        box.conf[
            0
        ].item()
    )


    bx1, by1, bx2, by2 = (
        box.xyxy[
            0
        ]
        .cpu()
        .numpy()
    )
```

```
            # ------------------------------------------------
            # Coordenadas del tile
            # -> coordenadas de la imagen completa
            # ------------------------------------------------

            bx1 += x1

            bx2 += x1

            by1 += y1

            by2 += y1


            detections.append({

                "confidence":
                    confidence,

                "bbox":
                    (
                        int(
                            bx1
                        ),
                        int(
                            by1
                        ),
                        int(
                            bx2
                        ),
                        int(
                            by2
                        )
                    )
            })


# ============================================================
# 7. ORDENAR DETECCIONES
# ============================================================

detections = sorted(

    detections,

    key=lambda x:
        x[
            "confidence"
```

```
        ],

    reverse=True
)


# ============================================================
# 8. VISUALIZACIÓN
# ============================================================

output = (
    image_rgb.copy()
)


for detection in (
    detections
):

    x1, y1, x2, y2 = (
        detection[
            "bbox"
        ]
    )


    confidence = (
        detection[
            "confidence"
        ]
    )


    cv2.rectangle(

        output,

        (
            x1,
            y1
        ),

        (
            x2,
            y2
        ),

        (
            0,
```

```
            255,
            0
        ),

        3
    )


    cv2.putText(

        output,

        f"pedestrian_light "
        f"{confidence:.2f}",

        (
            x1,
            max(
                20,
                y1 - 8
            )
        ),

        cv2.FONT_HERSHEY_SIMPLEX,

        0.6,

        (
            0,
            255,
            0
        ),

        2,

        cv2.LINE_AA
    )


print()
print(
    "Detecciones encontradas:",
    len(
        detections
    )
)


display(
```

```
        Image.fromarray(
            output
        )
    )


    return detections


print()
print(
    "✓ Función tiled_pedestrian_light_detection preparada."
)
```

```
================================================================
==
13.15 — INFERENCIA POR TILES
================================================================
==

Dataset:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final

Modelo V1:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_best.pt

¿Existe dataset?: True
¿Existe modelo V1?: True

✓ Modelo V1 cargado correctamente.
✓ No se utiliza la antigua carpeta training.
✓ No se realizará ningún entrenamiento.

✓ Función tiled_pedestrian_light_detection preparada.
```

In [ ]:

```
# ============================================================
# 13.16 ENTRENAMIENTO V2
# YOLO11n + MAYOR RESOLUCIÓN (1280)
# ============================================================

from ultralytics import YOLO
import os

BASE_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"

DATASET_DIR = os.path.join(
    BASE_DIR,
```

```
    "dataset_pedestrian_traffic_light",
    "dataset_yolo_final"
)

DATA_YAML = os.path.join(
    DATASET_DIR,
    "data.yaml"
)

# ------------------------------------------------------------
# Modelo base COCO
# ------------------------------------------------------------

model_v2 = YOLO(
    "yolo11n.pt"
)

# ------------------------------------------------------------
# Entrenamiento
# ------------------------------------------------------------

results_v2 = model_v2.train(

    data=DATA_YAML,

    # Cambio principal respecto al V1
    imgsz=1280,

    epochs=100,

    patience=20,

    batch=4,

    device="cpu",

    workers=0,

    pretrained=True,

    project=os.path.join(
        DATASET_DIR,
        "training"
    ),

    name="yolo11n_pedestrian_light_v2_1280",

    exist_ok=False,
```

```
    plots=True,

    verbose=True,

    seed=42
)
```

In [187]:

```
# ============================================================
# 13.16.1 CONSERVACIÓN PERMANENTE DEL MODELO ENTRENADO V2
#         VERIFICACIÓN DE LA COPIA ESTABLE
# ============================================================

from pathlib import Path
import hashlib
import json
from datetime import datetime


# ============================================================
# 1. RAÍZ DEL TFM Y CARPETA ESTABLE DE MODELOS
# ============================================================

TFM_ROOT = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)

MODELS_DIR = (
    TFM_ROOT
    / "MODELOS_YOLO_FINALES_TFM"
)


if not MODELS_DIR.is_dir():

    raise FileNotFoundError(
        f"No existe la carpeta estable de modelos:\n"
        f"{MODELS_DIR}"
    )


# ============================================================
# 2. MODELOS V2 CONSERVADOS
# ============================================================

V2_BEST_PATH = (
    MODELS_DIR
    / "pedestrian_light_v2_best.pt"
)
```

```
V2_LAST_PATH = (
    MODELS_DIR
    / "pedestrian_light_v2_last.pt"
)


V2_MODELS = {

    "V2_best":
        V2_BEST_PATH,

    "V2_last":
        V2_LAST_PATH
}


# ============================================================
# 3. FUNCIÓN SHA-256
# ============================================================

def sha256_file(path):

    h = hashlib.sha256()

    with open(
        path,
        "rb"
    ) as f:

        for block in iter(
            lambda:
                f.read(
                    1024 * 1024
                ),
            b""
        ):

            h.update(
                block
            )

    return h.hexdigest()


# ============================================================
# 4. VERIFICACIÓN DE LOS PESOS V2
# ============================================================

print("=" * 72)
```

```
print("13.16.1 — VERIFICACIÓN PERMANENTE DEL MODELO V2")
print("=" * 72)

print()
print("Carpeta estable:")
print(MODELS_DIR)

print()


manifest_v2 = {

    "verification_date":
        datetime.now().isoformat(),

    "model_version":
        "V2",

    "description":
        "YOLO11n especializado para semáforo peatonal "
        "entrenado a mayor resolución (1280).",

    "models_directory":
        str(
            MODELS_DIR
        ),

    "models":
        {}
}


missing_models = []


for model_name, model_path in V2_MODELS.items():

    exists = (
        model_path.is_file()
    )

    print(
        f"{model_name}:"
    )

    print(
        f"  {model_path}"
    )
```

```
print(
    f"  Existe: {exists}"
)


if not exists:

    missing_models.append(
        model_name
    )

    print()

    continue


size_mb = (
    model_path.stat().st_size
    /
    (1024 ** 2)
)


sha256 = (
    sha256_file(
        model_path
    )
)


print(
    f"  Tamaño: {size_mb:.2f} MB"
)

print(
    f"  SHA-256: {sha256}"
)

print()


manifest_v2[
    "models"
][
    model_name
] = {

    "path":
        str(
```

```
                model_path
            ),

        "size_mb":
            round(
                size_mb,
                3
            ),

        "sha256":
            sha256
    }


# ============================================================
# 5. COMPROBACIÓN FINAL
# ============================================================

if missing_models:

    raise FileNotFoundError(
        "Faltan los siguientes pesos del modelo V2: "
        +
        ", ".join(
            missing_models
        )
    )


# ============================================================
# 6. GUARDAR MANIFIESTO DEL MODELO V2
# ============================================================

manifest_v2_path = (
    MODELS_DIR
    / "manifest_modelo_v2.json"
)


with open(
    manifest_v2_path,
    "w",
    encoding="utf-8"
) as f:

    json.dump(
        manifest_v2,
        f,
        indent=4,
```

```
        ensure_ascii=False
    )


# ============================================================
# 7. VARIABLES DEFINITIVAS PARA USO POSTERIOR
# ============================================================

V2_MODEL_PATH_FINAL = (
    V2_BEST_PATH
)


# ============================================================
# 8. RESULTADO
# ============================================================

print("=" * 72)
print("MODELO V2 CONSERVADO Y VERIFICADO")
print("=" * 72)

print()

print(
    "✓ pedestrian_light_v2_best.pt"
)

print(
    "✓ pedestrian_light_v2_last.pt"
)

print()

print(
    "✓ Modelo definitivo V2:"
)

print(
    V2_MODEL_PATH_FINAL
)

print()

print(
    "✓ No se ha realizado ningún entrenamiento."
)

print(
```

```
    "✓ No se depende de la antigua carpeta training."
)

print(
    "✓ Los pesos V2 se cargan desde MODELOS_YOLO_FINALES_TFM."
)

print()

print(
    "Archivo de control:"
)

print(
    manifest_v2_path
)
================================================================================
13.16.1 — VERIFICACIÓN PERMANENTE DEL MODELO V2
================================================================================

Carpeta estable:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM

V2_best:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_best.pt
  Existe: True
  Tamaño: 5.30 MB
  SHA-256:
07ce72073add35eb0d7f8165e77d211612706716cc7c25980dee7e98ab0c273c

V2_last:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_last.pt
  Existe: True
  Tamaño: 5.30 MB
  SHA-256:
df02244123e9f0072b556984958b6a2ed693bddb848b45330736c7c5f5f107a0

================================================================================
MODELO V2 CONSERVADO Y VERIFICADO
================================================================================

✓ pedestrian_light_v2_best.pt
```

```
✓ pedestrian_light_v2_last.pt

✓ Modelo definitivo V2:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_best.pt

✓ No se ha realizado ningún entrenamiento.
✓ No se depende de la antigua carpeta training.
✓ Los pesos V2 se cargan desde MODELOS_YOLO_FINALES_TFM.

Archivo de control:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\manifest_modelo_v2.json
```

In [189]:

```
# ============================================================
# 13.17 COMPARACIÓN V1 vs V2 EN VAL
#       BÚSQUEDA DEL UMBRAL DE CONFIANZA
#       MODELOS YA ENTRENADOS — SIN REENTRENAR
# ============================================================

from pathlib import Path

import cv2
import pandas as pd

from ultralytics import YOLO
from IPython.display import display


# ============================================================
# 1. RUTAS DEFINITIVAS
# ============================================================

TFM_DIR = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)


DATASET_DIR = (
    TFM_DIR
    / "dataset_pedestrian_traffic_light"
    / "dataset_yolo_final"
)


MODELS_DIR = (
    TFM_DIR
    / "MODELOS_YOLO_FINALES_TFM"
```

```
)


# ------------------------------------------------------------
# Modelos entrenados definitivos
# ------------------------------------------------------------

V1_PATH = (
    MODELS_DIR
    / "pedestrian_light_v1_best.pt"
)


V2_PATH = (
    MODELS_DIR
    / "pedestrian_light_v2_best.pt"
)


# ------------------------------------------------------------
# Conjunto de validación
# ------------------------------------------------------------

VAL_IMAGES_DIR = (
    DATASET_DIR
    / "images"
    / "val"
)


VAL_LABELS_DIR = (
    DATASET_DIR
    / "labels"
    / "val"
)


# ============================================================
# 2. COMPROBACIONES
# ============================================================

print("=" * 72)
print("13.17 — COMPARACIÓN V1 vs V2 SOBRE VAL")
print("=" * 72)

print()

print("Dataset:")
print(DATASET_DIR)
```

```
print()

print("Modelo V1:")
print(V1_PATH)

print()

print("Modelo V2:")
print(V2_PATH)

print()

print("Imágenes VAL:")
print(VAL_IMAGES_DIR)

print()

print("Labels VAL:")
print(VAL_LABELS_DIR)

print()

print(
    "¿Existe V1?:",
    V1_PATH.is_file()
)

print(
    "¿Existe V2?:",
    V2_PATH.is_file()
)

print(
    "¿Existe images/val?:",
    VAL_IMAGES_DIR.is_dir()
)

print(
    "¿Existe labels/val?:",
    VAL_LABELS_DIR.is_dir()
)


# ============================================================
# 3. VALIDACIÓN DE RUTAS
# ============================================================

if not V1_PATH.is_file():
```

```
    raise FileNotFoundError(
        f"No existe el modelo V1:\n{V1_PATH}"
    )


if not V2_PATH.is_file():

    raise FileNotFoundError(
        f"No existe el modelo V2:\n{V2_PATH}"
    )


if not VAL_IMAGES_DIR.is_dir():

    raise FileNotFoundError(
        f"No existe la carpeta de imágenes VAL:\n"
        f"{VAL_IMAGES_DIR}"
    )


if not VAL_LABELS_DIR.is_dir():

    raise FileNotFoundError(
        f"No existe la carpeta de labels VAL:\n"
        f"{VAL_LABELS_DIR}"
    )


# ============================================================
# 4. CARGAR LOS DOS MODELOS YA ENTRENADOS
# ============================================================

model_v1 = YOLO(
    str(V1_PATH)
)


model_v2 = YOLO(
    str(V2_PATH)
)


print()

print(
    "✓ Modelo V1 cargado."
)
```

```
print(
    "✓ Modelo V2 cargado."
)

print(
    "✓ No se utiliza la antigua carpeta training."
)

print(
    "✓ No se realizará ningún entrenamiento."
)


# ============================================================
# 5. IoU ENTRE DOS BOXES
# ============================================================

def iou_boxes(a, b):

    ax1, ay1, ax2, ay2 = a
    bx1, by1, bx2, by2 = b


    ix1 = max(
        ax1,
        bx1
    )

    iy1 = max(
        ay1,
        by1
    )

    ix2 = min(
        ax2,
        bx2
    )

    iy2 = min(
        ay2,
        by2
    )


    iw = max(
        0,
        ix2 - ix1
    )
```

```
ih = max(
    0,
    iy2 - iy1
)


intersection = (
    iw * ih
)


area_a = (
    max(
        0,
        ax2 - ax1
    )
    *
    max(
        0,
        ay2 - ay1
    )
)


area_b = (
    max(
        0,
        bx2 - bx1
    )
    *
    max(
        0,
        by2 - by1
    )
)


union = (
    area_a
    +
    area_b
    -
    intersection
)


if union <= 0:

    return 0.0
```

```
    return (
        intersection
        /
        union
    )


# ============================================================
# 6. LEER GROUND TRUTH YOLO
# ============================================================

def read_gt_boxes(
    image_path,
    label_path
):

    image = cv2.imread(
        str(image_path)
    )


    if image is None:

        raise ValueError(
            f"No se pudo abrir la imagen:\n"
            f"{image_path}"
        )


    h, w = (
        image.shape[:2]
    )


    boxes = []


    label_path = Path(
        label_path
    )


    if not label_path.is_file():

        return boxes
```

```
with open(
    label_path,
    "r",
    encoding="utf-8"
) as f:

    for line in f:

        line = (
            line.strip()
        )


        if not line:

            continue


        parts = (
            line.split()
        )


        # Formato YOLO:
        # class xc yc width height
        if len(parts) < 5:

            continue


        xc = float(
            parts[1]
        )

        yc = float(
            parts[2]
        )

        bw = float(
            parts[3]
        )

        bh = float(
            parts[4]
        )


        x1 = (
            xc - bw / 2
```

```
            ) * w

            y1 = (
                yc - bh / 2
            ) * h

            x2 = (
                xc + bw / 2
            ) * w

            y2 = (
                yc + bh / 2
            ) * h


            boxes.append(
                (
                    x1,
                    y1,
                    x2,
                    y2
                )
            )


    return boxes


# ============================================================
# 7. EVALUACIÓN DE UN MODELO
# ============================================================

def evaluate_model(
    model,
    imgsz,
    conf_threshold,
    iou_threshold=0.50
):

    TP = 0
    FP = 0
    FN = 0


    image_names = sorted(
        [
            f.name
            for f in VAL_IMAGES_DIR.iterdir()
            if (
```

```
            f.is_file()
            and
            f.suffix.lower()
            in (
                ".jpg",
                ".jpeg",
                ".png"
            )
        )
    ]
)


for image_name in image_names:

    image_path = (
        VAL_IMAGES_DIR
        /
        image_name
    )


    label_path = (
        VAL_LABELS_DIR
        /
        (
            image_path.stem
            +
            ".txt"
        )
    )


    # ----------------------------------------------------
    # Ground truth
    # ----------------------------------------------------

    gt_boxes = (
        read_gt_boxes(
            image_path,
            label_path
        )
    )


    # ----------------------------------------------------
    # Inferencia
    # ----------------------------------------------------
```

```
result = model.predict(

    source=str(
        image_path
    ),

    imgsz=imgsz,

    conf=conf_threshold,

    verbose=False

)[0]


pred_boxes = []


if result.boxes is not None:

    for box in result.boxes:

        x1, y1, x2, y2 = (

            box.xyxy[
                0
            ]

            .cpu()

            .numpy()
        )


        pred_boxes.append(
            (
                float(x1),
                float(y1),
                float(x2),
                float(y2)
            )
        )


# ====================================================
# 8. MATCHING GREEDY GT <-> PREDICCIÓN
# ====================================================

matched_gt = set()
```

```
matched_pred = set()

candidates = []


for gi, gt in enumerate(
    gt_boxes
):

    for pi, pred in enumerate(
        pred_boxes
    ):

        iou = (
            iou_boxes(
                gt,
                pred
            )
        )


        if iou >= iou_threshold:

            candidates.append(
                (
                    iou,
                    gi,
                    pi
                )
            )


candidates.sort(
    reverse=True,
    key=lambda x: x[0]
)


for iou, gi, pi in candidates:

    if gi in matched_gt:

        continue


    if pi in matched_pred:

        continue
```

```
            matched_gt.add(
                gi
            )

            matched_pred.add(
                pi
            )


        TP += len(
            matched_gt
        )


        FP += (
            len(
                pred_boxes
            )
            -
            len(
                matched_pred
            )
        )


        FN += (
            len(
                gt_boxes
            )
            -
            len(
                matched_gt
            )
        )


# ============================================================
# 9. MÉTRICAS
# ============================================================

precision = (

    TP
    /
    (
        TP + FP
    )
```

```
    if (
        TP + FP
    ) > 0

    else 0.0
)


recall = (

    TP
    /
    (
        TP + FN
    )

    if (
        TP + FN
    ) > 0

    else 0.0
)


f1 = (

    2
    *
    precision
    *
    recall
    /
    (
        precision
        +
        recall
    )

    if (
        precision
        +
        recall
    ) > 0

    else 0.0
)
```

```
    return {

        "TP":
            TP,

        "FP":
            FP,

        "FN":
            FN,

        "Precision":
            precision,

        "Recall":
            recall,

        "F1":
            f1
    }


# ============================================================
# 10. COMPARACIÓN DE UMBRALES
# ============================================================

thresholds = [

    0.01,

    0.05,

    0.10,

    0.20,

    0.30,

    0.40
]


rows = []


for conf in thresholds:

    print(
        f"\nEvaluando conf={conf:.2f}..."
```

```
)


# ------------------------------------------------------------
# V1 — resolución 960
# ------------------------------------------------------------

result_v1 = evaluate_model(

    model_v1,

    imgsz=960,

    conf_threshold=conf
)


rows.append({

    "Modelo":
        "V1_960",

    "Conf":
        conf,

    **result_v1
})


# ------------------------------------------------------------
# V2 — resolución 1280
# ------------------------------------------------------------

result_v2 = evaluate_model(

    model_v2,

    imgsz=1280,

    conf_threshold=conf
)


rows.append({

    "Modelo":
        "V2_1280",

    "Conf":
```

```
            conf,

        **result_v2
    })


# ============================================================
# 11. TABLA COMPARATIVA
# ============================================================

comparison_df = pd.DataFrame(
    rows
)


print()

print("=" * 70)

print(
    "COMPARACIÓN V1 vs V2 SOBRE VAL"
)

print(
    "IoU mínimo = 0.50"
)

print("=" * 70)


display(
    comparison_df
)


# ============================================================
# 12. MEJOR CONFIGURACIÓN SEGÚN F1
# ============================================================

best_configuration = (

    comparison_df

    .sort_values(

        [
            "F1",
            "Recall",
            "Precision"
```

```
        ],

        ascending=[
            False,
            False,
            False
        ]
    )

    .iloc[
        0
    ]
)


print()

print("=" * 70)

print(
    "MEJOR CONFIGURACIÓN EN VAL"
)

print("=" * 70)


print(
    best_configuration
)


# ============================================================
# 13. VARIABLES PARA LAS CELDAS POSTERIORES
# ============================================================

# Compatibilidad con las celdas posteriores del notebook
V1_MODEL_PATH_FINAL = V1_PATH

V2_MODEL_PATH_FINAL = V2_PATH


# Evitamos que una variable BEST_MODEL_PATH antigua
# vuelva a apuntar a la desaparecida carpeta training.
BEST_MODEL_PATH = V1_MODEL_PATH_FINAL


print()

print("=" * 70)
```

```
print(
    "13.17 FINALIZADO"
)

print("=" * 70)

print(
    "✓ V1 y V2 cargados desde MODELOS_YOLO_FINALES_TFM."
)

print(
    "✓ Comparación realizada sobre VAL."
)

print(
    "✓ No se ha realizado ningún entrenamiento."
)

print(
    "✓ Las antiguas rutas training no se utilizan."
)
```

```
======================================================================
13.17 — COMPARACIÓN V1 vs V2 SOBRE VAL
======================================================================

Dataset:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final

Modelo V1:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_best.pt

Modelo V2:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_best.pt

Imágenes VAL:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\images\val

Labels VAL:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\labels\val
```

```
¿Existe V1?: True
¿Existe V2?: True
¿Existe images/val?: True
¿Existe labels/val?: True

✓ Modelo V1 cargado.
✓ Modelo V2 cargado.
✓ No se utiliza la antigua carpeta training.
✓ No se realizará ningún entrenamiento.

Evaluando conf=0.01...

Evaluando conf=0.05...

Evaluando conf=0.10...

Evaluando conf=0.20...

Evaluando conf=0.30...

Evaluando conf=0.40...

======================================================================
COMPARACIÓN V1 vs V2 SOBRE VAL
IoU mínimo = 0.50
======================================================================
```

	Modelo	Conf	TP	FP	FN	Precision	Recall	F1
0	V1_960	0.01	5	83	2	0.056818	0.714286	0.105263
1	V2_1280	0.01	1	195	6	0.005102	0.142857	0.009852
2	V1_960	0.05	3	17	4	0.150000	0.428571	0.222222
3	V2_1280	0.05	1	48	6	0.020408	0.142857	0.035714
4	V1_960	0.10	2	5	5	0.285714	0.285714	0.285714
5	V2_1280	0.10	1	30	6	0.032258	0.142857	0.052632
6	V1_960	0.20	1	1	6	0.500000	0.142857	0.222222
7	V2_1280	0.20	1	13	6	0.071429	0.142857	0.095238
8	V1_960	0.30	1	0	6	1.000000	0.142857	0.250000
9	V2_1280	0.30	1	6	6	0.142857	0.142857	0.142857
10	V1_960	0.40	0	0	7	0.000000	0.000000	0.000000
11	V2_1280	0.40	1	4	6	0.200000	0.142857	0.166667

```
======================================================================
MEJOR CONFIGURACIÓN EN VAL
======================================================================
Modelo         V1_960
```

```
Conf              0.1
TP                  2
FP                  5
FN                  5
Precision    0.285714
Recall       0.285714
F1           0.285714
Name: 4, dtype: object

========================================================================
13.17 FINALIZADO
========================================================================
✓ V1 y V2 cargados desde MODELOS_YOLO_FINALES_TFM.
✓ Comparación realizada sobre VAL.
✓ No se ha realizado ningún entrenamiento.
✓ Las antiguas rutas training no se utilizan.
```

In [190]:

```
# 13.18 Comprobación visual del modelo seleccionado en validación


from PIL import Image
import os
import cv2

FINAL_CONF = 0.30

val_images = sorted([
    f for f in os.listdir(VAL_IMAGES_DIR)
    if f.lower().endswith(
        (".jpg", ".jpeg", ".png")
    )
])

total_detections = 0

for image_name in val_images:

    image_path = os.path.join(
        VAL_IMAGES_DIR,
        image_name
    )

    result = model_v1.predict(
        source=image_path,
        imgsz=960,
        conf=FINAL_CONF,
        verbose=False
    )[0]
```

```
    n_detections = len(
        result.boxes
    )

    total_detections += n_detections

    print("\n" + "=" * 70)
    print("Imagen:", image_name)
    print(
        "Detecciones:",
        n_detections
    )

    annotated = result.plot()

    annotated_rgb = cv2.cvtColor(
        annotated,
        cv2.COLOR_BGR2RGB
    )

    display(
        Image.fromarray(
            annotated_rgb
        )
    )


print("\n" + "=" * 70)
print("RESUMEN FINAL")
print("=" * 70)

print(
    "Imágenes VAL:",
    len(val_images)
)

print(
    "Detecciones totales:",
    total_detections
)

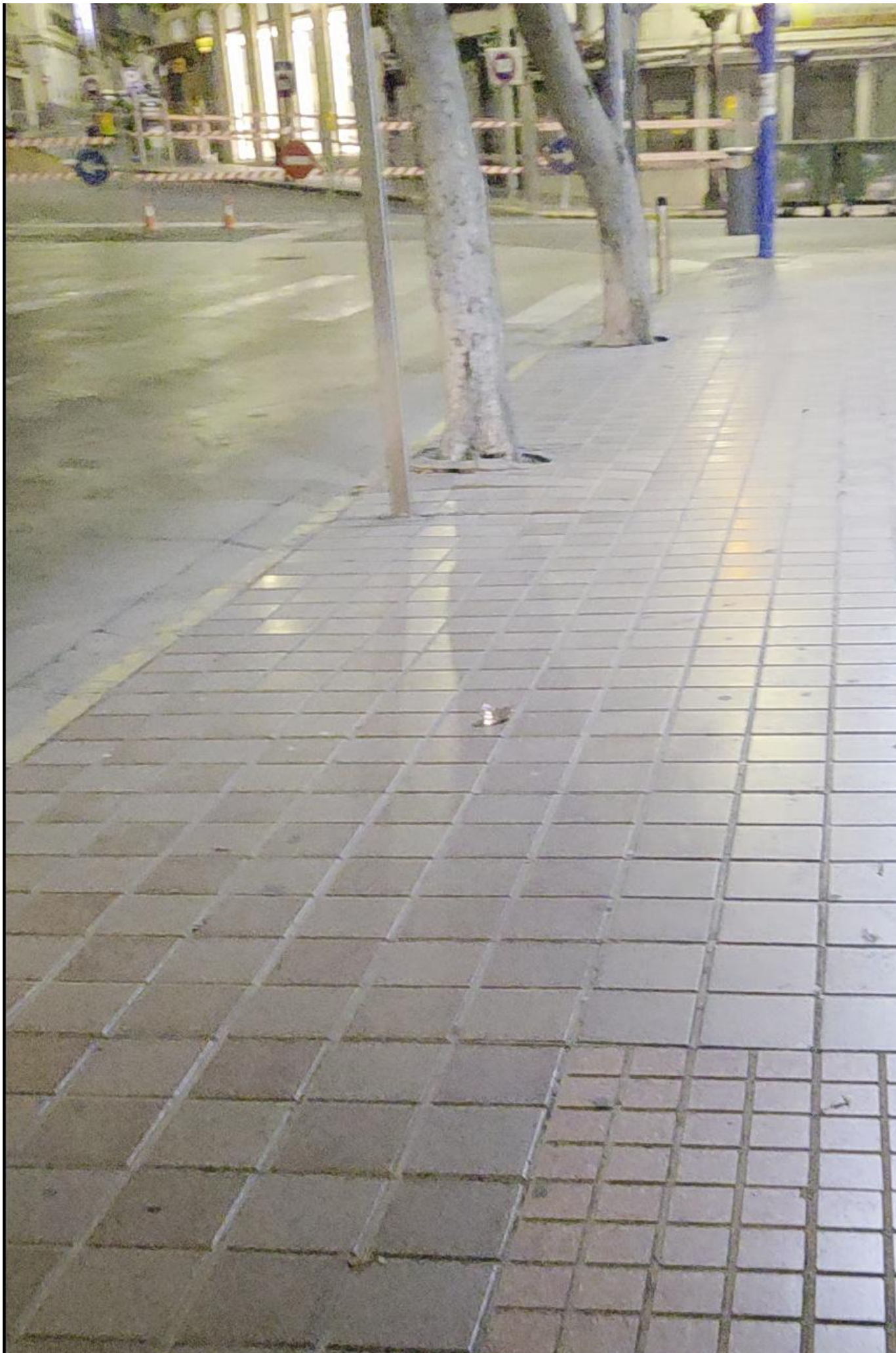
print(
    "Modelo seleccionado: V1 YOLO11n"
)

print(
    "imgsz: 960"
)
```

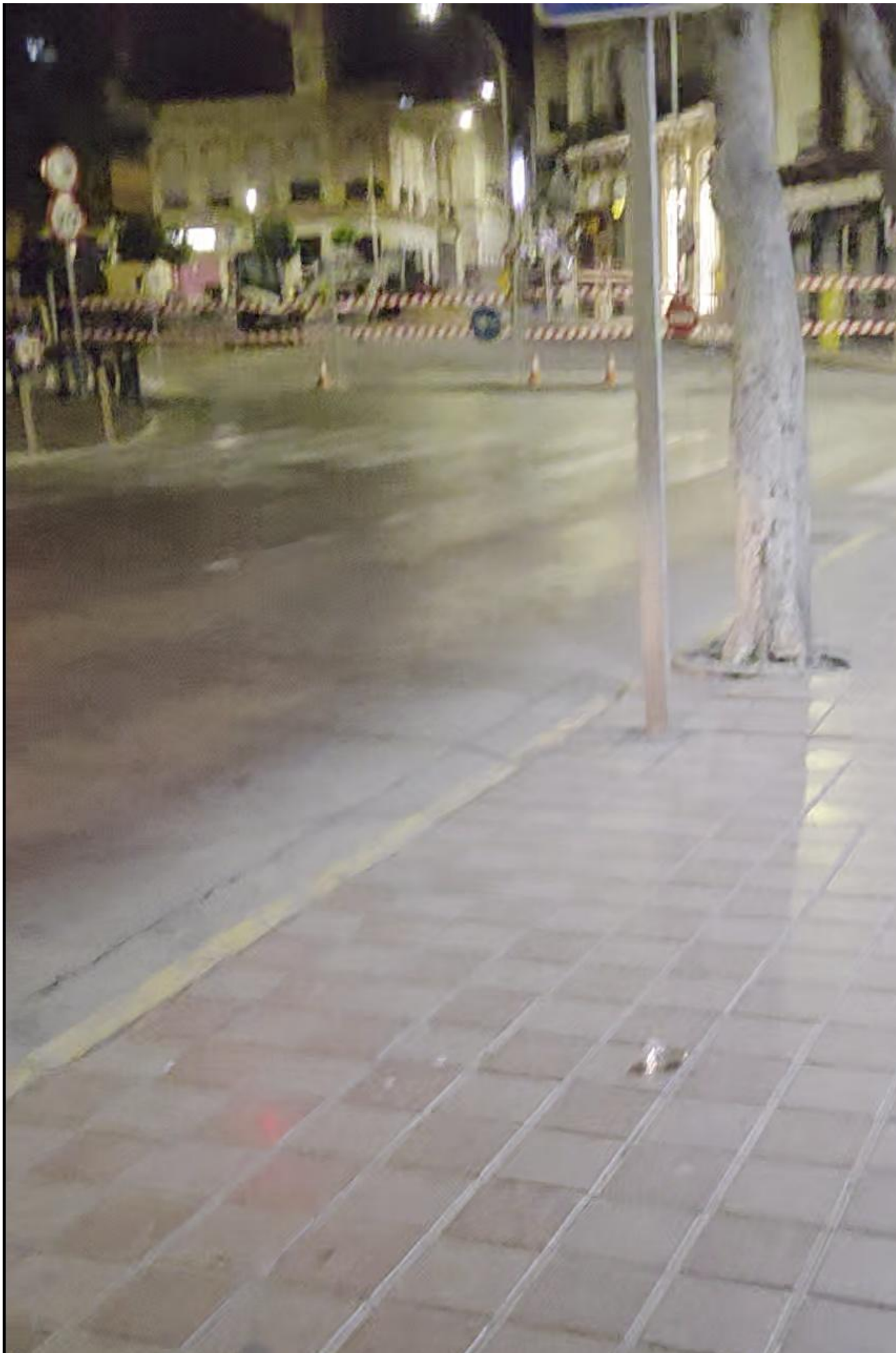
```
print(
    "conf final:",
    FINAL_CONF
)

print("=" * 70)
```

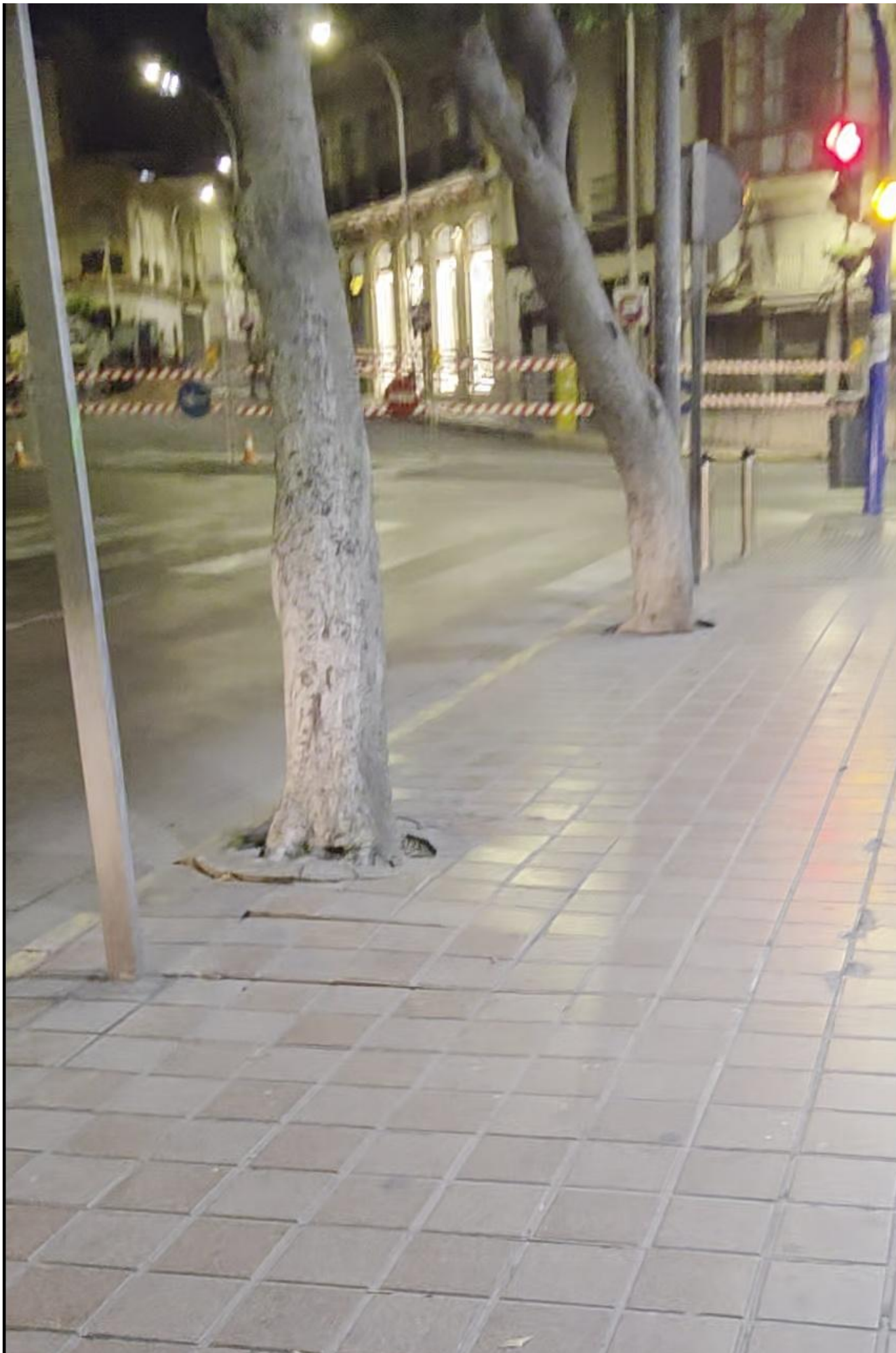
```
======================================================================
Imagen: 1000179711_frame_000000.jpg
Detecciones: 0
```



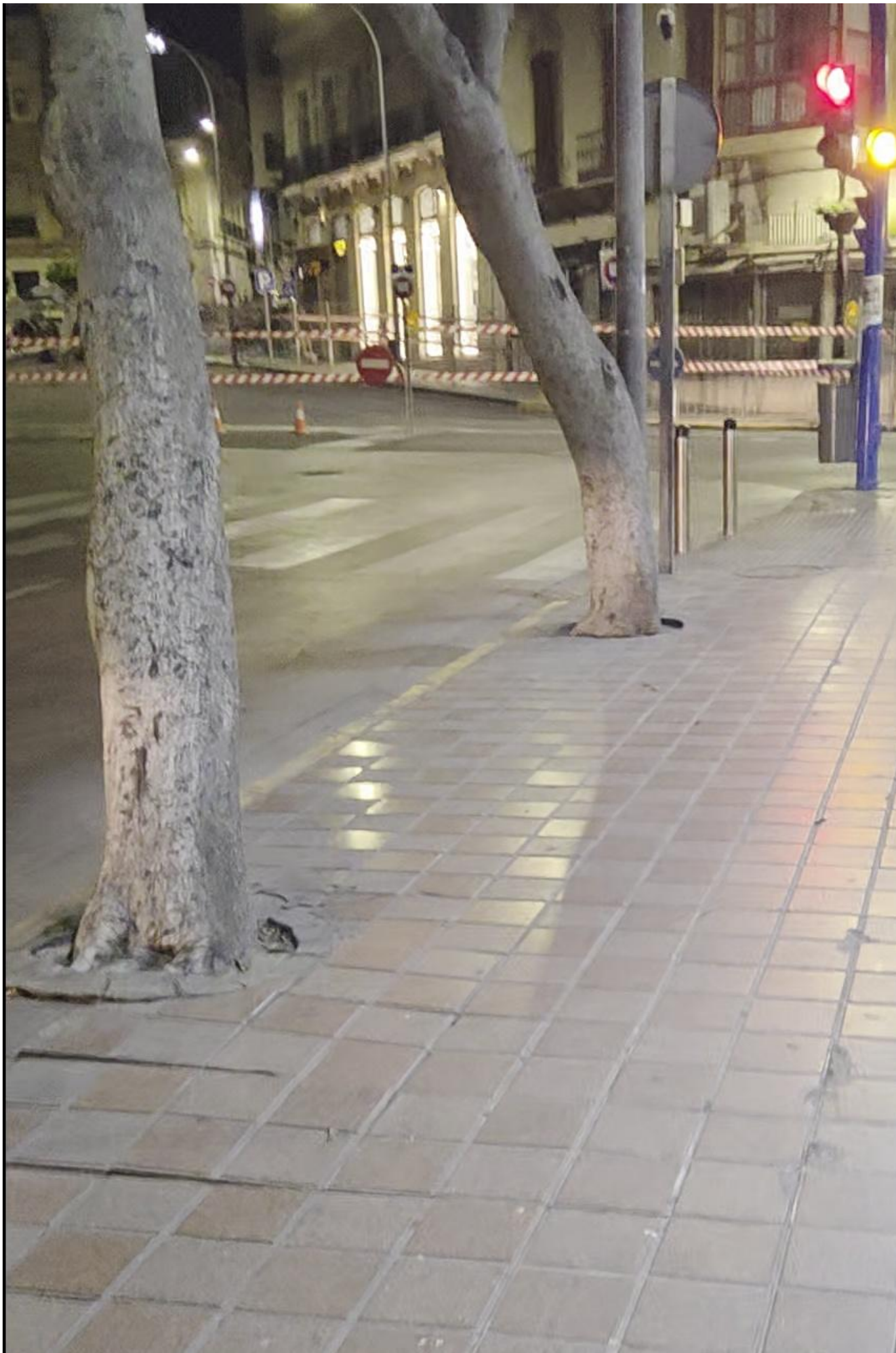
```
================================================================
Imagen: 1000179711_frame_000030.jpg
Detecciones: 0
```



```
======================================================================
Imagen: 1000179711_frame_000075.jpg
Detecciones: 0
```



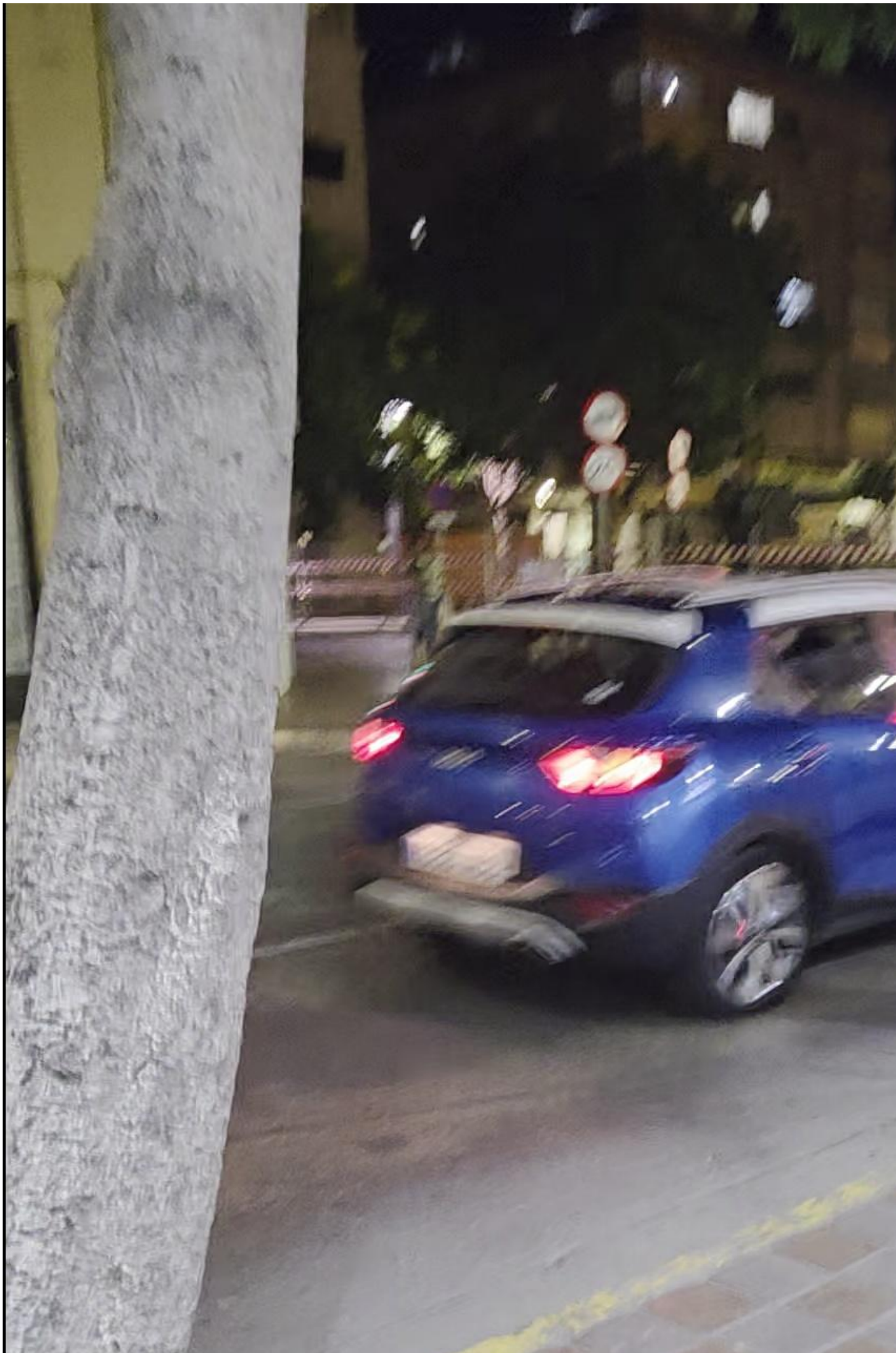
```
======================================================================
Imagen: 1000179711_frame_000105.jpg
Detecciones: 0
```



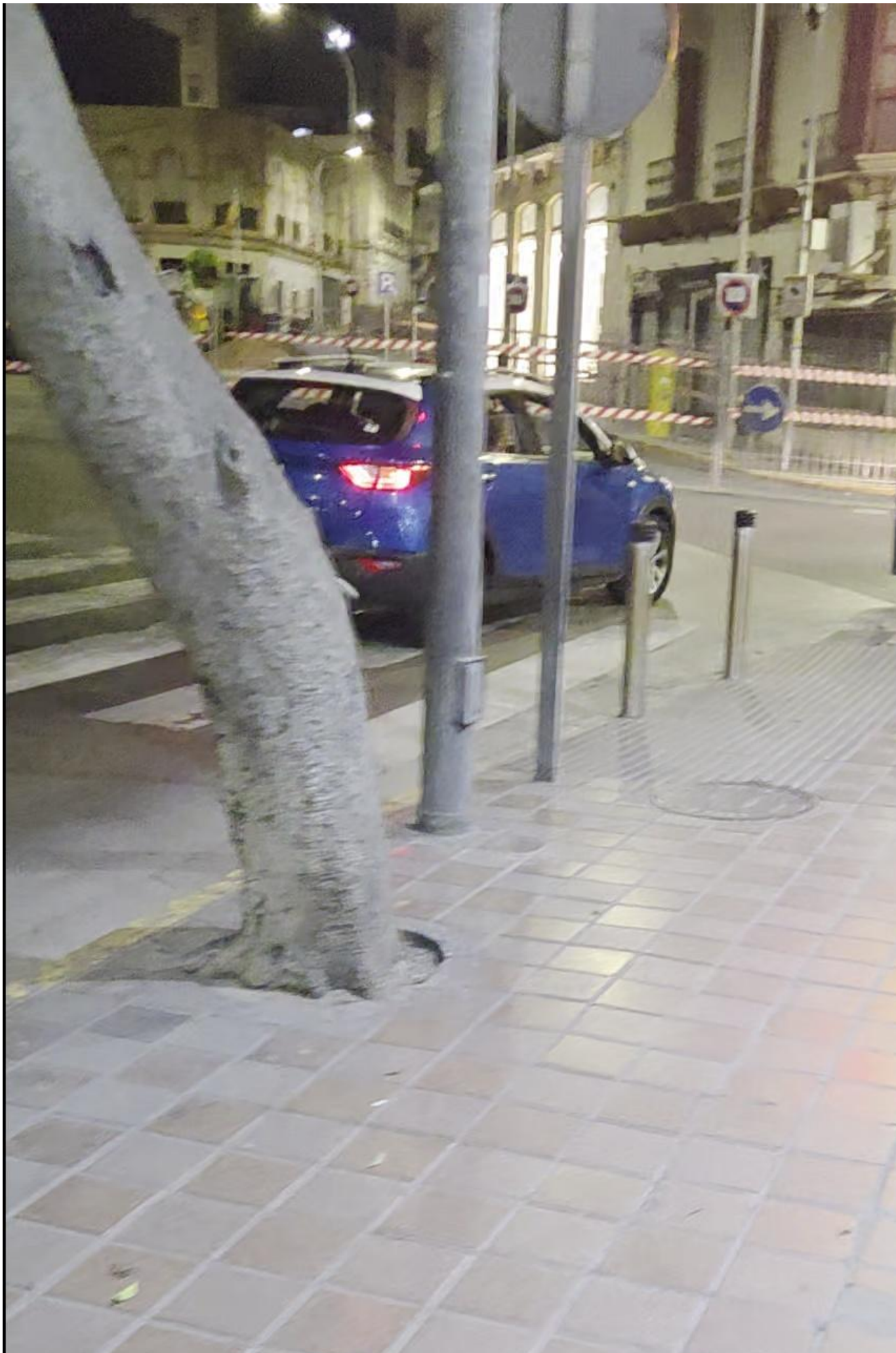
```
======================================================================
Imagen: 1000179711_frame_000150.jpg
Detecciones: 0
```



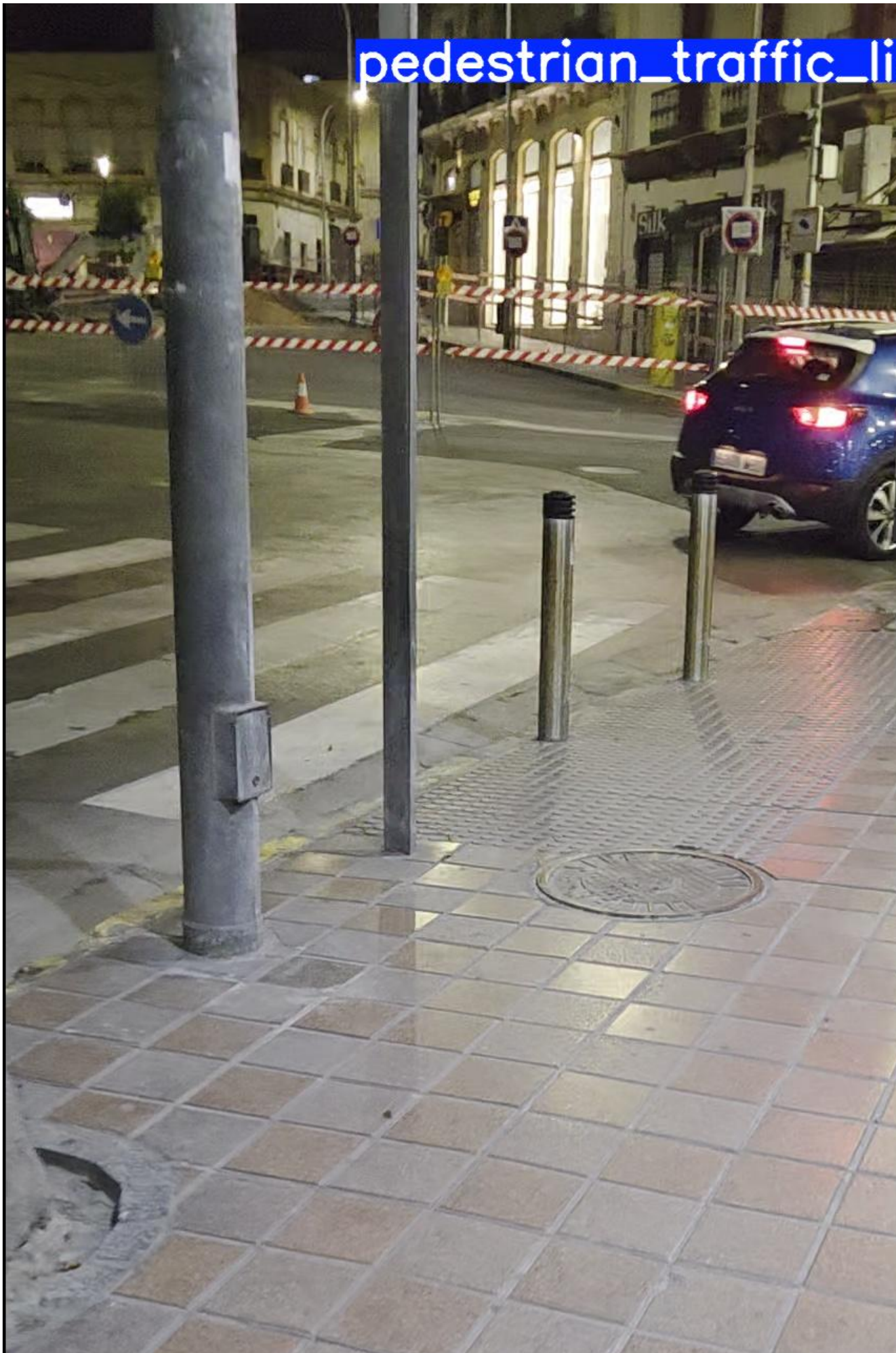
```
======================================================================
Imagen: 1000179711_frame_000180.jpg
Detecciones: 0
```



```
======================================================================
Imagen: 1000179711_frame_000225.jpg
Detecciones: 0
```



```
======================================================================
Imagen: 1000179711_frame_000270.jpg
Detecciones: 1
```


pedestrian_traffic_li

```
================================================================
Imagen: 1000179711_frame_000300.jpg
Detecciones: 0
```



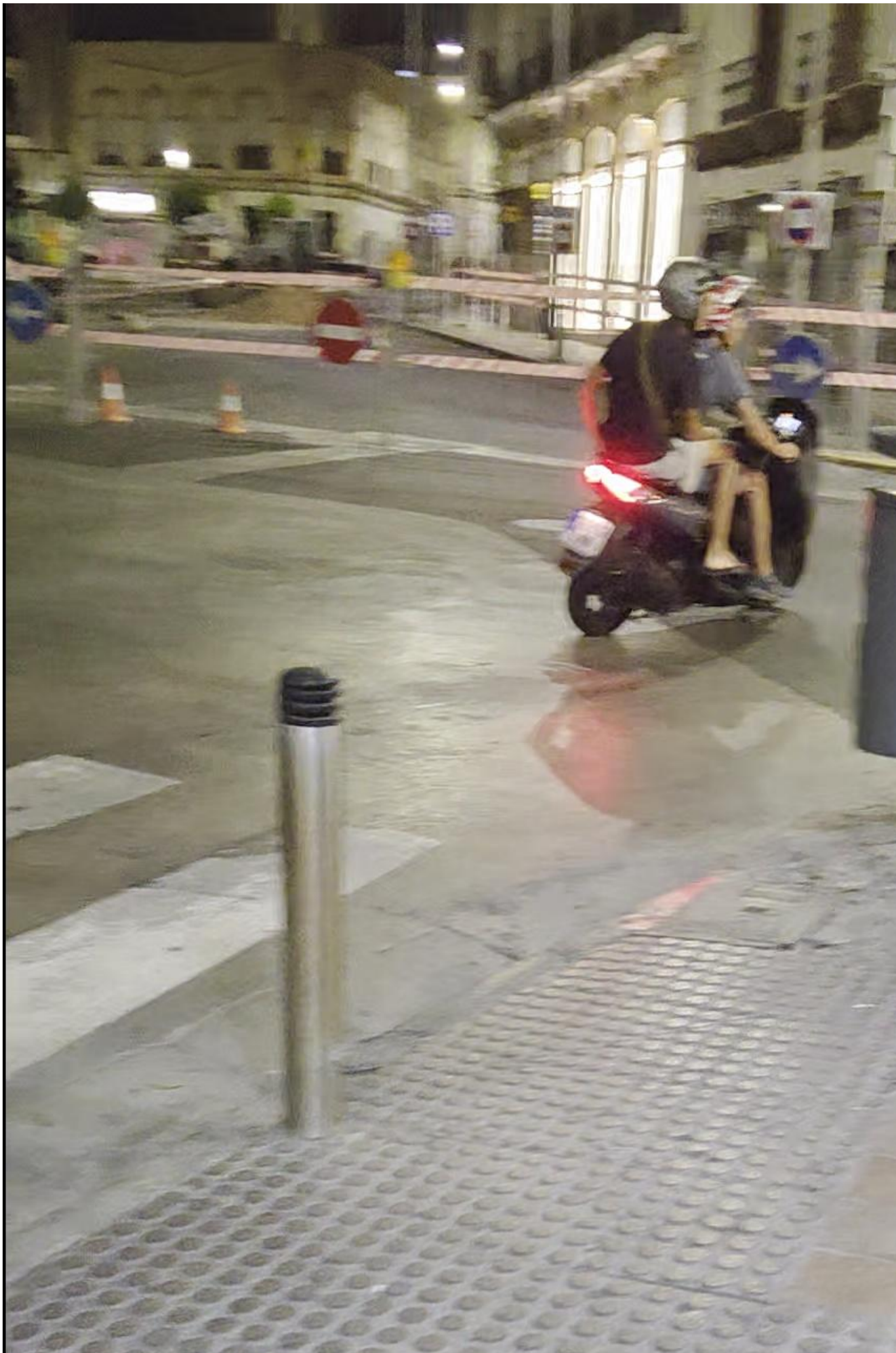
```
======================================================================
Imagen: 1000179711_frame_000345.jpg
Detecciones: 0
```


P
silk

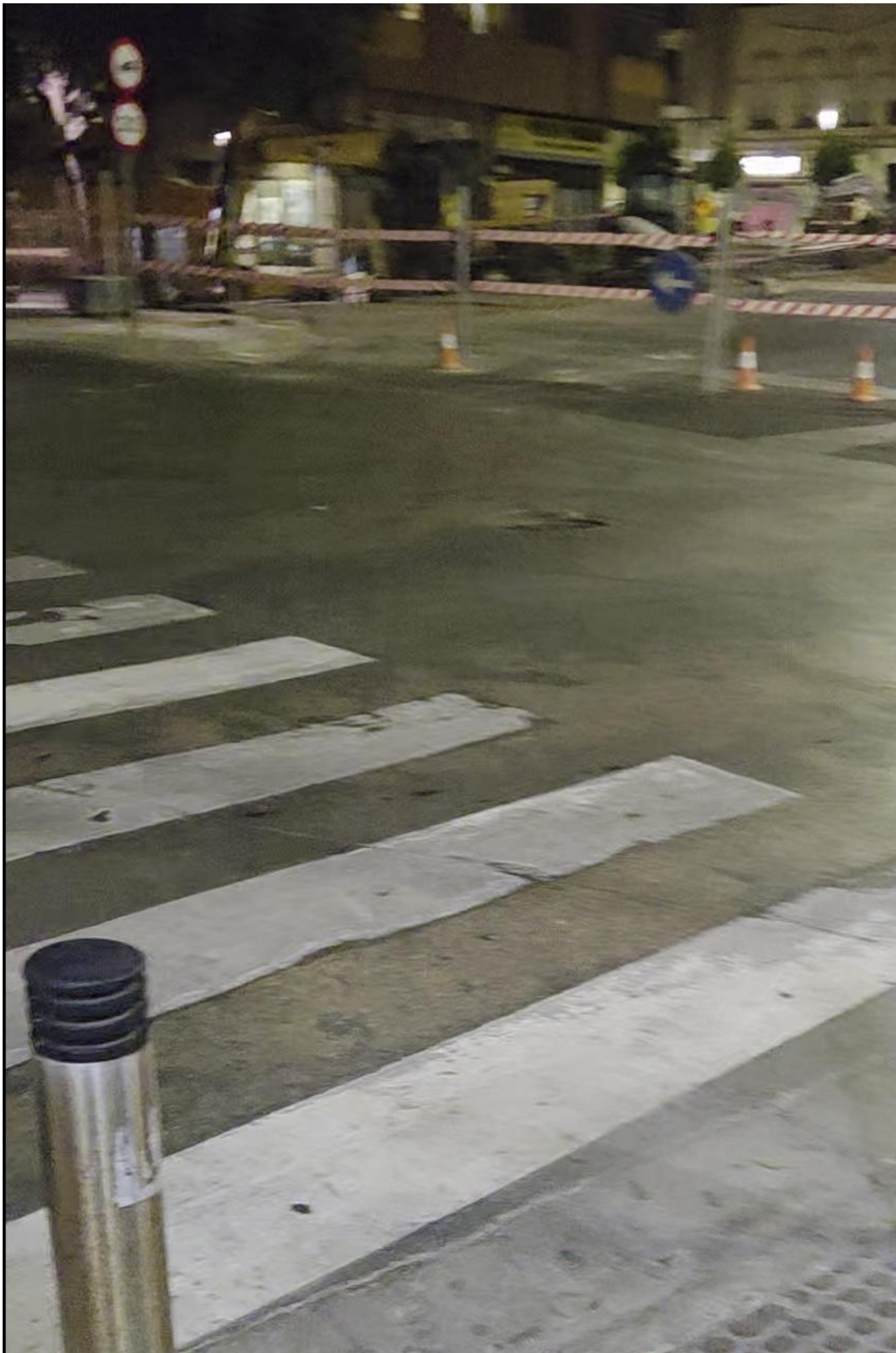
```
======================================================================
Imagen: 1000179711_frame_000375.jpg
Detecciones: 0
```



```
======================================================================
Imagen: 1000179711_frame_000420.jpg
Detecciones: 0
```



```
======================================================================
Imagen: 1000179711_frame_000450.jpg
Detecciones: 0
```



```
======================================================================
Imagen: 1000179711_frame_000495.jpg
Detecciones: 0
```


ME
DUTYFRE

```
================================================================
Imagen: 1000179711_frame_000540.jpg
Detecciones: 0
```



```
======================================================================
RESUMEN FINAL
======================================================================
Imágenes VAL: 15
Detecciones totales: 1
Modelo seleccionado: V1 YOLO11n
imgsz: 960
conf final: 0.3
======================================================================
```

14. Clasificación semántica del semáforo peatonal mediante CLIP

In [8]:

```
# 14.1 FILTRO SEMÁNTICO FINAL DE SEMÁFOROS PEATONALES
# YOLO V1 + CLIP


GREEN_FINAL_PROMPTS = [
    "a pedestrian crossing signal showing a green walking person",
    "a green pedestrian walk signal with a visible human figure",
    "a pedestrian traffic light indicating walk"
]

RED_FINAL_PROMPTS = [
    "a pedestrian crossing signal showing a red standing person",
    "a red pedestrian stop signal with a visible human figure",
    "a pedestrian traffic light indicating stop"
]

NON_ACTIONABLE_PROMPTS = [
    "a vehicle traffic light",
    "the side or back of a traffic signal",
    "a traffic signal without a visible pedestrian symbol"
]

FINAL_SIGNAL_PROMPTS = (
    GREEN_FINAL_PROMPTS
    + RED_FINAL_PROMPTS
    + NON_ACTIONABLE_PROMPTS
)


def classify_final_signal(crop):

    result = classify_with_clip(
        crop,
```

```
        FINAL_SIGNAL_PROMPTS
    )

    probs = result["all_probabilities"]

    # Promedio por familia para evitar sesgos
    green_score = np.mean([
        probs[p]
        for p in GREEN_FINAL_PROMPTS
    ])

    red_score = np.mean([
        probs[p]
        for p in RED_FINAL_PROMPTS
    ])

    non_actionable_score = np.mean([
        probs[p]
        for p in NON_ACTIONABLE_PROMPTS
    ])

    scores = {
        "GREEN": green_score,
        "RED": red_score,
        "NON_ACTIONABLE": non_actionable_score
    }

    best_label = max(
        scores,
        key=scores.get
    )

    return {
        "label": best_label,
        "green": green_score,
        "red": red_score,
        "non_actionable": non_actionable_score
    }


print("Filtro semántico preparado.")
```

```
Filtro semántico preparado.
```

In [11]:

```
# DATASET YOLO DEFINITIVO


import os

DATASET_DIR = (
```

```
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
    r"\dataset_pedestrian_traffic_light"
    r"\dataset_yolo_final"
)

TRAIN_IMAGES_DIR = os.path.join(
    DATASET_DIR,
    "images",
    "train"
)

TRAIN_LABELS_DIR = os.path.join(
    DATASET_DIR,
    "labels",
    "train"
)

VAL_IMAGES_DIR = os.path.join(
    DATASET_DIR,
    "images",
    "val"
)

VAL_LABELS_DIR = os.path.join(
    DATASET_DIR,
    "labels",
    "val"
)

TEST_IMAGES_DIR = os.path.join(
    DATASET_DIR,
    "images",
    "test"
)

TEST_LABELS_DIR = os.path.join(
    DATASET_DIR,
    "labels",
    "test"
)

print("DATASET_DIR:", DATASET_DIR)
print("TRAIN:", TRAIN_IMAGES_DIR)
print("VAL:", VAL_IMAGES_DIR)
print("TEST:", TEST_IMAGES_DIR)
DATASET_DIR: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final
```

```
TRAIN: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\images\trai
n
VAL: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\images\val
TEST: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\images\test
```

In [12]:

```
# 14.2 Preparación de ejemplos anotados para validación semántica

# BUSCAR EJEMPLOS POSITIVOS EN TRAIN

TRAIN_IMAGES_DIR = os.path.join(
    DATASET_DIR,
    "images",
    "train"
)

TRAIN_LABELS_DIR = os.path.join(
    DATASET_DIR,
    "labels",
    "train"
)

positive_train_images = []

for image_name in sorted(os.listdir(TRAIN_IMAGES_DIR)):

    if not image_name.lower().endswith(
        (".jpg", ".jpeg", ".png")
    ):
        continue

    stem = os.path.splitext(image_name)[0]

    label_path = os.path.join(
        TRAIN_LABELS_DIR,
        stem + ".txt"
    )

    if not os.path.exists(label_path):
        continue

    with open(
        label_path,
        "r",
        encoding="utf-8"
    ) as f:
```

```
        lines = [
            line.strip()
            for line in f
            if line.strip()
        ]

    if len(lines) > 0:
        positive_train_images.append(image_name)


print(
    "Imágenes positivas disponibles en TRAIN:",
    len(positive_train_images)
)

print(
    positive_train_images[:30]
)
```

```
Imágenes positivas disponibles en TRAIN: 69
['1000179707_frame_000000.jpg', '1000179707_frame_000042.jpg',
'1000179707_frame_000084.jpg', '1000179707_frame_000140.jpg',
'1000179707_frame_000182.jpg', '1000179707_frame_000238.jpg',
'1000179707_frame_000280.jpg', '1000179707_frame_000336.jpg',
'1000179707_frame_000378.jpg', '1000179707_frame_000574.jpg',
'1000179709_frame_000000.jpg', '1000179709_frame_000070.jpg',
'1000179709_frame_000098.jpg', '1000179709_frame_000140.jpg',
'1000179709_frame_000182.jpg', '1000179709_frame_000210.jpg',
'1000179709_frame_000252.jpg', '1000179709_frame_000294.jpg',
'1000179709_frame_000322.jpg', '1000179709_frame_000476.jpg',
'1000179709_frame_000518.jpg', '20260611_184739_frame_000000.jpg',
'20260611_184739_frame_000028.jpg',
'20260611_184739_frame_000056.jpg',
'20260611_184739_frame_000084.jpg',
'20260611_184739_frame_000112.jpg',
'20260611_184739_frame_000140.jpg',
'20260611_184739_frame_000168.jpg',
'20260611_184739_frame_000210.jpg',
'20260611_184739_frame_000238.jpg']
```

In [13]:

```
# MOSTRAR SEMÁFOROS PEATONALES ANOTADOS EN TRAIN


import matplotlib.pyplot as plt
import math

train_crops = []

for image_name in positive_train_images:
```

```
image_path = os.path.join(
    TRAIN_IMAGES_DIR,
    image_name
)

stem = os.path.splitext(image_name)[0]

label_path = os.path.join(
    TRAIN_LABELS_DIR,
    stem + ".txt"
)

image = cv2.imread(image_path)

if image is None:
    continue

image_rgb = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)

h, w = image.shape[:2]

with open(
    label_path,
    "r",
    encoding="utf-8"
) as f:

    lines = [
        line.strip()
        for line in f
        if line.strip()
    ]

for line in lines:

    parts = line.split()

    xc = float(parts[1])
    yc = float(parts[2])
    bw = float(parts[3])
    bh = float(parts[4])

    x1 = int((xc - bw / 2) * w)
    y1 = int((yc - bh / 2) * h)
    x2 = int((xc + bw / 2) * w)
    y2 = int((yc + bh / 2) * h)
```

```
        margin = 30

        x1 = max(0, x1 - margin)
        y1 = max(0, y1 - margin)
        x2 = min(w, x2 + margin)
        y2 = min(h, y2 + margin)

        crop = image_rgb[
            y1:y2,
            x1:x2
        ]

        train_crops.append({
            "id": len(train_crops),
            "image": image_name,
            "crop": crop
        })


# Mostrar los primeros 40
items_to_show = train_crops[:40]

cols = 5
rows = math.ceil(
    len(items_to_show) / cols
)

plt.figure(
    figsize=(15, rows * 3)
)

for i, item in enumerate(items_to_show):

    plt.subplot(
        rows,
        cols,
        i + 1
    )

    plt.imshow(
        item["crop"]
    )

    plt.title(
        f"ID {item['id']}",
        fontsize=10
    )
```

```
    plt.axis("off")

plt.tight_layout()
plt.show()

print(
    "Total de objetos anotados en TRAIN:",
    len(train_crops)
)
```

ID 0



ID 1



ID 2



ID 3



ID 4



ID 5



ID 6



ID 7



ID 8



ID 9



ID 10



ID 11



ID 12



ID 13



ID 14



ID 15



ID 16



ID 17



ID 18



ID 19



ID 20



ID 21



ID 22



ID 23



ID 24



ID 25



ID 26



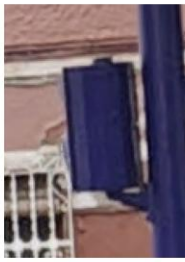
ID 27



ID 28



ID 29



ID 30



ID 31



ID 32



ID 33



ID 34



ID 35



ID 36



ID 37



ID 38



ID 39



```
Total de objetos anotados en TRAIN: 99
```

In [195]:

```
# 14.3 VALIDACIÓN SEMÁNTICA DE CLIP SOBRE CASOS CONTROLADOS


CONTROL_IDS = {
    12: "RED",
    31: "GREEN",
    26: "NON_ACTIONABLE"
}

for crop_id, expected in CONTROL_IDS.items():

    item = train_crops[crop_id]

    crop_pil = Image.fromarray(
        item["crop"]
    )

    result = classify_final_signal(
        crop_pil
    )

    print("\n" + "=" * 60)
    print("ID:", crop_id)
    print("Esperado:", expected)
    print("CLIP:", result["label"])

    print(
        f"GREEN={result['green']:.3f} | "
        f"RED={result['red']:.3f} | "
        f"NON_ACTIONABLE={result['non_actionable']:.3f}"
    )

    display(crop_pil)
```

```
============================================================
ID: 12
Esperado: RED
CLIP: RED
GREEN=0.037 | RED=0.275 | NON_ACTIONABLE=0.021
```



```
============================================================
ID: 31
Esperado: GREEN
CLIP: NON_ACTIONABLE
GREEN=0.127 | RED=0.062 | NON_ACTIONABLE=0.144
```



```
============================================================
ID: 26
Esperado: NON_ACTIONABLE
CLIP: NON_ACTIONABLE
GREEN=0.013 | RED=0.016 | NON_ACTIONABLE=0.305
```



14.4 Clasificación jerárquica con CLIP: accionabilidad y estado del semáforo

In [21]:

```
# CLIP JERÁRQUICO PARA SEMÁFORO PEATONAL
# ETAPA 1: ACTIONABLE / NON_ACTIONABLE
```

```
# ETAPA 2: GREEN / RED


ACTIONABLE_PROMPTS = [
    "a pedestrian crossing signal with a clearly visible human
figure",
    "a pedestrian traffic light showing a walking or standing person",
    "a pedestrian crossing light with a visible pedestrian symbol",
    "a traffic signal specifically for pedestrians with a human icon"
]

NON_ACTIONABLE_FINAL_PROMPTS = [
    "a red vehicle traffic light for cars",
    "a green vehicle traffic light for cars",
    "a yellow or amber vehicle traffic light for cars",
    "a vehicle traffic signal with circular lights",
    "a traffic light without a visible pedestrian human figure",
    "the side or back of a traffic light",
    "a traffic light housing viewed from the side",
    "a traffic signal that is not intended for pedestrians"
]

# ============================================================
# CLIP JERÁRQUICO MEJORADO
# ============================================================

ACTIONABILITY_MARGIN = 0.055


def classify_pedestrian_signal_hierarchical(crop):

    # ------------------------------------------------------
    # ETAPA 1: ACTIONABLE vs NON_ACTIONABLE
    # ------------------------------------------------------

    stage1_prompts = (
        ACTIONABLE_PROMPTS
        + NON_ACTIONABLE_FINAL_PROMPTS
    )

    stage1 = classify_with_clip(
        crop,
        stage1_prompts
    )

    probs1 = stage1["all_probabilities"]

    actionable_score = np.mean([
        probs1[p]
```

```
    for p in ACTIONABLE_PROMPTS
])

non_actionable_score = np.mean([
    probs1[p]
    for p in NON_ACTIONABLE_FINAL_PROMPTS
])

difference = (
    non_actionable_score
    - actionable_score
)

# Rechazar solamente cuando NON_ACTIONABLE
# domina de forma suficientemente clara
if difference >= ACTIONABILITY_MARGIN:

    return {
        "label": "NON_ACTIONABLE",
        "actionable": actionable_score,
        "non_actionable": non_actionable_score,
        "green": None,
        "red": None
    }

# ----------------------------------------------------------
# ETAPA 2: GREEN vs RED
# ----------------------------------------------------------

stage2_prompts = (
    STATE_GREEN_PROMPTS
    + STATE_RED_PROMPTS
)

stage2 = classify_with_clip(
    crop,
    stage2_prompts
)

probs2 = stage2["all_probabilities"]

green_score = np.mean([
    probs2[p]
    for p in STATE_GREEN_PROMPTS
])

red_score = np.mean([
    probs2[p]
    for p in STATE_RED_PROMPTS
```

```
    ])

    state = (
        "GREEN"
        if green_score > red_score
        else "RED"
    )

    return {
        "label": state,
        "actionable": actionable_score,
        "non_actionable": non_actionable_score,
        "green": green_score,
        "red": red_score
    }
```

In [24]:

```
# ============================================================
# PROMPTS PARA CLASIFICAR EL ESTADO PEATONAL
# ============================================================

STATE_GREEN_PROMPTS = [
    "a green pedestrian traffic light",
    "a green pedestrian crossing signal",
    "a green walking pedestrian signal",
    "a green walking person traffic signal",
    "a green walk signal"
]

STATE_RED_PROMPTS = [
    "a red pedestrian traffic light",
    "a red pedestrian crossing signal",
    "a red standing pedestrian signal",
    "a red stop pedestrian signal"
]

print("GREEN prompts:", len(STATE_GREEN_PROMPTS))
print("RED prompts:", len(STATE_RED_PROMPTS))
```

```
GREEN prompts: 5
RED prompts: 4
```

In [198]:

```
# ============================================================
# 14.5 Validación controlada de la estrategia CLIP jerárquica
# ============================================================

CONTROL_IDS = {
    12: "RED",
    31: "GREEN",
    26: "NON_ACTIONABLE"
}
```

```
for crop_id, expected in CONTROL_IDS.items():

    item = train_crops[crop_id]

    crop_pil = Image.fromarray(
        item["crop"]
    )

    result = classify_pedestrian_signal_hierarchical(
        crop_pil
    )

    print("\n" + "=" * 65)
    print("ID:", crop_id)
    print("Esperado:", expected)
    print("Resultado:", result["label"])

    print(
        f"ACTIONABLE={result['actionable']:.3f} | "
        f"NON_ACTIONABLE={result['non_actionable']:.3f}"
    )

    if result["green"] is not None:

        print(
            f"GREEN={result['green']:.3f} | "
            f"RED={result['red']:.3f}"
        )

    display(crop_pil)
```

```
=================================================================
ID: 12
Esperado: RED
Resultado: RED
ACTIONABLE=0.161 | NON_ACTIONABLE=0.044
GREEN=0.068 | RED=0.165
```



```
================================================================
ID: 31
Esperado: GREEN
Resultado: GREEN
ACTIONABLE=0.172 | NON_ACTIONABLE=0.039
GREEN=0.167 | RED=0.041
```



```
================================================================
ID: 26
Esperado: NON_ACTIONABLE
Resultado: RED
ACTIONABLE=0.123 | NON_ACTIONABLE=0.064
GREEN=0.087 | RED=0.142
```



In [199]:

```
# ============================================================
# 14.6 Inspección del pipeline YOLO + CLIP sobre el conjunto de
validación
# YOLO V1 sensible + CLIP jerárquico
# ============================================================
```

```
import os
import cv2
import math
import matplotlib.pyplot as plt
from PIL import Image
from collections import Counter

# Margen fijado previamente con TRAIN
ACTIONABILITY_MARGIN = 0.055

# YOLO se utiliza de forma sensible para generar candidatos
VAL_YOLO_CONF = 0.01

val_semantic_results = []

for image_name in sorted(os.listdir(VAL_IMAGES_DIR)):

    if not image_name.lower().endswith(
        (".jpg", ".jpeg", ".png")
    ):
        continue

    image_path = os.path.join(
        VAL_IMAGES_DIR,
        image_name
    )

    image = cv2.imread(image_path)

    if image is None:
        continue

    image_rgb = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )

    h, w = image.shape[:2]

    result = model_v1.predict(
        source=image_path,
        imgsz=960,
        conf=VAL_YOLO_CONF,
        verbose=False
    )[0]

    for box in result.boxes:
```

```
        x1, y1, x2, y2 = map(
            int,
            box.xyxy[0].tolist()
        )

        yolo_conf = float(
            box.conf[0].item()
        )

        # Añadimos contexto alrededor del candidato
        margin = 25

        x1c = max(0, x1 - margin)
        y1c = max(0, y1 - margin)
        x2c = min(w, x2 + margin)
        y2c = min(h, y2 + margin)

        crop = image_rgb[
            y1c:y2c,
            x1c:x2c
        ]

        if crop.size == 0:
            continue

        crop_pil = Image.fromarray(
            crop
        )

        clip_result = (
            classify_pedestrian_signal_hierarchical(
                crop_pil
            )
        )

        val_semantic_results.append({
            "id": len(val_semantic_results),
            "image": image_name,
            "crop": crop,
            "yolo_conf": yolo_conf,
            "label": clip_result["label"],
            "actionable": clip_result["actionable"],
            "non_actionable": clip_result["non_actionable"],
            "green": clip_result["green"],
            "red": clip_result["red"]
        })


# ============================================================
```

```
# MOSAICO
# ============================================================

cols = 5
rows = math.ceil(
    len(val_semantic_results) / cols
)

plt.figure(
    figsize=(15, max(3, rows * 3.2))
)

for i, item in enumerate(
    val_semantic_results
):

    plt.subplot(
        rows,
        cols,
        i + 1
    )

    plt.imshow(
        item["crop"]
    )

    plt.title(
        f"ID {item['id']} | "
        f"{item['label']}\n"
        f"YOLO {item['yolo_conf']:.3f}",
        fontsize=8
    )

    plt.axis("off")

plt.tight_layout()
plt.show()


# ============================================================
# RESUMEN
# ============================================================

counts = Counter(
    item["label"]
    for item in val_semantic_results
)

print("=" * 65)
```

```
print("VALIDACIÓN SEMÁNTICA SOBRE VAL")
print("=" * 65)

print(
    "Candidatos totales:",
    len(val_semantic_results)
)

for label in [
    "RED",
    "GREEN",
    "NON_ACTIONABLE"
]:
    print(
        f"{label}:",
        counts.get(label, 0)
    )
```



```
<Figure size 1500x5760 with 88 Axes>
==================================================================
VALIDACIÓN SEMÁNTICA SOBRE VAL
==================================================================
Candidatos totales: 88
RED: 17
GREEN: 67
NON_ACTIONABLE: 4
```

In [201]:

```
# ============================================================
# 14.7 Evaluación exploratoria del clasificador CLIP jerárquico sobre
casos revisados manualmente
# ============================================================

import pandas as pd

# ------------------------------------------------------------
# Correcciones manuales realizadas tras inspección visual
# ------------------------------------------------------------

MANUAL_CORRECTIONS = {
    1:  "NON_ACTIONABLE",
    3:  "RED",
    4:  "RED",
    5:  "RED",
    7:  "RED",
    8:  "NON_ACTIONABLE",
    11: "NON_ACTIONABLE",
    21: "NON_ACTIONABLE",
    23: "GREEN",
    24: "GREEN",
    27: "NON_ACTIONABLE"
```

```
}

# ------------------------------------------------------------
# Construir ground truth
# Los 29 restantes fueron validados como correctos visualmente
# ------------------------------------------------------------

evaluation_rows = []

for item in val_semantic_results:

    crop_id = item["id"]
    predicted = item["label"]

    true_label = MANUAL_CORRECTIONS.get(
        crop_id,
        predicted
    )

    evaluation_rows.append({
        "ID": crop_id,
        "Real": true_label,
        "Predicho": predicted,
        "Correcto": true_label == predicted
    })


evaluation_df = pd.DataFrame(
    evaluation_rows
)

# ------------------------------------------------------------
# Accuracy global
# ------------------------------------------------------------

accuracy = evaluation_df[
    "Correcto"
].mean()

print("=" * 65)
print("EVALUACIÓN CLIP JERÁRQUICO")
print("=" * 65)

print(
    f"Aciertos: "
    f"{evaluation_df['Correcto'].sum()}/"
    f"{len(evaluation_df)}"
)
```

```
print(
    f"Accuracy: "
    f"{accuracy:.3f} "
    f"({accuracy * 100:.1f} %)"
)

# ------------------------------------------------------------
# Matriz de confusión
# ------------------------------------------------------------

labels = [
    "RED",
    "GREEN",
    "NON_ACTIONABLE"
]

confusion = pd.crosstab(
    evaluation_df["Real"],
    evaluation_df["Predicho"],
    rownames=["REAL"],
    colnames=["PREDICHO"],
    dropna=False
)

confusion = confusion.reindex(
    index=labels,
    columns=labels,
    fill_value=0
)

print("\nMATRIZ DE CONFUSIÓN")
display(confusion)

# ------------------------------------------------------------
# Precision, Recall y F1 por clase
# ------------------------------------------------------------

metrics_rows = []

for label in labels:

    TP = (
        (evaluation_df["Real"] == label)
        &
        (evaluation_df["Predicho"] == label)
    ).sum()

    FP = (
        (evaluation_df["Real"] != label)
```

```
        &
        (evaluation_df["Predicho"] == label)
    ).sum()

    FN = (
        (evaluation_df["Real"] == label)
        &
        (evaluation_df["Predicho"] != label)
    ).sum()

    precision = (
        TP / (TP + FP)
        if (TP + FP) > 0
        else 0
    )

    recall = (
        TP / (TP + FN)
        if (TP + FN) > 0
        else 0
    )

    f1 = (
        2 * precision * recall
        / (precision + recall)
        if (precision + recall) > 0
        else 0
    )

    metrics_rows.append({
        "Clase": label,
        "TP": TP,
        "FP": FP,
        "FN": FN,
        "Precision": precision,
        "Recall": recall,
        "F1": f1
    })


metrics_df = pd.DataFrame(
    metrics_rows
)

print("\nMÉTRICAS POR CLASE")
display(metrics_df)

# ------------------------------------------------------------
# Mostrar únicamente los errores
```

```
# ------------------------------------------------------------

errors_df = evaluation_df[
    evaluation_df["Correcto"] == False
]

print("\nERRORES DETECTADOS")
display(errors_df)
```

```
======================================================================
EVALUACIÓN CLIP JERÁRQUICO
======================================================================
Aciertos: 79/88
Accuracy: 0.898 (89.8 %)
```

MATRIZ DE CONFUSIÓN

PREDICHO	RED	GREEN	NON_ACTIONABLE
REAL			
RED	16	4	0
GREEN	0	59	0
NON_ACTIONABLE	1	4	4

MÉTRICAS POR CLASE

	Clase	TP	FP	FN	Precision	Recall	F1
0	RED	16	1	4	0.941176	0.800000	0.864865
1	GREEN	59	8	0	0.880597	1.000000	0.936508
2	NON_ACTIONABLE	4	0	5	1.000000	0.444444	0.615385

ERRORES DETECTADOS

	ID	Real	Predicho	Correcto
1	1	NON_ACTIONABLE	GREEN	False
3	3	RED	GREEN	False
4	4	RED	GREEN	False
5	5	RED	GREEN	False
7	7	RED	GREEN	False
8	8	NON_ACTIONABLE	RED	False
11	11	NON_ACTIONABLE	GREEN	False
21	21	NON_ACTIONABLE	GREEN	False
27	27	NON_ACTIONABLE	GREEN	False

14.8 Ground truth final de los 40 casos revisados manualmente

In [14]:

```
# GROUND TRUTH FINAL DE LOS 40 CASOS REVISADOS MANUALMENTE

GROUND_TRUTH_40 = {
     0: "RED",
     1: "NON_ACTIONABLE",
     2: "RED",
     3: "RED",
     4: "RED",
     5: "RED",
     6: "NON_ACTIONABLE",
     7: "RED",
     8: "NON_ACTIONABLE",
     9: "NON_ACTIONABLE",

    10: "NON_ACTIONABLE",
    11: "NON_ACTIONABLE",
    12: "RED",
    13: "RED",
    14: "RED",
    15: "RED",
    16: "NON_ACTIONABLE",
    17: "RED",
    18: "NON_ACTIONABLE",
    19: "RED",

    20: "RED",
    21: "NON_ACTIONABLE",
    22: "NON_ACTIONABLE",
    23: "GREEN",
    24: "GREEN",
    25: "GREEN",
    26: "NON_ACTIONABLE",
    27: "NON_ACTIONABLE",
    28: "GREEN",
    29: "NON_ACTIONABLE",

    30: "NON_ACTIONABLE",
    31: "GREEN",
    32: "NON_ACTIONABLE",
    33: "GREEN",
    34: "NON_ACTIONABLE",
    35: "GREEN",
    36: "NON_ACTIONABLE",
    37: "GREEN",
    38: "NON_ACTIONABLE",
    39: "GREEN"
}
```

```
# ============================================================
# RECONSTRUIR evaluation_df
# ============================================================

evaluation_rows = []

for crop_id in range(40):

    item = train_crops[crop_id]

    real_label = GROUND_TRUTH_40[crop_id]

    evaluation_rows.append({
        "ID": crop_id,
        "Real": real_label,
        "crop": item["crop"]
    })

evaluation_df = pd.DataFrame(evaluation_rows)


print("Casos reconstruidos:", len(evaluation_df))
print("\nDistribución de clases:")
print(evaluation_df["Real"].value_counts())

display(
    evaluation_df[
        ["ID", "Real"]
    ]
)
Casos reconstruidos: 40

Distribución de clases:
Real
NON_ACTIONABLE    18
RED               13
GREEN              9
Name: count, dtype: int64
```

	ID	Real
0	0	RED
1	1	NON_ACTIONABLE
2	2	RED
3	3	RED
4	4	RED
5	5	RED

	ID	Real
6	6	NON_ACTIONABLE
7	7	RED
8	8	NON_ACTIONABLE
9	9	NON_ACTIONABLE
10	10	NON_ACTIONABLE
11	11	NON_ACTIONABLE
12	12	RED
13	13	RED
14	14	RED
15	15	RED
16	16	NON_ACTIONABLE
17	17	RED
18	18	NON_ACTIONABLE
19	19	RED
20	20	RED
21	21	NON_ACTIONABLE
22	22	NON_ACTIONABLE
23	23	GREEN
24	24	GREEN
25	25	GREEN
26	26	NON_ACTIONABLE
27	27	NON_ACTIONABLE
28	28	GREEN
29	29	NON_ACTIONABLE
30	30	NON_ACTIONABLE
31	31	GREEN
32	32	NON_ACTIONABLE
33	33	GREEN
34	34	NON_ACTIONABLE
35	35	GREEN
36	36	NON_ACTIONABLE
37	37	GREEN
38	38	NON_ACTIONABLE
39	39	GREEN

14.9 Clasificador supervisado de accionabilidad sobre embeddings visuales de CLIP

In [35]:

```
# ============================================================
# CLASIFICADOR SUPERVISADO ACTIONABLE / NON_ACTIONABLE
# SOBRE EMBEDDINGS VISUALES DE CLIP
# ============================================================

import numpy as np
import torch
from PIL import Image

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# ------------------------------------------------------------
# EXTRAER EMBEDDING VISUAL CLIP
# ------------------------------------------------------------

def get_clip_image_embedding(image):

    if not isinstance(image, Image.Image):
        image = Image.fromarray(image)

    inputs = clip_processor(
        images=image,
        return_tensors="pt"
    )

    device = next(
        clip_model.parameters()
    ).device

    pixel_values = inputs[
        "pixel_values"
    ].to(device)

    with torch.no_grad():

        # Encoder visual de CLIP
        vision_outputs = clip_model.vision_model(
            pixel_values=pixel_values
        )

        # Representación global de la imagen
        pooled_output = vision_outputs.pooler_output
```

```
        # Proyección al espacio multimodal de CLIP
        features = clip_model.visual_projection(
            pooled_output
        )

        # Normalización L2
        features = torch.nn.functional.normalize(
            features,
            p=2,
            dim=-1
        )

    return (
        features
        .cpu()
        .numpy()[0]
    )


# ------------------------------------------------------------
# CONSTRUIR X / y A PARTIR DE LOS 40 CASOS REVISADOS
# ------------------------------------------------------------

X = []
y = []
sample_ids = []

for _, row in evaluation_df.iterrows():

    crop_id = int(row["ID"])
    real_label = row["Real"]
    crop = row["crop"]

    # 0 = NON_ACTIONABLE
    # 1 = ACTIONABLE (RED o GREEN)
    binary_label = (
        0
        if real_label == "NON_ACTIONABLE"
        else 1
    )

    embedding = get_clip_image_embedding(
        crop
    )

    X.append(embedding)
    y.append(binary_label)
    sample_ids.append(crop_id)
```

```
X = np.asarray(X)
y = np.asarray(y)

# ------------------------------------------------------------
# COMPROBACIONES
# ------------------------------------------------------------

assert len(X) == 40, (
    f"ERROR: se esperaban 40 muestras y hay {len(X)}."
)

assert len(y) == 40
assert len(sample_ids) == 40

print("Muestras:", len(y))
print("ACTIONABLE:", int(y.sum()))
print(
    "NON_ACTIONABLE:",
    int((y == 0).sum())
)

# ------------------------------------------------------------
# VALIDACIÓN CRUZADA
# ------------------------------------------------------------

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

gate_model = LogisticRegression(
    max_iter=2000,
    class_weight="balanced",
    random_state=42
)

pred_cv = cross_val_predict(
    gate_model,
    X,
    y,
    cv=cv
)

print("\n" + "=" * 65)
print("CLIP + CLASIFICADOR LINEAL")
print("VALIDACIÓN CRUZADA SOBRE LOS 40 CASOS REVISADOS")
print("=" * 65)
```

```
print(
    "Accuracy:",
    round(
        accuracy_score(
            y,
            pred_cv
        ),
        3
    )
)

print("\nMatriz de confusión:")
print(
    confusion_matrix(
        y,
        pred_cv
    )
)

print("\nInforme:")
print(
    classification_report(
        y,
        pred_cv,
        target_names=[
            "NON_ACTIONABLE",
            "ACTIONABLE"
        ],
        digits=3
    )
)

# ------------------------------------------------------------
# ENTRENAR MODELO FINAL CON LOS 40
# ------------------------------------------------------------

gate_model.fit(
    X,
    y
)

print(
    "\nClasificador final entrenado."
)
Muestras: 40
ACTIONABLE: 22
NON_ACTIONABLE: 18
```

```
================================================================
CLIP + CLASIFICADOR LINEAL
VALIDACIÓN CRUZADA SOBRE LOS 40 CASOS REVISADOS
================================================================
Accuracy: 0.875

Matriz de confusión:
[[17  1]
 [ 4 18]]

Informe:
                precision    recall  f1-score   support

NON_ACTIONABLE      0.810     0.944     0.872        18
    ACTIONABLE      0.947     0.818     0.878        22

      accuracy                          0.875        40
     macro avg      0.878     0.881     0.875        40
  weighted avg      0.885     0.875     0.875        40


Clasificador final entrenado.
```

14.10 Guardado del Gate supervisado

In [211]:

```
# ================================================================
# GUARDAR GATE SUPERVISADO EN DISCO
# ================================================================

import os
import joblib

MODEL_DIR = os.path.join(
    r"C:\Users\juank\Documents\CARLOS\TFM",
    "models_finales"
)

os.makedirs(
    MODEL_DIR,
    exist_ok=True
)

GATE_MODEL_PATH = os.path.join(
    MODEL_DIR,
    "clip_gate_actionable.pkl"
)

joblib.dump(
```

```
    gate_model,
    GATE_MODEL_PATH
)

print("Gate model guardado en:")
print(GATE_MODEL_PATH)

print("\n¿Existe?")
print(os.path.isfile(GATE_MODEL_PATH))
```

```
Gate model guardado en:
C:\Users\juank\Documents\CARLOS\TFM\models_finales\clip_gate_actionabl
e.pkl

¿Existe?
True
```

14.11 Extensión temporal: seguimiento del semáforo peatonal en vídeo

In [22]:

```
# CONSISTENCIA TEMPORAL DEL SEMÁFORO PEATONAL
# YOLO V1 + TRACKING + CLIP EMBEDDINGS + GATE SUPERVISADO

import cv2
import numpy as np
import pandas as pd
from PIL import Image
from ultralytics import YOLO


# ------------------------------------------------------------
# CONFIGURACIÓN
# ------------------------------------------------------------

TEMPORAL_VIDEO_PATH = os.path.join(
    VIDEOS_DIR,
    "1000179711.mp4"
)

YOLO_TEMPORAL_CONF = 0.01
GATE_THRESHOLD = 0.50

# Procesamos 1 de cada 3 frames.
# En un vídeo ~30 FPS seguimos teniendo unas 10 observaciones/segundo.
FRAME_STEP = 30


# ------------------------------------------------------------
```

```
# CLASIFICACIÓN DEL ESTADO:
# SOLO GREEN vs RED
# ------------------------------------------------------------

def classify_state_red_green(crop):

    prompts = (
        STATE_GREEN_PROMPTS
        + STATE_RED_PROMPTS
    )

    result = classify_with_clip(
        crop,
        prompts
    )

    probs = result["all_probabilities"]

    green_score = np.mean([
        probs[p]
        for p in STATE_GREEN_PROMPTS
    ])

    red_score = np.mean([
        probs[p]
        for p in STATE_RED_PROMPTS
    ])

    # Normalización únicamente entre GREEN y RED
    total = green_score + red_score

    if total > 0:
        green_score /= total
        red_score /= total

    state = (
        "GREEN"
        if green_score > red_score
        else "RED"
    )

    return {
        "state": state,
        "green": green_score,
        "red": red_score
    }


# ------------------------------------------------------------
```

```
# MODELO V1 NUEVO PARA REINICIAR EL TRACKER
# ------------------------------------------------------------

temporal_model = YOLO(
    V1_PATH
)

cap = cv2.VideoCapture(
    TEMPORAL_VIDEO_PATH
)

fps = cap.get(
    cv2.CAP_PROP_FPS
)

frame_idx = 0

temporal_rows = []

print("=" * 70)
print("ANÁLISIS TEMPORAL DEL VÍDEO")
print("=" * 70)

print(
    "Vídeo:",
    TEMPORAL_VIDEO_PATH
)

print(
    "FPS:",
    fps
)


# ------------------------------------------------------------
# PROCESAMIENTO
# ------------------------------------------------------------

while True:

    ret, frame = cap.read()

    if not ret:
        break

    # Submuestreo temporal
    if frame_idx % FRAME_STEP != 0:
        frame_idx += 1
        continue
```

```
frame_rgb = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2RGB
)

h, w = frame.shape[:2]

# YOLO + ByteTrack
result = temporal_model.track(
    source=frame,
    persist=True,
    tracker="bytetrack.yaml",
    imgsz=960,
    conf=YOLO_TEMPORAL_CONF,
    verbose=False
)[0]

boxes = result.boxes

if boxes is None or len(boxes) == 0:
    frame_idx += 1
    continue

# IDs producidos por el tracker
if boxes.id is not None:

    track_ids = (
        boxes.id
        .int()
        .cpu()
        .tolist()
    )

else:

    track_ids = [
        None
    ] * len(boxes)

# ----------------------------------------------------------
# ANALIZAR CADA CANDIDATO
# ----------------------------------------------------------

for j, box in enumerate(boxes):

    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0].tolist()
```

```
)

yolo_conf = float(
    box.conf[0].item()
)

track_id = track_ids[j]

margin = 25

x1c = max(
    0,
    x1 - margin
)

y1c = max(
    0,
    y1 - margin
)

x2c = min(
    w,
    x2 + margin
)

y2c = min(
    h,
    y2 + margin
)

crop_array = frame_rgb[
    y1c:y2c,
    x1c:x2c
]

if crop_array.size == 0:
    continue

crop_pil = Image.fromarray(
    crop_array
)

# ------------------------------------------------
# CLIP EMBEDDING + GATE SUPERVISADO
# ------------------------------------------------

embedding = get_clip_image_embedding(
    crop_pil
)
```

```
candidate_X = embedding.reshape(
    1,
    -1
)

probabilities = gate_model.predict_proba(
    candidate_X
)[0]

actionable_index = list(
    gate_model.classes_
).index(1)

p_actionable = float(
    probabilities[
        actionable_index
    ]
)

actionable = (
    p_actionable >= GATE_THRESHOLD
)

# ------------------------------------------------------
# SOLO SI ES ACTIONABLE:
# GREEN vs RED
# ------------------------------------------------------

state = "NON_ACTIONABLE"
green_score = np.nan
red_score = np.nan

if actionable:

    state_result = classify_state_red_green(
        crop_pil
    )

    state = state_result[
        "state"
    ]

    green_score = state_result[
        "green"
    ]

    red_score = state_result[
        "red"
```

```
            ]

        # ----------------------------------------------------
        # GUARDAR EVIDENCIA DEL FRAME
        # ----------------------------------------------------

        temporal_rows.append({
            "frame": frame_idx,
            "time_s": (
                frame_idx / fps
                if fps > 0
                else np.nan
            ),
            "track_id": track_id,
            "yolo_conf": yolo_conf,
            "p_actionable": p_actionable,
            "actionable": actionable,
            "state": state,
            "green_score": green_score,
            "red_score": red_score,
            "x1": x1,
            "y1": y1,
            "x2": x2,
            "y2": y2
        })

    frame_idx += 1


cap.release()


# ------------------------------------------------------------
# DATAFRAME
# ------------------------------------------------------------

df_temporal_signal = pd.DataFrame(
    temporal_rows
)

print("\nProcesamiento terminado.")

print(
    "Observaciones:",
    len(df_temporal_signal)
)

print(
    "Tracks distintos:",
```

```
    df_temporal_signal[
        "track_id"
    ].nunique()
)

display(
    df_temporal_signal.head(20)
)
```

```
---------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
Cell In[22], line 82
     78 # MODELO V1 NUEVO PARA REINICIAR EL TRACKER
     79 # ------------------------------------------------------------
     80
     81 temporal_model = YOLO(
---> 82     V1_PATH
     83 )
     84
     85 cap = cv2.VideoCapture(

NameError: name 'V1_PATH' is not defined
```

In [213]:

```
print(
    "classify_state_red_green:",
    "OK" if "classify_state_red_green" in globals()
    else "FALTA"
)
```

```
classify_state_red_green: OK
```

In [214]:

```
print(
    "df_temporal_signal:",
    "OK" if "df_temporal_signal" in globals()
    else "FALTA"
)
```

```
df_temporal_signal: OK
```

14.12 Agregación y resumen temporal por track

In [215]:

```
# RESUMEN TEMPORAL POR TRACK


def summarize_track(group):

    total_obs = len(group)
```

```
actionable_hits = int(
    group["actionable"].sum()
)

actionable_ratio = (
    actionable_hits / total_obs
    if total_obs > 0
    else 0
)

actionable_group = group[
    group["actionable"]
]

if len(actionable_group) > 0:

    state_counts = (
        actionable_group[
            "state"
        ]
        .value_counts()
    )

    dominant_state = (
        state_counts.index[0]
    )

    state_consistency = (
        state_counts.iloc[0]
        / len(actionable_group)
    )

else:

    dominant_state = (
        "NON_ACTIONABLE"
    )

    state_consistency = 0.0

return pd.Series({
    "observations": total_obs,

    "first_frame":
        group["frame"].min(),

    "last_frame":
        group["frame"].max(),
```

```
        "mean_p_actionable":
            group[
                "p_actionable"
            ].mean(),

        "max_p_actionable":
            group[
                "p_actionable"
            ].max(),

        "actionable_hits":
            actionable_hits,

        "actionable_ratio":
            actionable_ratio,

        "dominant_state":
            dominant_state,

        "state_consistency":
            state_consistency
    })


# ============================================================
# SELECCIONAR SOLO OBSERVACIONES CON TRACK
# ============================================================

tracked_only = df_temporal_signal[
    df_temporal_signal["track_id"].notna()
].copy()


# ============================================================
# CONSTRUIR RESUMEN TEMPORAL
# ============================================================

if tracked_only.empty:

    # Mantener una estructura válida aunque ByteTrack
    # no haya generado ningún identificador temporal.
    track_summary = pd.DataFrame(
        columns=[
            "track_id",
            "observations",
            "first_frame",
            "last_frame",
            "mean_p_actionable",
            "max_p_actionable",
```

```
            "actionable_hits",
            "actionable_ratio",
            "dominant_state",
            "state_consistency",
        ]
    )

    track_summary = pd.DataFrame()
    print("=" * 75)
    print("RESUMEN TEMPORAL POR TRACK")
    print("=" * 75)
    print(
        "No se han obtenido tracks temporales para este vídeo "
        "(track_id ausente en todas las observaciones)."
    )

else:

    track_summary = (
        tracked_only
        .groupby("track_id")
        .apply(summarize_track)
        .reset_index()
    )

    track_summary = (
        track_summary
        .sort_values(
            [
                "actionable_ratio",
                "mean_p_actionable",
                "observations",
            ],
            ascending=[
                False,
                False,
                False,
            ],
        )
    )

    print("=" * 75)
    print("RESUMEN TEMPORAL POR TRACK")
    print("=" * 75)

    display(track_summary)
===========================================================================
=====
RESUMEN TEMPORAL POR TRACK
```

```
===========================================================================
=====
No se han obtenido tracks temporales para este vídeo (track_id ausente
en todas las observaciones).
```

14.13 Inspección visual de los tracks temporales

In [216]:

```
# INSPECCIÓN VISUAL DE LOS TRACKS TEMPORALES
# Se muestra el frame con mayor P(ACTIONABLE) de cada track

import matplotlib.pyplot as plt
import math
import cv2
from PIL import Image

track_examples = []

# Una fila representativa por track:
# la de mayor probabilidad ACTIONABLE
# Observaciones que sí tienen track asignado

tracked_only = df_temporal_signal[
    df_temporal_signal["track_id"].notna()
].copy()

print("=" * 75)
print("INSPECCIÓN VISUAL DE TRACKS TEMPORALES")
print("=" * 75)

if tracked_only.empty:
    print(
        "No hay tracks disponibles para inspección visual "
        "en este vídeo."
    )

else:
    # Una fila representativa por track:
    # la de mayor probabilidad ACTIONABLE
    best_rows = (
        tracked_only
        .sort_values(
            "p_actionable",
            ascending=False
        )
        .groupby("track_id")
        .head(1)
        .reset_index(drop=True)
    )
```

```
    rows = math.ceil(len(best_rows) / 4)
    cols = min(4, len(best_rows))

    plt.figure(
        figsize=(4 * cols, 3.5 * rows)
    )

    for i, (_, item) in enumerate(best_rows.iterrows()):

        plt.subplot(rows, cols, i + 1)

        plt.imshow(item["crop"])

        plt.title(
            f"Track {item['track_id']:.0f}\n"
            f"P={item['p_actionable']:.2f}",
            fontsize=9
        )

        plt.axis("off")

    plt.tight_layout()
    plt.show()
================================================================================
=====
INSPECCIÓN VISUAL DE TRACKS TEMPORALES
================================================================================
=====
No hay tracks disponibles para inspección visual en este vídeo.
```

14.14 Decisión temporal final por track

In [217]:

```
# ============================================================
# 14.14 DECISIÓN TEMPORAL FINAL POR TRACK
# ============================================================

MIN_ACTIONABLE_HITS = 3
MIN_ACTIONABLE_RATIO = 0.75
MIN_STATE_CONSISTENCY = 0.80


def temporal_decision(row):
    """
    Decide el estado temporal final de cada track.

    Para aceptar un estado GREEN o RED se exige:
    - número mínimo de observaciones ACTIONABLE;
```

```
    - proporción mínima de observaciones ACTIONABLE;
    - consistencia mínima del estado dominante.

    Si no se cumplen estas condiciones,
    el track se considera NON_ACTIONABLE.
    """

    valid = (
        row["actionable_hits"] >= MIN_ACTIONABLE_HITS
        and
        row["actionable_ratio"] >= MIN_ACTIONABLE_RATIO
        and
        row["state_consistency"] >= MIN_STATE_CONSISTENCY
    )

    if not valid:
        return "NON_ACTIONABLE"

    return row["dominant_state"]


print("=" * 75)
print("DECISIÓN TEMPORAL FINAL POR TRACK")
print("=" * 75)

# ------------------------------------------------------------
# Comprobar si existen tracks válidos
# ------------------------------------------------------------

if "track_summary" not in globals():

    print(
        "No se puede obtener una decisión temporal final "
        "porque no existe track_summary."
    )

elif track_summary.empty:

    print(
        "No se puede obtener una decisión temporal por track "
        "porque este vídeo no contiene tracks válidos."
    )

else:

    # ------------------------------------------------------------
    # Aplicar decisión temporal
    # ------------------------------------------------------------
```

```
    track_summary = track_summary.copy()

    track_summary["temporal_decision"] = (
        track_summary.apply(
            temporal_decision,
            axis=1
        )
    )

    # ----------------------------------------------------------
    # Mostrar resultado
    # ----------------------------------------------------------

    display_columns = [
        col for col in [
            "track_id",
            "observations",
            "actionable_hits",
            "actionable_ratio",
            "mean_p_actionable",
            "dominant_state",
            "state_consistency",
            "temporal_decision"
        ]
        if col in track_summary.columns
    ]

    display(
        track_summary[
            display_columns
        ]
    )

    print("\nResumen de decisiones:")

    print(
        track_summary[
            "temporal_decision"
        ].value_counts(
            dropna=False
        )
    )
================================================================================
DECISIÓN TEMPORAL FINAL POR TRACK
================================================================================
No se puede obtener una decisión temporal por track porque este vídeo
no contiene tracks válidos.
```

14.15 Asociación espacial de detecciones para el seguimiento temporal

In [307]:

```
# ============================================================
# TRACKING ESPACIAL PARA SEMÁFOROS PEQUEÑOS
# ============================================================

def assign_signal_tracks_spatial(
    df,
    max_frame_gap=6,
    max_dist_factor=2.5
):

    data = df.copy()

    data = (
        data
        .sort_values(
           ["frame", "p_actionable"],
           ascending=[True, False]
        )
        .reset_index(drop=True)
    )

    data["cx"] = (
        data["x1"] + data["x2"]
    ) / 2

    data["cy"] = (
        data["y1"] + data["y2"]
    ) / 2

    data["w"] = (
        data["x2"] - data["x1"]
    )

    data["h"] = (
        data["y2"] - data["y1"]
    )

    data["diag"] = np.sqrt(
        data["w"] ** 2
        +
        data["h"] ** 2
    )

    data["spatial_track_id"] = -1
```

```
active_tracks = {}
next_track_id = 0

for idx, row in data.iterrows():

    frame = int(row["frame"])

    best_track = None
    best_distance = np.inf

    for track_id, previous in active_tracks.items():

        frame_gap = (
            frame
            - previous["frame"]
        )

        if (
            frame_gap <= 0
            or frame_gap > max_frame_gap
        ):
            continue


        dx = row["cx"] - previous["cx"]
        dy = row["cy"] - previous["cy"]

        distance = np.sqrt(
            dx ** 2 + dy ** 2
        )

        reference_diag = max(
            row["diag"],
            previous["diag"],
            1
        )

        normalized_distance = (
            distance
            /
            reference_diag
        )

        if (
            normalized_distance <= max_dist_factor
            and
            normalized_distance < best_distance
        ):
```

```
                best_track = track_id
                best_distance = normalized_distance

        if best_track is None:

            best_track = next_track_id
            next_track_id += 1

        data.at[
            idx,
            "spatial_track_id"
        ] = best_track

        active_tracks[best_track] = {
            "frame": frame,
            "cx": row["cx"],
            "cy": row["cy"],
            "diag": row["diag"]
        }

    return data
```

14.16 Integración del pipeline temporal completo para vídeo

In [219]:

```
# PROCESAMIENTO TEMPORAL COMPLETO DE UN VÍDEO


import os
import cv2
import numpy as np
import pandas as pd
from PIL import Image
from ultralytics import YOLO


# ============================================================
# PARÁMETROS CONGELADOS
# ============================================================

YOLO_TEMPORAL_CONF = 0.01
GATE_THRESHOLD = 0.50

FRAME_STEP = 3

MIN_ACTIONABLE_HITS = 3
```

```
MIN_ACTIONABLE_RATIO = 0.75
MIN_STATE_CONSISTENCY = 0.80


def process_temporal_signal_video(video_path):

    # ----------------------------------------------------------
    # Reiniciar YOLO + tracker para cada vídeo
    # ----------------------------------------------------------

    temporal_model = YOLO(
        V1_PATH
    )

    cap = cv2.VideoCapture(
        video_path
    )

    fps = cap.get(
        cv2.CAP_PROP_FPS
    )

    frame_idx = 0
    temporal_rows = []

    # ==========================================================
    # PROCESAMIENTO FRAME A FRAME
    # ==========================================================

    while True:

        ret, frame = cap.read()

        if not ret:
            break

        # Submuestreo temporal
        if frame_idx % FRAME_STEP != 0:

            frame_idx += 1
            continue

        frame_rgb = cv2.cvtColor(
            frame,
            cv2.COLOR_BGR2RGB
        )

        h, w = frame.shape[:2]
```

```
result = temporal_model.track(
    source=frame,
    persist=True,
    tracker="bytetrack.yaml",
    imgsz=960,
    conf=YOLO_TEMPORAL_CONF,
    verbose=False
)[0]

boxes = result.boxes

if boxes is None or len(boxes) == 0:

    frame_idx += 1
    continue

# IDs del tracker
if boxes.id is not None:

    track_ids = (
        boxes.id
        .int()
        .cpu()
        .tolist()
    )

else:

    track_ids = [
        None
    ] * len(boxes)

# ====================================================
# ANALIZAR CANDIDATOS
# ====================================================

for j, box in enumerate(boxes):

    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0].tolist()
    )

    yolo_conf = float(
        box.conf[0].item()
    )

    track_id = track_ids[j]
```

```
margin = 25

x1c = max(0, x1 - margin)
y1c = max(0, y1 - margin)

x2c = min(w, x2 + margin)
y2c = min(h, y2 + margin)

crop_array = frame_rgb[
    y1c:y2c,
    x1c:x2c
]

if crop_array.size == 0:
    continue

crop_pil = Image.fromarray(
    crop_array
)

# ------------------------------------------------
# CLIP embedding + gate supervisado
# ------------------------------------------------

embedding = get_clip_image_embedding(
    crop_pil
)

X_candidate = embedding.reshape(
    1,
    -1
)

probabilities = gate_model.predict_proba(
    X_candidate
)[0]

actionable_index = list(
    gate_model.classes_
).index(1)

p_actionable = float(
    probabilities[
        actionable_index
    ]
)

actionable = (
    p_actionable >= GATE_THRESHOLD
```

```
)

# ------------------------------------------------
# GREEN / RED
# ------------------------------------------------

state = "NON_ACTIONABLE"

green_score = np.nan
red_score = np.nan

if actionable:

    state_result = (
        classify_state_red_green(
            crop_pil
        )
    )

    state = state_result[
        "state"
    ]

    green_score = state_result[
        "green"
    ]

    red_score = state_result[
        "red"
    ]

# ------------------------------------------------
# GUARDAR OBSERVACIÓN
# ------------------------------------------------

temporal_rows.append({
    "frame": frame_idx,

    "time_s":
        frame_idx / fps
        if fps > 0
        else np.nan,

    "track_id": track_id,

    "yolo_conf":
        yolo_conf,

    "p_actionable":
```

```
                p_actionable,

            "actionable":
                actionable,

            "state":
                state,

            "green_score":
                green_score,

            "red_score":
                red_score,

            "x1": x1,
            "y1": y1,
            "x2": x2,
            "y2": y2
        })

    frame_idx += 1


cap.release()


# ============================================================
# DATAFRAME DE OBSERVACIONES
# ============================================================

df_temporal = pd.DataFrame(
    temporal_rows
)

if len(df_temporal) == 0:

    return (
        df_temporal,
        pd.DataFrame()
    )


# Solo tracks válidos de ByteTrack
tracked = df_temporal[
    df_temporal[
        "track_id"
    ].notna()
].copy()
```

```
if len(tracked) == 0:

    return (
        df_temporal,
        pd.DataFrame()
    )


# ============================================================
# RESUMEN POR TRACK
# ============================================================

summary_rows = []

for track_id, group in tracked.groupby(
    "track_id"
):

    observations = len(group)

    actionable_hits = int(
        group[
            "actionable"
        ].sum()
    )

    actionable_ratio = (
        actionable_hits
        / observations
    )

    actionable_group = group[
        group["actionable"]
    ]

    if len(actionable_group) > 0:

        state_counts = (
            actionable_group[
                "state"
            ]
            .value_counts()
        )

        dominant_state = (
            state_counts.index[0]
        )
```

```
    state_consistency = (
        state_counts.iloc[0]
        / len(actionable_group)
    )

else:

    dominant_state = (
        "NON_ACTIONABLE"
    )

    state_consistency = 0.0


# ----------------------------------------------------
# DECISIÓN TEMPORAL
# ----------------------------------------------------

valid = (
    actionable_hits
    >= MIN_ACTIONABLE_HITS

    and

    actionable_ratio
    >= MIN_ACTIONABLE_RATIO

    and

    state_consistency
    >= MIN_STATE_CONSISTENCY
)

if valid:

    temporal_decision = (
        dominant_state
    )

else:

    temporal_decision = (
        "NON_ACTIONABLE"
    )


summary_rows.append({

    "track_id":
```

```
            track_id,

        "observations":
            observations,

        "first_frame":
            group["frame"].min(),

        "last_frame":
            group["frame"].max(),

        "mean_p_actionable":
            group[
                "p_actionable"
            ].mean(),

        "max_p_actionable":
            group[
                "p_actionable"
            ].max(),

        "actionable_hits":
            actionable_hits,

        "actionable_ratio":
            actionable_ratio,

        "dominant_state":
            dominant_state,

        "state_consistency":
            state_consistency,

        "temporal_decision":
            temporal_decision
    })


track_summary = pd.DataFrame(
    summary_rows
)


return (
    df_temporal,
    track_summary
)
```

14.17 Validación temporal sobre los 10 vídeos de prueba

In [220]:

```
# VALIDACIÓN TEMPORAL SOBRE LOS 10 VÍDEOS

TEMPORAL_RESULTS_DIR = os.path.join(
    VIDEOS_DIR,
    "temporal_signal_results"
)

os.makedirs(
    TEMPORAL_RESULTS_DIR,
    exist_ok=True
)


all_video_results = []


for i, video_name in enumerate(
    VIDEO_FILES,
    start=1
):

    video_path_test = os.path.join(
        VIDEOS_DIR,
        video_name
    )

    print("\n" + "=" * 75)

    print(
        f"VÍDEO {i}/{len(VIDEO_FILES)}"
    )

    print(
        video_name
    )

    print("=" * 75)


    df_video_temporal, df_video_tracks = (
        process_temporal_signal_video(
            video_path_test
        )
    )
```

```
# ----------------------------------------------------------
# GUARDAR RESULTADOS DETALLADOS
# ----------------------------------------------------------

base_name = os.path.splitext(
    video_name
)[0]

observations_csv = os.path.join(
    TEMPORAL_RESULTS_DIR,
    f"{base_name}_observations.csv"
)

tracks_csv = os.path.join(
    TEMPORAL_RESULTS_DIR,
    f"{base_name}_tracks.csv"
)


df_video_temporal.to_csv(
    observations_csv,
    index=False
)

df_video_tracks.to_csv(
    tracks_csv,
    index=False
)


# ----------------------------------------------------------
# RESUMEN DEL VÍDEO
# ----------------------------------------------------------

if len(df_video_tracks) > 0:

    green_tracks = int(
        (
            df_video_tracks[
                "temporal_decision"
            ]
            == "GREEN"
        ).sum()
    )

    red_tracks = int(
        (
```

```
            df_video_tracks[
                "temporal_decision"
            ]
            == "RED"
        ).sum()
    )

    non_actionable_tracks = int(
        (
            df_video_tracks[
                "temporal_decision"
            ]
            == "NON_ACTIONABLE"
        ).sum()
    )

    total_tracks = len(
        df_video_tracks
    )

else:

    green_tracks = 0
    red_tracks = 0
    non_actionable_tracks = 0
    total_tracks = 0


all_video_results.append({

    "video":
        video_name,

    "observations":
        len(df_video_temporal),

    "tracks":
        total_tracks,

    "GREEN":
        green_tracks,

    "RED":
        red_tracks,

    "NON_ACTIONABLE":
        non_actionable_tracks
})
```

```
    print(
        "Observaciones:",
        len(df_video_temporal)
    )

    print(
        "Tracks:",
        total_tracks
    )

    print(
        "GREEN:",
        green_tracks
    )

    print(
        "RED:",
        red_tracks
    )

    print(
        "NON_ACTIONABLE:",
        non_actionable_tracks
    )


# ============================================================
# TABLA GLOBAL
# ============================================================

df_temporal_summary_all = pd.DataFrame(
    all_video_results
)

print("\n" + "=" * 80)
print("RESULTADO TEMPORAL DE LOS 10 VÍDEOS")
print("=" * 80)

display(
    df_temporal_summary_all
)


# Guardar resumen general
summary_csv = os.path.join(
    TEMPORAL_RESULTS_DIR,
    "temporal_summary_all_videos.csv"
)
```

```
df_temporal_summary_all.to_csv(
    summary_csv,
    index=False
)

print(
    "\nResumen guardado en:",
    summary_csv
)

================================================================================
VÍDEO 1/10
20260611_184739.mp4
================================================================================
Observaciones: 275
Tracks: 12
GREEN: 5
RED: 0
NON_ACTIONABLE: 7

================================================================================
VÍDEO 2/10
20260611_184808.mp4
================================================================================
Observaciones: 123
Tracks: 3
GREEN: 0
RED: 2
NON_ACTIONABLE: 1

================================================================================
VÍDEO 3/10
20260611_185035.mp4
================================================================================
Observaciones: 66
Tracks: 0
GREEN: 0
RED: 0
NON_ACTIONABLE: 0

================================================================================
```

VÍDEO 4/10
20260611_185443~2.mp4
==========
Observaciones: 60
Tracks: 2
GREEN: 1
RED: 0
NON_ACTIONABLE: 1

==========
VÍDEO 5/10
20260611_185616.mp4
==========
Observaciones: 435
Tracks: 12
GREEN: 2
RED: 0
NON_ACTIONABLE: 10

==========
VÍDEO 6/10
1000179702.mp4
==========
Observaciones: 534
Tracks: 0
GREEN: 0
RED: 0
NON_ACTIONABLE: 0

==========
VÍDEO 7/10
1000179704.mp4
==========
Observaciones: 623
Tracks: 8
GREEN: 1
RED: 0
NON_ACTIONABLE: 7

==========
VÍDEO 8/10

```
1000179707.mp4
================================================================================
=====
Observaciones: 473
Tracks: 8
GREEN: 0
RED: 4
NON_ACTIONABLE: 4

================================================================================
=====
VÍDEO 9/10
1000179709.mp4
================================================================================
=====
Observaciones: 335
Tracks: 4
GREEN: 0
RED: 3
NON_ACTIONABLE: 1

================================================================================
=====
VÍDEO 10/10
1000179711.mp4
================================================================================
=====
Observaciones: 1436
Tracks: 1
GREEN: 1
RED: 0
NON_ACTIONABLE: 0

================================================================================
==========
RESULTADO TEMPORAL DE LOS 10 VÍDEOS
================================================================================
==========
```

	video	observations	tracks	GREEN	RED	NON_ACTIONABLE
0	20260611_184739.mp4	275	12	5	0	7
1	20260611_184808.mp4	123	3	0	2	1
2	20260611_185035.mp4	66	0	0	0	0
3	20260611_185443~2.mp4	60	2	1	0	1
4	20260611_185616.mp4	435	12	2	0	10
5	1000179702.mp4	534	0	0	0	0
6	1000179704.mp4	623	8	1	0	7

	video	observations	tracks	GREEN	RED	NON_ACTIONABLE
7	1000179707.mp4	473	8	0	4	4
8	1000179709.mp4	335	4	0	3	1
9	1000179711.mp4	1436	1	1	0	0

```
Resumen guardado en: C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\temporal_signal_results\temporal_summary_all_videos.csv
```

In [221]:

```
VIDEO_FILES = [
    "1000179702.mp4",
    "1000179704.mp4",
    "1000179707.mp4",
    "1000179709.mp4",
    "1000179711.mp4",
    "20260611_184739.mp4",
    "20260611_184808.mp4",
    "20260611_185035.mp4",
    "20260611_185443~2.mp4",
    "20260611_185616.mp4"
]

print("Vídeos seleccionados:", len(VIDEO_FILES))

for video in VIDEO_FILES:
    print(video)
```

```
Vídeos seleccionados: 10
1000179702.mp4
1000179704.mp4
1000179707.mp4
1000179709.mp4
1000179711.mp4
20260611_184739.mp4
20260611_184808.mp4
20260611_185035.mp4
20260611_185443~2.mp4
20260611_185616.mp4
```

14.18 Evaluación final a nivel de vídeo

In [222]:

```
# DECISIÓN FINAL DEL ESTADO DEL SEMÁFORO A NIVEL DE VÍDEO


import pandas as pd
from sklearn.metrics import accuracy_score, confusion_matrix

# ------------------------------------------------------------
```

```
# Ground truth de los 10 vídeos
# ------------------------------------------------------------

GROUND_TRUTH_VIDEO = {
    "1000179702.mp4": "GREEN",
    "1000179704.mp4": "GREEN",
    "1000179707.mp4": "RED",
    "1000179709.mp4": "RED",
    "1000179711.mp4": "NONE",
    "20260611_184739.mp4": "GREEN",
    "20260611_184808.mp4": "RED",
    "20260611_185035.mp4": "NONE",
    "20260611_185443~2.mp4": "GREEN",
    "20260611_185616.mp4": "RED"
}


# ------------------------------------------------------------
# Decisión conservadora a nivel de vídeo
# ------------------------------------------------------------

def decide_video_state(row):

    green = int(row["GREEN"])
    red = int(row["RED"])

    # No existe evidencia peatonal utilizable
    if green == 0 and red == 0:
        return "NONE"

    # Solo evidencia verde
    if green > 0 and red == 0:
        return "GREEN"

    # Solo evidencia roja
    if red > 0 and green == 0:
        return "RED"

    # Existen evidencias contradictorias
    # Solo decidimos si una domina claramente a la otra
    if green >= 2 * red:
        return "GREEN"

    if red >= 2 * green:
        return "RED"

    # En un sistema asistencial es preferible abstenerse
    # antes que forzar una decisión poco fiable
    return "UNKNOWN"
```

```
# ------------------------------------------------------------
# Aplicar decisión
# ------------------------------------------------------------

df_video_eval = df_temporal_summary_all.copy()

df_video_eval["Real"] = (
    df_video_eval["video"]
    .map(GROUND_TRUTH_VIDEO)
)

df_video_eval["Predicho"] = (
    df_video_eval.apply(
        decide_video_state,
        axis=1
    )
)

df_video_eval["Correcto"] = (
    df_video_eval["Real"]
    == df_video_eval["Predicho"]
)


# ------------------------------------------------------------
# Resultados
# ------------------------------------------------------------

print("=" * 75)
print("EVALUACIÓN FINAL DEL SEMÁFORO A NIVEL DE VÍDEO")
print("=" * 75)

display(
    df_video_eval[
        [
            "video",
            "GREEN",
            "RED",
            "NON_ACTIONABLE",
            "Real",
            "Predicho",
            "Correcto"
        ]
    ]
)
```

```
accuracy_video = df_video_eval["Correcto"].mean()

print("\nAccuracy global:")
print(
    f"{df_video_eval['Correcto'].sum()}/"
    f"{len(df_video_eval)} "
    f"= {accuracy_video:.3f} "
    f"({100 * accuracy_video:.1f} %)"
)


# ------------------------------------------------------------
# Casos problemáticos
# ------------------------------------------------------------

errors_video = df_video_eval[
    ~df_video_eval["Correcto"]
]

print("\nCasos no resueltos correctamente:")
display(
    errors_video[
        [
            "video",
            "GREEN",
            "RED",
            "NON_ACTIONABLE",
            "Real",
            "Predicho"
        ]
    ]
)
```

```
================================================================================
EVALUACIÓN FINAL DEL SEMÁFORO A NIVEL DE VÍDEO
================================================================================
```

	video	GREEN	RED	NON_ACTIONABLE	Real	Predicho	Correcto
0	20260611_184739.mp4	5	0	7	GREEN	GREEN	True
1	20260611_184808.mp4	0	2	1	RED	RED	True
2	20260611_185035.mp4	0	0	0	NONE	NONE	True
3	20260611_185443~2.mp4	1	0	1	GREEN	GREEN	True
4	20260611_185616.mp4	2	0	10	RED	GREEN	False
5	1000179702.mp4	0	0	0	GREEN	NONE	False
6	1000179704.mp4	1	0	7	GREEN	GREEN	True
7	1000179707.mp4	0	4	4	RED	RED	True

	video	GREEN	RED	NON_ACTIONABLE	Real	Predicho	Correcto
8	1000179709.mp4	0	3	1	RED	RED	True
9	1000179711.mp4	1	0	0	NONE	GREEN	False

```
Accuracy global:
7/10 = 0.700 (70.0 %)
```

```
Casos no resueltos correctamente:
```

	video	GREEN	RED	NON_ACTIONABLE	Real	Predicho
4	20260611_185616.mp4	2	0	10	RED	GREEN
5	1000179702.mp4	0	0	0	GREEN	NONE
9	1000179711.mp4	1	0	0	NONE	GREEN

14.19 Cuantificar evidencia útil perdida por falta de track_id

In [223]:

```
# CUANTIFICAR EVIDENCIA ÚTIL PERDIDA POR FALTA DE TRACK_ID


import os
import pandas as pd

VIDEOS_CHECK = [
    "1000179702",
    "1000179704",
    "20260611_185616"
]

for video_id in VIDEOS_CHECK:

    obs_path = os.path.join(
        TEMPORAL_RESULTS_DIR,
        f"{video_id}_observations.csv"
    )

    df = pd.read_csv(obs_path)

    total = len(df)

    actionable = df["actionable"].fillna(False).astype(bool)

    no_track = df["track_id"].isna()

    useful_no_track = actionable & no_track
```

```
    green_no_track = (
        useful_no_track
        & (df["state"] == "GREEN")
    )

    red_no_track = (
        useful_no_track
        & (df["state"] == "RED")
    )

    print("\n" + "=" * 70)
    print(video_id)
    print("=" * 70)

    print("Observaciones totales:", total)
    print("ACTIONABLE:", actionable.sum())

    print(
        "ACTIONABLE sin track_id:",
        useful_no_track.sum()
    )

    print(
        "GREEN útiles sin track_id:",
        green_no_track.sum()
    )

    print(
        "RED útiles sin track_id:",
        red_no_track.sum()
    )

    if actionable.sum() > 0:

        perdida = (
            useful_no_track.sum()
            / actionable.sum()
        )

        print(
            "Porcentaje de evidencia ACTIONABLE perdida:",
            f"{100 * perdida:.1f}%"
        )
```

```
======================================================================
1000179702
======================================================================
Observaciones totales: 534
ACTIONABLE: 188
```

```
ACTIONABLE sin track_id: 188
GREEN útiles sin track_id: 114
RED útiles sin track_id: 74
Porcentaje de evidencia ACTIONABLE perdida: 100.0%

======================================================================
1000179704
======================================================================
Observaciones totales: 623
ACTIONABLE: 114
ACTIONABLE sin track_id: 109
GREEN útiles sin track_id: 99
RED útiles sin track_id: 10
Porcentaje de evidencia ACTIONABLE perdida: 95.6%

======================================================================
20260611_185616
======================================================================
Observaciones totales: 435
ACTIONABLE: 318
ACTIONABLE sin track_id: 185
GREEN útiles sin track_id: 137
RED útiles sin track_id: 48
Porcentaje de evidencia ACTIONABLE perdida: 58.2%
```

14.20 Inspección visual de evidencia actionable sin track_id

In [224]:

```
# INSPECCIÓN VISUAL DE EVIDENCIA ACTIONABLE SIN TRACK_ID


import os
import cv2
import pandas as pd
import matplotlib.pyplot as plt

VIDEOS_INSPECCION = [
    "1000179702",
    "1000179704"
]

N_MUESTRAS = 15

for video_id in VIDEOS_INSPECCION:

    print("\n" + "=" * 80)
    print("VÍDEO:", video_id)
```

```
print("=" * 80)

obs_path = os.path.join(
    TEMPORAL_RESULTS_DIR,
    f"{video_id}_observations.csv"
)

video_path = os.path.join(
    VIDEOS_DIR,
    f"{video_id}.mp4"
)

df = pd.read_csv(obs_path)

# Solo evidencia que el gate considera ACTIONABLE
# pero que ByteTrack no ha asociado a ningún track
lost = df[
    (df["actionable"] == True)
    & (df["track_id"].isna())
].copy()

# Ordenar por confianza del gate
lost = lost.sort_values(
    "p_actionable",
    ascending=False
)

# Evitar repetir exactamente la misma detección
lost = lost.drop_duplicates(
    subset=["frame", "x1", "y1", "x2", "y2"]
)

lost = lost.head(N_MUESTRAS)

print("Candidatos mostrados:", len(lost))

cap = cv2.VideoCapture(video_path)

cols = 5
rows = (len(lost) + cols - 1) // cols

plt.figure(
    figsize=(16, 3.5 * rows)
)

for pos, (_, row) in enumerate(
    lost.iterrows(),
    start=1
):
```

```
        frame_id = int(row["frame"])

        cap.set(
            cv2.CAP_PROP_POS_FRAMES,
            frame_id
        )

        ok, frame = cap.read()

        if not ok:
            continue

        x1 = max(0, int(row["x1"]))
        y1 = max(0, int(row["y1"]))
        x2 = min(frame.shape[1], int(row["x2"]))
        y2 = min(frame.shape[0], int(row["y2"]))

        crop = frame[
            y1:y2,
            x1:x2
        ]

        crop = cv2.cvtColor(
            crop,
            cv2.COLOR_BGR2RGB
        )

        plt.subplot(
            rows,
            cols,
            pos
        )

        plt.imshow(crop)

        plt.title(
            f"frame={frame_id}\n"
            f"{row['state']} | "
            f"Pact={row['p_actionable']:.2f}\n"
            f"G={row['green_score']:.2f} "
            f"R={row['red_score']:.2f}"
        )

        plt.axis("off")

    cap.release()

    plt.tight_layout()
```

```
    plt.show()

================================================================================
==========
VÍDEO: 1000179702
================================================================================
==========
Candidatos mostrados: 15
<Figure size 1600x1050 with 15 Axes>

================================================================================
==========
VÍDEO: 1000179704
================================================================================
==========
Candidatos mostrados: 15
<Figure size 1600x1050 with 15 Axes>
```

14.21 Analizar distribución espacial de la evidencia sin track

In [225]:

```
# ANALIZAR DISTRIBUCIÓN ESPACIAL DE LA EVIDENCIA SIN TRACK

import os
import pandas as pd

VIDEOS_CHECK = [
    "1000179702",
    "1000179704"
]

for video_id in VIDEOS_CHECK:

    path = os.path.join(
        TEMPORAL_RESULTS_DIR,
        f"{video_id}_observations.csv"
    )

    df = pd.read_csv(path)

    df = df[
        (df["actionable"] == True)
        & (df["track_id"].isna())
    ].copy()

    # Centro de cada detección
    df["cx"] = (
```

```
    df["x1"] + df["x2"]
) / 2

df["cy"] = (
    df["y1"] + df["y2"]
) / 2

print("\n" + "=" * 75)
print("VÍDEO:", video_id)
print("=" * 75)

for estado in ["GREEN", "RED"]:

    aux = df[
        df["state"] == estado
    ]

    print(f"\n{estado}")
    print("Observaciones:", len(aux))

    if len(aux) > 0:

        print(
            "cx medio:",
            round(aux["cx"].mean(), 1)
        )

        print(
            "cy medio:",
            round(aux["cy"].mean(), 1)
        )

        print(
            "cx rango:",
            round(aux["cx"].min(), 1),
            "-",
            round(aux["cx"].max(), 1)
        )

        print(
            "cy rango:",
            round(aux["cy"].min(), 1),
            "-",
            round(aux["cy"].max(), 1)
        )

        print(
            "P(actionable) media:",
            round(aux["p_actionable"].mean(), 3)
```

```
            )

            print(
                "score semántico medio:",
                round(
                    (
                        aux["green_score"]
                        if estado == "GREEN"
                        else aux["red_score"]
                    ).mean(),
                    3
                )
            )

================================================================================
VÍDEO: 1000179702
================================================================================

GREEN
Observaciones: 114
cx medio: 751.5
cy medio: 359.6
cx rango: 148.0 - 1060.5
cy rango: 53.0 - 671.0
P(actionable) media: 0.655
score semántico medio: 0.68

RED
Observaciones: 74
cx medio: 687.1
cy medio: 330.3
cx rango: 54.0 - 1064.5
cy rango: 24.5 - 1209.5
P(actionable) media: 0.684
score semántico medio: 0.747

================================================================================
VÍDEO: 1000179704
================================================================================

GREEN
Observaciones: 99
cx medio: 667.1
cy medio: 296.9
cx rango: 15.5 - 1070.5
```

```
cy rango: 75.0 - 453.0
P(actionable) media: 0.639
score semántico medio: 0.765

RED
Observaciones: 10
cx medio: 632.5
cy medio: 299.6
cx rango: 31.0 - 930.5
cy rango: 99.0 - 690.5
P(actionable) media: 0.611
score semántico medio: 0.593
```

15. Refinamiento del pipeline a partir del análisis de errores

15.1 Ampliación del ground truth de entrenamiento

In [229]:

```
# AMPLIACIÓN DEL GROUND TRUTH DE TRAIN
# MOSTRAR IDs 40-59

%matplotlib inline

import matplotlib.pyplot as plt
import math

START_ID = 40
END_ID = 60

subset = [
    item
    for item in train_crops
    if START_ID <= int(item["id"]) < END_ID
]

cols = 5
rows = math.ceil(len(subset) / cols)

plt.figure(
    figsize=(15, 3.2 * rows)
)

for pos, item in enumerate(
    subset,
```

```
    start=1
):

    plt.subplot(
        rows,
        cols,
        pos
    )

    plt.imshow(
        item["crop"]
    )

    plt.title(
        f"ID {item['id']}",
        fontsize=10
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

ID 40



ID 41





ID 45



ID 46





ID 50



ID 51





ID 55



ID 56





15.2 Ampliación del ground truth de train

In [230]:

```
# ============================================================
# AMPLIACIÓN DEL GROUND TRUTH DE TRAIN
# MOSTRAR IDs 60-80
# ============================================================

import matplotlib.pyplot as plt
import math

START_ID = 60
END_ID = 80

subset = [
    item
    for item in train_crops
    if START_ID <= int(item["id"]) < END_ID
]

cols = 5
rows = math.ceil(len(subset) / cols)

plt.figure(
    figsize=(15, 3.2 * rows)
)

for pos, item in enumerate(
    subset,
    start=1
):

    plt.subplot(
        rows,
        cols,
        pos
    )

    plt.imshow(
        item["crop"]
    )

    plt.title(
        f"ID {item['id']}",
        fontsize=10
    )

    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

ID 60



ID 61





ID 65



ID 66





ID 70

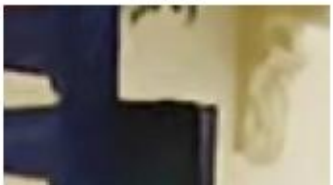


ID 71



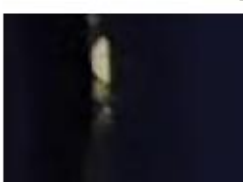


ID 75



ID 76





15.3 Ampliación del ground truth de train

In [231]:

```
# AMPLIACIÓN DEL GROUND TRUTH DE TRAIN
# MOSTRAR IDs 80-99

import matplotlib.pyplot as plt
import math

START_ID = 80
END_ID = 99

subset = [
    item
    for item in train_crops
    if START_ID <= int(item["id"]) < END_ID
]

cols = 5
rows = math.ceil(len(subset) / cols)

plt.figure(
    figsize=(15, 3.2 * rows)
)

for pos, item in enumerate(
    subset,
    start=1
):

    plt.subplot(
        rows,
        cols,
        pos
    )

    plt.imshow(
        item["crop"]
    )

    plt.title(
        f"ID {item['id']}",
        fontsize=10
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

ID 80



ID 81





ID 85



ID 86





ID 90



ID 91





ID 95



ID 96





15.4 Descripción manual de los semáforos peatonales

In [16]:

```
GROUND_TRUTH_40_59 = {
    40: "NON_ACTIONABLE",
    41: "GREEN",
    42: "NON_ACTIONABLE",
    43: "GREEN",
    44: "GREEN",

    45: "GREEN",
    46: "GREEN",
    47: "NON_ACTIONABLE",
    48: "GREEN",
    49: "NON_ACTIONABLE",

    50: "GREEN",
    51: "NON_ACTIONABLE",
    52: "GREEN",
    53: "NON_ACTIONABLE",
    54: "NON_ACTIONABLE",

    55: "RED",
    56: "RED",
    57: "RED",
    58: "RED",
    59: "RED"
}

GROUND_TRUTH_60_79 = {
    60: "RED",
    61: "RED",
    62: "RED",
    63: "RED",
    64: "RED",

    65: "RED",
    66: "RED",
    67: "RED",
    68: "NON_ACTIONABLE",
    69: "GREEN",

    70: "NON_ACTIONABLE",
    71: "GREEN",
    72: "NON_ACTIONABLE",
    73: "GREEN",
    74: "NON_ACTIONABLE",
```

```
    75: "GREEN",
    76: "GREEN",
    77: "NON_ACTIONABLE",
    78: "NON_ACTIONABLE",
    79: "GREEN"
}

GROUND_TRUTH_80_98 = {
    80: "NON_ACTIONABLE",
    81: "GREEN",
    82: "RED",
    83: "NON_ACTIONABLE",
    84: "NON_ACTIONABLE",

    85: "RED",
    86: "NON_ACTIONABLE",
    87: "RED",
    88: "NON_ACTIONABLE",
    89: "RED",

    90: "NON_ACTIONABLE",
    91: "RED",
    92: "RED",
    93: "RED",
    94: "RED",

    95: "RED",
    96: "RED",
    97: "RED",
    98: "RED"
}
```

15.5 Ejecución de la descripción del número total de estados de semáforos peatonales

In [17]:

```
# UNIR EL GROUND TRUTH COMPLETO DE LOS 99 CROPS

GROUND_TRUTH_99 = {}

GROUND_TRUTH_99.update(GROUND_TRUTH_40)
GROUND_TRUTH_99.update(GROUND_TRUTH_40_59)
GROUND_TRUTH_99.update(GROUND_TRUTH_60_79)
GROUND_TRUTH_99.update(GROUND_TRUTH_80_98)

print("Total de etiquetas:", len(GROUND_TRUTH_99))
```

```
# Comprobar IDs
ids_esperados = set(range(99))
ids_reales = set(GROUND_TRUTH_99.keys())

faltan = sorted(ids_esperados - ids_reales)
sobran = sorted(ids_reales - ids_esperados)

print("IDs que faltan:", faltan)
print("IDs inesperados:", sobran)

# Distribución de clases
from collections import Counter

conteo = Counter(GROUND_TRUTH_99.values())

print("\nDistribución:")
for clase, n in conteo.items():
    print(f"{clase}: {n}")

print("\nACTIONABLE:",
      conteo["RED"] + conteo["GREEN"])

print("NON_ACTIONABLE:",
      conteo["NON_ACTIONABLE"])
Total de etiquetas: 99
IDs que faltan: []
IDs inesperados: []

Distribución:
RED: 38
NON_ACTIONABLE: 37
GREEN: 24

ACTIONABLE: 62
NON_ACTIONABLE: 37
```

15.6 Entrenamiento y validación cruzada del Gate V2

In [36]:

```
# ============================================================
# GATE V2 - ENTRENAMIENTO CON LOS 99 CROPS DE TRAIN
# VALIDACIÓN CRUZADA ESTRATIFICADA DE 5 FOLDS
# ============================================================

import numpy as np

from sklearn.base import clone
```

```
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# ------------------------------------------------------------
# 1. Relacionar ID -> crop
# ------------------------------------------------------------

crops_by_id = {
    int(item["id"]): item["crop"]
    for item in train_crops
}

# ------------------------------------------------------------
# 2. Crear X e y con los 99 casos
# ------------------------------------------------------------

X_99 = []
y_99 = []

for crop_id in sorted(GROUND_TRUTH_99.keys()):

    real_label = GROUND_TRUTH_99[crop_id]

    binary_label = (
        0
        if real_label == "NON_ACTIONABLE"
        else 1
    )

    embedding = get_clip_image_embedding(
        crops_by_id[crop_id]
    )

    X_99.append(embedding)
    y_99.append(binary_label)

X_99 = np.asarray(X_99)
y_99 = np.asarray(y_99)

print("Muestras:", len(y_99))
print("Dimensión embedding:", X_99.shape)

print(
    "ACTIONABLE:",
    int((y_99 == 1).sum())
```

```
)

print(
    "NON_ACTIONABLE:",
    int((y_99 == 0).sum())
)

# ------------------------------------------------------------
# 3. Validación cruzada de 5 folds
# ------------------------------------------------------------

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

gate_cv = clone(gate_model)

y_pred_cv_99 = cross_val_predict(
    gate_cv,
    X_99,
    y_99,
    cv=cv
)

accuracy_99 = accuracy_score(
    y_99,
    y_pred_cv_99
)

cm_99 = confusion_matrix(
    y_99,
    y_pred_cv_99
)

print("\n" + "=" * 65)
print("GATE V2 - VALIDACIÓN CRUZADA SOBRE 99 CASOS")
print("=" * 65)

print(
    f"Accuracy: {accuracy_99:.4f} "
    f"({100 * accuracy_99:.1f} %)"
)

print("\nMatriz de confusión:")
print(cm_99)

print("\nInforme:")
```

```
print(
    classification_report(
        y_99,
        y_pred_cv_99,
        target_names=[
            "NON_ACTIONABLE",
            "ACTIONABLE"
        ],
        digits=3
    )
)

# ------------------------------------------------------------
# 4. Entrenar modelo final con los 99 casos
# ------------------------------------------------------------

gate_model_v2 = clone(gate_model)

gate_model_v2.fit(
    X_99,
    y_99
)

print("Gate V2 final entrenado con los 99 casos.")
Muestras: 99
Dimensión embedding: (99, 512)
ACTIONABLE: 62
NON_ACTIONABLE: 37

======================================================================
GATE V2 - VALIDACIÓN CRUZADA SOBRE 99 CASOS
======================================================================
Accuracy: 0.8687 (86.9 %)

Matriz de confusión:
[[34  3]
 [10 52]]

Informe:
                precision    recall  f1-score   support

NON_ACTIONABLE      0.773     0.919     0.840        37
    ACTIONABLE      0.945     0.839     0.889        62

      accuracy                          0.869        99
     macro avg      0.859     0.879     0.864        99
  weighted avg      0.881     0.869     0.870        99

Gate V2 final entrenado con los 99 casos.
```

15.7 Guardado del clasificador supervisado definitivo

In [236]:

```
# GUARDAR GATE V2 DEFINITIVO

import os
import joblib

GATE_V2_PATH = os.path.join(
    r"C:\Users\juank\Documents\CARLOS\TFM",
    "models_finales",
    "clip_gate_actionable_v2_99.pkl"
)

joblib.dump(
    gate_model_v2,
    GATE_V2_PATH
)

print("Gate V2 guardado en:")
print(GATE_V2_PATH)

print("\n¿Existe?")
print(os.path.isfile(GATE_V2_PATH))
```

```
Gate V2 guardado en:
C:\Users\juank\Documents\CARLOS\TFM\models_finales\clip_gate_actionabl
e_v2_99.pkl

¿Existe?
True
```

16 Mejora del seguimiento temporal de semáforos pequeños

16.1 Asociación espacio-temporal alternativa para semáforos pequeños

In [237]:

```
# TRACKING TEMPORAL ALTERNATIVO PARA SEMÁFOROS PEQUEÑOS
# Asociación por proximidad espacial + continuidad temporal

import os
import numpy as np
import pandas as pd
```

```
def assign_signal_tracks_spatial(
    df,
    max_frame_gap=6,
    max_dist_factor=2.5
):
    """
    Asocia detecciones de semáforo entre frames consecutivos
    usando distancia entre centros y tamaño de las cajas.

    No depende del track_id generado por ByteTrack.
    """

    data = df.copy()

    # Orden temporal
    data = data.sort_values(
        ["frame"]
    ).reset_index(drop=True)

    # Centro y tamaño
    data["cx"] = (
        data["x1"] + data["x2"]
    ) / 2

    data["cy"] = (
        data["y1"] + data["y2"]
    ) / 2

    data["w"] = (
        data["x2"] - data["x1"]
    )

    data["h"] = (
        data["y2"] - data["y1"]
    )

    data["diag"] = np.sqrt(
        data["w"] ** 2
        +
        data["h"] ** 2
    )

    # Nuevo identificador temporal
    data["spatial_track_id"] = -1

    active_tracks = {}
    next_track_id = 0
```

```
for idx, row in data.iterrows():

    frame = int(row["frame"])

    best_track = None
    best_distance = np.inf

    for track_id, previous in list(
        active_tracks.items()
    ):

        frame_gap = (
            frame
            -
            previous["frame"]
        )

        # Track demasiado antiguo
        if frame_gap > max_frame_gap:
            continue

        dx = (
            row["cx"]
            -
            previous["cx"]
        )

        dy = (
            row["cy"]
            -
            previous["cy"]
        )

        distance = np.sqrt(
            dx ** 2 + dy ** 2
        )

        reference_diag = max(
            row["diag"],
            previous["diag"],
            1
        )

        normalized_distance = (
            distance
            /
            reference_diag
        )
```

```
            if (
                normalized_distance
                <= max_dist_factor
                and normalized_distance
                < best_distance
            ):
                best_track = track_id
                best_distance = normalized_distance

        # Si no coincide con ningún track existente,
        # creamos uno nuevo
        if best_track is None:

            best_track = next_track_id
            next_track_id += 1

        data.at[
            idx,
            "spatial_track_id"
        ] = best_track

        active_tracks[best_track] = {
            "frame": frame,
            "cx": row["cx"],
            "cy": row["cy"],
            "diag": row["diag"]
        }

    return data
```

16.2 Prueba del tracking alternativo en el vídeo 1000179704

In [238]:

```
# ============================================================
# PRUEBA EN 1000179704
# ============================================================

video_id = "1000179704"

obs_path = os.path.join(
    TEMPORAL_RESULTS_DIR,
    f"{video_id}_observations.csv"
)

df_test_tracking = pd.read_csv(
    obs_path
```

```
)

# Nos interesa especialmente la evidencia aceptada por el gate
df_actionable = df_test_tracking[
    df_test_tracking["actionable"] == True
].copy()

df_actionable_spatial = (
    assign_signal_tracks_spatial(
        df_actionable,
        max_frame_gap=6,
        max_dist_factor=2.5
    )
)

print(
    "Observaciones ACTIONABLE:",
    len(df_actionable_spatial)
)

print(
    "Tracks espaciales creados:",
    df_actionable_spatial[
        "spatial_track_id"
    ].nunique()
)
Observaciones ACTIONABLE: 114
Tracks espaciales creados: 11
```

16.3 Resumen y evaluación de los nuevos tracks

In [239]:

```
# RESUMEN DE LOS NUEVOS TRACKS


track_spatial_summary = (
    df_actionable_spatial
    .groupby("spatial_track_id")
    .agg(
        observations=("frame", "count"),

        first_frame=("frame", "min"),
        last_frame=("frame", "max"),

        mean_p_actionable=(
            "p_actionable",
            "mean"
        ),
```

```
        green_hits=(
            "state",
            lambda x: (x == "GREEN").sum()
        ),

        red_hits=(
            "state",
            lambda x: (x == "RED").sum()
        ),

        mean_green_score=(
            "green_score",
            "mean"
        ),

        mean_red_score=(
            "red_score",
            "mean"
        )
    )
    .reset_index()
)

track_spatial_summary[
    "dominant_state"
] = np.where(
    track_spatial_summary["green_hits"]
    >
    track_spatial_summary["red_hits"],
    "GREEN",
    np.where(
        track_spatial_summary["red_hits"]
        >
        track_spatial_summary["green_hits"],
        "RED",
        "UNKNOWN"
    )
)

track_spatial_summary = (
    track_spatial_summary
    .sort_values(
        "observations",
        ascending=False
    )
)

display(
    track_spatial_summary
```

)

	spatial_track_id	observations	first_frame	last_frame	mean_p_actionable	green_hits	red_hits	r
10	10	53	564	636	0.685967	48	5	0
9	9	19	525	549	0.634976	19	0	0
7	7	17	474	501	0.585521	17	0	0
6	6	8	447	465	0.573117	8	0	0
3	3	6	72	81	0.586232	4	2	0
5	5	3	438	438	0.706911	0	3	0
4	4	3	405	405	0.693100	3	0	0
1	1	2	33	36	0.558034	2	0	0
0	0	1	21	21	0.543191	1	0	0
2	2	1	66	66	0.559371	1	0	0
8	8	1	510	510	0.550921	1	0	0

16.4 Tracking espacial corregido para semáforos pequeños

In [80]:

```
# ============================================================
# TRACKING ESPACIAL CORREGIDO PARA SEMÁFOROS PEQUEÑOS
# + RECONEXIÓN CONSERVADORA DE FRAGMENTOS
# ============================================================

def assign_signal_tracks_spatial(
    df,
    max_frame_gap=6,
    max_dist_factor=2.5,
    relink_max_gap=9,
    relink_dist_factor=1.0
):

    data = df.copy()

    data = (
        data
        .sort_values(
            ["frame", "p_actionable"],
            ascending=[True, False]
        )
        .reset_index(drop=True)
    )

    # ------------------------------------------------------
    # Centro y tamaño de cada detección
```

```
# ----------------------------------------------------------

data["cx"] = (
    data["x1"] + data["x2"]
) / 2

data["cy"] = (
    data["y1"] + data["y2"]
) / 2

data["w"] = (
    data["x2"] - data["x1"]
)

data["h"] = (
    data["y2"] - data["y1"]
)

data["diag"] = np.sqrt(
    data["w"]**2
    +
    data["h"]**2
)

data["spatial_track_id"] = -1

active_tracks = {}
next_track_id = 0

# ==========================================================
# 1. TRACKING NORMAL
# ==========================================================

for idx, row in data.iterrows():

    frame = int(row["frame"])

    best_track = None
    best_distance = np.inf

    for track_id, previous in active_tracks.items():

        frame_gap = (
            frame
            - previous["frame"]
        )

        # No asociar detecciones del mismo frame
        if frame_gap <= 0:
```

```
            continue

        # Ventana temporal normal
        if frame_gap > max_frame_gap:
            continue

        dx = (
            row["cx"]
            - previous["cx"]
        )

        dy = (
            row["cy"]
            - previous["cy"]
        )

        distance = np.sqrt(
            dx**2 + dy**2
        )

        reference_diag = max(
            row["diag"],
            previous["diag"],
            1
        )

        normalized_distance = (
            distance
            / reference_diag
        )

        if (
            normalized_distance
            <= max_dist_factor
            and
            normalized_distance
            < best_distance
        ):
            best_track = track_id
            best_distance = normalized_distance

    # Nuevo track
    if best_track is None:
        best_track = next_track_id
        next_track_id += 1

    data.at[
        idx,
        "spatial_track_id"
```

```
    ] = best_track

    active_tracks[best_track] = {
        "frame": frame,
        "cx": row["cx"],
        "cy": row["cy"],
        "diag": row["diag"]
    }

# ============================================================
# 2. RECONEXIÓN CONSERVADORA DE FRAGMENTOS
# ============================================================
#
# No ampliamos max_frame_gap globalmente.
# Solo intentamos unir tracks ya creados cuando:
#
# - ambos tienen al menos 3 observaciones;
# - al menos uno es un fragmento corto (3 observaciones);
# - la separación temporal está entre 7 y 9 frames;
# - ambos representan el mismo estado GREEN/RED;
# - sus extremos están espacialmente muy próximos.
# ============================================================

track_info = {}

for track_id, group in data.groupby(
    "spatial_track_id"
):

    group = (
        group
        .sort_values("frame")
    )

    observations = len(group)

    if observations < 3:
        continue

    green_hits = int(
        (group["state"] == "GREEN").sum()
    )

    red_hits = int(
        (group["state"] == "RED").sum()
    )

    if green_hits > red_hits:
        dominant_state = "GREEN"
```

```
    elif red_hits > green_hits:
        dominant_state = "RED"

    else:
        dominant_state = "UNKNOWN"

    first = group.iloc[0]
    last = group.iloc[-1]

    track_info[track_id] = {
        "observations": observations,
        "first": first,
        "last": last,
        "dominant_state": dominant_state
    }

merge_candidates = []

track_ids = list(track_info.keys())

for track_a in track_ids:

    info_a = track_info[track_a]

    for track_b in track_ids:

        if track_a == track_b:
            continue

        info_b = track_info[track_b]

        # B debe comenzar después de A
        gap = (
            int(info_b["first"]["frame"])
            -
            int(info_a["last"]["frame"])
        )

        # Solo huecos que el tracking normal no cubre
        if (
            gap <= max_frame_gap
            or
            gap > relink_max_gap
        ):
            continue

        # Al menos uno debe ser un fragmento muy corto
        if not (
```

```
    info_a["observations"]>= 4
    and
    info_b["observations"]== 3
):
    continue

# Mismo estado dominante
if (
    info_a["dominant_state"]
    != info_b["dominant_state"]
):
    continue

if (
    info_a["dominant_state"]
    not in ["GREEN", "RED"]
):
    continue

# También exigimos mismo estado en los extremos
if (
    info_a["last"]["state"]
    != info_b["first"]["state"]
):
    continue

dx = (
    info_b["first"]["cx"]
    -
    info_a["last"]["cx"]
)

dy = (
    info_b["first"]["cy"]
    -
    info_a["last"]["cy"]
)

distance = np.sqrt(
    dx**2 + dy**2
)

reference_diag = max(
    info_a["last"]["diag"],
    info_b["first"]["diag"],
    1
)

normalized_distance = (
```

```
            distance
            / reference_diag
        )

        if (
            normalized_distance
            <= relink_dist_factor
        ):
            merge_candidates.append(
                (
                    normalized_distance,
                    track_a,
                    track_b
                )
            )

# ------------------------------------------------------------
# Aplicar fusiones de forma 1 a 1,
# empezando por la asociación espacial más clara
# ------------------------------------------------------------

merge_candidates.sort()

used_sources = set()
used_targets = set()

for _, track_a, track_b in merge_candidates:

    if (
        track_a in used_sources
        or
        track_b in used_targets
    ):
        continue

    data.loc[
        data["spatial_track_id"] == track_b,
        "spatial_track_id"
    ] = track_a

    used_sources.add(track_a)
    used_targets.add(track_b)

# ------------------------------------------------------------
# Renumerar tracks después de las fusiones
# ------------------------------------------------------------

unique_ids = sorted(
    data["spatial_track_id"].unique()
```

```
    )

    id_map = {
        old_id: new_id
        for new_id, old_id
        in enumerate(unique_ids)
    }

    data["spatial_track_id"] = (
        data["spatial_track_id"]
        .map(id_map)
        .astype(int)
    )

    return data
```

16.5 Evaluación de los tracks espaciales corregidos

In [241]:

```
# RESUMEN DE TRACKS ESPACIALES


track_spatial_summary = (
    df_actionable_spatial
    .groupby("spatial_track_id")
    .agg(
        observations=("frame", "count"),
        first_frame=("frame", "min"),
        last_frame=("frame", "max"),

        mean_p_actionable=(
            "p_actionable",
            "mean"
        ),

        green_hits=(
            "state",
            lambda x: (x == "GREEN").sum()
        ),

        red_hits=(
            "state",
            lambda x: (x == "RED").sum()
        ),

        mean_green_score=(
            "green_score",
            "mean"
        ),
```

```
        mean_red_score=(
            "red_score",
            "mean"
        )
    )
    .reset_index()
)

def dominant_state(row):

    if row["green_hits"] > row["red_hits"]:
        return "GREEN"

    if row["red_hits"] > row["green_hits"]:
        return "RED"

    return "UNKNOWN"


track_spatial_summary["dominant_state"] = (
    track_spatial_summary.apply(
        dominant_state,
        axis=1
    )
)

track_spatial_summary = (
    track_spatial_summary
    .sort_values(
        ["observations", "mean_p_actionable"],
        ascending=[False, False]
    )
)

display(track_spatial_summary)
```

	spatial_track_id	observations	first_frame	last_frame	mean_p_actionable	green_hits	red_hits	r
10	10	53	564	636	0.685967	48	5	0
9	9	19	525	549	0.634976	19	0	0
7	7	17	474	501	0.585521	17	0	0
6	6	8	447	465	0.573117	8	0	0
3	3	6	72	81	0.586232	4	2	0
5	5	3	438	438	0.706911	0	3	0
4	4	3	405	405	0.693100	3	0	0
1	1	2	33	36	0.558034	2	0	0
2	2	1	66	66	0.559371	1	0	0

	spatial_track_id	observations	first_frame	last_frame	mean_p_actionable	green_hits	red_hits	r
8	8	1	510	510	0.550921	1	0	0
0	0	1	21	21	0.543191	1	0	0

In [242]:

```
# PRUEBA DEL MISMO TRACKING ESPACIAL EN 1000179702


video_id = "1000179702"

obs_path = os.path.join(
    TEMPORAL_RESULTS_DIR,
    f"{video_id}_observations.csv"
)

df_test_tracking_702 = pd.read_csv(obs_path)

df_actionable_702 = df_test_tracking_702[
    df_test_tracking_702["actionable"] == True
].copy()

df_actionable_spatial_702 = assign_signal_tracks_spatial(
    df_actionable_702,
    max_frame_gap=6,
    max_dist_factor=2.5
)

print(
    "Observaciones ACTIONABLE:",
    len(df_actionable_spatial_702)
)

print(
    "Tracks espaciales creados:",
    df_actionable_spatial_702[
        "spatial_track_id"
    ].nunique()
)


# ============================================================
# RESUMEN DE TRACKS
# ============================================================

track_summary_702 = (
    df_actionable_spatial_702
    .groupby("spatial_track_id")
    .agg(
        observations=("frame", "count"),
```

```
        first_frame=("frame", "min"),
        last_frame=("frame", "max"),

        mean_p_actionable=(
            "p_actionable",
            "mean"
        ),

        green_hits=(
            "state",
            lambda x: (x == "GREEN").sum()
        ),

        red_hits=(
            "state",
            lambda x: (x == "RED").sum()
        ),

        mean_green_score=(
            "green_score",
            "mean"
        ),

        mean_red_score=(
            "red_score",
            "mean"
        )
    )
    .reset_index()
)


def dominant_state_702(row):

    if row["green_hits"] > row["red_hits"]:
        return "GREEN"

    if row["red_hits"] > row["green_hits"]:
        return "RED"

    return "UNKNOWN"


track_summary_702["dominant_state"] = (
    track_summary_702.apply(
        dominant_state_702,
        axis=1
    )
)
```

```
# Solo como diagnóstico:
# marcar cuáles tienen persistencia temporal
track_summary_702["persistent"] = (
    track_summary_702["observations"] >= 2
)

track_summary_702 = (
    track_summary_702
    .sort_values(
        ["observations", "mean_p_actionable"],
        ascending=[False, False]
    )
)

display(track_summary_702)
Observaciones ACTIONABLE: 188
Tracks espaciales creados: 61
```

	spatial_track_id	observations	first_frame	last_frame	mean_p_actionable	green_hits	red_hits	n
50	50	13	519	555	0.767711	4	9	0
4	4	12	60	96	0.614050	8	4	0
9	9	8	84	111	0.711429	7	1	0
46	46	8	486	510	0.705406	4	4	0
45	45	7	483	501	0.695356	4	3	0
...	...	...	...	...	...	...	...	.
33	33	1	231	231	0.535145	1	0	0
25	25	1	162	162	0.530027	0	1	0
7	7	1	69	69	0.529609	0	1	0
26	26	1	162	162	0.525233	1	0	0
31	31	1	189	189	0.504423	1	0	0

61 rows × 11 columns

16.6 Inspección visual de la persistencia temporal

In [243]:

```
%matplotlib inline
import matplotlib.pyplot as plt

print("Backend:", plt.get_backend())
Backend: inline
```

In [244]:

```
# INSPECCIÓN VISUAL DE LOS TRACKS PERSISTENTES - 1000179702
```

```
import cv2
import matplotlib.pyplot as plt
import os
import math

video_id = "1000179702"

video_path = os.path.join(
    VIDEOS_DIR,
    f"{video_id}.mp4"
)

# Nos quedamos con tracks de al menos 3 observaciones
top_tracks = (
    track_summary_702[
        track_summary_702["observations"] >= 3
    ]
    .sort_values(
        "observations",
        ascending=False
    )
    .head(12)
)

track_ids = top_tracks[
    "spatial_track_id"
].tolist()

print("Tracks inspeccionados:")
print(track_ids)

cap = cv2.VideoCapture(video_path)

cols = 4
rows = math.ceil(len(track_ids) / cols)

plt.figure(
    figsize=(16, 4 * rows)
)

for pos, track_id in enumerate(
    track_ids,
    start=1
):

    # Observaciones pertenecientes al track
    aux = df_actionable_spatial_702[
        df_actionable_spatial_702[
            "spatial_track_id"
```

```
    ] == track_id
].copy()

# Elegimos como representante la observación
# con mayor P(ACTIONABLE)
representative = (
    aux
    .sort_values(
        "p_actionable",
        ascending=False
    )
    .iloc[0]
)

frame_id = int(
    representative["frame"]
)

cap.set(
    cv2.CAP_PROP_POS_FRAMES,
    frame_id
)

ok, frame = cap.read()

if not ok:
    continue

x1 = int(representative["x1"])
y1 = int(representative["y1"])
x2 = int(representative["x2"])
y2 = int(representative["y2"])

# Añadimos contexto alrededor del bbox
w = x2 - x1
h = y2 - y1

margin_x = int(w * 0.30)
margin_y = int(h * 0.30)

x1c = max(0, x1 - margin_x)
y1c = max(0, y1 - margin_y)

x2c = min(
    frame.shape[1],
    x2 + margin_x
)

y2c = min(
```

```
            frame.shape[0],
            y2 + margin_y
        )

        crop = frame[
            y1c:y2c,
            x1c:x2c
        ]

        crop = cv2.cvtColor(
            crop,
            cv2.COLOR_BGR2RGB
        )

        summary_row = (
            top_tracks[
                top_tracks[
                    "spatial_track_id"
                ] == track_id
            ]
            .iloc[0]
        )

        plt.subplot(
            rows,
            cols,
            pos
        )

        plt.imshow(crop)

        plt.title(
            f"Track {track_id}\n"
            f"obs={int(summary_row['observations'])} | "
            f"{summary_row['dominant_state']}\n"
            f"G={int(summary_row['green_hits'])} "
            f"R={int(summary_row['red_hits'])}"
        )

        plt.axis("off")

cap.release()

plt.tight_layout()
plt.show()
Tracks inspeccionados:
[50, 4, 9, 46, 45, 16, 11, 17, 51, 56, 0, 1]
```

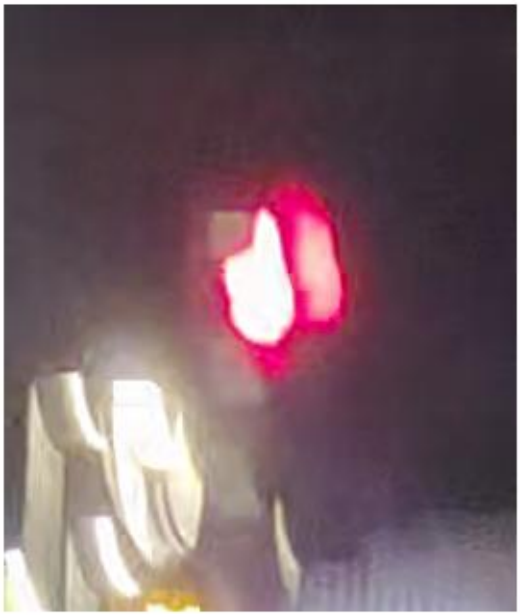
Track 50
obs=13 | RED
G=4 R=9



Track 4
obs=12 | GREEN
G=8 R=4



Track 45
obs=7 | GREEN
G=4 R=3



Track 16
obs=6 | GREEN
G=5 R=1



Track 51
obs=5 | GREEN
G=5 R=0



Track 56
obs=5 | RED
G=2 R=3



16.7 Reprocesamiento con el Gate V2 definitivo

In [245]:

```
# REPROCESAR 1000179702 CON GATE V2
# SIN SOBRESCRIBIR LOS RESULTADOS ANTIGUOS


import os

VIDEO_TEST_V2 = "1000179702.mp4"

video_path_v2 = os.path.join(
    VIDEOS_DIR,
    VIDEO_TEST_V2
)

print("Vídeo:")
print(video_path_v2)

print("\n¿Existe?")
print(os.path.isfile(video_path_v2))


# ------------------------------------------------------------
# Sustituir temporalmente el gate antiguo por Gate V2
# ------------------------------------------------------------

gate_model_backup = (
    gate_model
    if "gate_model" in globals()
    else None
)

gate_model = gate_model_v2


# ------------------------------------------------------------
# Procesar el vídeo
# ------------------------------------------------------------

df_v2_702, tracks_v2_702 = (
    process_temporal_signal_video(
        video_path_v2
    )
)


# ------------------------------------------------------------
# Restaurar el gate anterior
```

```
# ------------------------------------------------------------

if gate_model_backup is not None:
    gate_model = gate_model_backup


print("\n" + "=" * 70)
print("RESULTADO CON GATE V2")
print("=" * 70)

print(
    "Observaciones:",
    len(df_v2_702)
)

print(
    "ACTIONABLE:",
    int(df_v2_702["actionable"].sum())
)

print(
    "NON_ACTIONABLE:",
    int((~df_v2_702["actionable"]).sum())
)

print("\nEstados ACTIONABLE:")

print(
    df_v2_702[
        df_v2_702["actionable"] == True
    ]["state"].value_counts()
)
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\1000179702.mp4

¿Existe?
True

======================================================================
RESULTADO CON GATE V2
======================================================================
Observaciones: 534
ACTIONABLE: 97
NON_ACTIONABLE: 437

Estados ACTIONABLE:
state
GREEN    57
```

```
RED      40
Name: count, dtype: int64
```

16.8 Análisis de sensibilidad al umbral de confianza YOLO

Conclusión del análisis del detector especializado. El barrido del umbral de confianza mostró una elevada inestabilidad: con conf=0.01 se generó un número excesivo de candidatos, mientras que a partir de conf=0.02 desaparecieron las detecciones. Este comportamiento impedía establecer un umbral operativo robusto y motivó continuar evaluando alternativas antes de fijar la arquitectura definitiva.

17. Diagnóstico del detector YOLO especializado y reentrenamiento

17.1 Distribución de las confidencias del detector YOLO V1

In [246]:

```
print("ESTADÍSTICAS YOLO_CONF")
print("=" * 60)

print(
    df_v2_702["yolo_conf"].describe(
        percentiles=[
            0.01,
            0.05,
            0.10,
            0.25,
            0.50,
            0.75,
            0.90,
            0.95,
            0.99
        ]
    )
)

print("\nMáximo:")
print(df_v2_702["yolo_conf"].max())

print("\nMínimo:")
print(df_v2_702["yolo_conf"].min())
```

```
print("\n20 confidencias más altas:")
display(
    df_v2_702[
        [
            "frame",
            "yolo_conf",
            "actionable",
            "state",
            "x1",
            "y1",
            "x2",
            "y2"
        ]
    ]
    .sort_values(
        "yolo_conf",
        ascending=False
    )
    .head(20)
)
ESTADÍSTICAS YOLO_CONF
============================================================
count    534.000000
mean       0.036819
std        0.051292
min        0.010010
1%         0.010135
5%         0.010557
10%        0.011142
25%        0.013067
50%        0.018724
75%        0.036670
90%        0.080370
95%        0.123051
99%        0.280347
max        0.554667
Name: yolo_conf, dtype: float64

Máximo:
0.5546671152114868

Mínimo:
0.010009832680225372

20 confidencias más altas:
```

	frame	yolo_conf	actionable	state	x1	y1	x2	y2
464	525	0.554667	True	GREEN	748	553	824	642
185	135	0.342673	True	RED	873	285	976	385

	frame	yolo_conf	actionable	state	x1	y1	x2	y2
65	72	0.333761	False	NON_ACTIONABLE	836	286	889	352
402	474	0.329830	False	NON_ACTIONABLE	126	11	224	161
453	510	0.318647	False	NON_ACTIONABLE	445	437	523	525
501	552	0.283214	False	NON_ACTIONABLE	371	344	460	447
176	132	0.274526	True	GREEN	914	291	1021	378
137	111	0.274086	True	RED	894	424	953	503
271	240	0.251415	False	NON_ACTIONABLE	978	1116	1080	1353
283	261	0.211391	True	RED	145	166	179	219
467	528	0.199371	False	NON_ACTIONABLE	746	566	828	649
450	507	0.195231	True	GREEN	383	421	462	503
55	69	0.188733	False	NON_ACTIONABLE	824	285	877	356
152	123	0.170008	True	GREEN	975	273	1080	386
127	108	0.161094	True	GREEN	884	275	992	380
138	111	0.158505	False	NON_ACTIONABLE	879	511	912	572
23	24	0.148011	False	NON_ACTIONABLE	856	336	913	403
113	99	0.140147	False	NON_ACTIONABLE	951	401	1016	510
284	261	0.139955	False	NON_ACTIONABLE	0	205	28	284
83	84	0.137825	False	NON_ACTIONABLE	959	0	1080	234

17.2 Efecto de la resolución de entrada en el detector YOLO V1

In [247]:

```
import cv2
import numpy as np
from ultralytics import YOLO

# ============================================================
# DIAGNÓSTICO YOLO V1 EN UN ÚNICO FRAME
# ============================================================

TEST_FRAME = 525

cap = cv2.VideoCapture(video_path_v2)
cap.set(cv2.CAP_PROP_POS_FRAMES, TEST_FRAME)

ok, frame_test = cap.read()
cap.release()

print("Frame leído:", ok)
print("Forma:", frame_test.shape if ok else None)
```

```
test_model = YOLO(V1_PATH)

print("\nClases del modelo:")
print(test_model.names)

for imgsz in [640, 960, 1280]:

    result = test_model.predict(
        source=frame_test,
        imgsz=imgsz,
        conf=0.01,
        verbose=False
    )[0]

    if len(result.boxes) > 0:

        confs = (
            result.boxes.conf
            .cpu()
            .numpy()
        )

        print("\n" + "=" * 60)
        print("IMGSZ:", imgsz)
        print("Detecciones:", len(confs))
        print("Confianza máxima:", confs.max())
        print(
            "Top 10:",
            np.sort(confs)[::-1][:10]
        )

    else:

        print("\n" + "=" * 60)
        print("IMGSZ:", imgsz)
        print("Detecciones: 0")
Frame leído: True
Forma: (1920, 1080, 3)

Clases del modelo:
{0: 'pedestrian_traffic_light'}

============================================================
IMGSZ: 640
Detecciones: 5
Confianza máxima: 0.55252314
Top 10: [    0.55252     0.27221    0.060569    0.019239    0.010092]
```

```
============================================================
IMGSZ: 960
Detecciones: 12
Confianza máxima: 0.4843788
Top 10: [    0.48438     0.12393    0.099063    0.076427    0.063957
0.057095    0.036634    0.024854    0.015821    0.015802]

============================================================
IMGSZ: 1280
Detecciones: 11
Confianza máxima: 0.11397773
Top 10: [    0.11398    0.066545    0.053599    0.032456    0.023623
0.02171    0.020512    0.012223    0.011456    0.010513]
```

17.3 Inspección cualitativa del detector V1 sobre validación

In [248]:

```
# COMPROBAR YOLO V1 SOBRE UNA IMAGEN POSITIVA DE VALIDACIÓN


import os
import cv2
import numpy as np
from ultralytics import YOLO
from IPython.display import display
from PIL import Image

# Buscar una imagen de VAL que tenga anotación
positive_val_image = None

for label_name in sorted(os.listdir(VAL_LABELS_DIR)):

    if not label_name.endswith(".txt"):
        continue

    label_path = os.path.join(
        VAL_LABELS_DIR,
        label_name
    )

    # Debe contener al menos una anotación
    if os.path.getsize(label_path) == 0:
        continue

    stem = os.path.splitext(label_name)[0]

    for ext in [".jpg", ".jpeg", ".png"]:
```

```
        candidate = os.path.join(
            VAL_IMAGES_DIR,
            stem + ext
        )

        if os.path.isfile(candidate):
            positive_val_image = candidate
            break

    if positive_val_image is not None:
        break


print("Imagen positiva seleccionada:")
print(positive_val_image)

# ------------------------------------------------------------
# Inferencia
# ------------------------------------------------------------

model_test = YOLO(V1_PATH)

result = model_test.predict(
    source=positive_val_image,
    imgsz=640,
    conf=0.001,
    verbose=False
)[0]

print("\nDetecciones:", len(result.boxes))

if len(result.boxes) > 0:

    confs = result.boxes.conf.cpu().numpy()

    print("Confianza máxima:", confs.max())
    print(
        "Top 10:",
        np.sort(confs)[::-1][:10]
    )

    # Mostrar resultado
    annotated = result.plot()

    annotated_rgb = cv2.cvtColor(
        annotated,
        cv2.COLOR_BGR2RGB
    )
```

```
    display(
        Image.fromarray(
            annotated_rgb
        )
    )
Imagen positiva seleccionada:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\images\val\
1000179711_frame_000000.jpg

Detecciones: 300
Confianza máxima: 0.042889576
Top 10: [    0.04289    0.019837    0.016736    0.013573   0.0085223
0.005858   0.0058441   0.0057972   0.0056716   0.0056302]
```


pedestrian_traffic_light

17.4 Reentrenamiento experimental del detector YOLO11n V2 a 1280 px

In [249]:

```
# ============================================================
# 17.4 REENTRENAMIENTO EXPERIMENTAL DEL DETECTOR YOLO11n V2
# SEMÁFORO PEATONAL - IMGSZ 1280
# ============================================================

import os
import torch
from ultralytics import YOLO


# ------------------------------------------------------------
# 1. RUTAS
# ------------------------------------------------------------

TFM_DIR = r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"

DATASET_DIR = os.path.join(
    TFM_DIR,
    "dataset_pedestrian_traffic_light",
    "dataset_yolo_final"
)

TRAINING_DIR = os.path.join(
    DATASET_DIR,
    "training"
)

os.makedirs(TRAINING_DIR, exist_ok=True)


# ------------------------------------------------------------
# 2. DATA.YAML
# ------------------------------------------------------------

DATA_YAML = os.path.join(
    DATASET_DIR,
    "data.yaml"
)

# Si no existe, se reconstruye automáticamente
if not os.path.isfile(DATA_YAML):

    print("No existe data.yaml. Se va a crear automáticamente.")
```

```
    yaml_content = f"""path: {DATASET_DIR.replace(os.sep, "/")}
train: images/train
val: images/val
test: images/test

names:
  0: pedestrian_traffic_light
"""

    with open(
        DATA_YAML,
        "w",
        encoding="utf-8"
    ) as f:
        f.write(yaml_content)


print("=" * 70)
print("DATASET YOLO V2")
print("=" * 70)

print("\nDATASET_DIR:")
print(DATASET_DIR)
print("¿Existe?:", os.path.isdir(DATASET_DIR))

print("\nDATA_YAML:")
print(DATA_YAML)
print("¿Existe?:", os.path.isfile(DATA_YAML))


# ------------------------------------------------------------
# 3. COMPROBAR CARPETAS DEL DATASET
# ------------------------------------------------------------

dataset_folders = {
    "images/train": os.path.join(DATASET_DIR, "images", "train"),
    "images/val": os.path.join(DATASET_DIR, "images", "val"),
    "images/test": os.path.join(DATASET_DIR, "images", "test"),
    "labels/train": os.path.join(DATASET_DIR, "labels", "train"),
    "labels/val": os.path.join(DATASET_DIR, "labels", "val"),
    "labels/test": os.path.join(DATASET_DIR, "labels", "test"),
}

print("\n" + "=" * 70)
print("ESTRUCTURA DEL DATASET")
print("=" * 70)

for name, path in dataset_folders.items():
    estado = "OK" if os.path.isdir(path) else "NO ENCONTRADO"
```

```
    print(f"{name:20s}: {estado}")


# ------------------------------------------------------------
# 4. DISPOSITIVO
# ------------------------------------------------------------

DEVICE = 0 if torch.cuda.is_available() else "cpu"

print("\n" + "=" * 70)
print("DISPOSITIVO")
print("=" * 70)

if torch.cuda.is_available():
    print("GPU CUDA:", torch.cuda.get_device_name(0))
else:
    print("CPU")


# ------------------------------------------------------------
# 5. RUTA DEL MODELO V2 ENTRENADO
# ------------------------------------------------------------

V2_RUN_NAME = "yolo11n_pedestrian_light_v2_1280"

V2_PATH = os.path.join(
    TRAINING_DIR,
    V2_RUN_NAME,
    "weights",
    "best.pt"
)

print("\n" + "=" * 70)
print("COMPROBACIÓN DEL MODELO V2")
print("=" * 70)

print("\nV2_PATH:")
print(V2_PATH)

print("\n¿best.pt existe?")
print(os.path.isfile(V2_PATH))


# ------------------------------------------------------------
# 6. ENTRENAR SOLO SI NO EXISTE best.pt
# ------------------------------------------------------------

if os.path.isfile(V2_PATH):
```

```
    print("\n" + "=" * 70)
    print("MODELO V2 YA ENTRENADO")
    print("=" * 70)

    print(
        "Se utilizará directamente el modelo existente."
    )

    model_v2 = YOLO(V2_PATH)

    results_v2 = None

else:

    print("\n" + "=" * 70)
    print("NO SE HA ENCONTRADO EL MODELO V2 ENTRENADO")
    print("=" * 70)

    print(
        "Se iniciará el entrenamiento YOLO11n V2 a 1280 px."
    )

    model_v2 = YOLO("yolo11n.pt")

    results_v2 = model_v2.train(
        data=DATA_YAML,
        epochs=100,
        imgsz=1280,
        batch=4,
        patience=20,
        device=DEVICE,
        workers=0,
        project=TRAINING_DIR,
        name=V2_RUN_NAME,
        pretrained=True,
        single_cls=True,
        seed=42,
        deterministic=True,
        save=True,
        plots=True,
        verbose=True,
        exist_ok=True
    )


# ------------------------------------------------------------
# 7. COMPROBACIÓN FINAL
# ------------------------------------------------------------
```

```
print("\n" + "=" * 70)
print("RESULTADO 17.4")
print("=" * 70)

print("\nV2_PATH:")
print(V2_PATH)

print("\n¿best.pt existe?")
print(os.path.isfile(V2_PATH))

if os.path.isfile(V2_PATH):

    print(
        "\nOK: YOLO11n V2 preparado para continuar "
        "con las siguientes etapas."
    )

else:

    print(
        "\nATENCIÓN: best.pt todavía no está disponible."
    )
```

```
======================================================================
DATASET YOLO V2
======================================================================

DATASET_DIR:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final
¿Existe?: True

DATA_YAML:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\data.yaml
¿Existe?: True

======================================================================
ESTRUCTURA DEL DATASET
======================================================================
images/train        : OK
images/val          : OK
images/test         : OK
labels/train        : OK
labels/val          : OK
labels/test         : OK

======================================================================
DISPOSITIVO
======================================================================
```

```
CPU

======================================================================
COMPROBACIÓN DEL MODELO V2
======================================================================

V2_PATH:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\weights\best.pt

¿best.pt existe?
False

======================================================================
NO SE HA ENCONTRADO EL MODELO V2 ENTRENADO
======================================================================
Se iniciará el entrenamiento YOLO11n V2 a 1280 px.
New https://pypi.org/project/ultralytics/8.4.161 available  Update
with 'pip install -U ultralytics'
Ultralytics 8.4.33  Python-3.14.3 torch-2.11.0+cpu CPU (AMD Ryzen AI 7
350 w/ Radeon 860M)
engine\trainer: agnostic_nms=False, amp=True, angle=1.0,
augment=False, auto_augment=randaugment, batch=4, bgr=0.0, box=7.5,
cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5,
compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip,
cos_lr=False, cutmix=0.0,
data=C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\data.yaml,
degrees=0.0, deterministic=True, device=cpu, dfl=1.5, dnn=False,
dropout=0.0, dynamic=False, embed=None, end2end=None, epochs=100,
erasing=0.4, exist_ok=True, fliplr=0.5, flipud=0.0,
format=torchscript, fraction=1.0, freeze=None, half=False,
hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=1280, int8=False, iou=0.7,
keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolo11n.pt,
momentum=0.937, mosaic=1.0, multi_scale=0.0,
name=yolo11n_pedestrian_light_v2_1280, nbs=64, nms=False, opset=None,
optimize=False, optimizer=auto, overlap_mask=True, patience=20,
perspective=0.0, plots=True, pose=12.0, pretrained=True,
profile=False, project=C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training,
rect=False, resume=False, retina_masks=False, rle=1.0, save=True,
save_conf=False, save_crop=False,
save_dir=C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280, save_frames=False, save_json=False,
save_period=-1, save_txt=False, scale=0.5, seed=42, shear=0.0,
show=False, show_boxes=True, show_conf=True, show_labels=True,
```

```
simplify=True, single_cls=True, source=None, split=val,
stream_buffer=False, task=detect, time=None, tracker=botsort.yaml,
translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False,
warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8,
weight_decay=0.0005, workers=0, workspace=None
Overriding class names with single class.
Overriding model.yaml nc=80 with nc=1

                   from  n    params  module
arguments
  0                  -1  1       464  ultralytics.nn.modules.conv.Conv
[3, 16, 3, 2]
  1                  -1  1      4672  ultralytics.nn.modules.conv.Conv
[16, 32, 3, 2]
  2                  -1  1      6640
ultralytics.nn.modules.block.C3k2            [32, 64, 1, False, 0.25]
  3                  -1  1     36992  ultralytics.nn.modules.conv.Conv
[64, 64, 3, 2]
  4                  -1  1     26080
ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
  5                  -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
  6                  -1  1     87040
ultralytics.nn.modules.block.C3k2            [128, 128, 1, True]
  7                  -1  1    295424  ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]
  8                  -1  1    346112
ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
  9                  -1  1    164608
ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                  -1  1    249728
ultralytics.nn.modules.block.C2PSA           [256, 256, 1]
 11                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 12             [-1, 6]  1         0
ultralytics.nn.modules.conv.Concat           [1]
 13                  -1  1    111296
ultralytics.nn.modules.block.C3k2            [384, 128, 1, False]
 14                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 15             [-1, 4]  1         0
ultralytics.nn.modules.conv.Concat           [1]
 16                  -1  1     32096
ultralytics.nn.modules.block.C3k2            [256, 64, 1, False]
 17                  -1  1     36992  ultralytics.nn.modules.conv.Conv
[64, 64, 3, 2]
 18            [-1, 13]  1         0
ultralytics.nn.modules.conv.Concat           [1]
```

```
 19                  -1  1     86720
ultralytics.nn.modules.block.C3k2            [192, 128, 1, False]
 20                  -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
 21            [-1, 10]  1         0
ultralytics.nn.modules.conv.Concat           [1]
 22                  -1  1    378880
ultralytics.nn.modules.block.C3k2            [384, 256, 1, True]
 23        [16, 19, 22]  1    430867
ultralytics.nn.modules.head.Detect           [1, 16, None, [64, 128,
256]]
YOLO11n summary: 182 layers, 2,590,035 parameters, 2,590,019
gradients, 6.4 GFLOPs

Transferred 448/499 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
train: Fast image access  (ping: 0.30.1 ms, read: 838.4252.5 MB/s,
size: 561.6 KB)
train: Scanning C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\labels\trai
n... 69 images, 28 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━━━━━━━━━
97/97 1.3Kit/s 0.1s
train: New cache created: C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\labels\trai
n.cache
val: Fast image access  (ping: 0.30.0 ms, read: 681.8142.6 MB/s, size:
479.3 KB)
val: Scanning C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\labels\val.
.. 7 images, 8 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━━━━━━━━━ 15/15
1.5Kit/s 0.0s
val: New cache created: C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\labels\val.
cache
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and
'momentum' automatically...
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 81
weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Plotting labels to C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\labels.jpg...
Image sizes 1280 train, 1280 val
Using 0 dataloader workers
Logging results to C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280
Starting training for 100 epochs...
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      1/100         0G      1.888      14.35      1.438          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:172.0sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s4.9s
                   all         15          7   0.000222      0.143
0.000129   9.02e-05

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      2/100         0G      1.614      7.334      1.313          6
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.8sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.7s4.8s
                   all         15          7   0.000222      0.143
0.0146    0.00589

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      3/100         0G      1.786      8.376      1.423          5
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:151.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.7s4.7s
                   all         15          7          0          0
0          0

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      4/100         0G      1.838      6.666      1.461          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:151.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.7s4.7s
                   all         15          7          0          0
0          0

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      5/100         0G      1.732      11.16      1.401          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:151.8sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.5s/it 3.0s5.2s
                   all         15          7     0.0878      0.143
0.147      0.015

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
```

```
      6/100         0G      1.747      8.687      1.415          0
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.5s/it 2.9s5.2s
                   all         15          7   0.000667      0.429
0.0467     0.0114

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      7/100         0G      1.655      9.804      1.389          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:151.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.9s5.2s
                   all         15          7   0.000444      0.286
0.0183    0.00512

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      8/100         0G      1.688      7.357      1.476          4
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s5.1s
                   all         15          7   0.000222      0.143
0.00294   0.000881

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
      9/100         0G      1.622      6.273      1.419          4
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.7s4.8s
                   all         15          7   0.000444      0.286
0.00268    0.00107

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     10/100         0G      1.756      5.165       1.48          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s4.9s
                   all         15          7   0.000222      0.143
0.00355    0.00248

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     11/100         0G      1.638      4.981      1.429          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:151.7sss
```

```
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s5.0s
                   all         15          7   0.000222      0.143
0.00477    0.00239

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     12/100         0G      1.728      4.944      1.498          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s4.9s
                   all         15          7   0.000444      0.286
0.0249    0.00942

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     13/100         0G      1.482      4.765      1.278          0
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.6sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s4.9s
                   all         15          7   0.000889      0.571
0.0432     0.0153

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     14/100         0G      1.489      3.775      1.309          4
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.8s4.9s
                   all         15          7   0.000667      0.429
0.00409      0.002

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     15/100         0G      1.529       3.94      1.285          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.7s4.9s
                   all         15          7   0.000889      0.571
0.0129    0.00271

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     16/100         0G      1.386      3.652      1.289          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.6sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.6s4.6s
```

```
                   all         15          7   0.000667      0.429
0.0197     0.0054

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     17/100         0G      1.339       3.66      1.243          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 9.0s/it 3:441.7ss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.7s4.9s
                   all         15          7   0.000667      0.429
0.00968    0.00242

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     18/100         0G      1.437      3.118      1.237          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.7s4.7s
                   all         15          7      0.499      0.143
0.151     0.0167

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     19/100         0G      1.623      4.203      1.425          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.4s/it 2.7s4.8s
                   all         15          7   0.000889      0.571
0.0167    0.00538

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     20/100         0G      1.378      3.569      1.187          0
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.6sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.6s4.5s
                   all         15          7      0.905      0.143
0.153     0.0166

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     21/100         0G      1.375       2.79      1.213          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.6s4.6s
                   all         15          7    0.00111      0.714
0.0261     0.0103
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     22/100         0G      1.478       3.22      1.271          0
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.6s4.6s
                   all         15          7     0.0262      0.143
0.0122    0.00487

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     23/100         0G      1.386      2.596      1.233          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:171.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.5s4.5s
                   all         15          7      0.158      0.143
0.035     0.0157

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     24/100         0G      1.426      2.726      1.277          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:171.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.5s
                   all         15          7      0.303      0.143
0.148     0.0317

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     25/100         0G      1.334       2.43      1.204          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.4s
                   all         15          7      0.202      0.143
0.0484     0.0194

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     26/100         0G      1.344      2.509      1.228          4
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.3s
                   all         15          7     0.0862      0.143
0.0342     0.0122

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
```

```
     27/100         0G      1.337      2.194      1.205          5
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.4s
                   all         15          7     0.0746      0.143
0.0203    0.00952

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     28/100         0G       1.24      2.238       1.16          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:171.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.2s
                   all         15          7   0.000444      0.286
0.0083    0.00191

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     29/100         0G      1.363      2.167      1.166          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:171.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.3s
                   all         15          7   0.000444      0.286
0.00544    0.00114

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     30/100         0G      1.279      2.154      1.183          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:171.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.4s
                   all         15          7     0.0228      0.286
0.0143    0.00623

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     31/100         0G      1.381       2.14      1.229          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.1s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.4s
                   all         15          7     0.0411      0.143
0.0167    0.00511

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     32/100         0G      1.416      2.406      1.261          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 9.7s/it 4:021.8sss
```

```
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.4s
                   all         15          7     0.0772      0.143
0.0227    0.00599

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     33/100         0G      1.349          2      1.242          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.3s/it 2.5s4.4s
                   all         15          7   0.000444      0.286
0.00976    0.00336

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     34/100         0G      1.222      2.005      1.148          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.2s
                   all         15          7      0.111      0.143
0.0228    0.00911

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     35/100         0G      1.266      1.829      1.235          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.4s
                   all         15          7       0.13      0.143
0.0283     0.0137

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     36/100         0G      1.188      1.635      1.143          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.3s
                   all         15          7      0.112      0.143
0.0208    0.00845

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     37/100         0G      1.206       1.88      1.167          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.4s
```

```
                   all         15          7   0.000444      0.286
0.01    0.00375

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     38/100         0G      1.191      1.861      1.149          0
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.4s
                   all         15          7   0.000222      0.143
0.000191   5.74e-05

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     39/100         0G      1.183      1.756       1.16          3
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.4s
                   all         15          7   0.000444      0.286
0.00032    6.9e-05

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     40/100         0G       1.22      1.768       1.17          0
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.6sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.4s
                   all         15          7   0.000222      0.143
0.00139   0.000556

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     41/100         0G      1.179      2.411      1.164          4
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.5s4.4s
                   all         15          7          0          0
0          0

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     42/100         0G      1.306      1.641      1.211          1
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.3s
                   all         15          7   0.000222      0.143
0.000136   5.45e-05
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     43/100         0G       1.24      1.455      1.198          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.3s
                   all         15          7   0.000444      0.286
0.0141    0.00305

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
     44/100         0G      1.199      1.416      1.165          2
1280: 100% ━━━━━━━━━━━━━━━━━━━━ 25/25 3.0s/it 1:161.7sss
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.2s/it 2.4s4.3s
                   all         15          7   0.000667      0.429
0.0278     0.0125
EarlyStopping: Training stopped early as no improvement observed in
last 20 epochs. Best results observed at epoch 24, best model saved as
best.pt.
To update EarlyStopping(patience=20) pass a new patience value, i.e.
`patience=300` or use `patience=0` to disable EarlyStopping.

44 epochs completed in 1.051 hours.
Optimizer stripped from C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\weights\last.pt, 5.6MB
Optimizer stripped from C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\weights\best.pt, 5.6MB

Validating C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\weights\best.pt...
Ultralytics 8.4.33  Python-3.14.3 torch-2.11.0+cpu CPU (AMD Ryzen AI 7
350 w/ Radeon 860M)
YOLO11n summary (fused): 101 layers, 2,582,347 parameters, 0
gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R
mAP50  mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 2/2 1.1s/it 2.2s4.0s
                   all         15          7      0.304      0.143
0.148     0.0317
Speed: 2.0ms preprocess, 115.8ms inference, 0.0ms loss, 6.9ms
postprocess per image
Results saved to C:\Users\juank\OneDrive\Documentos\MSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280
```

```
======================================================================
RESULTADO 17.4
======================================================================

V2_PATH:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\weights\best.pt

¿best.pt existe?
True

OK: YOLO11n V2 preparado para continuar con las siguientes etapas.
```

In [250]:

```
# ============================================================
# 17.4.1 CONSERVACIÓN PERMANENTE DEL MODELO V2 REFINADO
# ============================================================

from pathlib import Path
import shutil
import hashlib


# ------------------------------------------------------------
# 1. Rutas
# ------------------------------------------------------------

TFM_DIR = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)

TRAINING_V2_DIR = (
    TFM_DIR
    / "dataset_pedestrian_traffic_light"
    / "dataset_yolo_final"
    / "training"
    / "yolo11n_pedestrian_light_v2_1280"
)

SOURCE_BEST = (
    TRAINING_V2_DIR
    / "weights"
    / "best.pt"
)

SOURCE_LAST = (
    TRAINING_V2_DIR
    / "weights"
    / "last.pt"
```

```
)


MODELS_DIR = (
    TFM_DIR
    / "MODELOS_YOLO_FINALES_TFM"
)

MODELS_DIR.mkdir(
    parents=True,
    exist_ok=True
)


DEST_BEST = (
    MODELS_DIR
    / "pedestrian_light_v2_refined_best.pt"
)

DEST_LAST = (
    MODELS_DIR
    / "pedestrian_light_v2_refined_last.pt"
)


# ------------------------------------------------------------
# 2. Comprobar pesos generados
# ------------------------------------------------------------

print("=" * 72)
print("17.4.1 — CONSERVACIÓN DEL V2 REFINADO")
print("=" * 72)

print()
print("Origen best.pt:")
print(SOURCE_BEST)

print()
print(
    "¿Existe best.pt?:",
    SOURCE_BEST.is_file()
)

print(
    "¿Existe last.pt?:",
    SOURCE_LAST.is_file()
)
```

```
if not SOURCE_BEST.is_file():

    raise FileNotFoundError(
        f"No existe best.pt:\n{SOURCE_BEST}"
    )


# ------------------------------------------------------------
# 3. Copiar pesos
# ------------------------------------------------------------

shutil.copy2(
    SOURCE_BEST,
    DEST_BEST
)


if SOURCE_LAST.is_file():

    shutil.copy2(
        SOURCE_LAST,
        DEST_LAST
    )


# ------------------------------------------------------------
# 4. SHA-256
# ------------------------------------------------------------

def sha256_file(path):

    h = hashlib.sha256()

    with open(
        path,
        "rb"
    ) as f:

        for block in iter(
            lambda: f.read(1024 * 1024),
            b""
        ):

            h.update(block)

    return h.hexdigest()


# ------------------------------------------------------------
```

```
# 5. Resultado
# ------------------------------------------------------------

print()
print("=" * 72)
print("V2 REFINADO GUARDADO PERMANENTEMENTE")
print("=" * 72)

print()
print("✓ BEST:")
print(DEST_BEST)

print()
print(
    "SHA-256:",
    sha256_file(DEST_BEST)
)

if DEST_LAST.is_file():

    print()
    print("✓ LAST:")
    print(DEST_LAST)

print()
print(
    "✓ El V2 original NO se ha sobrescrito."
)

print(
    "✓ El reentrenamiento 17.4 queda conservado por separado."
)
======================================================================
==
17.4.1 — CONSERVACIÓN DEL V2 REFINADO
======================================================================
==

Origen best.pt:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\dataset_pedestrian_traffic_light\dataset_yolo_final\training\yo
lo11n_pedestrian_light_v2_1280\weights\best.pt

¿Existe best.pt?: True
¿Existe last.pt?: True

======================================================================
==
```

```
V2 REFINADO GUARDADO PERMANENTEMENTE
================================================================
==

✓ BEST:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_refined_best.pt

SHA-256:
fa793fb305c2f70929e39bb57d391140ab4f0fe84b64976884b771bd17e31f23

✓ LAST:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_refined_last.pt

✓ El V2 original NO se ha sobrescrito.
✓ El reentrenamiento 17.4 queda conservado por separado.
```

17.5 Evaluación del detector YOLO11n V2 sobre validación

In [251]:

```
# Evaluación del detector YOLOn V2 sobre evaluación

import numpy as np
import cv2
from PIL import Image
from IPython.display import display
from ultralytics import YOLO

# Cargar mejor modelo V2
yolo_v2 = YOLO(V2_PATH)

# Inferencia sobre la imagen positiva de validación
result_v2 = yolo_v2.predict(
    source=positive_val_image,
    imgsz=1280,
    conf=0.001,
    verbose=False
)[0]

print("=" * 60)
print("PRUEBA FINAL YOLO V2")
print("=" * 60)

print("Detecciones:", len(result_v2.boxes))
```

```
if len(result_v2.boxes) > 0:

    confs = result_v2.boxes.conf.cpu().numpy()

    print("Confianza máxima:", float(confs.max()))
    print(
        "Top 10:",
        np.sort(confs)[::-1][:10]
    )

    # Mostrar resultado
    annotated = result_v2.plot()

    annotated_rgb = cv2.cvtColor(
        annotated,
        cv2.COLOR_BGR2RGB
    )

    display(
        Image.fromarray(annotated_rgb)
    )

else:
    print("No se han detectado objetos.")
============================================================
PRUEBA FINAL YOLO V2
============================================================
Detecciones: 300
Confianza máxima: 0.5203205347061157
Top 10: [    0.52032     0.12954     0.04122    0.031245    0.017774
0.012268    0.011564   0.0092114   0.0084562   0.0073388]
```



El reentrenamiento especializado V2 no corrigió la baja confianza ni la generación excesiva de candidatos observada en el detector anterior. Por este motivo se descartó su uso como detector principal y se adoptó YOLO11 preentrenado sobre COCO, utilizando la clase traffic light únicamente como generador de regiones candidatas. La discriminación de semáforo peatonal y su estado se delegó posteriormente al clasificador semántico basado en CLIP y al Gate V2 supervisado.

17.6 Selección del detector de candidatos basado en YOLO11 COCO

In [7]:

```
# ============================================================
# DETECTOR DEFINITIVO DE CANDIDATOS
# YOLO11 PREENTRENADO COCO
# ============================================================

from ultralytics import YOLO

yolo_signal_detector = YOLO("yolo11n.pt")

print("Detector definitivo cargado.")
print("Clase traffic light disponible:")

for class_id, class_name in yolo_signal_detector.names.items():
    if class_name == "traffic light":
        print(
            "ID:",
            class_id,
            "| Clase:",
            class_name
        )
```

```
Detector definitivo cargado.
Clase traffic light disponible:
ID: 9 | Clase: traffic light
```

18. Pipeline temporal definitivo del semáforo peatonal

18.1 Integración del pipeline final: YOLO11 COCO + CLIP + Gate V2 + RED/GREEN

In [19]:

```
# PIPELINE TEMPORAL FINAL DEL SEMÁFORO PEATONAL
# YOLO11 COCO + CLIP + GATE V2 + GREEN/RED
```

```
import os
import cv2
import numpy as np
import pandas as pd

from PIL import Image


def process_temporal_signal_video(video_path):

    # ----------------------------------------------------------
    # Comprobación rápida de dependencias
    # ----------------------------------------------------------

    required = [
        "yolo_signal_detector",
        "get_clip_image_embedding",
        "gate_model_v2",
        "classify_state_red_green"
    ]

    missing = [
        name
        for name in required
        if name not in globals()
    ]

    if missing:
        raise RuntimeError(
            f"Faltan estas variables/funciones: {missing}"
        )

    # ----------------------------------------------------------
    # Configuración
    # ----------------------------------------------------------

    FRAME_STEP = 3

    # YOLO actúa solo como generador de candidatos
    YOLO_SIGNAL_CONF = 0.05

    # Gate V2 decide si el crop es interpretable
    GATE_THRESHOLD = 0.50

    # Clase COCO "traffic light"
    TRAFFIC_LIGHT_CLASS = 9

    print("=" * 70)
    print("PIPELINE TEMPORAL FINAL")
```

```
print("=" * 70)

print("Vídeo:")
print(video_path)

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    raise RuntimeError(
        f"No se pudo abrir el vídeo: {video_path}"
    )

fps = cap.get(cv2.CAP_PROP_FPS)

total_frames = int(
    cap.get(cv2.CAP_PROP_FRAME_COUNT)
)

print("FPS:", fps)
print("Frames:", total_frames)

temporal_rows = []

frame_idx = 0
processed_frames = 0

# Índice de la clase positiva del Gate
gate_classes = list(gate_model_v2.classes_)

if 1 not in gate_classes:
    raise RuntimeError(
        f"El Gate no contiene la clase ACTIONABLE=1. "
        f"Clases encontradas: {gate_classes}"
    )

actionable_idx = gate_classes.index(1)

# ------------------------------------------------------------
# Procesamiento del vídeo
# ------------------------------------------------------------

while True:

    ret, frame = cap.read()

    if not ret:
        break

    # Procesar 1 de cada 3 frames
```

```
if frame_idx % FRAME_STEP != 0:
    frame_idx += 1
    continue

processed_frames += 1

# ------------------------------------------------------
# YOLO11 COCO: solo traffic light
# ------------------------------------------------------

result = yolo_signal_detector.predict(
    source=frame,
    classes=[TRAFFIC_LIGHT_CLASS],
    conf=YOLO_SIGNAL_CONF,
    verbose=False
)[0]

# Imagen PIL para realizar los crops
frame_rgb = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2RGB
)

frame_pil = Image.fromarray(frame_rgb)

# ------------------------------------------------------
# Analizar cada candidato
# ------------------------------------------------------

for box in result.boxes:

    yolo_conf = float(
        box.conf[0].item()
    )

    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0].tolist()
    )

    # Pequeño margen contextual
    margin = 20

    x1c = max(0, x1 - margin)
    y1c = max(0, y1 - margin)

    x2c = min(
        frame_pil.width,
        x2 + margin
```

```
)

y2c = min(
    frame_pil.height,
    y2 + margin
)

crop_pil = frame_pil.crop(
    (x1c, y1c, x2c, y2c)
)

if crop_pil.width == 0 or crop_pil.height == 0:
    continue

# ------------------------------------------------
# CLIP embedding
# ------------------------------------------------

embedding = get_clip_image_embedding(
    crop_pil
)

X_candidate = np.asarray(
    embedding
).reshape(1, -1)

# ------------------------------------------------
# Gate V2
# ------------------------------------------------

probabilities = (
    gate_model_v2.predict_proba(
        X_candidate
    )[0]
)

p_actionable = float(
    probabilities[actionable_idx]
)

actionable = (
    p_actionable >= GATE_THRESHOLD
)

# ------------------------------------------------
# Estado GREEN / RED
# ------------------------------------------------

if actionable:
```

```
            state_result = (
                classify_state_red_green(
                    crop_pil
                )
            )

            state = state_result.get(
                "state",
                "UNKNOWN"
            )

            green_score = state_result.get(
                "green",
                np.nan
            )

            red_score = state_result.get(
                "red",
                np.nan
            )

        else:

            state = "NON_ACTIONABLE"
            green_score = np.nan
            red_score = np.nan

        temporal_rows.append({

            "frame": frame_idx,

            "time_s": (
                frame_idx / fps
                if fps > 0
                else np.nan
            ),

            "track_id": np.nan,

            "yolo_conf": yolo_conf,

            "p_actionable": p_actionable,

            "actionable": actionable,

            "state": state,

            "green_score": green_score,
```

```
            "red_score": red_score,

            "x1": x1,
            "y1": y1,
            "x2": x2,
            "y2": y2
        })

    frame_idx += 1

    if processed_frames % 50 == 0:

        print(
            f"Frames analizados: "
            f"{processed_frames}"
        )

cap.release()

# ----------------------------------------------------------
# DataFrame final
# ----------------------------------------------------------

df_final_signal = pd.DataFrame(
    temporal_rows
)

print("\n" + "=" * 70)
print("PROCESAMIENTO FINALIZADO")
print("=" * 70)

print(
    "Observaciones:",
    len(df_final_signal)
)

if len(df_final_signal) > 0:

    print(
        "ACTIONABLE:",
        int(
            df_final_signal[
                "actionable"
            ].sum()
        )
    )

    print(
```

```
            "NON_ACTIONABLE:",
            int(
                (~df_final_signal[
                    "actionable"
                ]).sum()
            )
        )

        print("\nEstados:")

        print(
            df_final_signal[
                "state"
            ].value_counts()
        )

    return df_final_signal
```

18.2 Validación inicial del pipeline final sobre vídeo

In [52]:

```
video_test_final = os.path.join(
    VIDEOS_DIR,
    "20260611_185616.mp4"
)

df_test_final = process_temporal_signal_video(
    video_test_final
)
```

```
======================================================================
PIPELINE TEMPORAL FINAL
======================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_185616.mp4
FPS: 29.997528845141506
Frames: 611
Frames analizados: 50
Frames analizados: 100
Frames analizados: 150
Frames analizados: 200

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 461
ACTIONABLE: 308
```

```
NON_ACTIONABLE: 153

Estados:
state
GREEN             218
NON_ACTIONABLE    153
RED                90
Name: count, dtype: int64
```

18.3 Consistencia temporal y asociación espacial de tracks

In [81]:

```
# ============================================================
# CONSISTENCIA TEMPORAL FINAL POR TRACK
# ============================================================

# Nos quedamos únicamente con candidatos aceptados por el Gate
df_actionable_final = df_test_final[
    df_test_final["actionable"] == True
].copy()

print("Candidatos ACTIONABLE:", len(df_actionable_final))

# ------------------------------------------------------------
# Asociación espacial-temporal
# ------------------------------------------------------------

df_tracks_final = assign_signal_tracks_spatial(
    df_actionable_final,
    max_frame_gap=6,
    max_dist_factor=2.5
)

print(
    "Tracks creados:",
    df_tracks_final["spatial_track_id"].nunique()
)

# ------------------------------------------------------------
# Resumen de cada track
# ------------------------------------------------------------

def summarize_final_track(group):

    observations = len(group)

    green_hits = int(
```

```
        (group["state"] == "GREEN").sum()
    )

    red_hits = int(
        (group["state"] == "RED").sum()
    )

    if green_hits > red_hits:
        dominant_state = "GREEN"

    elif red_hits > green_hits:
        dominant_state = "RED"

    else:
        dominant_state = "UNKNOWN"

    consistency = (
        max(green_hits, red_hits) / observations
        if observations > 0
        else 0
    )

    return pd.Series({
        "observations": observations,
        "first_frame": group["frame"].min(),
        "last_frame": group["frame"].max(),
        "mean_p_actionable": group["p_actionable"].mean(),
        "green_hits": green_hits,
        "red_hits": red_hits,
        "dominant_state": dominant_state,
        "state_consistency": consistency
    })


track_summary_final = (
    df_tracks_final
    .groupby("spatial_track_id")
    .apply(summarize_final_track)
    .reset_index()
)

# Track fiable:
# al menos 3 observaciones y 60 % de consistencia
track_summary_final["persistent"] = (
    (track_summary_final["observations"] >= 3)
    &
    (track_summary_final["state_consistency"] >= 0.60)
)
```

```
track_summary_final = (
    track_summary_final
    .sort_values(
        [
            "persistent",
            "observations",
            "state_consistency",
            "mean_p_actionable"
        ],
        ascending=[False, False, False, False]
    )
)

display(track_summary_final)
Candidatos ACTIONABLE: 308
Tracks creados: 55
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\385943514.py:73:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)
```

	spatial_track_id	observations	first_frame	last_frame	mean_p_actionable	green_hits	red_hits	c
20	20	30	249	336	0.760253	25	5	C
44	44	26	492	570	0.774075	25	1	C
46	46	26	513	603	0.782447	24	2	C
9	9	26	120	198	0.713495	6	20	F
33	33	25	423	504	0.767041	24	1	C
34	34	19	423	483	0.757359	17	2	C
25	25	15	312	357	0.765995	9	6	C
1	1	13	45	90	0.555333	0	13	F
24	24	12	297	336	0.762785	10	2	C
17	17	10	219	249	0.740269	9	1	C
26	26	10	330	357	0.761145	7	3	C
23	23	8	264	288	0.753098	7	1	C
45	45	7	492	513	0.763771	6	1	C
39	39	6	462	480	0.764579	6	0	C
14	14	6	177	195	0.735252	4	2	C
18	18	5	219	234	0.727891	3	2	C
53	53	3	582	588	0.797430	3	0	C
19	19	3	234	240	0.757640	3	0	C

	spatial_track_id	observations	first_frame	last_frame	mean_p_actionable	green_hits	red_hits	c
35	35	3	423	429	0.705629	0	3	F
2	2	3	48	57	0.523472	0	3	F
12	12	3	144	150	0.678035	1	2	F
13	13	3	150	156	0.604443	1	2	F
50	50	2	558	561	0.782336	2	0	C
54	54	2	597	600	0.771641	2	0	C
21	21	2	264	267	0.758054	2	0	C
40	40	2	462	468	0.744332	2	0	C
32	32	2	411	414	0.720333	2	0	C
11	11	2	132	135	0.645284	0	2	F
10	10	2	120	123	0.587697	0	2	F
6	6	2	99	102	0.560870	0	2	F
7	7	2	99	102	0.551477	0	2	F
5	5	2	87	90	0.530247	0	2	F
28	28	2	354	357	0.768508	1	1	U
16	16	2	195	198	0.739072	1	1	U
37	37	2	459	462	0.717195	1	1	U
51	51	1	558	558	0.778468	1	0	C
49	49	1	549	549	0.776798	1	0	C
48	48	1	540	540	0.775376	1	0	C
52	52	1	570	570	0.772667	1	0	C
38	38	1	462	462	0.772191	1	0	C
31	31	1	411	411	0.767605	1	0	C
27	27	1	333	333	0.752133	0	1	F
47	47	1	531	531	0.749144	1	0	C
22	22	1	264	264	0.737950	1	0	C
29	29	1	357	357	0.731852	1	0	C
41	41	1	477	477	0.729567	1	0	C
42	42	1	477	477	0.724469	1	0	C
30	30	1	357	357	0.699812	1	0	C
15	15	1	177	177	0.679940	1	0	C
43	43	1	477	477	0.654507	1	0	C
8	8	1	120	120	0.632259	0	1	F
36	36	1	423	423	0.572076	0	1	F
3	3	1	66	66	0.566917	1	0	C
4	4	1	78	78	0.527197	0	1	F
0	0	1	15	15	0.524385	0	1	F

18.4 Inspección visual de los tracks persistentes

In [70]:

```
# VISUALIZACIÓN DE TRACKS PERSISTENTES


import cv2
import matplotlib.pyplot as plt
from PIL import Image

persistent_ids = (
    track_summary_final[
        track_summary_final["persistent"] == True
    ]["spatial_track_id"]
    .tolist()
)

print("Tracks persistentes:", persistent_ids)

cap = cv2.VideoCapture(video_test_final)

fig, axes = plt.subplots(
    len(persistent_ids),
    1,
    figsize=(6, 4 * len(persistent_ids))
)

if len(persistent_ids) == 1:
    axes = [axes]

for ax, track_id in zip(axes, persistent_ids):

    group = df_tracks_final[
        df_tracks_final["spatial_track_id"] == track_id
    ]

    # Elegir observación con mayor p_actionable
    row = group.loc[
        group["p_actionable"].idxmax()
    ]

    frame_number = int(row["frame"])

    cap.set(
        cv2.CAP_PROP_POS_FRAMES,
        frame_number
    )

    ret, frame = cap.read()
```

```
    if not ret:
        continue

    x1 = int(row["x1"])
    y1 = int(row["y1"])
    x2 = int(row["x2"])
    y2 = int(row["y2"])

   # Ampliar el contexto alrededor del track
    h, w = frame.shape[:2]

    box_w = x2 - x1
    box_h = y2 - y1

    margin_x = int(box_w * 1.5)
    margin_y = int(box_h * 1.5)

    x1c = max(0, x1 - margin_x)
    y1c = max(0, y1 - margin_y)

    x2c = min(w, x2 + margin_x)
    y2c = min(h, y2 + margin_y)

    crop = frame[y1c:y2c, x1c:x2c]

    crop_rgb = cv2.cvtColor(
        crop,
        cv2.COLOR_BGR2RGB
    )

    summary = track_summary_final[
        track_summary_final["spatial_track_id"] == track_id
    ].iloc[0]

    ax.imshow(crop_rgb)

    ax.set_title(
        f"Track {track_id} | "
        f"{summary['dominant_state']} | "
        f"obs={int(summary['observations'])} | "
        f"consistencia={summary['state_consistency']:.2f}"
    )

    ax.axis("off")

cap.release()

plt.tight_layout()
```

```
plt.show()
Tracks persistentes: [20, 44, 9, 33, 46, 34, 25, 1, 24, 17, 26, 23,
45, 39, 14, 18, 53, 19, 35, 2, 55, 12, 13]
```























18.5 Clasificación semántica final de los tracks persistentes

In [71]:

```
# ============================================================
# CLASIFICACIÓN SEMÁNTICA FINAL:
# PEATONAL / NO PEATONAL
# ============================================================

PEDESTRIAN_TRACK_PROMPTS = [
    "a pedestrian crossing signal with a visible human figure",
    "a green pedestrian traffic light showing a walking person",
    "a red pedestrian traffic light showing a standing person",
    "a pedestrian countdown signal with a person icon"
]

NON_PEDESTRIAN_TRACK_PROMPTS = [
    "a vehicle traffic light with circular lamps",
    "an amber circular traffic signal",
    "the back or side of a traffic light housing",
    "a traffic signal without a human figure"
]

TRACK_TYPE_PROMPTS = (
    PEDESTRIAN_TRACK_PROMPTS
    + NON_PEDESTRIAN_TRACK_PROMPTS
)


def classify_track_type(crop):

    result = classify_with_clip(
        crop,
        TRACK_TYPE_PROMPTS
    )

    probs = result["all_probabilities"]

    # Promedio por familia para evitar sesgos
    pedestrian_score = np.mean([
        probs[p]
        for p in PEDESTRIAN_TRACK_PROMPTS
    ])

    non_pedestrian_score = np.mean([
        probs[p]
        for p in NON_PEDESTRIAN_TRACK_PROMPTS
    ])
```

```
    total = (
        pedestrian_score
        + non_pedestrian_score
    )

    if total > 0:
        pedestrian_score /= total
        non_pedestrian_score /= total

    # Zona de incertidumbre
    margin = abs(
        pedestrian_score
        - non_pedestrian_score
    )

    if margin < 0.05:
        label = "UNKNOWN"
    elif pedestrian_score > non_pedestrian_score:
        label = "PEDESTRIAN"
    else:
        label = "NON_PEDESTRIAN"

    return {
        "type": label,
        "pedestrian_score": pedestrian_score,
        "non_pedestrian_score": non_pedestrian_score
    }
```

18.6 Filtrado semántico de los tracks persistentes

In [82]:

```
# FILTRO SEMÁNTICO FINAL DE TRACKS PERSISTENTES


import cv2
import pandas as pd
from PIL import Image

if "classify_track_type" not in globals():
    raise RuntimeError(
        "Falta classify_track_type. Ejecuta antes la celda "
        "VALIDACIÓN SEMÁNTICA FINAL DE TRACKS PERSISTENTES."
    )

persistent_tracks = track_summary_final[
    track_summary_final["persistent"] == True
].copy()
```

```
cap = cv2.VideoCapture(video_test_final)

semantic_rows = []

for _, summary in persistent_tracks.iterrows():

    track_id = int(summary["spatial_track_id"])

    group = df_tracks_final[
        df_tracks_final["spatial_track_id"] == track_id
    ].copy()

    # Observación más fiable del track
    row = group.loc[
        group["p_actionable"].idxmax()
    ]

    frame_number = int(row["frame"])

    cap.set(
        cv2.CAP_PROP_POS_FRAMES,
        frame_number
    )

    ret, frame = cap.read()

    if not ret:
        continue

    x1 = int(row["x1"])
    y1 = int(row["y1"])
    x2 = int(row["x2"])
    y2 = int(row["y2"])

    # ---------------------------------------------------------
    # Crop ampliado para conservar contexto
    # ---------------------------------------------------------

    h, w = frame.shape[:2]

    box_w = x2 - x1
    box_h = y2 - y1

    margin_x = int(box_w * 1.5)
    margin_y = int(box_h * 1.5)

    x1c = max(0, x1 - margin_x)
    y1c = max(0, y1 - margin_y)
```

```
x2c = min(w, x2 + margin_x)
y2c = min(h, y2 + margin_y)

crop = frame[
    y1c:y2c,
    x1c:x2c
]

crop_rgb = cv2.cvtColor(
    crop,
    cv2.COLOR_BGR2RGB
)

crop_pil = Image.fromarray(
    crop_rgb
)

# ----------------------------------------------------------
# CLIP: peatonal / no peatonal
# ----------------------------------------------------------

semantic = classify_track_type(
    crop_pil
)

semantic_rows.append({

    "spatial_track_id": track_id,

    "first_frame": int(
        summary["first_frame"]
    ),

    "last_frame": int(
        summary["last_frame"]
    ),

    "observations": int(
        summary["observations"]
    ),

    "dominant_state":
        summary["dominant_state"],

    "state_consistency":
        summary["state_consistency"],

    "track_type":
        semantic["type"],
```

```
        "pedestrian_score":
            semantic["pedestrian_score"],

        "non_pedestrian_score":
            semantic["non_pedestrian_score"]
    })

cap.release()

df_final_pedestrian_tracks = pd.DataFrame(
    semantic_rows
)

# Tracks que finalmente consideramos semáforos peatonales
df_final_pedestrian_tracks[
    "valid_pedestrian_signal"
] = (
    df_final_pedestrian_tracks[
        "track_type"
    ] == "PEDESTRIAN"
)

df_final_pedestrian_tracks = (
    df_final_pedestrian_tracks
    .sort_values(
        [
            "valid_pedestrian_signal",
            "first_frame"
        ],
        ascending=[False, True]
    )
)

display(
    df_final_pedestrian_tracks
)

print("\n" + "=" * 70)
print("TRACKS PEATONALES FINALES")
print("=" * 70)

display(
    df_final_pedestrian_tracks[
        df_final_pedestrian_tracks[
            "valid_pedestrian_signal"
        ] == True
    ][
        [
```

```
            "spatial_track_id",
            "first_frame",
            "last_frame",
            "observations",
            "dominant_state",
            "state_consistency",
            "pedestrian_score"
        ]
    ]
)
```

	spatial_track_id	first_frame	last_frame	observations	dominant_state	state_consistency	
19	2	48	57	3	RED	1.000000	PEDES
3	9	120	198	26	RED	0.769231	PEDES
20	12	144	150	3	RED	0.666667	PEDES
21	13	150	156	3	RED	0.666667	PEDES
14	14	177	195	6	GREEN	0.666667	PEDES
9	17	219	249	10	GREEN	0.900000	PEDES
15	18	219	234	5	GREEN	0.600000	PEDES
17	19	234	240	3	GREEN	1.000000	PEDES
0	20	249	336	30	GREEN	0.833333	PEDES
11	23	264	288	8	GREEN	0.875000	PEDES
8	24	297	336	12	GREEN	0.833333	PEDES
6	25	312	357	15	GREEN	0.600000	PEDES
10	26	330	357	10	GREEN	0.700000	PEDES
4	33	423	504	25	GREEN	0.960000	PEDES
5	34	423	483	19	GREEN	0.894737	PEDES
18	35	423	429	3	RED	1.000000	PEDES
13	39	462	480	6	GREEN	1.000000	PEDES
1	44	492	570	26	GREEN	0.961538	PEDES
12	45	492	513	7	GREEN	0.857143	PEDES
2	46	513	603	26	GREEN	0.923077	PEDES
16	53	582	588	3	GREEN	1.000000	PEDES
7	1	45	90	13	RED	1.000000	NON_P

```
======================================================================
TRACKS PEATONALES FINALES
======================================================================
```

	spatial_track_id	first_frame	last_frame	observations	dominant_state	state_consistency	pedestr
19	2	48	57	3	RED	1.000000	0.6351
3	9	120	198	26	RED	0.769231	0.6431
20	12	144	150	3	RED	0.666667	0.9635

	spatial_track_id	first_frame	last_frame	observations	dominant_state	state_consistency	pedestr
21	13	150	156	3	RED	0.666667	0.8972
14	14	177	195	6	GREEN	0.666667	0.9498
9	17	219	249	10	GREEN	0.900000	0.9194
15	18	219	234	5	GREEN	0.600000	0.9734
17	19	234	240	3	GREEN	1.000000	0.9780
0	20	249	336	30	GREEN	0.833333	0.9489
11	23	264	288	8	GREEN	0.875000	0.7035
8	24	297	336	12	GREEN	0.833333	0.9597
6	25	312	357	15	GREEN	0.600000	0.9909
10	26	330	357	10	GREEN	0.700000	0.8709
4	33	423	504	25	GREEN	0.960000	0.8891
5	34	423	483	19	GREEN	0.894737	0.8562
18	35	423	429	3	RED	1.000000	0.9092
13	39	462	480	6	GREEN	1.000000	0.9399
1	44	492	570	26	GREEN	0.961538	0.7256
12	45	492	513	7	GREEN	0.857143	0.9706
2	46	513	603	26	GREEN	0.923077	0.8846
16	53	582	588	3	GREEN	1.000000	0.9209

18.7 Integración del análisis final del semáforo peatonal por vídeo

In [83]:

```
# ============================================================
# ANÁLISIS FINAL DEL SEMÁFORO PEATONAL EN UN VÍDEO
# ============================================================

def analyze_final_pedestrian_signal(video_path):

    # 1. YOLO + Gate + GREEN/RED
    df_signal = process_temporal_signal_video(
        video_path
    )

    if len(df_signal) == 0:
        return pd.DataFrame(), pd.DataFrame()

    # 2. Solo candidatos ACTIONABLE
    df_actionable = df_signal[
        df_signal["actionable"] == True
    ].copy()
```

```
if len(df_actionable) == 0:
    return df_signal, pd.DataFrame()

# 3. Tracking espacial-temporal
df_tracks = assign_signal_tracks_spatial(
    df_actionable,
    max_frame_gap=6,
    max_dist_factor=2.5
)

# 4. Resumen temporal
track_summary = (
    df_tracks
    .groupby("spatial_track_id")
    .apply(summarize_final_track)
    .reset_index()
)

track_summary["persistent"] = (
    (track_summary["observations"] >= 3)
    &
    (track_summary["state_consistency"] >= 0.60)
)

persistent_tracks = track_summary[
    track_summary["persistent"] == True
].copy()

if len(persistent_tracks) == 0:
    return df_signal, pd.DataFrame()

# 5. Validación semántica PEDESTRIAN / NON_PEDESTRIAN
cap = cv2.VideoCapture(video_path)

semantic_rows = []

for _, summary in persistent_tracks.iterrows():

    track_id = int(
        summary["spatial_track_id"]
    )

    group = df_tracks[
        df_tracks["spatial_track_id"] == track_id
    ]

    # Observación más fiable
    row = group.loc[
```

```
    group["p_actionable"].idxmax()
]

frame_number = int(row["frame"])

cap.set(
    cv2.CAP_PROP_POS_FRAMES,
    frame_number
)

ret, frame = cap.read()

if not ret:
    continue

x1 = int(row["x1"])
y1 = int(row["y1"])
x2 = int(row["x2"])
y2 = int(row["y2"])

h, w = frame.shape[:2]

box_w = x2 - x1
box_h = y2 - y1

# Contexto ampliado
margin_x = int(box_w * 1.5)
margin_y = int(box_h * 1.5)

x1c = max(0, x1 - margin_x)
y1c = max(0, y1 - margin_y)

x2c = min(w, x2 + margin_x)
y2c = min(h, y2 + margin_y)

crop = frame[
    y1c:y2c,
    x1c:x2c
]

if crop.size == 0:
    continue

crop_rgb = cv2.cvtColor(
    crop,
    cv2.COLOR_BGR2RGB
)

crop_pil = Image.fromarray(
```

```
            crop_rgb
        )

        semantic = classify_track_type(
            crop_pil
        )

        semantic_rows.append({

            "video": os.path.basename(video_path),

            "spatial_track_id": track_id,

            "first_frame":
                int(summary["first_frame"]),

            "last_frame":
                int(summary["last_frame"]),

            "observations":
                int(summary["observations"]),

            "dominant_state":
                summary["dominant_state"],

            "state_consistency":
                float(summary["state_consistency"]),

            "track_type":
                semantic["type"],

            "pedestrian_score":
                semantic["pedestrian_score"],

            "non_pedestrian_score":
                semantic["non_pedestrian_score"]
        })

    cap.release()

    df_semantic = pd.DataFrame(
        semantic_rows
    )

    if len(df_semantic) == 0:
        return df_signal, df_semantic

    # Solo semáforos peatonales finales
    df_pedestrian = df_semantic[
```

```
        df_semantic["track_type"] == "PEDESTRIAN"
    ].copy()

    return df_signal, df_pedestrian
```

18.8 Procesamiento final del conjunto de vídeos de evaluación

In [84]:

```
VIDEO_FILES = [
    "1000179702.mp4",
    "1000179704.mp4",
    "1000179707.mp4",
    "1000179709.mp4",
    "1000179711.mp4",
    "20260611_184739.mp4",
    "20260611_184808.mp4",
    "20260611_185035.mp4",
    "20260611_185443~2.mp4",
    "20260611_185616.mp4"
]

print("Lista definitiva de vídeos:")

for video in VIDEO_FILES:
    ruta = os.path.join(VIDEOS_DIR, video)
    print(
        "✓" if os.path.isfile(ruta) else "✗",
        video
    )
```

```
Lista definitiva de vídeos:
✓ 1000179702.mp4
✓ 1000179704.mp4
✓ 1000179707.mp4
✓ 1000179709.mp4
✓ 1000179711.mp4
✓ 20260611_184739.mp4
✓ 20260611_184808.mp4
✓ 20260611_185035.mp4
✓ 20260611_185443~2.mp4
✓ 20260611_185616.mp4
```

In [85]:

```
# ============================================================
# 18.8 PROCESAMIENTO FINAL DE LOS 10 VÍDEOS
# ============================================================
```

```
import os
import pandas as pd


# ------------------------------------------------------------
# 1. COMPROBAR DEPENDENCIAS
# ------------------------------------------------------------

required = [
    "VIDEOS_DIR",
    "VIDEO_FILES",
    "analyze_final_pedestrian_signal"
]

missing = [
    name
    for name in required
    if name not in globals()
]

if missing:
    raise RuntimeError(
        f"Faltan estas variables/funciones: {missing}"
    )


# ------------------------------------------------------------
# 2. COMPROBAR LOS 10 VÍDEOS
# ------------------------------------------------------------

video_paths = [
    os.path.join(
        VIDEOS_DIR,
        video_file
    )
    for video_file in VIDEO_FILES
]

print("=" * 70)
print("PROCESAMIENTO FINAL DE LOS 10 VÍDEOS")
print("=" * 70)

print("\nDirectorio:")
print(VIDEOS_DIR)

print(
    "\nVídeos del conjunto final:",
    len(VIDEO_FILES)
```

```
)


# ------------------------------------------------------------
# 3. ESTRUCTURAS DE RESULTADOS
# ------------------------------------------------------------

all_pedestrian_tracks = []
video_summary_rows = []
failed_videos = []


# ------------------------------------------------------------
# 4. PROCESAR LOS 10 VÍDEOS
# ------------------------------------------------------------

for i, (video_file, video_path) in enumerate(
    zip(VIDEO_FILES, video_paths),
    start=1
):

    video_name = os.path.basename(
        video_path
    )

    print("\n" + "=" * 70)

    print(
        f"PROCESANDO VÍDEO {i}/{len(VIDEO_FILES)}: "
        f"{video_name}"
    )

    print("=" * 70)


    # ------------------------------------------------------------
    # Comprobar existencia
    # ------------------------------------------------------------

    if not os.path.isfile(video_path):

        print(
            "NO ENCONTRADO:",
            video_path
        )

        failed_videos.append({
            "video": video_name,
            "error": "Archivo no encontrado"
```

```
        })

        continue


    try:

        # ----------------------------------------------------
        # Pipeline final:
        # YOLO + Gate + GREEN/RED +
        # tracking espacial + validación semántica
        # ----------------------------------------------------

        df_signal, df_pedestrian = (
            analyze_final_pedestrian_signal(
                video_path
            )
        )


        # Protección por si alguna función devuelve None
        if df_signal is None:
            df_signal = pd.DataFrame()

        if df_pedestrian is None:
            df_pedestrian = pd.DataFrame()


        # ----------------------------------------------------
        # Tracks peatonales obtenidos
        # ----------------------------------------------------

        if len(df_pedestrian) > 0:

            df_pedestrian = (
                df_pedestrian.copy()
            )

            # IMPORTANTE:
            # si "video" ya existe, se actualiza;
            # si no existe, se crea.
            # NO usamos insert(), porque provocaba:
            # ValueError: cannot insert video, already exists
            df_pedestrian["video"] = (
                video_name
            )

            # Dejar "video" como primera columna
            ordered_columns = (
```

```
    ["video"]
    + [
        col
        for col
        in df_pedestrian.columns
        if col != "video"
    ]
)

df_pedestrian = (
    df_pedestrian[
        ordered_columns
    ]
)


# Guardar tracks del vídeo
all_pedestrian_tracks.append(
    df_pedestrian
)


# Número de tracks GREEN
if (
    "dominant_state"
    in df_pedestrian.columns
):

    green_tracks = int(
        (
            df_pedestrian[
                "dominant_state"
            ]
            == "GREEN"
        ).sum()
    )

    red_tracks = int(
        (
            df_pedestrian[
                "dominant_state"
            ]
            == "RED"
        ).sum()
    )

else:

    green_tracks = 0
```

```
        red_tracks = 0


else:

    green_tracks = 0
    red_tracks = 0


# ------------------------------------------------------
# Resumen del vídeo
# ------------------------------------------------------

video_summary_rows.append({

    "video":
        video_name,

    "candidates":
        len(df_signal),

    "pedestrian_tracks":
        len(df_pedestrian),

    "green_tracks":
        green_tracks,

    "red_tracks":
        red_tracks
})


print("\nResumen del vídeo:")

print(
    "Candidatos:",
    len(df_signal)
)

print(
    "Tracks peatonales:",
    len(df_pedestrian)
)

print(
    "Tracks GREEN:",
    green_tracks
)
```

```
        print(
            "Tracks RED:",
            red_tracks
        )


    except Exception as exc:

        print(
            f"\nERROR procesando "
            f"{video_name}: {exc}"
        )

        failed_videos.append({
            "video": video_name,
            "error": str(exc)
        })


# ------------------------------------------------------------
# 5. DATAFRAME RESUMEN DE LOS 10 VÍDEOS
# ------------------------------------------------------------

df_video_summary = pd.DataFrame(
    video_summary_rows
)


# ------------------------------------------------------------
# 6. UNIR TODOS LOS TRACKS PEATONALES
# ------------------------------------------------------------

if all_pedestrian_tracks:

    df_all_pedestrian_tracks = (
        pd.concat(
            all_pedestrian_tracks,
            ignore_index=True
        )
    )

else:

    df_all_pedestrian_tracks = (
        pd.DataFrame()
    )


# ------------------------------------------------------------
```

```
# 7. RESULTADO FINAL
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("RESUMEN FINAL DE LOS 10 VÍDEOS")
print("=" * 70)

display(
    df_video_summary
)


print("\n" + "=" * 70)
print("TRACKS PEATONALES TOTALES")
print("=" * 70)

print(
    "Total:",
    len(df_all_pedestrian_tracks)
)


# ------------------------------------------------------------
# 8. VÍDEOS CON ERROR
# ------------------------------------------------------------

if failed_videos:

    print("\n" + "=" * 70)
    print("VÍDEOS CON ERROR")
    print("=" * 70)

    df_failed_videos = pd.DataFrame(
        failed_videos
    )

    display(
        df_failed_videos
    )

else:

    df_failed_videos = pd.DataFrame()

    print(
        "\nOK: los 10 vídeos se han "
        "procesado sin errores."
    )
======================================================================
```

```
PROCESAMIENTO FINAL DE LOS 10 VÍDEOS
======================================================================

Directorio:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM\VÍDEOS

Vídeos del conjunto final: 10

======================================================================
PROCESANDO VÍDEO 1/10: 1000179702.mp4
======================================================================
======================================================================
PIPELINE TEMPORAL FINAL
======================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\1000179702.mp4
FPS: 29.88811243138673
Frames: 607
Frames analizados: 50
Frames analizados: 100
Frames analizados: 150
Frames analizados: 200

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 291
ACTIONABLE: 79
NON_ACTIONABLE: 212

Estados:
state
NON_ACTIONABLE    212
RED                42
GREEN              37
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 291
Tracks peatonales: 1
```

```
Tracks GREEN: 1
Tracks RED: 0

================================================================================
PROCESANDO VÍDEO 2/10: 1000179704.mp4
================================================================================
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\1000179704.mp4
FPS: 30.04085247032597
Frames: 778
Frames analizados: 50
Frames analizados: 100
Frames analizados: 150
Frames analizados: 200
Frames analizados: 250

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 88
ACTIONABLE: 9
NON_ACTIONABLE: 79

Estados:
state
NON_ACTIONABLE    79
GREEN              8
RED                1
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 88
Tracks peatonales: 1
Tracks GREEN: 1
Tracks RED: 0

================================================================================
```

```
PROCESANDO VÍDEO 3/10: 1000179707.mp4
======================================================================
======================================================================
PIPELINE TEMPORAL FINAL
======================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\1000179707.mp4
FPS: 29.978597151643687
Frames: 684
Frames analizados: 50
Frames analizados: 100
Frames analizados: 150
Frames analizados: 200

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 119
ACTIONABLE: 45
NON_ACTIONABLE: 74

Estados:
state
NON_ACTIONABLE    74
RED               45
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 119
Tracks peatonales: 2
Tracks GREEN: 0
Tracks RED: 2

======================================================================
PROCESANDO VÍDEO 4/10: 1000179709.mp4
======================================================================
======================================================================
PIPELINE TEMPORAL FINAL
======================================================================
Vídeo:
```

```
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\1000179709.mp4
FPS: 29.99828368028695
Frames: 536
Frames analizados: 50
Frames analizados: 100
Frames analizados: 150

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 314
ACTIONABLE: 129
NON_ACTIONABLE: 185

Estados:
state
NON_ACTIONABLE    185
RED               121
GREEN               8
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 314
Tracks peatonales: 9
Tracks GREEN: 1
Tracks RED: 8

================================================================================
PROCESANDO VÍDEO 5/10: 1000179711.mp4
================================================================================
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\1000179711.mp4
FPS: 30.014545928225616
Frames: 553
Frames analizados: 50
Frames analizados: 100
```

```
Frames analizados: 150

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 385
ACTIONABLE: 95
NON_ACTIONABLE: 290

Estados:
state
NON_ACTIONABLE    290
RED                86
GREEN               9
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 385
Tracks peatonales: 14
Tracks GREEN: 1
Tracks RED: 13

================================================================================
PROCESANDO VÍDEO 6/10: 20260611_184739.mp4
================================================================================
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184739.mp4
FPS: 29.995415447254988
Frames: 434
Frames analizados: 50
Frames analizados: 100

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 974
ACTIONABLE: 466
```

```
NON_ACTIONABLE: 508

Estados:
state
NON_ACTIONABLE    508
GREEN             293
RED               173
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 974
Tracks peatonales: 30
Tracks GREEN: 23
Tracks RED: 7

======================================================================
PROCESANDO VÍDEO 7/10: 20260611_184808.mp4
======================================================================
======================================================================
PIPELINE TEMPORAL FINAL
======================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184808.mp4
FPS: 29.999395616571828
Frames: 364
Frames analizados: 50
Frames analizados: 100

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 252
ACTIONABLE: 116
NON_ACTIONABLE: 136

Estados:
state
NON_ACTIONABLE    136
RED                97
GREEN              19
```

```
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 252
Tracks peatonales: 8
Tracks GREEN: 0
Tracks RED: 8

======================================================================
PROCESANDO VÍDEO 8/10: 20260611_185035.mp4
======================================================================
======================================================================
PIPELINE TEMPORAL FINAL
======================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_185035.mp4
FPS: 30.08422849320503
Frames: 273
Frames analizados: 50

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 11
ACTIONABLE: 0
NON_ACTIONABLE: 11

Estados:
state
NON_ACTIONABLE    11
Name: count, dtype: int64

Resumen del vídeo:
Candidatos: 11
Tracks peatonales: 0
Tracks GREEN: 0
Tracks RED: 0

======================================================================
PROCESANDO VÍDEO 9/10: 20260611_185443~2.mp4
```

```
================================================================================
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_185443~2.mp4
FPS: 29.733811591466868
Frames: 91

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 75
ACTIONABLE: 20
NON_ACTIONABLE: 55

Estados:
state
NON_ACTIONABLE    55
GREEN             20
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 75
Tracks peatonales: 3
Tracks GREEN: 3
Tracks RED: 0

================================================================================
PROCESANDO VÍDEO 10/10: 20260611_185616.mp4
================================================================================
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_185616.mp4
FPS: 29.997528845141506
Frames: 611
Frames analizados: 50
```

```
Frames analizados: 100
Frames analizados: 150
Frames analizados: 200

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 461
ACTIONABLE: 308
NON_ACTIONABLE: 153

Estados:
state
GREEN             218
NON_ACTIONABLE    153
RED                90
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

Resumen del vídeo:
Candidatos: 461
Tracks peatonales: 21
Tracks GREEN: 16
Tracks RED: 5

================================================================================
RESUMEN FINAL DE LOS 10 VÍDEOS
================================================================================
```

	video	candidates	pedestrian_tracks	green_tracks	red_tracks
0	1000179702.mp4	291	1	1	0
1	1000179704.mp4	88	1	1	0
2	1000179707.mp4	119	2	0	2
3	1000179709.mp4	314	9	1	8
4	1000179711.mp4	385	14	1	13
5	20260611_184739.mp4	974	30	23	7
6	20260611_184808.mp4	252	8	0	8
7	20260611_185035.mp4	11	0	0	0
8	20260611_185443~2.mp4	75	3	3	0
9	20260611_185616.mp4	461	21	16	5

```
======================================================================
TRACKS PEATONALES TOTALES
======================================================================
Total: 89

OK: los 10 vídeos se han procesado sin errores.
```

18.9 Guardado de los resultados finales del módulo de semáforos

In [86]:

```
# GUARDAR RESULTADOS FINALES DEL MÓDULO DE SEMÁFOROS


import os

SIGNAL_RESULTS_DIR = os.path.join(
    r"C:\Users\juank\Documents\CARLOS\TFM",
    "resultados_semaforo_final"
)

os.makedirs(
    SIGNAL_RESULTS_DIR,
    exist_ok=True
)

summary_path = os.path.join(
    SIGNAL_RESULTS_DIR,
    "resumen_semaforos_por_video.csv"
)

tracks_path = os.path.join(
    SIGNAL_RESULTS_DIR,
    "tracks_semaforos_peatonales.csv"
)

df_video_summary.to_csv(
    summary_path,
    index=False
)

df_all_pedestrian_tracks.to_csv(
    tracks_path,
    index=False
)

print("RESULTADOS GUARDADOS")
```

```
print("=" * 60)

print("Vídeos procesados:", len(df_video_summary))
print("Candidatos totales:", df_video_summary["candidates"].sum())
print("Tracks peatonales:",
df_video_summary["pedestrian_tracks"].sum())
print("Tracks GREEN:", df_video_summary["green_tracks"].sum())
print("Tracks RED:", df_video_summary["red_tracks"].sum())

print("\nArchivos:")
print(summary_path)
print(tracks_path)
RESULTADOS GUARDADOS
============================================================
Vídeos procesados: 10
Candidatos totales: 2970
Tracks peatonales: 89
Tracks GREEN: 46
Tracks RED: 43

Archivos:
C:\Users\juank\Documents\CARLOS\TFM\resultados_semaforo_final\resumen_
semaforos_por_video.csv
C:\Users\juank\Documents\CARLOS\TFM\resultados_semaforo_final\tracks_s
emaforos_peatonales.csv
```

19. Estimación de variables dinámicas de los vehículos

19.1 Cálculo de variables dinámicas a partir del tracking

In [107]:

```
# VARIABLES DINÁMICAS DE VEHÍCULOS
# A PARTIR DEL TRACKING YA CALCULADO


import numpy as np
import pandas as pd

# Recuperar el tracking global si el kernel se ha reiniciado
if "df_all_tracking" not in globals():

    tracking_combined_csv_path = os.path.join(
        OUTPUT_DIR,
        "dataset_completo_tracking.csv"
```

```
    )

    df_all_tracking = pd.read_csv(
        tracking_combined_csv_path
    )

    print(
        "Tracking global cargado:",
        len(df_all_tracking),
        "registros"
    )

    print(
        "Vídeos incluidos:",
        df_all_tracking["video"].nunique()
    )

VEHICLE_CLASSES = [
    "car",
    "truck",
    "bus",
    "motorcycle",
    "bicycle"
]

# ------------------------------------------------------------
# 1. Seleccionar únicamente vehículos
# ------------------------------------------------------------

df_vehicle_dynamic = df_all_tracking[
    df_all_tracking["class_name"].isin(
        VEHICLE_CLASSES
    )
].copy()

df_vehicle_dynamic = df_vehicle_dynamic.sort_values(
    [
        "video",
        "track_id",
        "frame"
    ]
).reset_index(drop=True)


# ------------------------------------------------------------
# 2. Diferencias temporales dentro de cada trayectoria
# ------------------------------------------------------------

group_cols = [
```

```
    "video",
    "track_id"
]

g = df_vehicle_dynamic.groupby(
    group_cols,
    sort=False
)

df_vehicle_dynamic["delta_t"] = (
    g["time_seconds"].diff()
)

df_vehicle_dynamic["delta_x_norm"] = (
    g["center_x_norm"].diff()
)

df_vehicle_dynamic["delta_y_norm"] = (
    g["center_y_norm"].diff()
)

df_vehicle_dynamic["displacement_norm"] = np.sqrt(
    df_vehicle_dynamic["delta_x_norm"] ** 2
    +
    df_vehicle_dynamic["delta_y_norm"] ** 2
)


# ------------------------------------------------------------
# 3. Velocidad aparente normalizada
# ------------------------------------------------------------

df_vehicle_dynamic["speed_norm"] = (
    df_vehicle_dynamic["displacement_norm"]
    /
    df_vehicle_dynamic["delta_t"].replace(
        0,
        np.nan
    )
)


# ------------------------------------------------------------
# 4. Variación del área de la bounding box
# Indicador visual de aproximación / alejamiento
# ------------------------------------------------------------

previous_area = (
    g["bbox_area_norm"].shift(1)
```

```
)

df_vehicle_dynamic["area_change"] = (
    df_vehicle_dynamic["bbox_area_norm"]
    -
    previous_area
)

df_vehicle_dynamic["area_change_ratio"] = (
    df_vehicle_dynamic["bbox_area_norm"]
    /
    previous_area.replace(0, np.nan)
    -
    1
)


# ------------------------------------------------------------
# 5. Limpiar primeras observaciones de cada track
# ------------------------------------------------------------

dynamic_cols = [
    "delta_x_norm",
    "delta_y_norm",
    "displacement_norm",
    "speed_norm",
    "area_change",
    "area_change_ratio"
]

df_vehicle_dynamic[
    dynamic_cols
] = df_vehicle_dynamic[
    dynamic_cols
].fillna(0)


print("=" * 70)
print("VARIABLES DINÁMICAS DE VEHÍCULOS")
print("=" * 70)

print(
    "Observaciones de vehículos:",
    len(df_vehicle_dynamic)
)

print(
    "Tracks distintos:",
    df_vehicle_dynamic[
```

```
        ["video", "track_id"]
    ].drop_duplicates().shape[0]
)

print("\nClases:")
print(
    df_vehicle_dynamic[
        "class_name"
    ].value_counts()
)

display(
    df_vehicle_dynamic[
        [
            "video",
            "frame",
            "track_id",
            "class_name",
            "center_x_norm",
            "center_y_norm",
            "bbox_area_norm",
            "displacement_norm",
            "speed_norm",
            "area_change_ratio"
        ]
    ].head(20)
)
```

```
======================================================================
VARIABLES DINÁMICAS DE VEHÍCULOS
======================================================================
Observaciones de vehículos: 9441
Tracks distintos: 301

Clases:
class_name
car           8659
truck          407
bicycle        323
motorcycle      43
bus              9
Name: count, dtype: int64
```

	video	frame	track_id	class_name	center_x_norm	center_y_norm	bbox_area_norm	displ
0	1000179702	0	1	car	0.61481	0.52865	0.461111	0.000
1	1000179702	1	1	car	0.61528	0.53542	0.463754	0.006
2	1000179702	2	1	car	0.61481	0.54063	0.466715	0.005
3	1000179702	3	1	car	0.61574	0.54479	0.464792	0.004
4	1000179702	4	1	car	0.61713	0.55521	0.459925	0.010

	video	frame	track_id	class_name	center_x_norm	center_y_norm	bbox_area_norm	displ
5	1000179702	5	1	car	0.61667	0.55755	0.462474	0.002
6	1000179702	6	1	car	0.61898	0.55885	0.460862	0.002
7	1000179702	7	1	car	0.61852	0.56667	0.470145	0.007
8	1000179702	8	1	car	0.61204	0.58490	0.501535	0.019
9	1000179702	9	1	car	0.60139	0.60781	0.554274	0.025
10	1000179702	10	1	car	0.61435	0.59297	0.511761	0.019
11	1000179702	11	1	car	0.62222	0.59583	0.491478	0.008
12	1000179702	12	1	car	0.63333	0.59375	0.456414	0.011
13	1000179702	13	1	car	0.63889	0.59401	0.440852	0.005
14	1000179702	14	1	car	0.64259	0.59323	0.432234	0.003
15	1000179702	15	1	car	0.64491	0.59427	0.430903	0.002
16	1000179702	16	1	car	0.64769	0.59297	0.425693	0.003
17	1000179702	17	1	car	0.65139	0.59010	0.419395	0.004
18	1000179702	18	1	car	0.65417	0.58802	0.416045	0.003
19	1000179702	19	1	car	0.65694	0.58672	0.413763	0.003

19.2 Resumen dinámico por trayectoria de vehículo

In [108]:

```
# RESUMEN DINÁMICO POR TRAYECTORIA DE VEHÍCULO


df_vehicle_track_summary = (
    df_vehicle_dynamic
    .groupby(
        ["video", "track_id"],
        as_index=False
    )
    .agg(
        class_name=(
            "class_name",
            "first"
        ),

        observations=(
            "frame",
            "size"
        ),

        first_frame=(
            "frame",
```

```
    "min"
),

last_frame=(
    "frame",
    "max"
),

first_time=(
    "time_seconds",
    "min"
),

last_time=(
    "time_seconds",
    "max"
),

start_area=(
    "bbox_area_norm",
    "first"
),

end_area=(
    "bbox_area_norm",
    "last"
),

mean_area=(
    "bbox_area_norm",
    "mean"
),

mean_speed=(
    "speed_norm",
    "mean"
),

median_speed=(
    "speed_norm",
    "median"
),

max_speed=(
    "speed_norm",
    "max"
),

mean_displacement=(
```

```
            "displacement_norm",
            "mean"
        ),

        max_displacement=(
            "displacement_norm",
            "max"
        ),

        mean_area_change=(
            "area_change_ratio",
            "mean"
        ),

        median_area_change=(
            "area_change_ratio",
            "median"
        )
    )
)


# ============================================================
# VARIABLES DERIVADAS DE LA TRAYECTORIA
# ============================================================

df_vehicle_track_summary[
    "duration_s"
] = (
    df_vehicle_track_summary["last_time"]
    -
    df_vehicle_track_summary["first_time"]
)


# Cambio neto del tamaño aparente del vehículo
df_vehicle_track_summary[
    "net_area_growth"
] = np.where(

    df_vehicle_track_summary[
        "start_area"
    ] > 0,

    (
        df_vehicle_track_summary[
            "end_area"
        ]
        /
```

```
        df_vehicle_track_summary[
            "start_area"
        ]
    ) - 1,

    np.nan
)


print("=" * 70)
print("RESUMEN DINÁMICO DE TRAYECTORIAS")
print("=" * 70)

print(
    "Trayectorias:",
    len(df_vehicle_track_summary)
)

print("\nClases:")
print(
    df_vehicle_track_summary[
        "class_name"
    ].value_counts()
)


display(
    df_vehicle_track_summary[
        [
            "video",
            "track_id",
            "class_name",
            "observations",
            "duration_s",
            "mean_speed",
            "max_speed",
            "start_area",
            "end_area",
            "net_area_growth"
        ]
    ]
    .sort_values(
        "net_area_growth",
        ascending=False
    )
    .head(20)
)
```

```
print("\n" + "=" * 70)
print("DISTRIBUCIÓN DE VARIABLES DINÁMICAS")
print("=" * 70)

display(
    df_vehicle_track_summary[
        [
            "observations",
            "duration_s",
            "mean_speed",
            "max_speed",
            "net_area_growth"
        ]
    ].describe(
        percentiles=[
            0.10,
            0.25,
            0.50,
            0.75,
            0.90,
            0.95
        ]
    )
)
```

```
======================================================================
RESUMEN DINÁMICO DE TRAYECTORIAS
======================================================================
Trayectorias: 301

Clases:
class_name
car           272
truck          17
motorcycle      7
bicycle         5
Name: count, dtype: int64
```

	video	track_id	class_name	observations	duration_s	mean_speed	max_speed	st
222	20260611_185035	68	car	74	2.426	0.380803	0.815736	0.
181	20260611_184808	117	car	116	3.833	0.254522	0.584368	0.
178	20260611_184808	83	car	54	1.800	0.485378	1.026471	0.
202	20260611_185035	32	car	79	2.592	0.355289	0.766596	0.
292	20260611_185616	190	car	119	3.934	0.119912	0.287302	0.
96	1000179709	80	car	30	0.966	0.689667	1.297952	0.
65	1000179707	95	car	254	8.973	0.162508	1.212151	0.
177	20260611_184808	76	car	66	2.166	0.412056	0.877030	0.
298	20260611_185616	207	car	63	2.067	0.137289	0.716538	0.

	video	track_id	class_name	observations	duration_s	mean_speed	max_speed	st
234	20260611_185035	116	car	55	1.795	0.104813	0.264434	0.
244	20260611_185443~2	12	car	85	2.825	0.073174	0.136391	0.
141	20260611_184739	85	car	30	1.000	0.097836	0.212951	0.
90	1000179709	40	car	24	0.767	0.873207	1.501729	0.
38	1000179704	85	car	23	0.732	0.182312	0.455950	0.
5	1000179702	34	car	68	2.275	0.359484	0.669496	0.
67	1000179707	100	car	279	9.307	0.150817	3.051620	0.
49	1000179707	21	car	88	3.869	0.180895	1.273448	0.
86	1000179709	30	car	27	0.866	0.839021	1.439331	0.
95	1000179709	68	car	34	1.100	0.698890	1.389355	0.
230	20260611_185035	91	car	94	3.092	0.107964	0.459606	0.

```
======================================================================
DISTRIBUCIÓN DE VARIABLES DINÁMICAS
======================================================================
```

	observations	duration_s	mean_speed	max_speed	net_area_growth
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	31.365449	1.176080	0.168515	0.432350	0.975736
std	50.322884	1.801767	0.197815	0.522119	5.213497
min	1.000000	0.000000	0.000000	0.000000	-0.913762
10%	1.000000	0.000000	0.000000	0.000000	-0.389981
25%	3.000000	0.067000	0.054441	0.101282	-0.105625
50%	12.000000	0.500000	0.110684	0.261698	0.000000
75%	39.000000	1.633000	0.193367	0.525865	0.249238
90%	85.000000	3.062000	0.380803	1.178485	1.434891
95%	114.000000	3.869000	0.616705	1.502232	3.912578
max	347.000000	12.135000	1.512770	3.051620	60.176683

19.3 Clasificación dinámica de las trayectorias de vehículos

In [109]:

```
# CLASIFICACIÓN DINÁMICA DE TRAYECTORIAS DE VEHÍCULOS

import numpy as np

df_vehicle_risk = df_vehicle_track_summary.copy()

# ----------------------------------------------------------
# 1. Eliminar trayectorias demasiado cortas
```

```
# ------------------------------------------------------------

df_vehicle_risk["valid_track"] = (
    df_vehicle_risk["observations"] >= 5
)

df_valid = df_vehicle_risk[
    df_vehicle_risk["valid_track"]
].copy()


# ------------------------------------------------------------
# 2. Umbral de movimiento
# Percentil 25 de la velocidad media
# ------------------------------------------------------------

MOVING_THRESHOLD = df_valid[
    "mean_speed"
].quantile(0.25)


# ------------------------------------------------------------
# 3. Crecimiento robusto del área
# Logaritmo del cociente final/inicial
# ------------------------------------------------------------

eps = 1e-6

df_vehicle_risk["log_area_growth"] = np.log(
    (
        df_vehicle_risk["end_area"] + eps
    )
    /
    (
        df_vehicle_risk["start_area"] + eps
    )
)

APPROACH_THRESHOLD = (
    df_vehicle_risk.loc[
        df_vehicle_risk["valid_track"],
        "log_area_growth"
    ]
    .quantile(0.75)
)


# ------------------------------------------------------------
# 4. Clasificación dinámica
```

```
# ------------------------------------------------------------

df_vehicle_risk["moving"] = (
    df_vehicle_risk["valid_track"]
    &
    (
        df_vehicle_risk["mean_speed"]
        > MOVING_THRESHOLD
    )
)

df_vehicle_risk["approaching"] = (
    df_vehicle_risk["moving"]
    &
    (
        df_vehicle_risk["log_area_growth"]
        > APPROACH_THRESHOLD
    )
    &
    (
        df_vehicle_risk["end_area"]
        > df_vehicle_risk["start_area"]
    )
)


# ------------------------------------------------------------
# 5. Estado dinámico interpretable
# ------------------------------------------------------------

def vehicle_dynamic_state(row):

    if not row["valid_track"]:
        return "INSUFFICIENT"

    if row["approaching"]:
        return "APPROACHING"

    if row["moving"]:
        return "MOVING"

    return "STABLE"


df_vehicle_risk["dynamic_state"] = (
    df_vehicle_risk.apply(
        vehicle_dynamic_state,
        axis=1
    )
```

```
)


# ------------------------------------------------------------
# RESULTADOS
# ------------------------------------------------------------

print("=" * 70)
print("CLASIFICACIÓN DINÁMICA DE VEHÍCULOS")
print("=" * 70)

print(
    f"Umbral MOVING: {MOVING_THRESHOLD:.4f}"
)

print(
    f"Umbral APPROACHING (log área): "
    f"{APPROACH_THRESHOLD:.4f}"
)

print("\nEstados:")
print(
    df_vehicle_risk[
        "dynamic_state"
    ].value_counts()
)

print("\nPor clase:")
display(
    pd.crosstab(
        df_vehicle_risk["class_name"],
        df_vehicle_risk["dynamic_state"]
    )
)

display(
    df_vehicle_risk[
        [
            "video",
            "track_id",
            "class_name",
            "observations",
            "mean_speed",
            "start_area",
            "end_area",
            "log_area_growth",
            "dynamic_state"
        ]
    ]
```

```
    .sort_values(
        [
            "dynamic_state",
            "mean_speed"
        ],
        ascending=[True, False]
    )
)
```

```
======================================================================
CLASIFICACIÓN DINÁMICA DE VEHÍCULOS
======================================================================
Umbral MOVING: 0.0908
Umbral APPROACHING (log área): 0.5639

Estados:
dynamic_state
INSUFFICIENT    105
MOVING          102
STABLE           49
APPROACHING      45
Name: count, dtype: int64

Por clase:
```

dynamic_state	APPROACHING	INSUFFICIENT	MOVING	STABLE
class_name				
bicycle	0	2	3	0
car	44	87	96	45
motorcycle	1	3	3	0
truck	0	13	0	4

	video	track_id	class_name	observations	mean_speed	start_area	end_area	log_
106	1000179709	119	car	14	0.942618	0.136227	0.269939	0.68
83	1000179709	25	car	25	0.878759	0.022492	0.076489	1.22
90	1000179709	40	car	24	0.873207	0.009028	0.057955	1.85
86	1000179709	30	car	27	0.839021	0.028878	0.132200	1.52
91	1000179709	44	car	23	0.782687	0.068542	0.216867	1.15
...	...	...	...	...	...	...	...	...
225	20260611_185035	79	truck	9	0.038066	0.045897	0.046519	0.01
219	20260611_185035	63	car	18	0.037150	0.027831	0.024595	-0.12
249	20260611_185443~2	21	car	19	0.034981	0.010069	0.004653	-0.7
48	1000179707	19	car	6	0.032569	0.002234	0.001753	-0.24
283	20260611_185616	151	car	14	0.025257	0.002876	0.003013	0.04

301 rows × 9 columns

20. Fusión multimodal y estimación final del riesgo de cruce

20.1 Fusión temporal y reglas de decisión de riesgo

In [110]:

```
# ============================================================
# FUSIÓN FINAL:
# SEMÁFORO PEATONAL + DINÁMICA DE VEHÍCULOS
# ============================================================

import os
import pandas as pd
import numpy as np


# ------------------------------------------------------------
# 1. NORMALIZAR IDENTIFICADOR DE VÍDEO
# ------------------------------------------------------------

def normalize_video_id(video):
    name = os.path.basename(str(video))
    return os.path.splitext(name)[0]


df_signals_fusion = df_all_pedestrian_tracks.copy()
df_vehicles_fusion = df_vehicle_risk.copy()

df_signals_fusion["video_id"] = (
    df_signals_fusion["video"]
    .apply(normalize_video_id)
)

df_vehicles_fusion["video_id"] = (
    df_vehicles_fusion["video"]
    .apply(normalize_video_id)
)


# ------------------------------------------------------------
# 2. EXCLUIR EL VÍDEO NO PERTENECIENTE AL CONJUNTO FINAL
# ------------------------------------------------------------

EXCLUDED_VIDEO = "1000179710"

df_signals_fusion = df_signals_fusion[
    df_signals_fusion["video_id"] != EXCLUDED_VIDEO
```

```
].copy()

df_vehicles_fusion = df_vehicles_fusion[
    df_vehicles_fusion["video_id"] != EXCLUDED_VIDEO
].copy()


# ------------------------------------------------------------
# 3. FUSIÓN TEMPORAL
# ------------------------------------------------------------

fusion_rows = []

for _, signal in df_signals_fusion.iterrows():

    video_id = signal["video_id"]

    signal_start = int(signal["first_frame"])
    signal_end = int(signal["last_frame"])

    # Vehículos pertenecientes al mismo vídeo
    vehicles_video = df_vehicles_fusion[
        df_vehicles_fusion["video_id"] == video_id
    ].copy()

    # Solapamiento temporal entre track de semáforo y vehículo
    overlapping = vehicles_video[
        (vehicles_video["last_frame"] >= signal_start)
        &
        (vehicles_video["first_frame"] <= signal_end)
    ].copy()

    # Solo tracks con información suficiente
    valid_overlapping = overlapping[
        overlapping["valid_track"] == True
    ].copy()

    approaching = valid_overlapping[
        valid_overlapping["dynamic_state"] == "APPROACHING"
    ]

    moving = valid_overlapping[
        valid_overlapping["dynamic_state"] == "MOVING"
    ]

    stable = valid_overlapping[
        valid_overlapping["dynamic_state"] == "STABLE"
    ]
```

```
n_vehicles = len(valid_overlapping)
n_approaching = len(approaching)
n_moving = len(moving)
n_stable = len(stable)

# ---------------------------------------------------------
# 4. REGLAS DE DECISIÓN
# ---------------------------------------------------------

signal_state = signal["dominant_state"]

if signal_state == "RED":

    risk_level = "HIGH"
    recommendation = "WAIT"

elif signal_state == "GREEN":

    if n_approaching > 0:

        risk_level = "HIGH"
        recommendation = "CAUTION"

    elif n_moving > 0:

        risk_level = "MEDIUM"
        recommendation = "CAUTION"

    else:

        risk_level = "LOW"
        recommendation = "CROSS_WITH_CAUTION"

else:

    risk_level = "HIGH"
    recommendation = "WAIT"


# ---------------------------------------------------------
# 5. GUARDAR EVENTO DE DECISIÓN
# ---------------------------------------------------------

fusion_rows.append({

    "video": signal["video"],
    "video_id": video_id,

    "signal_track":
```

```
            int(signal["spatial_track_id"]),

        "signal_first_frame":
            signal_start,

        "signal_last_frame":
            signal_end,

        "signal_state":
            signal_state,

        "signal_consistency":
            float(signal["state_consistency"]),

        "pedestrian_score":
            float(signal["pedestrian_score"]),

        "vehicles_overlap":
            n_vehicles,

        "approaching_vehicles":
            n_approaching,

        "moving_vehicles":
            n_moving,

        "stable_vehicles":
            n_stable,

        "risk_level":
            risk_level,

        "recommendation":
            recommendation
    })


df_final_decisions = pd.DataFrame(
    fusion_rows
)


# ------------------------------------------------------------
# 6. RESULTADOS
# ------------------------------------------------------------

print("=" * 70)
print("FUSIÓN FINAL DE RIESGO")
print("=" * 70)
```

```
print(
    "Eventos de decisión:",
    len(df_final_decisions)
)

print("\nEstados del semáforo:")
print(
    df_final_decisions[
        "signal_state"
    ].value_counts()
)

print("\nNiveles de riesgo:")
print(
    df_final_decisions[
        "risk_level"
    ].value_counts()
)

print("\nRecomendaciones:")
print(
    df_final_decisions[
        "recommendation"
    ].value_counts()
)

display(
    df_final_decisions[
        [
            "video",
            "signal_state",
            "vehicles_overlap",
            "approaching_vehicles",
            "moving_vehicles",
            "risk_level",
            "recommendation"
        ]
    ]
)
```

```
======================================================================
FUSIÓN FINAL DE RIESGO
======================================================================
Eventos de decisión: 89

Estados del semáforo:
signal_state
GREEN    46
RED      43
```

```
Name: count, dtype: int64

Niveles de riesgo:
risk_level
HIGH      74
MEDIUM    13
LOW        2
Name: count, dtype: int64

Recomendaciones:
recommendation
CAUTION               44
WAIT                  43
CROSS_WITH_CAUTION     2
Name: count, dtype: int64
```

	video	signal_state	vehicles_overlap	approaching_vehicles	moving_vehicles	ris
0	1000179702.mp4	GREEN	3	1	0	HI
1	1000179704.mp4	GREEN	2	1	1	HI
2	1000179707.mp4	RED	4	0	2	HI
3	1000179707.mp4	RED	1	1	0	HI
4	1000179709.mp4	RED	2	0	2	HI
...	...	...	...	...	...	...
84	20260611_185616.mp4	GREEN	3	3	0	HI
85	20260611_185616.mp4	GREEN	10	6	3	HI
86	20260611_185616.mp4	GREEN	5	4	1	HI
87	20260611_185616.mp4	GREEN	10	6	3	HI
88	20260611_185616.mp4	GREEN	7	5	1	HI

89 rows × 7 columns

20.2 Guardado de los resultados de riesgo y recomendación

In [111]:

```
# GUARDAR RESULTADOS FINALES DE RIESGO Y RECOMENDACIÓN


FINAL_RESULTS_DIR = os.path.join(
    r"C:\Users\juank\Documents\CARLOS\TFM",
    "resultados_finales"
)

os.makedirs(
    FINAL_RESULTS_DIR,
    exist_ok=True
```

```
)

final_decisions_path = os.path.join(
    FINAL_RESULTS_DIR,
    "decisiones_finales_riesgo.csv"
)

df_final_decisions.to_csv(
    final_decisions_path,
    index=False
)

print("=" * 70)
print("PIPELINE FINAL COMPLETADO")
print("=" * 70)

print("Eventos:", len(df_final_decisions))

print("\nRiesgo:")
print(df_final_decisions["risk_level"].value_counts())

print("\nRecomendación:")
print(df_final_decisions["recommendation"].value_counts())

print("\nArchivo guardado:")
print(final_decisions_path)
```

```
======================================================================
PIPELINE FINAL COMPLETADO
======================================================================
Eventos: 89

Riesgo:
risk_level
HIGH      74
MEDIUM    13
LOW        2
Name: count, dtype: int64

Recomendación:
recommendation
CAUTION               44
WAIT                  43
CROSS_WITH_CAUTION     2
Name: count, dtype: int64

Archivo guardado:
C:\Users\juank\Documents\CARLOS\TFM\resultados_finales\decisiones_fina
les_riesgo.csv
```

20.3 Resumen final por vídeo del conjunto de evaluación

In [112]:

```
# ============================================================
# 20.3 CONSTRUCCIÓN DEL RESUMEN FINAL POR VÍDEO
# ============================================================

import pandas as pd


# ------------------------------------------------------------
# 1. COMPROBAR QUE EXISTEN LAS DECISIONES FINALES
# ------------------------------------------------------------

if "df_final_decisions" not in globals():
    raise RuntimeError(
        "Falta df_final_decisions. "
        "Ejecuta primero la celda de fusión final de riesgo."
    )


# ------------------------------------------------------------
# 2. RESUMEN DE EVENTOS POR VÍDEO
# ------------------------------------------------------------

summary_rows = []

for video_name, group in df_final_decisions.groupby("video"):

    summary_rows.append({

        "video":
            video_name,

        "events":
            len(group),

        "green_events":
            int(
                (group["signal_state"] == "GREEN").sum()
            ),

        "red_events":
            int(
                (group["signal_state"] == "RED").sum()
            ),
```

```
        "risk_high":
            int(
                (group["risk_level"] == "HIGH").sum()
            ),

        "risk_medium":
            int(
                (group["risk_level"] == "MEDIUM").sum()
            ),

        "risk_low":
            int(
                (group["risk_level"] == "LOW").sum()
            ),

        "wait":
            int(
                (group["recommendation"] == "WAIT").sum()
            ),

        "caution":
            int(
                (group["recommendation"] == "CAUTION").sum()
            ),

        "cross_with_caution":
            int(
                (
                    group["recommendation"]
                    == "CROSS_WITH_CAUTION"
                ).sum()
            )
    })


df_final_summary = pd.DataFrame(
    summary_rows
)


# ------------------------------------------------------------
# 3. ASEGURAR QUE APAREZCAN LOS 10 VÍDEOS
# ------------------------------------------------------------

base_videos = (
    df_video_summary[
        df_video_summary["video"]
        != "1000179710.mp4"
    ][["video"]]
```

```
    .drop_duplicates()
    .copy()
)


# ------------------------------------------------------------
# 4. INCORPORAR EL RESUMEN DE DECISIONES
# ------------------------------------------------------------

df_final_summary_10 = (
    base_videos
    .merge(
        df_final_summary,
        on="video",
        how="left"
    )
    .fillna(0)
)


# ------------------------------------------------------------
# 5. CONVERTIR COLUMNAS NUMÉRICAS A ENTEROS
# ------------------------------------------------------------

numeric_cols = [
    "events",
    "green_events",
    "red_events",
    "risk_high",
    "risk_medium",
    "risk_low",
    "wait",
    "caution",
    "cross_with_caution"
]

df_final_summary_10[numeric_cols] = (
    df_final_summary_10[numeric_cols]
    .astype(int)
)


# ------------------------------------------------------------
# 6. MOSTRAR RESULTADO
# ------------------------------------------------------------

print("=" * 70)
print("RESUMEN FINAL POR VÍDEO")
print("=" * 70)
```

```
display(
    df_final_summary_10
)

print(
    "\nNúmero de vídeos:",
    len(df_final_summary_10)
)

assert len(df_final_summary_10) == 10, (
    f"Se esperaban 10 vídeos y hay "
    f"{len(df_final_summary_10)}."
)
```

```
======================================================================
RESUMEN FINAL POR VÍDEO
======================================================================
```

	video	events	green_events	red_events	risk_high	risk_medium	risk_low	wait
0	1000179702.mp4	1	1	0	1	0	0	0
1	1000179704.mp4	1	1	0	1	0	0	0
2	1000179707.mp4	2	0	2	2	0	0	2
3	1000179709.mp4	9	1	8	8	1	0	8
4	1000179711.mp4	14	1	13	14	0	0	13
5	20260611_184739.mp4	30	23	7	22	8	0	7
6	20260611_184808.mp4	8	0	8	8	0	0	8
7	20260611_185035.mp4	0	0	0	0	0	0	0
8	20260611_185443~2.mp4	3	3	0	0	2	1	0
9	20260611_185616.mp4	21	16	5	18	2	1	5

```
Número de vídeos: 10
```

20.4 Guardado del resumen final por vídeo

In [113]:

```
# ============================================================
# GUARDAR RESUMEN FINAL DE LOS 10 VÍDEOS
# ============================================================

summary_final_path = os.path.join(
    FINAL_RESULTS_DIR,
    "resumen_final_por_video.csv"
)

df_final_summary_10.to_csv(
    summary_final_path,
```

```
    index=False
)

print("=" * 70)
print("RESUMEN FINAL GUARDADO")
print("=" * 70)
print("Vídeos:", len(df_final_summary_10))
print("Eventos:", df_final_summary_10["events"].sum())
print("\nArchivo:")
print(summary_final_path)
```

```
======================================================================
RESUMEN FINAL GUARDADO
======================================================================
Vídeos: 10
Eventos: 89

Archivo:
C:\Users\juank\Documents\CARLOS\TFM\resultados_finales\resumen_final_p
or_video.csv
```

21. Análisis y visualización de los resultados finales

21.1 Distribución global de los niveles de riesgo

In [114]:

```
# ============================================================
# GRÁFICA 1. DISTRIBUCIÓN GLOBAL DE NIVELES DE RIESGO
# ============================================================

import matplotlib.pyplot as plt

risk_counts = (
    df_final_decisions["risk_level"]
    .value_counts()
    .reindex(["LOW", "MEDIUM", "HIGH"], fill_value=0)
)

plt.figure(figsize=(7, 5))

bars = plt.bar(
    risk_counts.index,
    risk_counts.values
)

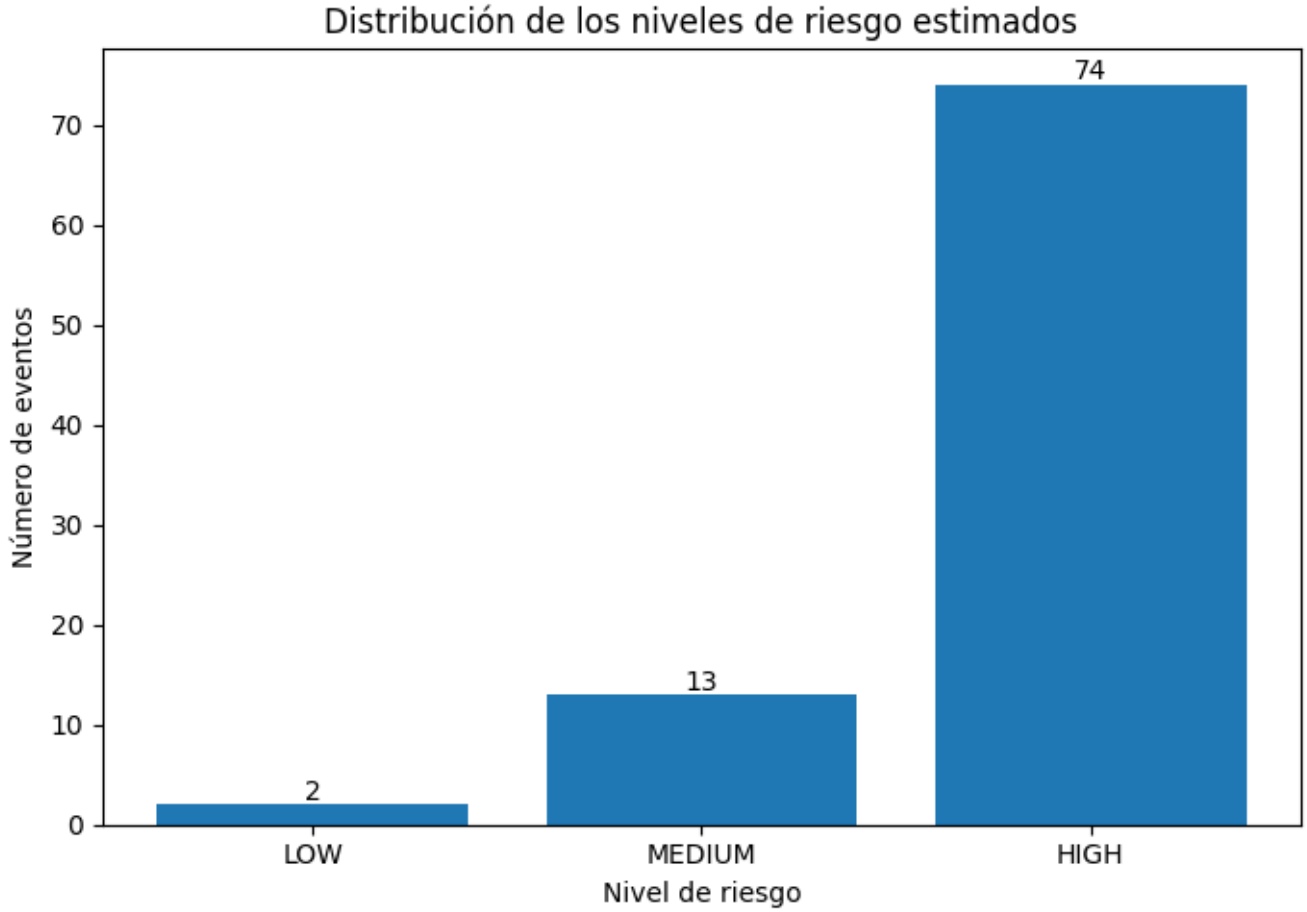
plt.title(
    "Distribución de los niveles de riesgo estimados"
```

```
)

plt.xlabel("Nivel de riesgo")
plt.ylabel("Número de eventos")

for bar, value in zip(bars, risk_counts.values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.5,
        str(value),
        ha="center"
    )

plt.tight_layout()
plt.show()
```



In [ ]:

```
## 21.2 Distribución de las recomendaciones finales al peatón
```

In [115]:

```
# ============================================================
# GRÁFICA 2. RECOMENDACIONES FINALES AL PEATÓN
# ============================================================

recommendation_counts = (
    df_final_decisions["recommendation"]
```

```
    .value_counts()
    .reindex(
        [
            "WAIT",
            "CAUTION",
            "CROSS_WITH_CAUTION"
        ],
        fill_value=0
    )
)

plt.figure(figsize=(8, 5))

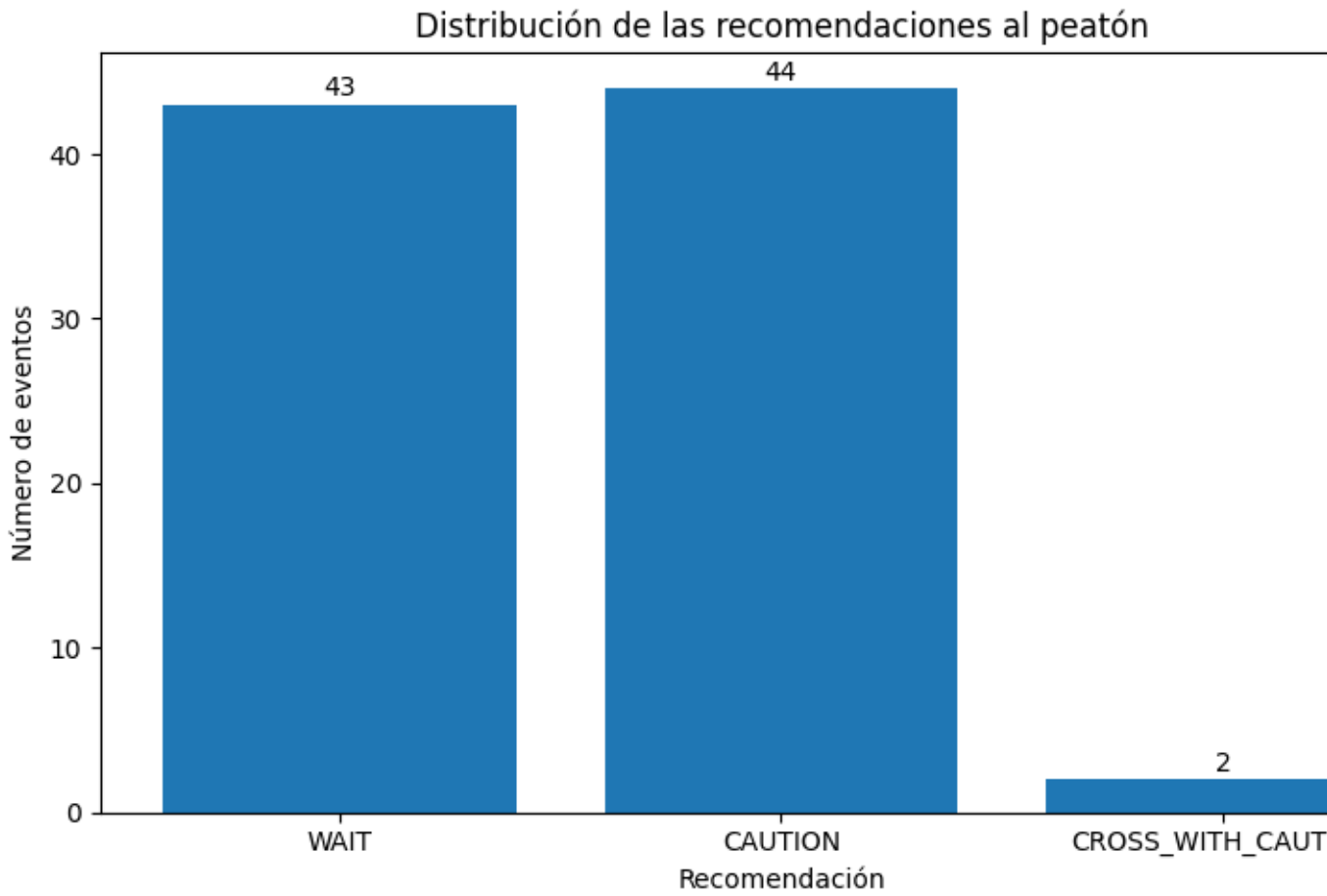
bars = plt.bar(
    recommendation_counts.index,
    recommendation_counts.values
)

plt.title(
    "Distribución de las recomendaciones al peatón"
)

plt.xlabel("Recomendación")
plt.ylabel("Número de eventos")

for bar, value in zip(
    bars,
    recommendation_counts.values
):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.5,
        str(value),
        ha="center"
    )

plt.tight_layout()
plt.show()
```



21.3 Distribución de los niveles de riesgo por vídeo

In [116]:

```
# GRÁFICA 3. DISTRIBUCIÓN DE NIVELES DE RIESGO POR VÍDEO

import numpy as np
import matplotlib.pyplot as plt

df_plot = df_final_summary_10.copy()

# Etiquetas más limpias para el eje X
video_labels = (
    df_plot["video"]
    .str.replace(".mp4", "", regex=False)
)

low = df_plot["risk_low"].values
medium = df_plot["risk_medium"].values
high = df_plot["risk_high"].values

x = np.arange(len(df_plot))

plt.figure(figsize=(13, 6))
```

```
plt.bar(
    x,
    low,
    label="LOW"
)

plt.bar(
    x,
    medium,
    bottom=low,
    label="MEDIUM",
    color="orange"
)

plt.bar(
    x,
    high,
    bottom=low + medium,
    label="HIGH",
    color="red"
)

# Número total de eventos encima de cada barra
totals = low + medium + high

for i, total in enumerate(totals):
    if total > 0:
        plt.text(
            i,
            total + 0.3,
            str(int(total)),
            ha="center"
        )

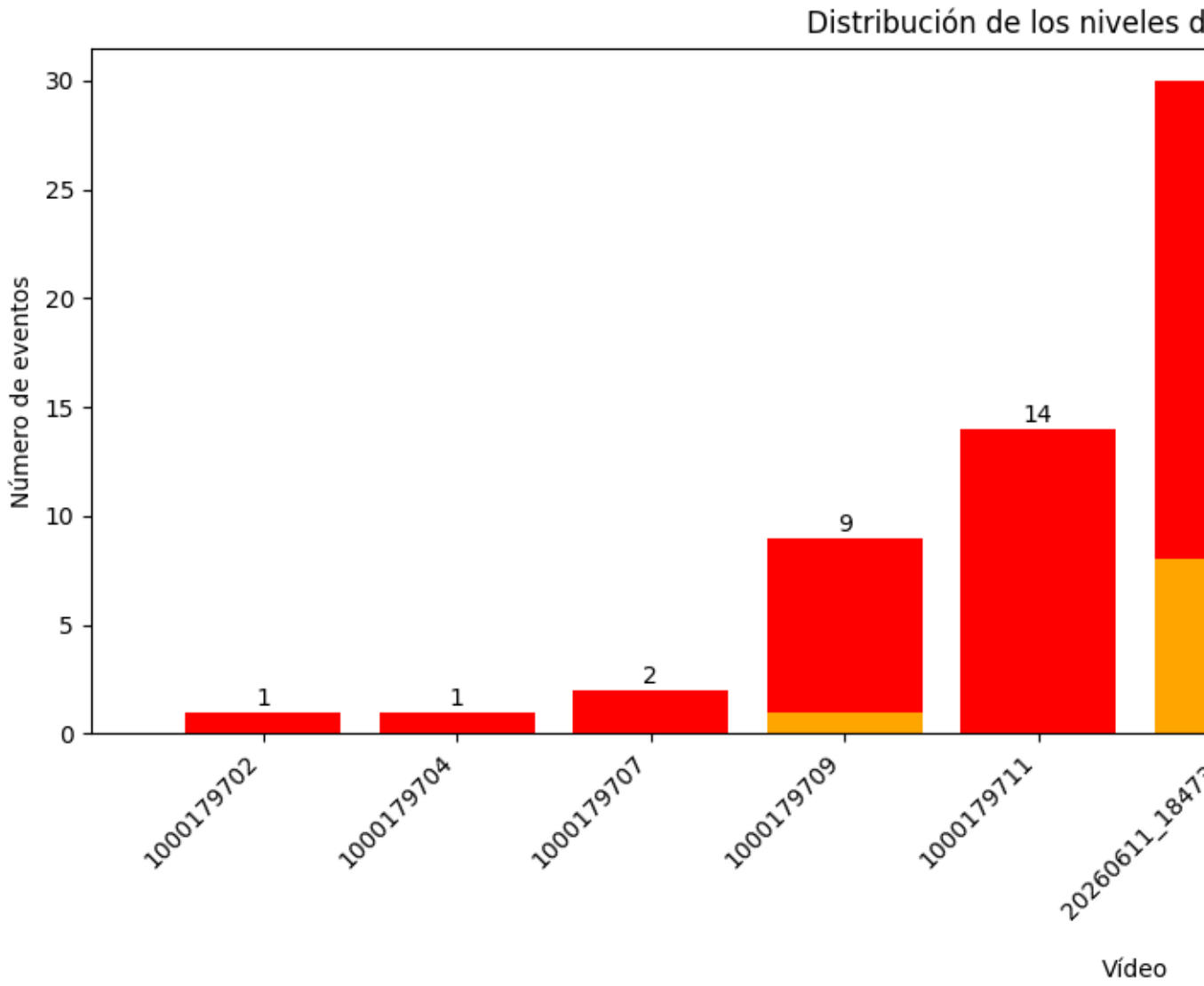
plt.title(
    "Distribución de los niveles de riesgo por vídeo"
)

plt.xlabel("Vídeo")
plt.ylabel("Número de eventos")

plt.xticks(
    x,
    video_labels,
    rotation=45,
    ha="right"
)
```

```
plt.legend(
    title="Nivel de riesgo"
)

plt.tight_layout()
plt.show()
```



21.4 Distribución de los estados del semáforo peatonal por vídeo

In [117]:

```
# GRÁFICA 4. ESTADOS DEL SEMÁFORO PEATONAL POR VÍDEO


df_plot = df_final_summary_10.copy()

video_labels = (
    df_plot["video"]
```

```
    .str.replace(".mp4", "", regex=False)
)

green = df_plot["green_events"].values
red = df_plot["red_events"].values

x = np.arange(len(df_plot))

plt.figure(figsize=(13, 6))

plt.bar(
    x,
    green,
    label="GREEN",
    color="green"
)

plt.bar(
    x,
    red,
    bottom=green,
    label="RED",
    color="red"
)

totals = green + red

for i, total in enumerate(totals):
    if total > 0:
        plt.text(
            i,
            total + 0.3,
            str(int(total)),
            ha="center"
        )

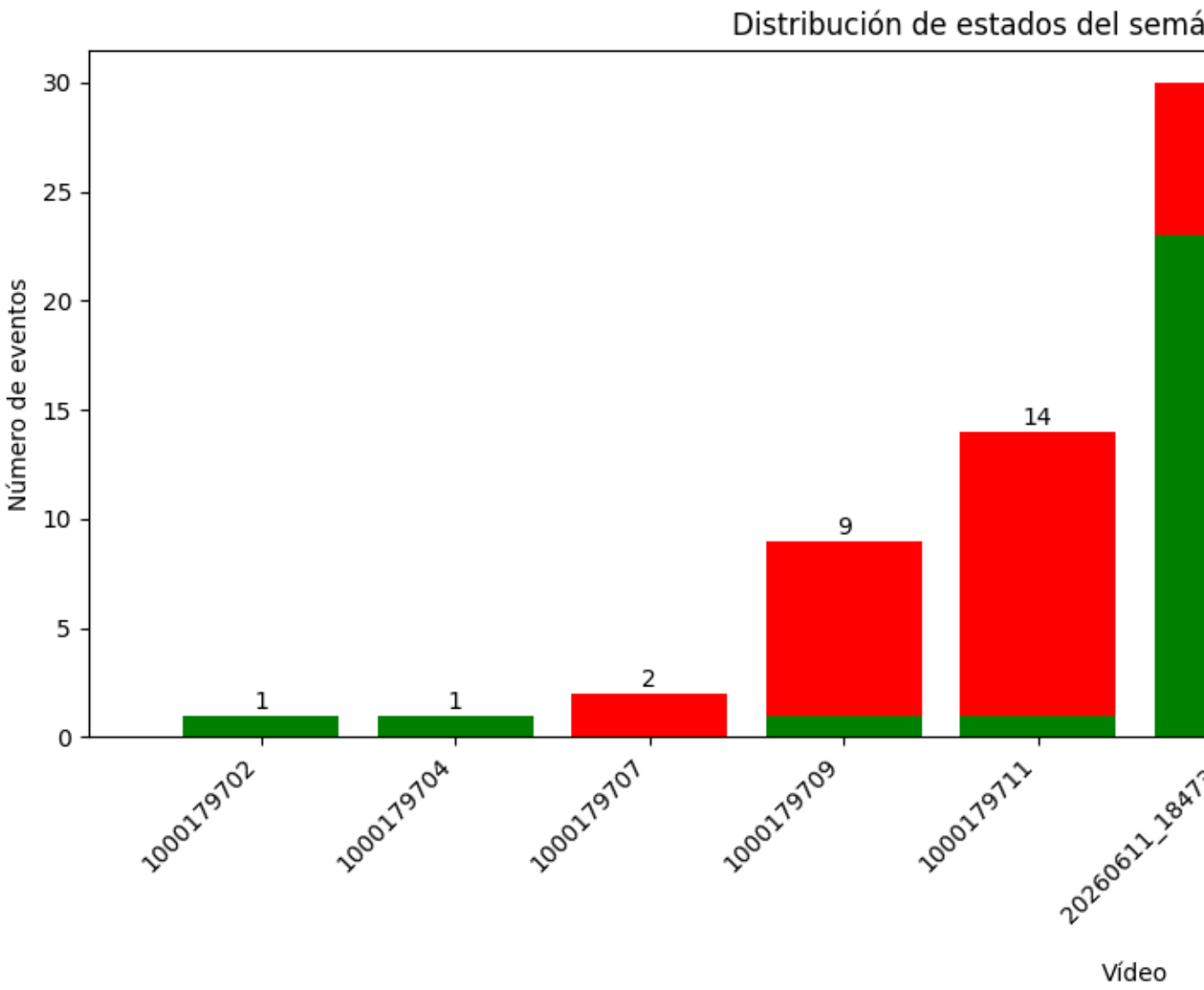
plt.title(
    "Distribución de estados del semáforo peatonal por vídeo"
)

plt.xlabel("Vídeo")
plt.ylabel("Número de eventos")

plt.xticks(
    x,
    video_labels,
    rotation=45,
    ha="right"
)
```

```
plt.legend(
    title="Estado del semáforo"
)

plt.tight_layout()
plt.show()
```



21.5 Distribución de los estados dinámicos de los vehículos

In [118]:

```
# GRÁFICA 5. ESTADOS DINÁMICOS DE LOS VEHÍCULOS


dynamic_counts = (
    df_vehicle_risk["dynamic_state"]
    .value_counts()
```

```
    .reindex(
        [
            "STABLE",
            "MOVING",
            "APPROACHING",
            "INSUFFICIENT"
        ],
        fill_value=0
    )
)

plt.figure(figsize=(8, 5))

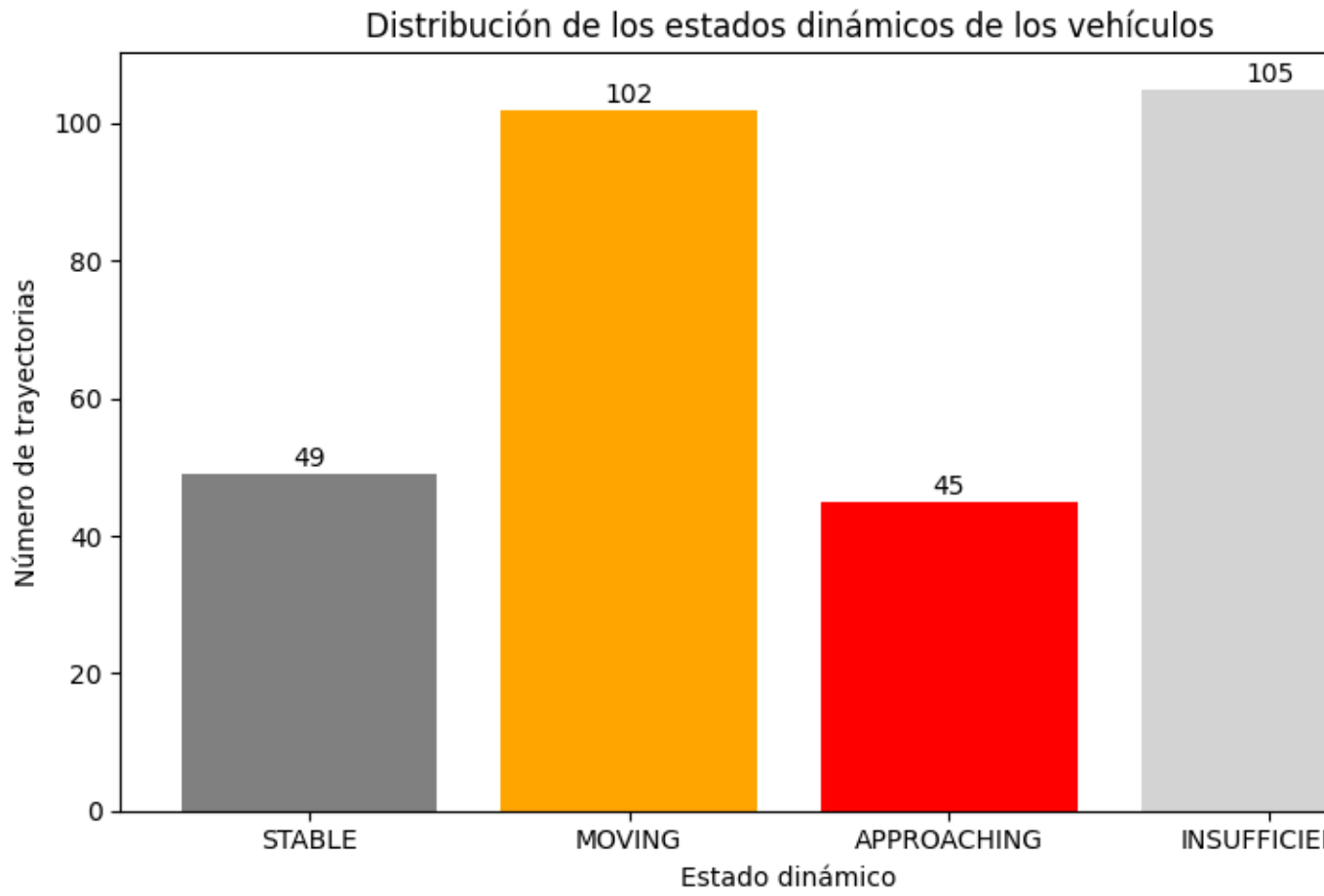
bars = plt.bar(
    dynamic_counts.index,
    dynamic_counts.values,
    color=[
        "gray",
        "orange",
        "red",
        "lightgray"
    ]
)

plt.title(
    "Distribución de los estados dinámicos de los vehículos"
)

plt.xlabel("Estado dinámico")
plt.ylabel("Número de trayectorias")

for bar, value in zip(
    bars,
    dynamic_counts.values
):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 1,
        str(value),
        ha="center"
    )

plt.tight_layout()
plt.show()
```



22.1 Preparación de la tabla de evaluación manual

In [119]:

```
# EVALUACIÓN MANUAL FINAL DE LOS EVENTOS


df_manual_eval = df_final_decisions.copy().reset_index(drop=True)

# Identificador único del evento
df_manual_eval.insert(
    0,
    "event_id",
    range(1, len(df_manual_eval) + 1)
)

# Frame representativo: centro del intervalo del semáforo
df_manual_eval["review_frame"] = (
    (
        df_manual_eval["signal_first_frame"]
        + df_manual_eval["signal_last_frame"]
    ) / 2
).round().astype(int)
```

```
# ------------------------------------------------------------
# COLUMNAS PARA ANOTACIÓN MANUAL
# ------------------------------------------------------------

df_manual_eval["manual_signal_state"] = ""
df_manual_eval["manual_risk_level"] = ""
df_manual_eval["manual_recommendation"] = ""
df_manual_eval["manual_notes"] = ""

# Ordenar columnas para facilitar la revisión
manual_columns = [
    "event_id",
    "video",
    "video_id",
    "review_frame",

    "signal_track",
    "signal_first_frame",
    "signal_last_frame",

    "signal_state",
    "signal_consistency",
    "pedestrian_score",

    "vehicles_overlap",
    "approaching_vehicles",
    "moving_vehicles",

    "risk_level",
    "recommendation",

    "manual_signal_state",
    "manual_risk_level",
    "manual_recommendation",
    "manual_notes"
]

df_manual_eval = df_manual_eval[manual_columns]

display(df_manual_eval.head(20))

print("=" * 70)
print("TABLA DE EVALUACIÓN MANUAL PREPARADA")
print("=" * 70)
print("Eventos:", len(df_manual_eval))
```

	event_id	video	video_id	review_frame	signal_track	signal_first_frame	signal_last_fr
0	1	1000179702.mp4	1000179702	177	5	165	189
1	2	1000179704.mp4	1000179704	598	0	594	603
2	3	1000179707.mp4	1000179707	208	5	192	225
3	4	1000179707.mp4	1000179707	554	11	549	558
4	5	1000179709.mp4	1000179709	69	0	45	93
5	6	1000179709.mp4	1000179709	189	9	171	207
6	7	1000179709.mp4	1000179709	304	19	300	309
7	8	1000179709.mp4	1000179709	471	21	465	477
8	9	1000179709.mp4	1000179709	471	22	465	477
9	10	1000179709.mp4	1000179709	483	23	477	489
10	11	1000179709.mp4	1000179709	482	24	477	486
11	12	1000179709.mp4	1000179709	496	25	483	510
12	13	1000179709.mp4	1000179709	495	27	486	504
13	14	1000179711.mp4	1000179711	99	3	66	132
14	15	1000179711.mp4	1000179711	75	4	72	78
15	16	1000179711.mp4	1000179711	78	5	75	81
16	17	1000179711.mp4	1000179711	100	6	93	108
17	18	1000179711.mp4	1000179711	104	7	99	108
18	19	1000179711.mp4	1000179711	122	10	117	126
19	20	1000179711.mp4	1000179711	122	11	117	126

==========

```
TABLA DE EVALUACIÓN MANUAL PREPARADA
======================================================================
Eventos: 89
```

22.2 Visor y herramienta de anotación manual de eventos

In [120]:

```
# VISOR Y ANOTACIÓN MANUAL DE EVENTOS


import os
import cv2
from PIL import Image
from IPython.display import display

# Archivo donde se irá guardando la evaluación manual
MANUAL_EVAL_PATH = os.path.join(
    FINAL_RESULTS_DIR,
    "evaluacion_manual_eventos.csv"
)


def show_event(event_id, offset=5):
    """
    Muestra tres frames del evento:
    - unos frames antes
    - frame central
    - unos frames después

    No muestra la predicción del sistema para evitar sesgo.
    """

    rows = df_manual_eval[
        df_manual_eval["event_id"] == event_id
    ]

    if len(rows) == 0:
        print("Event ID no encontrado.")
        return

    row = rows.iloc[0]

    video_name = row["video"]
    review_frame = int(row["review_frame"])

    # Buscar el vídeo
    if os.path.isfile(str(video_name)):
```

```
    video_path = str(video_name)
else:
    video_path = os.path.join(
        VIDEOS_DIR,
        str(video_name)
    )

if not os.path.isfile(video_path):
    print("No se encuentra el vídeo:")
    print(video_path)
    return

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("No se pudo abrir el vídeo.")
    return

total_frames = int(
    cap.get(cv2.CAP_PROP_FRAME_COUNT)
)

frames_to_show = [
    max(0, review_frame - offset),
    review_frame,
    min(
        total_frames - 1,
        review_frame + offset
    )
]

print("=" * 70)
print("EVALUACIÓN MANUAL")
print("=" * 70)
print("Evento:", event_id)
print("Vídeo:", video_name)
print("Frame central:", review_frame)
print(
    "Frames mostrados:",
    frames_to_show
)
print("=" * 70)

for frame_number in frames_to_show:

    cap.set(
        cv2.CAP_PROP_POS_FRAMES,
        frame_number
    )
```

```
        ok, frame = cap.read()

        if not ok:
            continue

        frame_rgb = cv2.cvtColor(
            frame,
            cv2.COLOR_BGR2RGB
        )

        image = Image.fromarray(
            frame_rgb
        )

        print(
            f"\nFrame {frame_number}"
        )

        display(image)

    cap.release()


def annotate_event(
    event_id,
    state,
    note=""
):
    """
    Guarda la anotación manual del estado del semáforo.
    Estados permitidos:
    GREEN, RED, UNKNOWN
    """

    state = state.upper()

    valid_states = [
        "GREEN",
        "RED",
        "UNKNOWN"
    ]

    if state not in valid_states:
        print(
            "Estado no válido. Usa:",
            valid_states
        )
        return
```

```
    mask = (
        df_manual_eval["event_id"]
        == event_id
    )

    if mask.sum() == 0:
        print("Event ID no encontrado.")
        return

    df_manual_eval.loc[
        mask,
        "manual_signal_state"
    ] = state

    df_manual_eval.loc[
        mask,
        "manual_notes"
    ] = note

    # Guardar automáticamente
    df_manual_eval.to_csv(
        MANUAL_EVAL_PATH,
        index=False
    )

    reviewed = (
        df_manual_eval[
            "manual_signal_state"
        ] != ""
    ).sum()

    print(
        f"Evento {event_id} → {state}"
    )

    print(
        f"Progreso: {reviewed}/"
        f"{len(df_manual_eval)}"
    )

    print(
        "Evaluación guardada automáticamente."
    )

def next_unreviewed():
    pending = df_manual_eval[
        df_manual_eval["manual_signal_state"] == ""
    ]
```

```
    if len(pending) == 0:
        print("Evaluación manual completada.")
        return

    next_id = int(pending.iloc[0]["event_id"])

    print(
        f"Siguiente evento: {next_id} "
        f"| Pendientes: {len(pending)}"
    )

    show_event(next_id)

print("Visor preparado.")
print("Eventos:", len(df_manual_eval))
print("Archivo de evaluación:")
print(MANUAL_EVAL_PATH)
```

```
Visor preparado.
Eventos: 89
Archivo de evaluación:
C:\Users\juank\Documents\CARLOS\TFM\resultados_finales\evaluacion_manu
al_eventos.csv
```

22.3 Registro del ground truth manual de los eventos

In [121]:

```
annotate_event(1, "GREEN")
annotate_event(2, "GREEN")
annotate_event(3, "GREEN")
annotate_event(4, "RED")
annotate_event(5, "RED")
annotate_event(6, "RED")
annotate_event(7, "RED")
annotate_event(8, "GREEN")
annotate_event(9, "GREEN")
annotate_event(10, "GREEN")
annotate_event(11, "GREEN")
annotate_event(12, "GREEN")
annotate_event(13, "GREEN")
annotate_event(14, "UNKNOWN")
annotate_event(15, "UNKNOWN")
annotate_event(16, "UNKNOWN")
annotate_event(17, "UNKNOWN")
annotate_event(18, "UNKNOWN")
annotate_event(19, "UNKNOWN")
annotate_event(20, "UNKNOWN")
```

```
annotate_event(21, "UNKNOWN")
annotate_event(22, "UNKNOWN")
annotate_event(23, "UNKNOWN")
annotate_event(24, "GREEN")
annotate_event(25, "GREEN")
annotate_event(26, "UNKNOWN")
annotate_event(27, "UNKNOWN")
annotate_event(28, "UNKNOWN")
annotate_event(29, "UNKNOWN")
annotate_event(30, "GREEN")
annotate_event(31, "GREEN")
annotate_event(32, "GREEN")
annotate_event(33, "GREEN")
annotate_event(34, "GREEN")
annotate_event(35, "GREEN")
annotate_event(36, "GREEN")
annotate_event(37, "GREEN")
annotate_event(38, "GREEN")
annotate_event(39, "GREEN")
annotate_event(40, "GREEN")
annotate_event(41, "GREEN")
annotate_event(42, "GREEN")
annotate_event(43, "GREEN")
annotate_event(44, "GREEN")
annotate_event(45, "GREEN")
annotate_event(46, "GREEN")
annotate_event(47, "GREEN")
annotate_event(48, "GREEN")
annotate_event(49, "GREEN")
annotate_event(50, "GREEN")
annotate_event(51, "GREEN")
annotate_event(52, "GREEN")
annotate_event(53, "GREEN")
annotate_event(54, "GREEN")
annotate_event(55, "GREEN")
annotate_event(56, "GREEN")
annotate_event(57, "GREEN")
annotate_event(58, "GREEN")
annotate_event(59, "GREEN")
annotate_event(60, "RED")
annotate_event(61, "RED")
annotate_event(62, "RED")
annotate_event(63, "RED")
annotate_event(64, "RED")
annotate_event(65, "RED")
annotate_event(66, "GREEN")
annotate_event(67, "GREEN")
annotate_event(68, "GREEN")
annotate_event(69, "RED")
```

```
annotate_event(70, "RED")
annotate_event(71, "RED")
annotate_event(72, "RED")
annotate_event(73, "RED")
annotate_event(74, "RED")
annotate_event(75, "RED")
annotate_event(76, "RED")
annotate_event(77, "RED")
annotate_event(78, "RED")
annotate_event(79, "RED")
annotate_event(80, "RED")
annotate_event(81, "RED")
annotate_event(82, "RED")
annotate_event(83, "RED")
annotate_event(84, "RED")
annotate_event(85, "RED")
annotate_event(86, "RED")
annotate_event(87, "RED")
annotate_event(88, "RED")
annotate_event(89, "RED")
Evento 1 → GREEN
Progreso: 1/89
Evaluación guardada automáticamente.
Evento 2 → GREEN
Progreso: 2/89
Evaluación guardada automáticamente.
Evento 3 → GREEN
Progreso: 3/89
Evaluación guardada automáticamente.
Evento 4 → RED
Progreso: 4/89
Evaluación guardada automáticamente.
Evento 5 → RED
Progreso: 5/89
Evaluación guardada automáticamente.
Evento 6 → RED
Progreso: 6/89
Evaluación guardada automáticamente.
Evento 7 → RED
Progreso: 7/89
Evaluación guardada automáticamente.
Evento 8 → GREEN
Progreso: 8/89
Evaluación guardada automáticamente.
Evento 9 → GREEN
Progreso: 9/89
Evaluación guardada automáticamente.
Evento 10 → GREEN
Progreso: 10/89
```

Evaluación guardada automáticamente.
Evento 11 → GREEN
Progreso: 11/89
Evaluación guardada automáticamente.
Evento 12 → GREEN
Progreso: 12/89
Evaluación guardada automáticamente.
Evento 13 → GREEN
Progreso: 13/89
Evaluación guardada automáticamente.
Evento 14 → UNKNOWN
Progreso: 14/89
Evaluación guardada automáticamente.
Evento 15 → UNKNOWN
Progreso: 15/89
Evaluación guardada automáticamente.
Evento 16 → UNKNOWN
Progreso: 16/89
Evaluación guardada automáticamente.
Evento 17 → UNKNOWN
Progreso: 17/89
Evaluación guardada automáticamente.
Evento 18 → UNKNOWN
Progreso: 18/89
Evaluación guardada automáticamente.
Evento 19 → UNKNOWN
Progreso: 19/89
Evaluación guardada automáticamente.
Evento 20 → UNKNOWN
Progreso: 20/89
Evaluación guardada automáticamente.
Evento 21 → UNKNOWN
Progreso: 21/89
Evaluación guardada automáticamente.
Evento 22 → UNKNOWN
Progreso: 22/89
Evaluación guardada automáticamente.
Evento 23 → UNKNOWN
Progreso: 23/89
Evaluación guardada automáticamente.
Evento 24 → GREEN
Progreso: 24/89
Evaluación guardada automáticamente.
Evento 25 → GREEN
Progreso: 25/89
Evaluación guardada automáticamente.
Evento 26 → UNKNOWN
Progreso: 26/89
Evaluación guardada automáticamente.

Evento 27 → UNKNOWN
Progreso: 27/89
Evaluación guardada automáticamente.
Evento 28 → UNKNOWN
Progreso: 28/89
Evaluación guardada automáticamente.
Evento 29 → UNKNOWN
Progreso: 29/89
Evaluación guardada automáticamente.
Evento 30 → GREEN
Progreso: 30/89
Evaluación guardada automáticamente.
Evento 31 → GREEN
Progreso: 31/89
Evaluación guardada automáticamente.
Evento 32 → GREEN
Progreso: 32/89
Evaluación guardada automáticamente.
Evento 33 → GREEN
Progreso: 33/89
Evaluación guardada automáticamente.
Evento 34 → GREEN
Progreso: 34/89
Evaluación guardada automáticamente.
Evento 35 → GREEN
Progreso: 35/89
Evaluación guardada automáticamente.
Evento 36 → GREEN
Progreso: 36/89
Evaluación guardada automáticamente.
Evento 37 → GREEN
Progreso: 37/89
Evaluación guardada automáticamente.
Evento 38 → GREEN
Progreso: 38/89
Evaluación guardada automáticamente.
Evento 39 → GREEN
Progreso: 39/89
Evaluación guardada automáticamente.
Evento 40 → GREEN
Progreso: 40/89
Evaluación guardada automáticamente.
Evento 41 → GREEN
Progreso: 41/89
Evaluación guardada automáticamente.
Evento 42 → GREEN
Progreso: 42/89
Evaluación guardada automáticamente.
Evento 43 → GREEN

Progreso: 43/89
Evaluación guardada automáticamente.
Evento 44 → GREEN
Progreso: 44/89
Evaluación guardada automáticamente.
Evento 45 → GREEN
Progreso: 45/89
Evaluación guardada automáticamente.
Evento 46 → GREEN
Progreso: 46/89
Evaluación guardada automáticamente.
Evento 47 → GREEN
Progreso: 47/89
Evaluación guardada automáticamente.
Evento 48 → GREEN
Progreso: 48/89
Evaluación guardada automáticamente.
Evento 49 → GREEN
Progreso: 49/89
Evaluación guardada automáticamente.
Evento 50 → GREEN
Progreso: 50/89
Evaluación guardada automáticamente.
Evento 51 → GREEN
Progreso: 51/89
Evaluación guardada automáticamente.
Evento 52 → GREEN
Progreso: 52/89
Evaluación guardada automáticamente.
Evento 53 → GREEN
Progreso: 53/89
Evaluación guardada automáticamente.
Evento 54 → GREEN
Progreso: 54/89
Evaluación guardada automáticamente.
Evento 55 → GREEN
Progreso: 55/89
Evaluación guardada automáticamente.
Evento 56 → GREEN
Progreso: 56/89
Evaluación guardada automáticamente.
Evento 57 → GREEN
Progreso: 57/89
Evaluación guardada automáticamente.
Evento 58 → GREEN
Progreso: 58/89
Evaluación guardada automáticamente.
Evento 59 → GREEN
Progreso: 59/89

Evaluación guardada automáticamente.
Evento 60 → RED
Progreso: 60/89
Evaluación guardada automáticamente.
Evento 61 → RED
Progreso: 61/89
Evaluación guardada automáticamente.
Evento 62 → RED
Progreso: 62/89
Evaluación guardada automáticamente.
Evento 63 → RED
Progreso: 63/89
Evaluación guardada automáticamente.
Evento 64 → RED
Progreso: 64/89
Evaluación guardada automáticamente.
Evento 65 → RED
Progreso: 65/89
Evaluación guardada automáticamente.
Evento 66 → GREEN
Progreso: 66/89
Evaluación guardada automáticamente.
Evento 67 → GREEN
Progreso: 67/89
Evaluación guardada automáticamente.
Evento 68 → GREEN
Progreso: 68/89
Evaluación guardada automáticamente.
Evento 69 → RED
Progreso: 69/89
Evaluación guardada automáticamente.
Evento 70 → RED
Progreso: 70/89
Evaluación guardada automáticamente.
Evento 71 → RED
Progreso: 71/89
Evaluación guardada automáticamente.
Evento 72 → RED
Progreso: 72/89
Evaluación guardada automáticamente.
Evento 73 → RED
Progreso: 73/89
Evaluación guardada automáticamente.
Evento 74 → RED
Progreso: 74/89
Evaluación guardada automáticamente.
Evento 75 → RED
Progreso: 75/89
Evaluación guardada automáticamente.

```
Evento 76 → RED
Progreso: 76/89
Evaluación guardada automáticamente.
Evento 77 → RED
Progreso: 77/89
Evaluación guardada automáticamente.
Evento 78 → RED
Progreso: 78/89
Evaluación guardada automáticamente.
Evento 79 → RED
Progreso: 79/89
Evaluación guardada automáticamente.
Evento 80 → RED
Progreso: 80/89
Evaluación guardada automáticamente.
Evento 81 → RED
Progreso: 81/89
Evaluación guardada automáticamente.
Evento 82 → RED
Progreso: 82/89
Evaluación guardada automáticamente.
Evento 83 → RED
Progreso: 83/89
Evaluación guardada automáticamente.
Evento 84 → RED
Progreso: 84/89
Evaluación guardada automáticamente.
Evento 85 → RED
Progreso: 85/89
Evaluación guardada automáticamente.
Evento 86 → RED
Progreso: 86/89
Evaluación guardada automáticamente.
Evento 87 → RED
Progreso: 87/89
Evaluación guardada automáticamente.
Evento 88 → RED
Progreso: 88/89
Evaluación guardada automáticamente.
Evento 89 → RED
Progreso: 89/89
Evaluación guardada automáticamente.
```

22.4 Evaluación final del estado del semáforo peatonal

In [122]:

```
# EVALUACIÓN FINAL DEL ESTADO DEL SEMÁFORO PEATONAL
```

```
import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Copia de seguridad
df_eval_final = df_manual_eval.copy()

# Normalizar etiquetas
df_eval_final["manual_signal_state"] = (
    df_eval_final["manual_signal_state"]
    .astype(str)
    .str.strip()
    .str.upper()
)

df_eval_final["signal_state"] = (
    df_eval_final["signal_state"]
    .astype(str)
    .str.strip()
    .str.upper()
)

# ------------------------------------------------------------
# 1. COMPROBAR QUE LOS 89 ESTÁN REVISADOS
# ------------------------------------------------------------

reviewed = (
    df_eval_final["manual_signal_state"] != ""
).sum()

print("=" * 70)
print("EVALUACIÓN MANUAL COMPLETADA")
print("=" * 70)

print("Eventos revisados:", reviewed)
print("Total de eventos:", len(df_eval_final))

print("\nGround truth manual:")
print(
    df_eval_final[
        "manual_signal_state"
    ].value_counts()
)

# ------------------------------------------------------------
```

```
# 2. SEPARAR CASOS INTERPRETABLES
# ------------------------------------------------------------

df_evaluable = df_eval_final[
    df_eval_final["manual_signal_state"].isin(
        ["GREEN", "RED"]
    )
].copy()

df_unknown = df_eval_final[
    df_eval_final["manual_signal_state"] == "UNKNOWN"
].copy()

print("\n" + "=" * 70)
print("COBERTURA DE LA EVALUACIÓN")
print("=" * 70)

print(
    "Casos GREEN/RED evaluables:",
    len(df_evaluable)
)

print(
    "Casos UNKNOWN:",
    len(df_unknown)
)

print(
    "Porcentaje evaluable:",
    round(
        100 * len(df_evaluable) /
        len(df_eval_final),
        2
    ),
    "%"
)

print(
    "Porcentaje UNKNOWN:",
    round(
        100 * len(df_unknown) /
        len(df_eval_final),
        2
    ),
    "%"
)

# ------------------------------------------------------------
# 3. MÉTRICAS GREEN / RED
```

```
# ------------------------------------------------------------

y_true = df_evaluable[
    "manual_signal_state"
]

y_pred = df_evaluable[
    "signal_state"
]

accuracy = accuracy_score(
    y_true,
    y_pred
)

cm = confusion_matrix(
    y_true,
    y_pred,
    labels=["GREEN", "RED"]
)

print("\n" + "=" * 70)
print("RESULTADOS DE CLASIFICACIÓN GREEN / RED")
print("=" * 70)

print(
    "Accuracy:",
    round(accuracy, 4)
)

print("\nMatriz de confusión:")
print(
    pd.DataFrame(
        cm,
        index=[
            "REAL_GREEN",
            "REAL_RED"
        ],
        columns=[
            "PRED_GREEN",
            "PRED_RED"
        ]
    )
)

print("\nInforme de clasificación:")

print(
    classification_report(
```

```
        y_true,
        y_pred,
        labels=["GREEN", "RED"],
        target_names=[
            "GREEN",
            "RED"
        ],
        digits=3,
        zero_division=0
    )
)
================================================================================
EVALUACIÓN MANUAL COMPLETADA
================================================================================
Eventos revisados: 89
Total de eventos: 89

Ground truth manual:
manual_signal_state
GREEN      44
RED        31
UNKNOWN    14
Name: count, dtype: int64

================================================================================
COBERTURA DE LA EVALUACIÓN
================================================================================
Casos GREEN/RED evaluables: 75
Casos UNKNOWN: 14
Porcentaje evaluable: 84.27 %
Porcentaje UNKNOWN: 15.73 %

================================================================================
RESULTADOS DE CLASIFICACIÓN GREEN / RED
================================================================================
Accuracy: 0.5733

Matriz de confusión:
            PRED_GREEN  PRED_RED
REAL_GREEN          28        16
REAL_RED            16        15

Informe de clasificación:
              precision    recall  f1-score   support

       GREEN      0.636     0.636     0.636        44
         RED      0.484     0.484     0.484        31

    accuracy                          0.573        75
```

```
   macro avg      0.560     0.560     0.560        75
weighted avg      0.573     0.573     0.573        75
```

22.5 Guardado de la evaluación manual final

In [123]:

```
# GUARDAR EVALUACIÓN MANUAL FINAL

final_manual_path = os.path.join(
    FINAL_RESULTS_DIR,
    "evaluacion_manual_final.csv"
)

df_eval_final.to_csv(
    final_manual_path,
    index=False
)

print("=" * 70)
print("EVALUACIÓN MANUAL FINAL GUARDADA")
print("=" * 70)
print("Eventos totales:", len(df_eval_final))
print("Casos evaluables:", len(df_evaluable))
print("Casos UNKNOWN:", len(df_unknown))
print("Accuracy GREEN/RED:", round(accuracy, 4))
print("\nArchivo:")
print(final_manual_path)
```

```
======================================================================
EVALUACIÓN MANUAL FINAL GUARDADA
======================================================================
Eventos totales: 89
Casos evaluables: 75
Casos UNKNOWN: 14
Accuracy GREEN/RED: 0.5733

Archivo:
C:\Users\juank\Documents\CARLOS\TFM\resultados_finales\evaluacion_manu
al_final.csv
```

22.6 Matriz de confusión de la clasificación del estado del semáforo peatonal

In [124]:

```
# MATRIZ DE CONFUSIÓN - SEMÁFORO PEATONAL


import matplotlib.pyplot as plt
```

```
from sklearn.metrics import ConfusionMatrixDisplay

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["GREEN", "RED"]
)

fig, ax = plt.subplots(figsize=(6, 5))

disp.plot(
    ax=ax,
    cmap="Blues",
    values_format="d",
    colorbar=False
)

plt.title(
    "Matriz de confusión del estado del semáforo peatonal"
)

plt.xlabel("Estado predicho")
plt.ylabel("Estado real")

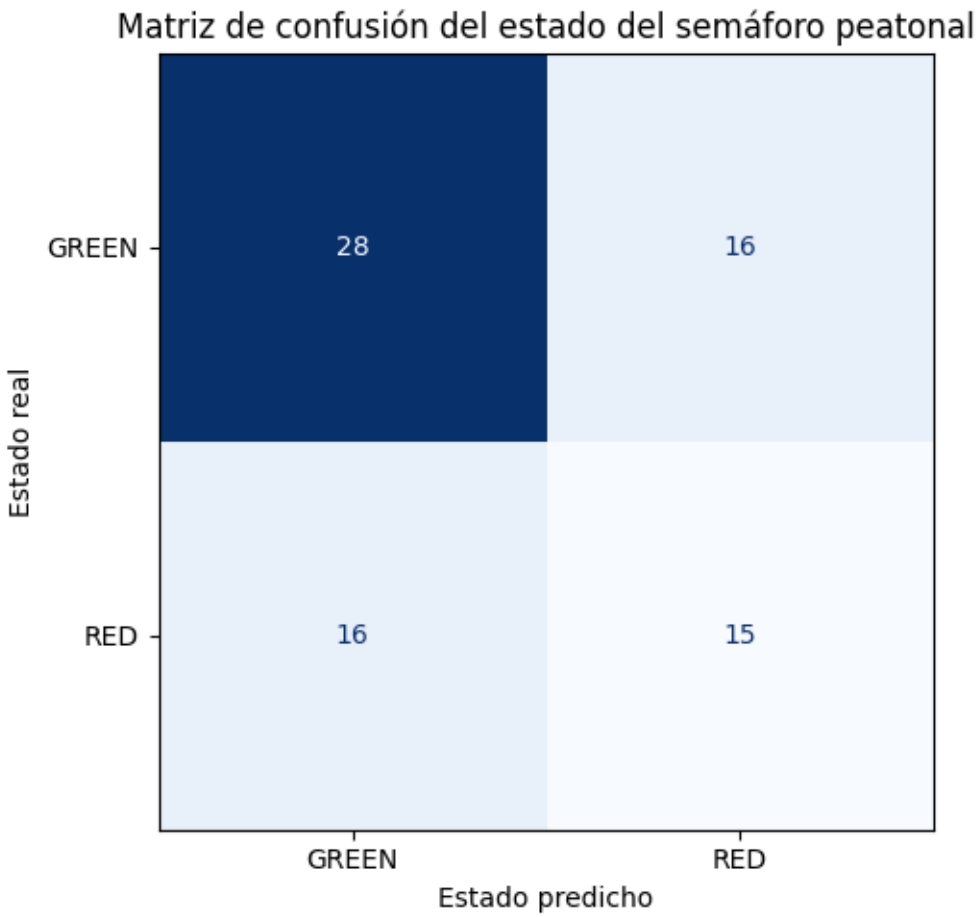
plt.tight_layout()

# Guardar figura
confusion_path = os.path.join(
    FINAL_RESULTS_DIR,
    "matriz_confusion_semaforo_peatonal.png"
)

plt.savefig(
    confusion_path,
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print("Matriz guardada en:")
print(confusion_path)
```




```
Matriz guardada en:
C:\Users\juank\Documents\CARLOS\TFM\resultados_finales\matriz_confusio
n_semaforo_peatonal.png
```

23. Demostración integrada del prototipo de asistencia al cruce

En este apartado se presenta una demostración del funcionamiento integrado del prototipo sobre tres secuencias representativas del conjunto experimental.

Se seleccionan tres escenarios complementarios:

- **Escenario 1 — RED / WAIT:** detección de señal peatonal en rojo y recomendación de espera.
- **Escenario 2 — GREEN / CAUTION:** señal verde, pero existencia de contexto de tráfico que aconseja precaución.
- **Escenario 3 — GREEN / CROSS_WITH_CAUTION:** señal verde y condiciones más favorables para realizar el cruce con precaución.

El objetivo es mostrar que la decisión final no depende exclusivamente del estado del semáforo, sino de la integración de información visual, temporal y contextual.

In [125]:

```
# 23.1 SELECCIÓN DE LOS ESCENARIOS DEMOSTRATIVOS


DEMO_VIDEOS = {
    "RED_WAIT": "20260611_184808.mp4",
    "GREEN_CAUTION": "20260611_184739.mp4",
    "GREEN_CROSS_WITH_CAUTION": "20260611_185443~2.mp4"
}

print("=" * 70)
print("VÍDEOS SELECCIONADOS PARA LA DEMOSTRACIÓN")
print("=" * 70)

for scenario, video in DEMO_VIDEOS.items():
    print(f"{scenario:28s} -> {video}")
```

```
======================================================================
VÍDEOS SELECCIONADOS PARA LA DEMOSTRACIÓN
======================================================================
RED_WAIT                     -> 20260611_184808.mp4
GREEN_CAUTION                -> 20260611_184739.mp4
GREEN_CROSS_WITH_CAUTION     -> 20260611_185443~2.mp4
```

23.2 DEMOSTRACIÓN INTEGRADA DEL PROTOTIPO

In [126]:

```
# ============================================================
# 23.2 DEMOSTRACIÓN INTEGRADA DEL PROTOTIPO
# ============================================================

import os
import pandas as pd
import matplotlib.pyplot as plt


# ------------------------------------------------------------
# Recuperación de los resultados finales
# ------------------------------------------------------------

if "DEMO_VIDEOS" not in globals():
    raise RuntimeError(
        "No existe DEMO_VIDEOS. Ejecuta primero el apartado 23.1."
    )


# Si el DataFrame final ya está en memoria, se reutiliza.
if "df_final_decisions" not in globals():

    candidate_paths = []
```

```
    # Ruta definida previamente en el notebook
    if "final_decisions_path" in globals():
        candidate_paths.append(final_decisions_path)

    # Directorio final definido previamente
    if "FINAL_RESULTS_DIR" in globals():
        candidate_paths.append(
            os.path.join(
                FINAL_RESULTS_DIR,
                "decisiones_finales_riesgo.csv"
            )
        )

    # Ruta utilizada por el pipeline final
    candidate_paths.append(

r"C:\Users\juank\Documents\CARLOS\TFM\resultados_finales\decisiones_fi
nales_riesgo.csv"
    )

    final_path = next(
        (path for path in candidate_paths if os.path.exists(path)),
        None
    )

    if final_path is None:
        raise FileNotFoundError(
            "No se ha encontrado decisiones_finales_riesgo.csv. "
            "Comprueba la carpeta resultados_finales."
        )

    df_final_decisions = pd.read_csv(final_path)


print("✓ Resultados finales cargados correctamente.")
print(f"✓ Eventos finales disponibles: {len(df_final_decisions)}")


# ------------------------------------------------------------
# Recomendación representativa buscada en cada escenario
# ------------------------------------------------------------

DEMO_TARGETS = {
    "RED_WAIT": "WAIT",
    "GREEN_CAUTION": "CAUTION",
    "GREEN_CROSS_WITH_CAUTION": "CROSS_WITH_CAUTION"
}
```

```
# ------------------------------------------------------------
# Selección automática del evento representativo
# ------------------------------------------------------------

demo_rows = []

for scenario, video in DEMO_VIDEOS.items():

    target_recommendation = DEMO_TARGETS[scenario]

    candidates = df_final_decisions[
        (df_final_decisions["video"] == video) &
        (df_final_decisions["recommendation"] ==
target_recommendation)
    ].copy()

    if candidates.empty:
        print(
            f"⚠ No se encontró un evento compatible para "
            f"{scenario} -> {video}"
        )
        continue

    # Seleccionar el evento con mayor consistencia de señal.
    # En caso de empate, priorizar mayor pedestrian_score.
    candidates = candidates.sort_values(
        ["signal_consistency", "pedestrian_score"],
        ascending=False
    )

    row = candidates.iloc[0]

    demo_rows.append({
        "scenario": scenario,
        "video": video,
        "signal_state": row["signal_state"],
        "signal_consistency": row["signal_consistency"],
        "risk_level": row["risk_level"],
        "recommendation": row["recommendation"],
        "approaching_vehicles": row["approaching_vehicles"],
        "moving_vehicles": row["moving_vehicles"],
        "stable_vehicles": row["stable_vehicles"],
        "first_frame": row["signal_first_frame"],
        "last_frame": row["signal_last_frame"]
    })


df_demo = pd.DataFrame(demo_rows)
```

```
# ------------------------------------------------------------
# Resumen tabular
# ------------------------------------------------------------

print("=" * 78)
print("DEMOSTRACIÓN INTEGRADA DEL PROTOTIPO DE ASISTENCIA AL CRUCE")
print("=" * 78)

display(
    df_demo[
        [
            "scenario",
            "video",
            "signal_state",
            "risk_level",
            "recommendation",
            "approaching_vehicles",
            "moving_vehicles",
            "stable_vehicles"
        ]
    ]
)


# ------------------------------------------------------------
# Visualización para la demostración
# ------------------------------------------------------------

fig, axes = plt.subplots(
    1,
    len(df_demo),
    figsize=(16, 5)
)

if len(df_demo) == 1:
    axes = [axes]


signal_colors = {
    "RED": "#d62728",
    "GREEN": "#2ca02c",
    "UNKNOWN": "#7f7f7f"
}

risk_colors = {
    "LOW": "#2ca02c",
    "MEDIUM": "#ffbf00",
    "HIGH": "#d62728"
```

```
}


for ax, (_, row) in zip(axes, df_demo.iterrows()):

    ax.axis("off")

    signal = row["signal_state"]
    risk = row["risk_level"]
    recommendation = row["recommendation"]

    # Semáforo
    ax.scatter(
        0.5,
        0.78,
        s=3000,
        color=signal_colors.get(signal, "#7f7f7f"),
        edgecolor="black"
    )

    ax.text(
        0.5,
        0.56,
        f"SEÑAL: {signal}",
        ha="center",
        va="center",
        fontsize=14,
        fontweight="bold"
    )

    # Riesgo
    ax.text(
        0.5,
        0.40,
        f"RIESGO: {risk}",
        ha="center",
        va="center",
        fontsize=14,
        fontweight="bold",
        color=risk_colors.get(risk, "black")
    )

    # Recomendación
    ax.text(
        0.5,
        0.23,
        recommendation.replace("_", " "),
        ha="center",
        va="center",
```

```
        fontsize=15,
        fontweight="bold",
        bbox=dict(
            boxstyle="round,pad=0.5",
            facecolor="white",
            edgecolor="black"
        )
    )

    ax.set_title(
        row["scenario"].replace("_", " "),
        fontsize=13,
        fontweight="bold",
        pad=15
    )

    ax.text(
        0.5,
        0.05,
        row["video"],
        ha="center",
        va="center",
        fontsize=9
    )

    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)

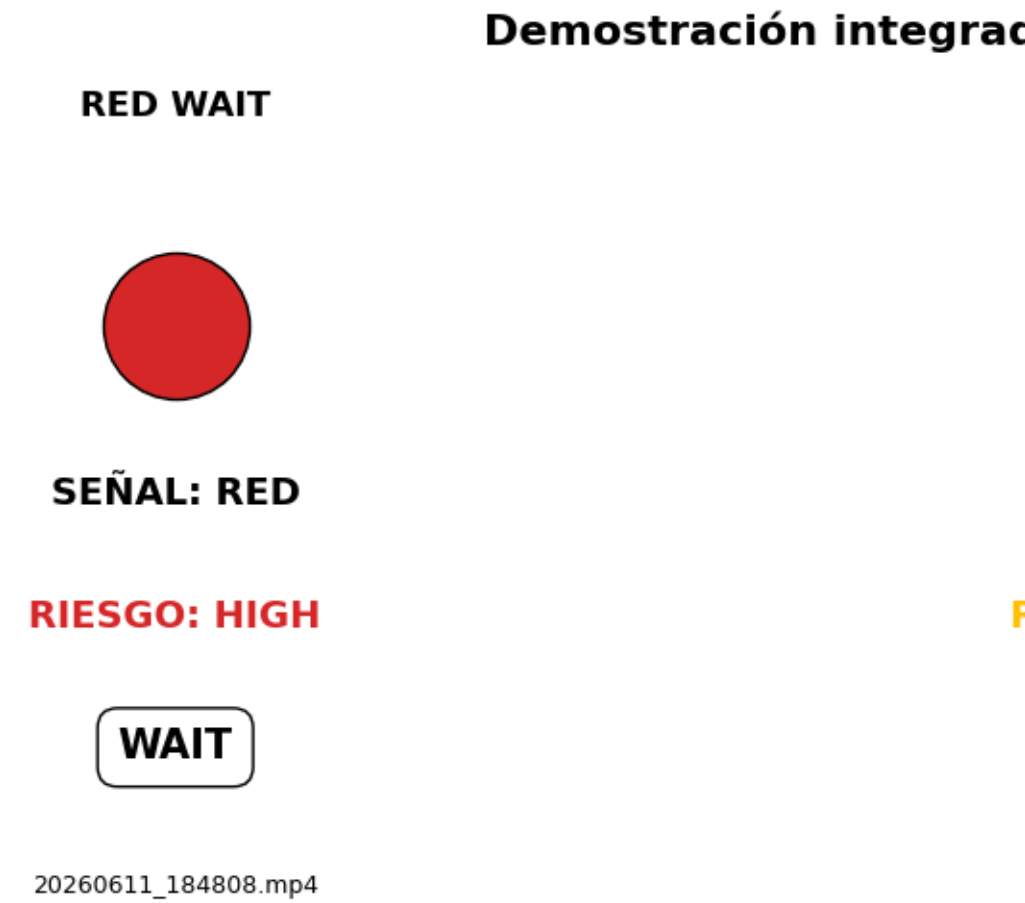

plt.suptitle(
    "Demostración integrada del sistema de asistencia al cruce",
    fontsize=16,
    fontweight="bold"
)

plt.tight_layout()
plt.show()
```

```
✓ Resultados finales cargados correctamente.
✓ Eventos finales disponibles: 89
================================================================================
DEMOSTRACIÓN INTEGRADA DEL PROTOTIPO DE ASISTENCIA AL CRUCE
================================================================================
```

	scenario	video	signal_state	risk_level	re
0	RED_WAIT	20260611_184808.mp4	RED	HIGH	WAIT
1	GREEN_CAUTION	20260611_184739.mp4	GREEN	MEDIUM	CAUTION

	scenario	video	signal_state	risk_level	re
2	GREEN_CROSS_WITH_CAUTION	20260611_185443~2.mp4	GREEN	LOW	CROSS_WI



23.3 Interpretación de los escenarios demostrativos

Los tres escenarios seleccionados permiten observar el comportamiento integrado del prototipo ante situaciones de distinta criticidad.

- **RED / WAIT:** la presencia de señal peatonal roja conduce a un nivel de riesgo alto y a una recomendación de espera, independientemente de la dinámica observada.
- **GREEN / CAUTION:** aunque la señal peatonal es verde, la presencia de vehículos en movimiento incrementa el riesgo hasta un nivel medio, por lo que el sistema recomienda proceder con precaución.
- **GREEN / CROSS WITH CAUTION:** cuando la señal es verde y no se detectan vehículos aproximándose ni en movimiento, el riesgo disminuye a un nivel bajo y el sistema permite el cruce manteniendo una recomendación conservadora de precaución.

La comparación evidencia que la decisión final no depende exclusivamente del estado del semáforo peatonal, sino de la integración de la información visual, temporal y dinámica del entorno.

23.4 Demostración visual del prototipo integrado sobre vídeos reales

Como demostración final del sistema, se generan tres vídeos representativos en los que se visualizan conjuntamente las distintas etapas del prototipo: detección de objetos mediante YOLO, seguimiento temporal, análisis dinámico de los vehículos, interpretación del semáforo peatonal, estimación del nivel de riesgo y recomendación de asistencia al cruce.

Los tres escenarios corresponden a situaciones de espera ante señal roja, cruce con precaución ante señal verde y presencia de tráfico, y cruce con precaución ante señal verde y condiciones dinámicas favorables.

La visualización utiliza los resultados obtenidos previamente por el pipeline experimental. Por tanto, representa de forma temporal e integrada el funcionamiento del prototipo sobre secuencias reales, aunque no constituye todavía una implementación online en tiempo real.

In [127]:

```
# ============================================================
# 23.4 DEMOSTRACIÓN VISUAL INTEGRADA SOBRE TRES VÍDEOS REALES
#
# PROTOTIPO BASE:
# YOLO11 COCO + TRACKING + CLIP + GATE V2
# + DINÁMICA VEHICULAR + FUSIÓN DE RIESGO
#
# NOTA:
# En 23.4 NO se utilizan los modelos YOLO especializados V1/V2.
# Esos se incorporan en 23.5.
# ============================================================

import os
import cv2
import math
import numpy as np
import pandas as pd

from pathlib import Path
from ultralytics import YOLO
from IPython.display import display, Video


# ============================================================
# 1. RUTAS
# ============================================================

TFM_DIR = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
```

```
)

VIDEOS_DIR = (
    TFM_DIR
    / "VÍDEOS"
)

PROTOTYPE_BASE_DIR = (
    TFM_DIR
    / "PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE"
)

PROTOTYPE_BASE_DIR.mkdir(
    parents=True,
    exist_ok=True
)


# ============================================================
# 2. COMPROBACIÓN DEL MÓDULO CLIP + GATE V2
# ============================================================

if "analyze_final_pedestrian_signal" not in globals():

    raise RuntimeError(
        "No existe analyze_final_pedestrian_signal. "
        "Ejecuta previamente el módulo de interpretación "
        "del semáforo peatonal con CLIP + Gate V2."
    )


print(
    "✓ Módulo CLIP + Gate V2 disponible."
)


# ============================================================
# 3. MODELOS YOLO COCO
# ============================================================

# Modelo para objetos + tracking
yolo_objects_coco = YOLO(
    "yolo11n.pt"
)


# Detector COCO que usa el módulo del semáforo
if "yolo_signal_detector" in globals():
```

```
    yolo_signal_coco = (
        yolo_signal_detector
    )

else:

    yolo_signal_coco = YOLO(
        "yolo11n.pt"
    )


print(
    "✓ YOLO11 COCO preparado."
)


# ============================================================
# 4. CLASES COCO DE INTERÉS
# ============================================================

COCO_CLASSES_E2E = [

    0,   # person

    1,   # bicycle

    2,   # car

    3,   # motorcycle

    5,   # bus

    7,   # truck

    9    # traffic light
]


VEHICLE_CLASSES = {

    "bicycle",

    "car",

    "motorcycle",

    "bus",

    "truck"
```

```
}


# ============================================================
# 5. YOLO + TRACKING
# ============================================================

def process_objects_and_tracking_e2e(
    video_path
):

    video_path = Path(
        video_path
    )

    if not video_path.is_file():

        raise FileNotFoundError(
            f"No existe el vídeo:\n{video_path}"
        )


    cap = cv2.VideoCapture(
        str(
            video_path
        )
    )

    fps = cap.get(
        cv2.CAP_PROP_FPS
    )

    width = int(
        cap.get(
            cv2.CAP_PROP_FRAME_WIDTH
        )
    )

    height = int(
        cap.get(
            cv2.CAP_PROP_FRAME_HEIGHT
        )
    )

    total_frames = int(
        cap.get(
            cv2.CAP_PROP_FRAME_COUNT
        )
    )
```

```
cap.release()


print()
print(
    "[1/6] YOLO11 COCO + tracking..."
)

print(
    f"Vídeo: {video_path.name}"
)

print(
    f"FPS: {fps:.2f}"
)

print(
    f"Frames: {total_frames}"
)


rows = []


results = (
    yolo_objects_coco.track(

        source=str(
            video_path
        ),

        stream=True,

        persist=True,

        tracker="bytetrack.yaml",

        classes=COCO_CLASSES_E2E,

        conf=0.20,

        verbose=False
    )
)


for frame_idx, result in enumerate(
    results
```

```
):

    boxes = (
        result.boxes
    )

    if boxes is None:
        continue


    frame_h, frame_w = (
        result.orig_shape
    )


    for i in range(
        len(boxes)
    ):

        cls_id = int(
            boxes.cls[
                i
            ].item()
        )

        class_name = str(
            yolo_objects_coco.names[
                cls_id
            ]
        )


        confidence = float(
            boxes.conf[
                i
            ].item()
        )


        x1, y1, x2, y2 = [

            float(v)

            for v in boxes.xyxy[
                i
            ].tolist()
        ]
```

```
if (
    boxes.id is not None
):

    track_id = int(
        boxes.id[
            i
        ].item()
    )

else:

    track_id = np.nan


center_x = (
    x1 + x2
) / 2.0

center_y = (
    y1 + y2
) / 2.0


bbox_area = max(
    0.0,
    x2 - x1
) * max(
    0.0,
    y2 - y1
)


frame_area = max(
    1.0,
    frame_w * frame_h
)


rows.append({

    "video":
        video_path.name,

    "frame":
        frame_idx,

    "time_s":
        (
```

```
                    frame_idx / fps
                    if fps > 0
                    else 0
                ),

            "track_id":
                track_id,

            "class_id":
                cls_id,

            "class_name":
                class_name,

            "confidence":
                confidence,

            "x1":
                x1,

            "y1":
                y1,

            "x2":
                x2,

            "y2":
                y2,

            "center_x_norm":
                center_x / frame_w,

            "center_y_norm":
                center_y / frame_h,

            "bbox_area_norm":
                bbox_area / frame_area,

            "frame_width":
                frame_w,

            "frame_height":
                frame_h
        })


df_tracking = pd.DataFrame(
    rows
)
```

```
    print(
        "✓ Detecciones/tracks:",
        len(
            df_tracking
        )
    )


    if (
        not df_tracking.empty
        and
        "track_id"
        in df_tracking.columns
    ):

        n_ids = (
            df_tracking[
                "track_id"
            ]
            .dropna()
            .nunique()
        )

    else:

        n_ids = 0


    print(
        "✓ IDs distintos:",
        n_ids
    )


    return (
        df_tracking,
        fps,
        total_frames,
        width,
        height
    )


# ============================================================
# 6. DINÁMICA VEHICULAR
# ============================================================
```

```
def build_vehicle_dynamics_e2e(
    df_tracking
):

    print()
    print(
        "[2/6] Análisis dinámico de vehículos..."
    )


    if (
        df_tracking is None
        or
        df_tracking.empty
    ):

        return pd.DataFrame()


    vehicles = (
        df_tracking[
            df_tracking[
                "class_name"
            ]
            .astype(str)
            .str.lower()
            .isin(
                VEHICLE_CLASSES
            )
        ]
        .copy()
    )


    vehicles = (
        vehicles[
            vehicles[
                "track_id"
            ]
            .notna()
        ]
        .copy()
    )


    if vehicles.empty:

        print(
            "✓ No hay tracks de vehículos."
```

```
    )

    return pd.DataFrame()


vehicles = (
    vehicles.sort_values(
        [
            "track_id",
            "frame"
        ]
    )
)


rows = []


for track_id, group in (
    vehicles.groupby(
        "track_id"
    )
):

    group = (
        group.sort_values(
            "frame"
        )
    )


    observations = len(
        group
    )


    first_frame = int(
        group[
            "frame"
        ].iloc[0]
    )

    last_frame = int(
        group[
            "frame"
        ].iloc[-1]
    )
```

```
first_area = float(
    group[
        "bbox_area_norm"
    ].iloc[0]
)

last_area = float(
    group[
        "bbox_area_norm"
    ].iloc[-1]
)


dx = (
    group[
        "center_x_norm"
    ]
    .diff()
)

dy = (
    group[
        "center_y_norm"
    ]
    .diff()
)


dt = (
    group[
        "time_s"
    ]
    .diff()
    .replace(
        0,
        np.nan
    )
)


distance = np.sqrt(
    dx ** 2
    +
    dy ** 2
)


speed = (
    distance
```

```
    /
    dt
)


mean_speed = float(
    speed
    .replace(
        [
            np.inf,
            -np.inf
        ],
        np.nan
    )
    .median()
)


if not np.isfinite(
    mean_speed
):

    mean_speed = 0.0


area_growth = (
    (
        last_area
        + 1e-8
    )
    /
    (
        first_area
        + 1e-8
    )
)


valid_track = (
    observations >= 5
)


moving = (
    valid_track
    and
    mean_speed > 0.015
)
```

```
approaching = (
    moving
    and
    area_growth > 1.12
)


if not valid_track:

    dynamic_state = (
        "INSUFFICIENT"
    )

elif approaching:

    dynamic_state = (
        "APPROACHING"
    )

elif moving:

    dynamic_state = (
        "MOVING"
    )

else:

    dynamic_state = (
        "STABLE"
    )


rows.append({

    "track_id":
        int(
            track_id
        ),

    "class_name":
        str(
            group[
                "class_name"
            ].iloc[0]
        ),

    "observations":
        observations,
```

```
            "first_frame":
                first_frame,

            "last_frame":
                last_frame,

            "mean_speed":
                mean_speed,

            "first_area":
                first_area,

            "last_area":
                last_area,

            "area_growth":
                area_growth,

            "valid_track":
                valid_track,

            "moving":
                moving,

            "approaching":
                approaching,

            "dynamic_state":
                dynamic_state
        })


df_dynamic = pd.DataFrame(
    rows
)


print(
    "✓ Tracks vehiculares:",
    len(
        df_dynamic
    )
)


if not df_dynamic.empty:

    print(
```

```
            df_dynamic[
                "dynamic_state"
            ]
            .value_counts()
            .to_dict()
        )


    return df_dynamic


# ============================================================
# 7. ACCESO FLEXIBLE A COLUMNAS DE SEMÁFORO
# ============================================================

def _first_available(
    row,
    names,
    default=None
):

    for name in names:

        if (
            name in row.index
            and
            pd.notna(
                row[
                    name
                ]
            )
        ):

            return row[
                name
            ]

    return default


# ============================================================
# 8. FUSIÓN CONTEXTUAL Y DECISIÓN
# ============================================================

def fuse_crossing_information_e2e(
    df_signal,
    df_pedestrian,
    df_dynamic
):
```

```
print()
print(
    "[4/6] Fusión contextual y decisión..."
)


# Preferencia por tracks peatonales finales.
# Si no existen, usar resultados temporales.
if (
    df_pedestrian is not None
    and
    not df_pedestrian.empty
):

    source = (
        df_pedestrian.copy()
    )

elif (
    df_signal is not None
    and
    not df_signal.empty
):

    source = (
        df_signal.copy()
    )

else:

    print(
        "⚠ No hay señal peatonal interpretable."
    )

    return pd.DataFrame()


decisions = []


for _, row in (
    source.iterrows()
):


    first_frame = int(
        _first_available(
```

```
        row,

        [
            "first_frame",
            "signal_first_frame",
            "frame"
        ],

        0
    )
)


last_frame = int(
    _first_available(

        row,

        [
            "last_frame",
            "signal_last_frame",
            "frame"
        ],

        first_frame
    )
)


signal_state = str(
    _first_available(

        row,

        [
            "dominant_state",
            "signal_state",
            "state"
        ],

        "UNKNOWN"
    )
).upper()


signal_consistency = (
    _first_available(

        row,
```

```
        [
            "state_consistency",
            "signal_consistency"
        ],

        np.nan
    )
)


pedestrian_score = (
    _first_available(

        row,

        [
            "pedestrian_score",
            "score"
        ],

        np.nan
    )
)


# ----------------------------------------------------
# Vehículos activos durante el intervalo
# ----------------------------------------------------

if (
    df_dynamic is not None
    and
    not df_dynamic.empty
):

    overlap = (
        df_dynamic[
            (
                df_dynamic[
                    "last_frame"
                ]
                >= first_frame
            )
            &
            (
                df_dynamic[
                    "first_frame"
                ]
```

```
                <= last_frame
            )
            &
            (
                df_dynamic[
                    "valid_track"
                ]
            )
        ]
        .copy()
    )

else:

    overlap = (
        pd.DataFrame()
    )


if overlap.empty:

    n_approaching = 0

    n_moving = 0

    n_stable = 0


else:

    n_approaching = int(
        (
            overlap[
                "dynamic_state"
            ]
            ==
            "APPROACHING"
        ).sum()
    )


    n_moving = int(
        (
            overlap[
                "dynamic_state"
            ]
            ==
            "MOVING"
        ).sum()
```

```
    )


    n_stable = int(
        (
            overlap[
                "dynamic_state"
            ]
            ==
            "STABLE"
        ).sum()
    )


# ------------------------------------------------------
# Reglas de decisión
# ------------------------------------------------------

if signal_state == "RED":

    risk_level = (
        "HIGH"
    )

    recommendation = (
        "WAIT"
    )


elif signal_state == "GREEN":

    if (
        n_approaching > 0
    ):

        risk_level = (
            "HIGH"
        )

        recommendation = (
            "CAUTION"
        )


    elif (
        n_moving > 0
    ):

        risk_level = (
```

```
                "MEDIUM"
            )

            recommendation = (
                "CAUTION"
            )


        else:

            risk_level = (
                "LOW"
            )

            recommendation = (
                "CROSS_WITH_CAUTION"
            )


    else:

        risk_level = (
            "MEDIUM"
        )

        recommendation = (
            "CAUTION"
        )


    decisions.append({

        "signal_first_frame":
            first_frame,

        "signal_last_frame":
            last_frame,

        "signal_state":
            signal_state,

        "signal_consistency":
            signal_consistency,

        "pedestrian_score":
            pedestrian_score,

        "approaching_vehicles":
            n_approaching,
```

```
            "moving_vehicles":
                n_moving,

            "stable_vehicles":
                n_stable,

            "risk_level":
                risk_level,

            "recommendation":
                recommendation
        })


    df_decisions = pd.DataFrame(
        decisions
    )


    print(
        "✓ Decisiones generadas:",
        len(
            df_decisions
        )
    )


    return df_decisions


# ============================================================
# 9. TRADUCCIONES
# ============================================================

def signal_es(
    value
):

    mapping = {

        "RED":
            "ROJO",

        "GREEN":
            "VERDE",

        "UNKNOWN":
            "DESCONOCIDO"
```

```
    }

    value = str(
        value
    ).upper()

    return mapping.get(
        value,
        value
    )


def risk_es(
    value
):

    mapping = {

        "HIGH":
            "ALTO",

        "MEDIUM":
            "MEDIO",

        "LOW":
            "BAJO"
    }

    value = str(
        value
    ).upper()

    return mapping.get(
        value,
        value
    )


def recommendation_es(
    value
):

    mapping = {

        "WAIT":
            "ESPERAR",

        "CAUTION":
            "PRECAUCION",
```

```
        "CROSS_WITH_CAUTION":
            "CRUZAR CON PRECAUCION"
    }

    value = str(
        value
    ).upper()

    return mapping.get(
        value,
        value.replace(
            "_",
            " "
        )
    )


# ============================================================
# 10. DECISIÓN ACTIVA PARA UN FRAME
# ============================================================

def decision_for_frame_e2e(
    df_decisions,
    frame_idx
):

    if (
        df_decisions is None
        or
        df_decisions.empty
    ):

        return None


    active = (
        df_decisions[
            (
                df_decisions[
                    "signal_first_frame"
                ]
                <= frame_idx
            )
            &
            (
                df_decisions[
                    "signal_last_frame"
                ]
```

```
                >= frame_idx
            )
        ]
    )


    if not active.empty:

        return active.iloc[
            0
        ]


    distances = (
        df_decisions[
            "signal_first_frame"
        ]
        .sub(
            frame_idx
        )
        .abs()
    )


    return (
        df_decisions.loc[
            distances.idxmin()
        ]
    )


# ============================================================
# 11. PANEL VISUAL
# ============================================================

def draw_assistance_panel_e2e(
    frame,
    scenario,
    frame_idx,
    total_frames,
    fps,
    decision,
    n_persons,
    n_vehicles,
    n_traffic_lights,
    prototype_mode="BASE"
):

    height, width = (
```

```
    frame.shape[:2]
)


panel_width = min(
    820,
    int(
        width * 0.44
    )
)

panel_height = min(
    470,
    int(
        height * 0.48
    )
)


x1 = (
    width
    - panel_width
    - 25
)

y1 = 25

x2 = (
    width
    - 25
)

y2 = (
    y1
    + panel_height
)


overlay = (
    frame.copy()
)


cv2.rectangle(
    overlay,
    (
        x1,
        y1
    ),
```

```
        (
            x2,
            y2
        ),
        (
            25,
            25,
            25
        ),
        -1
    )


    cv2.addWeighted(
        overlay,
        0.72,
        frame,
        0.28,
        0,
        frame
    )


    WHITE = (
        255,
        255,
        255
    )

    GRAY = (
        190,
        190,
        190
    )

    YELLOW = (
        0,
        220,
        255
    )

    GREEN = (
        0,
        220,
        0
    )

    ORANGE = (
        0,
```

```
    165,
    255
)

RED = (
    0,
    0,
    255
)


def put(
    text,
    y_pos,
    color=WHITE,
    scale=0.65,
    thickness=2
):

    cv2.putText(

        frame,

        text,

        (
            x1 + 22,
            y_pos
        ),

        cv2.FONT_HERSHEY_SIMPLEX,

        scale,

        color,

        thickness,

        cv2.LINE_AA
    )


if decision is None:

    signal_state = (
        "UNKNOWN"
    )

    risk_level = (
```

```
            "UNKNOWN"
        )

        recommendation = (
            "CAUTION"
        )

        approaching = 0

        moving = 0

        stable = 0


    else:

        signal_state = str(
            decision.get(
                "signal_state",
                "UNKNOWN"
            )
        ).upper()


        risk_level = str(
            decision.get(
                "risk_level",
                "UNKNOWN"
            )
        ).upper()


        recommendation = str(
            decision.get(
                "recommendation",
                "CAUTION"
            )
        ).upper()


        approaching = int(
            decision.get(
                "approaching_vehicles",
                0
            )
        )

        moving = int(
            decision.get(
```

```
            "moving_vehicles",
            0
        )
    )

    stable = int(
        decision.get(
            "stable_vehicles",
            0
        )
    )


if risk_level == "HIGH":

    risk_color = RED

elif risk_level == "MEDIUM":

    risk_color = ORANGE

elif risk_level == "LOW":

    risk_color = GREEN

else:

    risk_color = WHITE


# ------------------------------------------------------------
# Títulos diferentes para 23.4 y 23.5
# ------------------------------------------------------------

if (
    str(
        prototype_mode
    ).upper()
    ==
    "HYBRID"
):

    signal_detector_title = (
        "DETECTOR SEMAFORO: "
        "COCO + YOLO V1 + YOLO V2"
    )

else:
```

```
    signal_detector_title = (
        "DETECTOR SEMAFORO: "
        "YOLO11 COCO"
    )


y = (
    y1 + 40
)


put(
    "PROTOTIPO INTEGRADO DE ASISTENCIA AL CRUCE",
    y,
    WHITE,
    0.78,
    2
)


y += 38


put(
    "Escenario: "
    + str(
        scenario
    ).replace(
        "_",
        " "
    ),
    y,
    WHITE,
    0.64,
    1
)


y += 30


time_s = (
    frame_idx / fps
    if fps > 0
    else 0
)


put(
```

```
    f"Tiempo: {time_s:.1f} s   |   "
    f"Frame: {frame_idx}/{total_frames}",
    y,
    GRAY,
    0.56,
    1
)


# ------------------------------------------------------
# Percepción general
# ------------------------------------------------------

y += 39


put(
    "PERCEPCION DE OBJETOS: "
    "YOLO11 COCO + TRACKING",
    y,
    YELLOW,
    0.57,
    2
)


y += 28


put(
    f"Personas: {n_persons}   |   "
    f"Vehiculos: {n_vehicles}   |   "
    f"Semaforos COCO: {n_traffic_lights}",
    y,
    WHITE,
    0.52,
    1
)


# ------------------------------------------------------
# Semáforo
# ------------------------------------------------------

y += 37


put(
    signal_detector_title,
```

```
    y,
    YELLOW,
    0.55,
    2
)


y += 28


put(
    "INTERPRETACION: CLIP + GATE V2",
    y,
    YELLOW,
    0.55,
    2
)


y += 30


put(
    "SEMAFORO PEATONAL: "
    + signal_es(
        signal_state
    ),
    y,
    WHITE,
    0.66,
    2
)


# ------------------------------------------------------
# Dinámica
# ------------------------------------------------------

y += 30


put(
    f"Dinamica: aprox. {approaching}   |   "
    f"mov. {moving}   |   "
    f"estables {stable}",
    y,
    GRAY,
    0.52,
    1
```

```
    )


    # ----------------------------------------------------
    # Riesgo
    # ----------------------------------------------------

    y += 38


    put(
        "RIESGO: "
        + risk_es(
            risk_level
        )
        + f" ({risk_level})",
        y,
        risk_color,
        0.75,
        2
    )


    # ----------------------------------------------------
    # Recomendación
    # ----------------------------------------------------

    y += 38


    put(
        "ASISTENCIA: "
        + recommendation_es(
            recommendation
        ),
        y,
        risk_color,
        0.70,
        2
    )


# ============================================================
# 12. GENERACIÓN DEL VÍDEO FINAL
# ============================================================

def create_end_to_end_video(
    video_path,
    df_tracking,
```

```
    df_dynamic,
    df_decisions,
    output_path,
    scenario,
    prototype_mode="BASE"
):

    print()
    print(
        "[5/6] Generación del vídeo final..."
    )


    video_path = Path(
        video_path
    )

    output_path = Path(
        output_path
    )


    output_path.parent.mkdir(
        parents=True,
        exist_ok=True
    )


    cap = cv2.VideoCapture(
        str(
            video_path
        )
    )


    if not cap.isOpened():

        raise RuntimeError(
            f"No se puede abrir:\n{video_path}"
        )


    fps = cap.get(
        cv2.CAP_PROP_FPS
    )

    total_frames = int(
        cap.get(
            cv2.CAP_PROP_FRAME_COUNT
```

```
    )
)

width = int(
    cap.get(
        cv2.CAP_PROP_FRAME_WIDTH
    )
)

height = int(
    cap.get(
        cv2.CAP_PROP_FRAME_HEIGHT
    )
)


writer = cv2.VideoWriter(

    str(
        output_path
    ),

    cv2.VideoWriter_fourcc(
        *"mp4v"
    ),

    fps,

    (
        width,
        height
    )
)


dynamic_map = {}


if (
    df_dynamic is not None
    and
    not df_dynamic.empty
):

    dynamic_map = (

        df_dynamic

        .set_index(
```

```
            "track_id"
        )[
            "dynamic_state"
        ]

        .to_dict()
    )


frame_idx = 0


while True:

    ret, frame = (
        cap.read()
    )


    if not ret:
        break


    # ----------------------------------------------------
    # Tracks del frame
    # ----------------------------------------------------

    if (
        df_tracking is not None
        and
        not df_tracking.empty
    ):

        frame_tracks = (

            df_tracking[
                df_tracking[
                    "frame"
                ]
                ==
                frame_idx
            ]

        )

    else:

        frame_tracks = (
            pd.DataFrame()
```

```
    )


n_persons = 0

n_vehicles = 0

n_traffic_lights = 0


for _, row in (
    frame_tracks.iterrows()
):

    class_name = str(
        row[
            "class_name"
        ]
    ).lower()


    if class_name == "person":

        n_persons += 1


    if class_name in VEHICLE_CLASSES:

        n_vehicles += 1


    if class_name == "traffic light":

        n_traffic_lights += 1


    x1 = int(
        row[
            "x1"
        ]
    )

    y1b = int(
        row[
            "y1"
        ]
    )

    x2 = int(
```

```
    row[
        "x2"
    ]
)

y2b = int(
    row[
        "y2"
    ]
)


track_id = (
    row.get(
        "track_id",
        np.nan
    )
)


confidence = float(
    row.get(
        "confidence",
        0
    )
)


if class_name in VEHICLE_CLASSES:

    state = (
        dynamic_map.get(
            int(
                track_id
            )
            if pd.notna(
                track_id
            )
            else -1,
            "STABLE"
        )
    )


    if state == "APPROACHING":

        color = (
            0,
            0,
```

```
                255
            )


        elif state == "MOVING":

            color = (
                0,
                165,
                255
            )


        else:

            color = (
                0,
                220,
                0
            )


    elif class_name == "person":

        state = ""

        color = (
            0,
            220,
            255
        )


    elif class_name == "traffic light":

        state = ""

        color = (
            255,
            255,
            255
        )


    else:

        state = ""

        color = (
```

```
            190,
            190,
            190
        )


    cv2.rectangle(

        frame,

        (
            x1,
            y1b
        ),

        (
            x2,
            y2b
        ),

        color,

        2
    )


    label = (
        class_name
    )


    if pd.notna(
        track_id
    ):

        label += (
            f" ID:{int(track_id)}"
        )


    label += (
        f" {confidence:.2f}"
    )


    cv2.putText(

        frame,
```

```
        label,

        (
            x1,
            max(
                20,
                y1b - 7
            )
        ),

        cv2.FONT_HERSHEY_SIMPLEX,

        0.48,

        color,

        2,

        cv2.LINE_AA
    )


    if state:

        cv2.putText(

            frame,

            state,

            (
                x1,
                min(
                    height - 10,
                    y2b + 22
                )
            ),

            cv2.FONT_HERSHEY_SIMPLEX,

            0.48,

            color,

            2,

            cv2.LINE_AA
        )
```

```
# ------------------------------------------------------
# Decisión
# ------------------------------------------------------

decision = (
    decision_for_frame_e2e(
        df_decisions,
        frame_idx
    )
)


# ------------------------------------------------------
# Panel
# ------------------------------------------------------

draw_assistance_panel_e2e(

    frame=frame,

    scenario=scenario,

    frame_idx=frame_idx,

    total_frames=total_frames,

    fps=fps,

    decision=decision,

    n_persons=n_persons,

    n_vehicles=n_vehicles,

    n_traffic_lights=n_traffic_lights,

    prototype_mode=prototype_mode
)


# ------------------------------------------------------
# Información superior izquierda
# ------------------------------------------------------

time_s = (
    frame_idx / fps
    if fps > 0
    else 0
```

```
    )


    cv2.putText(

        frame,

        f"Frame: {frame_idx} | "
        f"Tiempo: {time_s:.2f} s",

        (
            15,
            35
        ),

        cv2.FONT_HERSHEY_SIMPLEX,

        0.70,

        (
            255,
            255,
            255
        ),

        2,

        cv2.LINE_AA
    )


    writer.write(
        frame
    )


    frame_idx += 1


cap.release()

writer.release()


print(
    "✓ Vídeo generado:"
)

print(
```

```
        output_path
    )


    return str(
        output_path
    )


# ============================================================
# 13. FUNCIÓN PRINCIPAL REUTILIZABLE
# ============================================================

def run_crossing_assistance_prototype(
    video_path,
    signal_detector=None,
    prototype_mode="BASE",
    scenario="END_TO_END",
    output_root=None
):

    video_path = Path(
        video_path
    )


    if signal_detector is None:

        signal_detector = (
            yolo_signal_coco
        )


    if output_root is None:

        output_root = (
            PROTOTYPE_BASE_DIR
        )


    output_root = Path(
        output_root
    )


    video_id = (
        video_path.stem
    )
```

```
output_dir = (
    output_root
    / video_id
)


output_dir.mkdir(
    parents=True,
    exist_ok=True
)


print()
print("=" * 78)

print(
    "PROTOTIPO END-TO-END DE ASISTENCIA AL CRUCE"
)

print("=" * 78)

print(
    "Vídeo:",
    video_path.name
)

print(
    "Modo:",
    prototype_mode
)


# ============================================================
# 1. YOLO + TRACKING
# ============================================================

(
    df_tracking,
    fps,
    total_frames,
    width,
    height
) = process_objects_and_tracking_e2e(
    video_path
)


# ============================================================
```

```
# 2. DINÁMICA
# ============================================================

df_dynamic = (
    build_vehicle_dynamics_e2e(
        df_tracking
    )
)


# ============================================================
# 3. SEMÁFORO PEATONAL
# ============================================================

print()
print(
    "[3/6] Interpretación del semáforo "
    "mediante detector + CLIP + Gate V2..."
)


original_signal_detector = (
    globals().get(
        "yolo_signal_detector",
        None
    )
)


globals()[
    "yolo_signal_detector"
] = signal_detector


try:

    signal_result = (
        analyze_final_pedestrian_signal(
            str(
                video_path
            )
        )
    )


finally:

    if (
        original_signal_detector
```

```
        is not None
    ):

        globals()[
            "yolo_signal_detector"
        ] = (
            original_signal_detector
        )

    else:

        globals().pop(
            "yolo_signal_detector",
            None
        )


# ----------------------------------------------------------
# La función existente devuelve normalmente:
# df_signal, df_pedestrian
# ----------------------------------------------------------

if (
    isinstance(
        signal_result,
        tuple
    )
    and
    len(
        signal_result
    ) >= 2
):

    df_signal = (
        signal_result[
            0
        ]
    )

    df_pedestrian = (
        signal_result[
            1
        ]
    )


elif isinstance(
    signal_result,
    pd.DataFrame
```

```
):

    df_signal = (
        signal_result
    )

    df_pedestrian = (
        pd.DataFrame()
    )


else:

    raise RuntimeError(
        "Formato inesperado devuelto por "
        "analyze_final_pedestrian_signal."
    )


# ============================================================
# 4. FUSIÓN
# ============================================================

df_decisions = (
    fuse_crossing_information_e2e(

        df_signal,

        df_pedestrian,

        df_dynamic
    )
)


# ============================================================
# 5. GUARDAR CSV
# ============================================================

df_tracking.to_csv(

    output_dir
    / "tracking.csv",

    index=False
)


df_dynamic.to_csv(
```

```
    output_dir
    / "vehicle_dynamics.csv",

    index=False
)


if (
    df_signal is not None
    and
    isinstance(
        df_signal,
        pd.DataFrame
    )
):

    df_signal.to_csv(

        output_dir
        / "signal_temporal.csv",

        index=False
    )


if (
    df_pedestrian is not None
    and
    isinstance(
        df_pedestrian,
        pd.DataFrame
    )
):

    df_pedestrian.to_csv(

        output_dir
        / "pedestrian_signal.csv",

        index=False
    )


df_decisions.to_csv(

    output_dir
    / "decisions.csv",
```

```
    index=False
)


# ============================================================
# 6. VÍDEO FINAL
# ============================================================

print()
print(
    "[6/6] Generación del prototipo visual..."
)


if str(
    prototype_mode
).upper() == "HYBRID":

    final_name = (
        f"{video_id}_PROTOTIPO_HIBRIDO.mp4"
    )

else:

    final_name = (
        f"{video_id}_PROTOTIPO_BASE.mp4"
    )


final_video_path = (
    output_dir
    / final_name
)


create_end_to_end_video(

    video_path=video_path,

    df_tracking=df_tracking,

    df_dynamic=df_dynamic,

    df_decisions=df_decisions,

    output_path=final_video_path,

    scenario=scenario,
```

```
    prototype_mode=prototype_mode
)


print()
print("=" * 78)

print(
    "✓ EJECUCIÓN END-TO-END FINALIZADA"
)

print("=" * 78)

print(
    "Carpeta:"
)

print(
    output_dir
)


return {

    "video":
        str(
            video_path
        ),

    "output_dir":
        str(
            output_dir
        ),

    "video_final":
        str(
            final_video_path
        ),

    "tracking":
        df_tracking,

    "vehicle_dynamics":
        df_dynamic,

    "signal_candidates":
        df_signal,

    "pedestrian_signals":
```

```
            df_pedestrian,

        "decisions":
            df_decisions
    }


print()
print("=" * 72)
print(
    "23.4 - MOTOR BASE END-TO-END PREPARADO"
)
print("=" * 72)

print(
    "✓ YOLO11 COCO + tracking"
)

print(
    "✓ Dinámica vehicular"
)

print(
    "✓ CLIP + Gate V2"
)

print(
    "✓ Fusión de riesgo"
)

print(
    "✓ Generación de vídeo"
)

print(
    "✓ run_crossing_assistance_prototype disponible"
)

video_base_test = (
    VIDEOS_DIR
    / "20260611_184739.mp4"
)


resultado_base_test = (
    run_crossing_assistance_prototype(

        video_path=video_base_test,
```

```
        signal_detector=yolo_signal_coco,

        prototype_mode="BASE",

        scenario="GREEN_CAUTION",

        output_root=PROTOTYPE_BASE_DIR
    )
)


print()
print(
    "Vídeo final:"
)

print(
    resultado_base_test[
        "video_final"
    ]
)
✓ Módulo CLIP + Gate V2 disponible.
✓ YOLO11 COCO preparado.

================================================================================
23.4 - MOTOR BASE END-TO-END PREPARADO
================================================================================
✓ YOLO11 COCO + tracking
✓ Dinámica vehicular
✓ CLIP + Gate V2
✓ Fusión de riesgo
✓ Generación de vídeo
✓ run_crossing_assistance_prototype disponible

================================================================================
PROTOTIPO END-TO-END DE ASISTENCIA AL CRUCE
================================================================================
Vídeo: 20260611_184739.mp4
Modo: BASE

[1/6] YOLO11 COCO + tracking...
Vídeo: 20260611_184739.mp4
```

```
FPS: 30.00
Frames: 434
✓ Detecciones/tracks: 2622
✓ IDs distintos: 98

[2/6] Análisis dinámico de vehículos...
✓ Tracks vehiculares: 67
{'MOVING': 25, 'APPROACHING': 23, 'INSUFFICIENT': 19}

[3/6] Interpretación del semáforo mediante detector + CLIP + Gate
V2...
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184739.mp4
FPS: 29.995415447254988
Frames: 434
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
```

```
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
Frames analizados: 50
Frames analizados: 100

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 974
ACTIONABLE: 466
NON_ACTIONABLE: 508

Estados:
state
NON_ACTIONABLE    508
GREEN             293
RED               173
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

[4/6] Fusión contextual y decisión...
✓ Decisiones generadas: 30

[6/6] Generación del prototipo visual...
```

```
[5/6] Generación del vídeo final...
✓ Vídeo generado:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184739\2026
0611_184739_PROTOTIPO_BASE.mp4

================================================================================
✓ EJECUCIÓN END-TO-END FINALIZADA
================================================================================
Carpeta:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184739

Vídeo final:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184739\2026
0611_184739_PROTOTIPO_BASE.mp4
```

23.5 Prototipo integrado híbrido de extremo a extremo

En este apartado se implementa la versión ampliada del prototipo de asistencia al cruce.

A diferencia de la demostración 23.4, basada en el detector YOLO11 preentrenado sobre COCO, esta versión combina tres fuentes de candidatos para la detección del semáforo peatonal:

- YOLO11 preentrenado sobre COCO.
- Modelo YOLO especializado V1.
- Modelo YOLO especializado V2.

Las detecciones obtenidas se fusionan antes de aplicar la validación semántica mediante CLIP y Gate V2. Posteriormente se estima el estado RED/GREEN y se integra temporalmente con el análisis de vehículos, tracking, dinámica de movimiento y reglas de decisión.

Los modelos especializados actúan como detectores complementarios y no determinan directamente la recomendación final del sistema.

23.5.1 Carga de los detectores del prototipo híbrido

In [129]:

```
# 23.5.1 CARGA DE LOS DETECTORES DEL PROTOTIPO HÍBRIDO


from pathlib import Path
from ultralytics import YOLO
import os


# ------------------------------------------------------------
# Raíz actual del TFM
# ------------------------------------------------------------

TFM_DIR = Path(
    r"C:\Users\juank\OneDrive\Documentos\MÁSTER EN IA\TFM"
)

MODELS_DIR = (
    TFM_DIR
    / "MODELOS_YOLO_FINALES_TFM"
)


# ------------------------------------------------------------
# Localizador robusto de los modelos especializados
# ------------------------------------------------------------

def localizar_modelo_yolo(
    nombre_estable,
    nombre_run
):

    candidatos = [

        # Copia permanente recomendada
        MODELS_DIR
        / nombre_estable,

        # Posible ubicación de training actual
        TFM_DIR
        / "dataset_pedestrian_traffic_light"
        / "dataset_yolo_final"
        / "training"
        / nombre_run
        / "weights"
        / "best.pt",

        # Ubicación histórica
        Path(
            r"C:\Users\juank\Documents\CARLOS\TFM"
```

```
        )
        / "dataset_pedestrian_traffic_light"
        / "dataset_yolo_final"
        / "training"
        / nombre_run
        / "weights"
        / "best.pt"
    ]

    for path in candidatos:

        if path.is_file():
            return path

    raise FileNotFoundError(
        "\nNo se ha localizado el modelo:\n"
        f"{nombre_estable}\n"
        f"Run esperado: {nombre_run}"
    )


# ------------------------------------------------------------
# V1
# ------------------------------------------------------------

V1_MODEL_PATH = localizar_modelo_yolo(
    "pedestrian_light_v1_best.pt",
    "yolo11_pedestrian_light"
)


# ------------------------------------------------------------
# V2
# ------------------------------------------------------------

V2_MODEL_PATH = localizar_modelo_yolo(
    "pedestrian_light_v2_best.pt",
    "yolo11n_pedestrian_light_v2_1280"
)


# ------------------------------------------------------------
# YOLO COCO
# ------------------------------------------------------------

if "yolo_signal_detector" in globals():

    yolo_signal_coco = (
        yolo_signal_detector
```

```
    )

else:

    yolo_signal_coco = YOLO(
        "yolo11n.pt"
    )


# ------------------------------------------------------------
# Modelos especializados
# ------------------------------------------------------------

yolo_signal_v1 = YOLO(
    str(V1_MODEL_PATH)
)

yolo_signal_v2 = YOLO(
    str(V2_MODEL_PATH)
)


# ------------------------------------------------------------
# Comprobación
# ------------------------------------------------------------

print("=" * 72)
print("DETECTORES DEL PROTOTIPO HÍBRIDO")
print("=" * 72)

print(
    "\n✓ YOLO COCO:"
)
print(
    "  yolo11n.pt"
)

print(
    "\n✓ YOLO especializado V1:"
)
print(
    f"  {V1_MODEL_PATH}"
)

print(
    "\n✓ YOLO especializado V2:"
)
print(
    f"  {V2_MODEL_PATH}"
```

```
)

print()
print(
    "✓ LOS TRES DETECTORES ESTÁN CARGADOS"
)
```

```
======================================================================
==
DETECTORES DEL PROTOTIPO HÍBRIDO
======================================================================
==

✓ YOLO COCO:
  yolo11n.pt

✓ YOLO especializado V1:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v1_best.pt

✓ YOLO especializado V2:
  C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\MODELOS_YOLO_FINALES_TFM\pedestrian_light_v2_best.pt

✓ LOS TRES DETECTORES ESTÁN CARGADOS
```

23.5.2 Detector híbrido de semáforo peatonal

In [130]:

```
# 23.5.2 DETECTOR HÍBRIDO DE SEMÁFORO PEATONAL
# COCO + YOLO V1 + YOLO V2


import torch


# ------------------------------------------------------------
# Configuración
# ------------------------------------------------------------

HYBRID_COCO_CONF = 0.05

HYBRID_V1_CONF = 0.05
HYBRID_V2_CONF = 0.05

HYBRID_IOU_MERGE = 0.50

HYBRID_MAX_DET = 20
```

```
# ------------------------------------------------------------
# IoU entre dos bounding boxes
# ------------------------------------------------------------

def bbox_iou_xyxy(a, b):

    ax1, ay1, ax2, ay2 = a
    bx1, by1, bx2, by2 = b

    inter_x1 = max(ax1, bx1)
    inter_y1 = max(ay1, by1)

    inter_x2 = min(ax2, bx2)
    inter_y2 = min(ay2, by2)

    inter_w = max(
        0.0,
        inter_x2 - inter_x1
    )

    inter_h = max(
        0.0,
        inter_y2 - inter_y1
    )

    intersection = (
        inter_w * inter_h
    )

    area_a = max(
        0.0,
        ax2 - ax1
    ) * max(
        0.0,
        ay2 - ay1
    )

    area_b = max(
        0.0,
        bx2 - bx1
    ) * max(
        0.0,
        by2 - by1
    )

    union = (
        area_a
        + area_b
```

```
        - intersection
    )

    if union <= 0:
        return 0.0

    return intersection / union


# ------------------------------------------------------------
# Convertir resultados YOLO en candidatos comunes
# ------------------------------------------------------------

def extraer_candidatos_yolo(
    result,
    source_name
):

    candidates = []

    if result.boxes is None:
        return candidates

    for box in result.boxes:

        xyxy = [
            float(v)
            for v in box.xyxy[0].tolist()
        ]

        conf = float(
            box.conf[0].item()
        )

        candidates.append({

            "xyxy": xyxy,

            "conf": conf,

            "sources": [
                source_name
            ]
        })

    return candidates


# ------------------------------------------------------------
```

```
# Fusionar detecciones solapadas
# ------------------------------------------------------------

def fusionar_candidatos(
    candidates,
    iou_threshold=0.50
):

    candidates = sorted(
        candidates,
        key=lambda x: x["conf"],
        reverse=True
    )

    fused = []

    for candidate in candidates:

        matched = False

        for kept in fused:

            iou = bbox_iou_xyxy(
                candidate["xyxy"],
                kept["xyxy"]
            )

            if iou >= iou_threshold:

                for source in candidate[
                    "sources"
                ]:

                    if source not in kept[
                        "sources"
                    ]:
                        kept[
                            "sources"
                        ].append(
                            source
                        )

                # Conservamos la caja
                # de mayor confianza
                if (
                    candidate["conf"]
                    > kept["conf"]
                ):
```

```
                    kept["xyxy"] = (
                        candidate["xyxy"]
                    )

                    kept["conf"] = (
                        candidate["conf"]
                    )

                matched = True
                break

        if not matched:

            fused.append(
                candidate.copy()
            )

    return fused


# ------------------------------------------------------------
# Clases proxy compatibles con process_temporal_signal_video
# ------------------------------------------------------------

class HybridBox:

    def __init__(
        self,
        xyxy,
        conf,
        sources
    ):

        self.xyxy = torch.tensor(
            [xyxy],
            dtype=torch.float32
        )

        self.conf = torch.tensor(
            [conf],
            dtype=torch.float32
        )

        self.sources = sources


class HybridResult:

    def __init__(self, boxes):
```

```
        self.boxes = boxes


# ------------------------------------------------------------
# Detector híbrido
# ------------------------------------------------------------

class HybridSignalDetector:

    def __init__(
        self,
        coco_model,
        v1_model,
        v2_model
    ):

        self.coco_model = coco_model
        self.v1_model = v1_model
        self.v2_model = v2_model

        self.reset_stats()


    def reset_stats(self):

        self.stats = {
            "COCO": 0,
            "V1": 0,
            "V2": 0,
            "FUSED": 0
        }


    def predict(
        self,
        source,
        classes=None,
        conf=None,
        verbose=False,
        **kwargs
    ):

        # --------------------------------------------
        # 1. YOLO COCO
        # --------------------------------------------

        coco_result = (
            self.coco_model.predict(
```

```
        source=source,
        classes=[9],
        conf=HYBRID_COCO_CONF,
        max_det=HYBRID_MAX_DET,
        verbose=False
    )[0]
)

coco_candidates = (
    extraer_candidatos_yolo(
        coco_result,
        "COCO"
    )
)


# ------------------------------------------
# 2. YOLO especializado V1
# ------------------------------------------

v1_result = (
    self.v1_model.predict(
        source=source,
        conf=HYBRID_V1_CONF,
        imgsz=960,
        max_det=HYBRID_MAX_DET,
        verbose=False
    )[0]
)

v1_candidates = (
    extraer_candidatos_yolo(
        v1_result,
        "V1"
    )
)


# ------------------------------------------
# 3. YOLO especializado V2
# ------------------------------------------

v2_result = (
    self.v2_model.predict(
        source=source,
        conf=HYBRID_V2_CONF,
        imgsz=1280,
        max_det=HYBRID_MAX_DET,
        verbose=False
```

```
    )[0]
)

v2_candidates = (
    extraer_candidatos_yolo(
        v2_result,
        "V2"
    )
)


# ------------------------------------------
# Estadísticas
# ------------------------------------------

self.stats["COCO"] += len(
    coco_candidates
)

self.stats["V1"] += len(
    v1_candidates
)

self.stats["V2"] += len(
    v2_candidates
)


# ------------------------------------------
# 4. Unión
# ------------------------------------------

all_candidates = (
    coco_candidates
    + v1_candidates
    + v2_candidates
)


# ------------------------------------------
# 5. Fusión espacial
# ------------------------------------------

fused_candidates = (
    fusionar_candidatos(
        all_candidates,
        iou_threshold=(
            HYBRID_IOU_MERGE
        )
```

```
            )
        )

        self.stats["FUSED"] += len(
            fused_candidates
        )


        boxes = [

            HybridBox(
                candidate["xyxy"],
                candidate["conf"],
                candidate["sources"]
            )

            for candidate
            in fused_candidates
        ]

        return [
            HybridResult(
                boxes
            )
        ]


# ------------------------------------------------------------
# Instanciar detector
# ------------------------------------------------------------

hybrid_signal_detector = (
    HybridSignalDetector(
        yolo_signal_coco,
        yolo_signal_v1,
        yolo_signal_v2
    )
)

print(
    "✓ Detector híbrido "
    "COCO + V1 + V2 preparado."
)
```

```
✓ Detector híbrido COCO + V1 + V2 preparado.
```

In [131]:

```
# ============================================================
# 23.5.3 CONEXIÓN DEL DETECTOR HÍBRIDO
# CON EL PIPELINE TEMPORAL EXISTENTE
# ============================================================
```

```
# Guardamos la función original
analyze_final_pedestrian_signal_base = (
    analyze_final_pedestrian_signal
)


def analyze_final_pedestrian_signal_hybrid(
    video_path
):

    global yolo_signal_detector

    original_detector = (
        yolo_signal_detector
    )

    hybrid_signal_detector.reset_stats()

    try:

        # Sustitución temporal:
        # el resto del pipeline no cambia
        yolo_signal_detector = (
            hybrid_signal_detector
        )

        result = (
            analyze_final_pedestrian_signal_base(
                video_path
            )
        )

    finally:

        # Restauramos COCO puro para que 23.4
        # siga siendo reproducible
        yolo_signal_detector = (
            original_detector
        )


    print()
    print("=" * 72)
    print(
        "CANDIDATOS DEL DETECTOR HÍBRIDO"
    )
    print("=" * 72)
```

```
    for key, value in (
        hybrid_signal_detector
        .stats
        .items()
    ):

        print(
            f"{key:10s}: {value}"
        )

    return result


print(
    "✓ Pipeline temporal híbrido conectado."
)
```

```
✓ Pipeline temporal híbrido conectado.
```

23.5.4 Prototipo end to end híbrido

In [132]:

```
# ============================================================
# 23.5.4 PROTOTIPO END-TO-END HÍBRIDO
# ============================================================


# ------------------------------------------------------------
# Guardar la función base del prototipo
# ------------------------------------------------------------

if "run_crossing_assistance_prototype" not in globals():
    raise RuntimeError(
        "No existe run_crossing_assistance_prototype. "
        "Ejecuta previamente la celda donde se define "
        "el prototipo end-to-end original."
    )


run_crossing_assistance_prototype_base = (
    run_crossing_assistance_prototype
)


# ------------------------------------------------------------
# Prototipo híbrido
# ------------------------------------------------------------
```

```
def run_crossing_assistance_prototype_hybrid(
    *args,
    **kwargs
):

    global analyze_final_pedestrian_signal
    global PROTOTYPE_PANEL_MODE


    # ------------------------------------------------------
    # Guardar configuración original
    # ------------------------------------------------------

    original_analyzer = (
        analyze_final_pedestrian_signal
    )

    original_panel_mode = globals().get(
        "PROTOTYPE_PANEL_MODE",
        "BASE"
    )


    try:

        # ------------------------------------------------
        # ACTIVAR PIPELINE HÍBRIDO
        # COCO + V1 + V2
        # ------------------------------------------------

        analyze_final_pedestrian_signal = (
            analyze_final_pedestrian_signal_hybrid
        )


        # ------------------------------------------------
        # ACTIVAR PANEL HÍBRIDO
        # ------------------------------------------------

        PROTOTYPE_PANEL_MODE = (
            "HYBRID"
        )


        # ------------------------------------------------
        # EJECUTAR PROTOTIPO END-TO-END
        # ------------------------------------------------

        result = (
```

```
            run_crossing_assistance_prototype_base(
                *args,
                **kwargs
            )
        )


    finally:

        # ------------------------------------------------
        # RESTAURAR PIPELINE BASE
        # ------------------------------------------------

        analyze_final_pedestrian_signal = (
            original_analyzer
        )


        # ------------------------------------------------
        # RESTAURAR PANEL BASE
        # ------------------------------------------------

        PROTOTYPE_PANEL_MODE = (
            original_panel_mode
        )


    return result


print(
    "✓ Prototipo híbrido end-to-end preparado."
)

print(
    "✓ Panel híbrido COCO + V1 + V2 activable."
)
```

```
✓ Prototipo híbrido end-to-end preparado.
✓ Panel híbrido COCO + V1 + V2 activable.
```

In [133]:

```
video_hybrid_test = os.path.join(
    VIDEOS_DIR,
    "20260611_184739.mp4"
)

resultado_hybrid_test = (
    run_crossing_assistance_prototype_hybrid(
        video_hybrid_test
```

```
    )
)

================================================================================
========
PROTOTIPO END-TO-END DE ASISTENCIA AL CRUCE
================================================================================
========
Vídeo: 20260611_184739.mp4
Modo: BASE

[1/6] YOLO11 COCO + tracking...
Vídeo: 20260611_184739.mp4
FPS: 30.00
Frames: 434
✓ Detecciones/tracks: 2621
✓ IDs distintos: 97

[2/6] Análisis dinámico de vehículos...
✓ Tracks vehiculares: 67
{'MOVING': 26, 'APPROACHING': 22, 'INSUFFICIENT': 19}

[3/6] Interpretación del semáforo mediante detector + CLIP + Gate
V2...
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184739.mp4
FPS: 29.995415447254988
Frames: 434
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
```

```
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
Frames analizados: 50
Frames analizados: 100

======================================================================
PROCESAMIENTO FINALIZADO
======================================================================
Observaciones: 1103
ACTIONABLE: 478
NON_ACTIONABLE: 625

Estados:
state
NON_ACTIONABLE    625
GREEN             331
RED               147
Name: count, dtype: int64
```

```
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

================================================================================
CANDIDATOS DEL DETECTOR HÍBRIDO
================================================================================
COCO      : 974
V1        : 378
V2        : 716
FUSED     : 1103

[4/6] Fusión contextual y decisión...
✓ Decisiones generadas: 24

[6/6] Generación del prototipo visual...

[5/6] Generación del vídeo final...
✓ Vídeo generado:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184739\2026
0611_184739_PROTOTIPO_BASE.mp4

================================================================================
========
✓ EJECUCIÓN END-TO-END FINALIZADA
================================================================================
========
Carpeta:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184739
```

23.5.5 Prueba 2/3 — escenario RED / WAIT

In [134]:

```
# 23.5.5 PRUEBA 2/3 — ESCENARIO RED / WAIT


video_hybrid_red_wait = os.path.join(
    VIDEOS_DIR,
    "20260611_184808.mp4"
```

```
)

print("=" * 72)
print("PROTOTIPO HÍBRIDO — ESCENARIO 2/3")
print("RED / WAIT")
print("=" * 72)

print("Vídeo:")
print(video_hybrid_red_wait)

print(
    "\n¿Existe?:",
    os.path.isfile(video_hybrid_red_wait)
)

resultado_hybrid_red_wait = (
    run_crossing_assistance_prototype_hybrid(
        video_hybrid_red_wait
    )
)
========================================================================
==
PROTOTIPO HÍBRIDO — ESCENARIO 2/3
RED / WAIT
========================================================================
==
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184808.mp4

¿Existe?: True

========================================================================
========
PROTOTIPO END-TO-END DE ASISTENCIA AL CRUCE
========================================================================
========
Vídeo: 20260611_184808.mp4
Modo: BASE

[1/6] YOLO11 COCO + tracking...
Vídeo: 20260611_184808.mp4
FPS: 30.00
Frames: 364
✓ Detecciones/tracks: 1340
✓ IDs distintos: 35

[2/6] Análisis dinámico de vehículos...
✓ Tracks vehiculares: 18
```

```
{'APPROACHING': 10, 'MOVING': 5, 'INSUFFICIENT': 3}

[3/6] Interpretación del semáforo mediante detector + CLIP + Gate
V2...
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_184808.mp4
FPS: 29.999395616571828
Frames: 364
C:\Users\juank\AppData\Local\Programs\Python\Python312\Lib\site-
packages\numpy\lib\_nanfunctions_impl.py:1241: RuntimeWarning: Mean of
empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
Frames analizados: 50
Frames analizados: 100

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
Observaciones: 317
ACTIONABLE: 109
NON_ACTIONABLE: 208

Estados:
state
NON_ACTIONABLE    208
RED                77
GREEN              32
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

================================================================================
==
CANDIDATOS DEL DETECTOR HÍBRIDO
================================================================================
==
COCO      : 252
V1        : 100
V2        : 220
```

```
FUSED     : 317

[4/6] Fusión contextual y decisión...
✓ Decisiones generadas: 2

[6/6] Generación del prototipo visual...

[5/6] Generación del vídeo final...
✓ Vídeo generado:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184808\2026
0611_184808_PROTOTIPO_BASE.mp4

================================================================================
========
✓ EJECUCIÓN END-TO-END FINALIZADA
================================================================================
========
Carpeta:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_184808
```

23.5.6 Prueba 3/3 — GREEN / CROSS WITH CAUTION

In [135]:

```
# 23.5.6 PRUEBA 3/3 — GREEN / CROSS WITH CAUTION


video_hybrid_cross = os.path.join(
    VIDEOS_DIR,
    "20260611_185443~2.mp4"
)

print("=" * 72)
print("PROTOTIPO HÍBRIDO — ESCENARIO 3/3")
print("GREEN / CROSS WITH CAUTION")
print("=" * 72)

print("Vídeo:")
print(video_hybrid_cross)

print(
    "\n¿Existe?:",
    os.path.isfile(video_hybrid_cross)
)

resultado_hybrid_cross = (
```

```
    run_crossing_assistance_prototype_hybrid(
        video_hybrid_cross
    )
)
================================================================================
PROTOTIPO HÍBRIDO — ESCENARIO 3/3
GREEN / CROSS WITH CAUTION
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_185443~2.mp4

¿Existe?: True

================================================================================
PROTOTIPO END-TO-END DE ASISTENCIA AL CRUCE
================================================================================
Vídeo: 20260611_185443~2.mp4
Modo: BASE

[1/6] YOLO11 COCO + tracking...
Vídeo: 20260611_185443~2.mp4
FPS: 29.73
Frames: 91
✓ Detecciones/tracks: 547
✓ IDs distintos: 16

[2/6] Análisis dinámico de vehículos...
✓ Tracks vehiculares: 14
{'MOVING': 8, 'INSUFFICIENT': 5, 'APPROACHING': 1}

[3/6] Interpretación del semáforo mediante detector + CLIP + Gate
V2...
================================================================================
PIPELINE TEMPORAL FINAL
================================================================================
Vídeo:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\VÍDEOS\20260611_185443~2.mp4
FPS: 29.733811591466868
Frames: 91

================================================================================
PROCESAMIENTO FINALIZADO
================================================================================
```

```
Observaciones: 151
ACTIONABLE: 46
NON_ACTIONABLE: 105

Estados:
state
NON_ACTIONABLE    105
GREEN              46
Name: count, dtype: int64
C:\Users\juank\AppData\Local\Temp\ipykernel_20084\119949934.py:34:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  .apply(summarize_final_track)

================================================================================
CANDIDATOS DEL DETECTOR HÍBRIDO
================================================================================
COCO      : 75
V1        : 80
V2        : 209
FUSED     : 151

[4/6] Fusión contextual y decisión...
✓ Decisiones generadas: 6

[6/6] Generación del prototipo visual...

[5/6] Generación del vídeo final...
✓ Vídeo generado:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_185443~2\20
260611_185443~2_PROTOTIPO_BASE.mp4

================================================================================
✓ EJECUCIÓN END-TO-END FINALIZADA
================================================================================
Carpeta:
C:\Users\juank\OneDrive\Documentos\MÁSTER EN
IA\TFM\PROTOTIPO_FINAL_YOLO_COCO_ASISTENCIA_CRUCE\20260611_185443~2
```